HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

BÁO CÁO TIỂU LUẬN 03

ĐỀ TÀI: PHÂN TÍCH CÁC HỆ THỐNG THÔNG MINH – TỪ HỌC MÁY, CNN, RNN ĐẾN KNOWLEDGE GRAPH, RAG VÀ HỆ ĐA TÁC TỬ

Lớp:	D23CTPM01
Nhóm:	Nhóm lớp CT – Nhóm 07
Họ và tên:	Nguyễn Duy Nghĩa
Mã sinh viên:	B23DCCN600
Giảng viên hướng dẫn:	PGS.TS Trần Đình Quế
Học kỳ:	Học kỳ 1 năm học 2026 – 2027

Hà Nội, Tháng 11/2026

Mục lục

Danh mục Hình vẽ v

Danh mục Bảng biểu ix

Mở đầu 1

1 Lịch sử phát triển của trí tuệ nhân tạo 3
1.1 Tổng quan . . . 3
1.2 Khởi nguồn và kỷ nguyên logic biểu tượng (1940 – 1960) . . . 4
1.2.1 Máy tính có tư duy và phép thử Turing . . . 4
1.2.2 Nơ-ron hình thức và Hội thảo Dartmouth . . . 4
1.2.3 Thành công bước đầu và trường phái biểu tượng . . . 5
1.3 Mùa đông AI lần thứ nhất (1974 – 1980) . . . 5
1.4 Hệ chuyên gia và mùa đông AI lần thứ hai (1980 – 1993) . . . 6
1.4.1 Kỷ nguyên vàng của hệ chuyên gia . . . 6
1.4.2 Sự tái sinh của mạng nơ-ron . . . 6
1.4.3 Mùa đông thứ hai . . . 6
1.5 Kỷ nguyên học máy thống kê (1993 – 2011) . . . 8
1.6 Cuộc cách mạng học sâu (2012 – 2020) . . . 8
1.6.1 ImageNet 2012 và tiến hoá kiến trúc thị giác . . . 8
1.6.2 Từ RNN đến Transformer . . . 9
1.7 AI tạo sinh và kỷ nguyên tác tử (2020 – 2026) . . . 9
1.7.1 Mô hình ngôn ngữ lớn và định luật tỷ lệ . . . 9
1.7.2 Suy luận, mô hình nhỏ và giới hạn của mô hình thuần tham số . . . 9
1.7.3 Tác tử tự chủ và hệ đa tác tử . . . 9
1.7.4 An toàn, quản trị và AGI . . . 9
1.8 Kết luận chương . . . 10

2 Các kỹ thuật học máy cơ bản 11
2.1 Cơ sở lý thuyết . . . 11
2.1.1 Bài toán học có giám sát và quy trình . . . 11
2.1.2 Hồi quy tuyến tính, Ridge và Lasso . . . 12
2.1.3 Hồi quy Logistic và Softmax . . . 13
2.1.4 Cây quyết định và phương pháp tổ hợp . . . 13
2.1.5 Máy vector hỗ trợ và các thuật toán khác . . . 14
2.1.6 Đánh giá mô hình và quá khớp . . . 14
2.2 Khảo sát và tiền xử lý ba bộ dữ liệu . . . 17
2.2.1 Tiểu đường: sàng lọc nguy cơ . . . 17

2.2.2 Giá nhà: định giá bất động sản . . . 19
2.2.3 Nhận xét khách hàng: phân loại văn bản . . . 21
2.3 Cài đặt và kết quả các mô hình học máy . . . 23
2.3.1 Đường ống tiền xử lý và mô hình . . . 23
2.3.2 Kết quả trên tiểu đường . . . 23
2.3.3 Kết quả trên giá nhà . . . 30
2.3.4 Kết quả trên nhận xét khách hàng . . . 30
2.4 Mạng nơ-ron đa tầng trên ba nền tảng . . . 33
2.4.1 Kiến trúc và lan truyền ngược . . . 33
2.4.2 Hiện thực bằng NumPy . . . 33
2.4.3 Hiện thực bằng Keras và PyTorch . . . 33
2.4.4 Đối chuẩn ba nền tảng . . . 35
2.5 Triển khai Web App . . . 36
2.6 Tổng kết chương . . . 38

3 Mạng nơ-ron tích chập (CNN) 39
3.1 Cơ sở lý thuyết . . . 39
3.1.1 Phép tích chập hai chiều . . . 39
3.1.2 Pooling, vùng tiếp nhận và hàm kích hoạt . . . 40
3.1.3 Tiến hoá kiến trúc . . . 40
3.2 Khảo sát dữ liệu . . . 43
3.3 Hiện thực trên ba nền tảng . . . 45
3.3.1 Hai họ mô hình . . . 45
3.3.2 Tích chập từ đầu bằng NumPy . . . 46
3.3.3 Khối dư bằng PyTorch và Keras . . . 47
3.4 Đánh giá và đối chuẩn . . . 51
3.4.1 Kết quả tổng hợp . . . 51
3.4.2 Kết quả chi tiết theo bộ dữ liệu . . . 51
3.4.3 Chi phí tính toán . . . 54
3.4.4 Bên trong mô hình . . . 54
3.4.5 Diễn giải mô hình bằng Grad-CAM . . . 54
3.5 Triển khai Web App . . . 56
3.6 Tổng kết chương . . . 57

4 Mạng nơ-ron hồi quy (RNN) 58
4.1 Cơ sở lý thuyết . . . 58
4.1.1 Simple RNN và đồ thị tính toán mở rộng . . . 58
4.1.2 BPTT và tiêu biến gradient . . . 58
4.1.3 Cắt chuẩn gradient . . . 59
4.1.4 LSTM và GRU . . . 60
4.1.5 BiLSTM và chống quá khớp . . . 60

4.2 Khảo sát dữ liệu . . . 62
4.2.1 Doanh thu ngày của sàn thương mại điện tử . . . 62
4.2.2 Cổ phiếu Apple . . . 63
4.2.3 Quỹ vàng GLD . . . 64
4.2.4 Tiền xử lý và cửa sổ trượt . . . 64
4.3 Hiện thực trên ba nền tảng . . . 66
4.3.1 RNN từ đầu bằng NumPy . . . 66
4.3.2 Keras và PyTorch . . . 68
4.4 Đánh giá và đối chuẩn . . . 70
4.4.1 Kết quả trên ba chuỗi . . . 70
4.4.2 Mô hình nào thắng dự báo ngây thơ? . . . 73
4.4.3 PyTorch so với Keras . . . 73
4.4.4 Ảnh hưởng của cách chuẩn hoá . . . 74
4.4.5 Độ dài cửa sổ . . . 75
4.4.6 Dự báo nhiều bước . . . 75
4.5 Triển khai Web App . . . 76
4.6 Tổng kết chương . . . 77

5 Knowledge Graph, RAG và tác tử thông minh 78
5.1 Giới hạn của mô hình ngôn ngữ thuần tuý . . . 78
5.2 Knowledge Graph và biểu diễn tri thức . . . 78
5.2.1 Mô hình bộ ba . . . 78
5.2.2 Truy xuất thưa, dày và theo đồ thị . . . 78
5.3 RAG nâng cao . . . 80
5.3.1 Kiến trúc và các thế hệ . . . 80
5.3.2 Chiến lược chia đoạn . . . 80
5.3.3 Sinh có căn cứ và đánh giá . . . 81
5.4 Thiết kế hệ thống thực nghiệm . . . 82
5.4.1 Mô hình hoá KB và KG . . . 83
5.4.2 Bộ truy xuất và bộ câu hỏi đánh giá . . . 83
5.5 Kiến trúc tác tử ReAct đơn lẻ . . . 88
5.6 Đánh giá thực nghiệm . . . 90
5.6.1 Chất lượng truy xuất . . . 90
5.6.2 Chất lượng câu trả lời . . . 92
5.7 Triển khai và điểm nghẽn của tác tử đơn . . . 94
5.8 Tổng kết chương . . . 95

6 Hệ đa tác tử (Multi-Agent Systems) 96
6.1 Tổng quan và cơ sở lý thuyết . . . 96
6.1.1 Từ tác tử đơn sang đa tác tử . . . 96
6.1.2 Cơ sở hình thức . . . 96

6.2 Kiến trúc phối hợp và giao thức 97
6.2.1 Các cấu trúc phối hợp 97
6.2.2 Giao thức truyền thông 97
6.2.3 Các framework hiện đại 98
6.3 Thực nghiệm: đa tác tử giải toán lời văn GSM8K 98
6.3.1 Bài toán và câu hỏi nghiên cứu 98
6.3.2 Thiết kế bốn hệ 99
6.4 Đánh giá định lượng: tác tử đơn và đa tác tử 102
6.5 Triển khai Web App 106
6.6 Tổng kết chương 107

Kết luận **108**

Tài liệu tham khảo **110**

Danh mục Hình vẽ

1 Quy trình thực nghiệm chung của tiểu luận 2

1.1 Mười giai đoạn phát triển của AI; chiều cao khối biểu diễn định tính mức kỳ vọng và đầu tư; hai khối thấp số 4 và 6 là hai "mùa đông" 3
1.2 Quan hệ bao hàm giữa AI, học máy, học sâu và AI tạo sinh; con số là năm hình thành thuật ngữ hoặc bước ngoặt tương ứng 4
1.3 Nơ-ron McCulloch–Pitts (1943) và quy tắc học Perceptron (1958): tổng có trọng số đi qua hàm ngưỡng; trọng số được cập nhật theo sai số 5
1.4 AND và OR tách được bằng một đường thẳng, XOR thì không: giới hạn cốt lõi của Perceptron một tầng 6
1.5 Kiến trúc hệ chuyên gia: tri thức chuyên gia được kỹ sư tri thức mã hoá thành luật; bộ suy diễn áp dụng luật lên dữ kiện 7
1.6 Chu kỳ kỳ vọng và tài trợ cho AI (minh hoạ định tính): hai mùa đông xen giữa các đợt hưng phấn 7
1.7 Ba trụ cột của học sâu: dữ liệu lớn, phần cứng song song và thuật toán cải tiến . 8
1.8 Vòng lặp của một tác tử AI: mục tiêu, lập kế hoạch, hành động bằng công cụ, quan sát kết quả 10

2.1 Quy trình học máy có giám sát tám bước; vòng đứt nét là vòng lặp điều chỉnh . 11
2.2 Hồi quy tuyến tính một biến: đường $\hat{y} = w_0 + w_1 x$ cực tiểu tổng bình phương phần dư 12
2.3 Cây quyết định độ sâu 2 huấn luyện trên hồ sơ tiểu đường; ngưỡng và tỷ lệ được đọc từ mô hình thật 13
2.4 Bagging (trái): các cây độc lập trên mẫu bootstrap, kết quả bỏ phiếu. Boosting (phải): mỗi cây sửa sai cho các cây trước 14
2.5 Kernel trick: hai vòng tròn đồng tâm không tách được trong $\mathbb{R}^2$ trở nên tách được khi ánh xạ lên chiều cao hơn 15
2.6 Ma trận nhầm lẫn và bốn chỉ số phân loại suy ra từ nó 15
2.7 Sai số Train giảm đơn điệu theo bậc đa thức, còn sai số Test có dạng chữ U . . 16
2.8 Dữ liệu tiểu đường: mất cân bằng lớp và tương quan giữa các thuộc tính số với nhãn 18
2.9 Phân bố bốn thuộc tính liên tục theo nhãn (mật độ chuẩn hoá) 18
2.10 Tỷ lệ tiểu đường theo giới tính, tiền sử hút thuốc, tăng huyết áp và bệnh tim . . 19
2.11 Phân bố giá nhà (trái) và giá trung bình theo số phòng ngủ (phải) 20
2.12 Quan hệ giữa giá và các thuộc tính số; đường đỏ là xu hướng tuyến tính 20
2.13 Phân bố giá theo thành phố, mức độ nội thất, đối tượng thuê và nguồn nước . . 21
2.14 Trái: phân bố số sao. Phải: tỷ lệ giới thiệu theo số sao 22
2.15 Đám mây từ của nhận xét được giới thiệu (trái) và không được giới thiệu (phải) 22

2.16 Khảo sát dữ liệu nhận xét khách hàng 22
2.17 Đường ROC (trái) và Precision–Recall (phải) trên tập Test; chú giải ghi AUC và AP 27
2.18 Ma trận nhầm lẫn của các mô hình trên tập Test ở ngưỡng chọn trên Validation (tỷ lệ theo hàng, số lượng trong ngoặc) 28
2.19 Độ quan trọng đặc trưng: hoán vị (mức giảm ROC-AUC trên Validation) và gain của XGBoost 29
2.20 Giá dự đoán so với giá thật trên tập Test của Lasso và Random Forest 30
2.21 Hệ số của Linear Regression (đơn vị tiền trên một độ lệch chuẩn của thuộc tính số, hoặc so với hạng mục tham chiếu) 31
2.22 Trái: ma trận nhầm lẫn của Logistic Regression trên Test. Phải: Softmax Regression dự đoán số sao 32
2.23 Các từ/cụm từ có hệ số dương (giới thiệu) và âm (không giới thiệu) lớn nhất của Logistic Regression 32
2.24 MLP cho dữ liệu bảng: ba lớp ẩn 128–64–32 (Linear, BatchNorm, ReLU, Dropout) và một đầu ra 33
2.25 Trái: mất mát Train/Validation của MLP theo epoch (dừng sớm). Phải: biểu đồ hiệu chuẩn xác suất trên tập Test 36
2.26 Kiến trúc triển khai: trình duyệt gửi JSON tới Web App, Web App gọi dịch vụ dự đoán đã nạp mô hình 36
2.27 Giao diện Web App cho hai thực nghiệm dữ liệu bảng 37
2.28 Giao diện Web App phân loại nhận xét khách hàng 38

3.1 Cấu trúc tổng quát của một CNN phân loại ảnh màu: hai khối Conv – ReLU – Pool trích đặc trưng, tiếp theo là các tầng kết nối đầy đủ 39
3.2 Phép tích chập trên ảnh 6×6 với kernel phát hiện cạnh dọc 3×3, bước 1, không đệm 40
3.3 Max pooling và average pooling 2×2, bước 2 trên bản đồ 4×4 41
3.4 Kiến trúc LeNet-5: C1, C3 là tích chập 5×5; S2, S4 là giảm mẫu; F5, F6 và tầng ra kết nối đầy đủ 41
3.5 Kiến trúc VGG-16: năm khối gồm 2–3 tầng Conv 3×3, sau mỗi khối là Max Pool, cuối cùng là ba tầng FC 42
3.6 Khối dư: nhánh $F(\mathbf{x})$ cộng với lối tắt, cho gradient đi thẳng về các tầng đầu . . 42
3.7 (a) Tích chập chuẩn; (b) tích chập tách được theo chiều sâu của MobileNet . . . 43
3.8 Ảnh ngẫu nhiên theo lớp của hai bộ ảnh xám 44
3.9 Ảnh ngẫu nhiên theo lớp của hai bộ ảnh màu 44
3.10 Tăng cường dữ liệu cho CIFAR-10: cột đầu là ảnh gốc, các cột sau là biến thể . 45
3.11 Hai họ mô hình trong thực nghiệm: Basic CNN và Improved CNN 46
3.12 Bản đồ nhiệt độ chính xác: bốn bộ dữ liệu (hàng) × năm mô hình (cột) 51
3.13 Đường cong học trên CIFAR-10 53
3.14 Ma trận nhầm lẫn trên tập kiểm tra CIFAR-10 54
3.15 Thời gian huấn luyện của các mô hình (giây, thang log) 55

3.16 Số tham số của các mô hình (thang log) . . . 55
3.17 Bên trong Basic CNN PyTorch trên MNIST: bộ lọc đã học (a) và bản đồ đặc trưng (b) . . . 56
3.18 Grad-CAM của Improved CNN PyTorch trên Rock/Paper/Scissors . . . 56
3.19 Giao diện Web App thị giác máy tính . . . 57

4.1 Mở rộng RNN theo thời gian: cùng một ô với cùng trọng số được lặp lại ở mọi bước . . . 58
4.2 BPTT: mất mát tại mỗi bước lan truyền ngược qua toàn bộ chuỗi . . . 59
4.3 Chuẩn gradient theo khoảng cách thời gian (thang log) . . . 59
4.4 Ô LSTM với ba cổng và trạng thái ô chạy xuyên suốt . . . 60
4.5 Ô GRU với cổng cập nhật và cổng đặt lại . . . 61
4.6 RNN hai chiều: trạng thái hai chiều được ghép để tạo đầu ra . . . 61
4.7 Chuỗi doanh thu ngày của sàn thương mại điện tử . . . 62
4.8 Tính mùa vụ của doanh thu theo ngày trong tuần và theo tháng . . . 63
4.9 Giá đóng cửa và khối lượng giao dịch của cổ phiếu AAPL . . . 63
4.10 Lợi suất ngày của cổ phiếu AAPL . . . 64
4.11 Giá quỹ vàng GLD trong giai đoạn khảo sát . . . 64
4.12 Tương quan giữa quỹ GLD và các cột thị trường . . . 65
4.13 Cửa sổ trượt: mỗi khung màu là một mẫu đầu vào, ô đỏ là giá trị cần dự báo . . . 65
4.14 RNN NumPy: MSE Train/Validation theo epoch (trái) và dự báo một bước trên test (phải) . . . 67
4.15 Dự báo doanh thu trên tập test của các mô hình PyTorch . . . 71
4.16 AAPL: phóng to dự báo trên một đoạn của tập test . . . 71
4.17 Dự báo giá quỹ vàng GLD trên tập test của các mô hình PyTorch . . . 72
4.18 Tỷ số RMSE mô hình / RMSE naive trên test cho 8 mô hình $\times$ 3 thực nghiệm . . . 73
4.19 Dự báo của LSTM với Min–Max toàn cục (đỏ) và chuẩn hoá tương đối (xanh, đứt) trên test AAPL (trái) và vàng (phải) . . . 74
4.20 RMSE theo tầm dự báo $h = 1..10$ trên ba thực nghiệm . . . 76
4.21 Giao diện Web App dự báo chuỗi thời gian . . . 77

5.1 Giải phẫu bộ ba (chủ thể, quan hệ, đối tượng) và ví dụ của ba miền dữ liệu trong chương . . . 79
5.2 Cùng một dữ kiện giao dịch biểu diễn theo LPG và theo RDF . . . 79
5.3 Hợp nhất thứ hạng RRF từ ba nguồn BM25, dense và đồ thị (trọng số đồ thị $w_G = 2$) . . . 80
5.4 Kiến trúc RAG ba làn: nạp tri thức, truy xuất lai và sinh có căn cứ . . . 80
5.5 Năm thế hệ RAG: độ phức tạp, khả năng suy luận và chi phí tăng dần . . . 81
5.6 Kiến trúc năm tầng của hệ thống KG – RAG trong chương . . . 82
5.7 Lược đồ KG thị trường: ngày – tháng – năm, chuỗi `next_day` và nút khái niệm biến động . . . 85

5.8 Đồ thị con của hai KG: hồ sơ nối tới các nút khái niệm dùng chung; ngày giao dịch nối theo chuỗi `next_day` . . . 86
5.9 Tác tử định tuyến: liên kết thực thể quyết định công cụ chính xác hay RAG . . 89
5.10 Recall@1, Recall@10, MRR và nDCG@10 của mười cấu hình truy xuất 91
5.11 Recall@1 với câu hỏi khớp mặt chữ và câu hỏi diễn đạt lại . . . 91
5.12 MRR@10 theo miền và bộ truy xuất . . . 92
5.13 Độ chính xác đáp án của bốn hệ trên ba miền . . . 93
5.14 Giao diện chatbot trên Web App . . . 94
5.15 Chatbot quỹ vàng GLD: câu hỏi tổng hợp năm 2013 . . . 95

6.1 Vòng lặp của một tác tử dựa trên LLM: quan sát, suy luận, hành động trên môi trường . . . 96
6.2 Bốn mẫu tổ chức hệ đa tác tử: điều phối – thợ, đường ống, tranh luận và bảng đen 97
6.3 Hai giao thức bổ sung cho nhau: MCP nối tác tử với công cụ, A2A nối tác tử với tác tử . . . 98
6.4 Bốn hệ được so sánh, từ một lượt gọi đến một nhóm năm vai trò có cổng đồng thuận . . . 99
6.5 Trình tự thông điệp của hệ đa tác tử ở nhánh bất đồng . . . 101
6.6 Độ chính xác của bốn hệ; thanh lỗi là khoảng tin cậy 95% bootstrap . . . 103
6.7 Chi phí trung bình mỗi câu: số lượt gọi LLM, số token (vào + ra) và thời gian . 103
6.8 Cổng đồng thuận: tỷ lệ câu hai Solver khớp nhau (trái) và độ chính xác ở từng nhánh, của cả hệ và cận trên oracle (phải) . . . 104
6.9 Cơ cấu lỗi của bốn hệ . . . 104
6.10 Hệ đa tác tử giải bài toán lời văn trên Web App: đáp án và vết từng vai trò . . . 107

Danh mục Bảng biểu

1.1 Sự chuyển dịch mô hình tính toán qua các giai đoạn 10

2.1 Thống kê mô tả các thuộc tính số của dữ liệu tiểu đường (trước làm sạch) . . . 17
2.2 Thống kê mô tả các thuộc tính số của dữ liệu giá nhà 19
2.3 Đối chiếu cài đặt NumPy từ đầu với scikit-learn trên tập Test 23
2.4 Tiểu đường: kết quả trên tập Test, ngưỡng cố định từ Validation 24
2.5 Precision, Recall, F1 ở ngưỡng 0,5 và ở ngưỡng tối ưu F1 (*) chọn trên Validation 27
2.6 Kiểm định chéo 5-fold trên tập Train: trung bình $\pm$ độ lệch chuẩn 29
2.7 Giá nhà: kết quả hồi quy trên tập Test 30
2.8 Nhận xét khách hàng: kết quả trên tập Test 31
2.9 Đối chuẩn MLP trên ba nền tảng và ba tập dữ liệu (tập Test) 35
2.10 Tóm tắt ba thực nghiệm của Chương 2 38

3.1 Bốn bộ dữ liệu ảnh của chương sau khi tải và chia tập 43
3.2 Kiến trúc Basic CNN cho ảnh xám 28×28 45
3.3 Độ chính xác (%) trên tập kiểm tra của mọi mô hình trên bốn bộ dữ liệu 51
3.4 Mức cải thiện của Improved CNN so với Basic CNN trong cùng framework . . 52
3.5 Kết quả các mô hình trên tập kiểm tra MNIST 52
3.6 Kết quả các mô hình trên tập kiểm tra Fashion-MNIST 52
3.7 Kết quả các mô hình trên tập kiểm tra Rock/Paper/Scissors 53
3.8 Kết quả chi tiết trên tập kiểm tra CIFAR-10 53

4.1 Ba bộ dữ liệu chuỗi thời gian của chương 62
4.2 Số tham số và tốc độ huấn luyện của bốn kiến trúc 68
4.3 E-commerce: kết quả dự báo doanh thu ngày trên tập test 70
4.4 AAPL: kết quả dự báo giá đóng cửa trên tập test 70
4.5 Vàng GLD: kết quả dự báo giá đóng cửa trên tập test 72
4.6 RMSE trên test của cùng kiến trúc trong hai framework 73
4.7 Ảnh hưởng của cách chuẩn hoá tới dự báo trên test 74
4.8 Tỷ số RMSE/naive theo độ dài cửa sổ 75

5.1 So sánh đặc tính vận hành của LLM đơn lẻ, fine-tuning và hệ RAG 78
5.2 So sánh các chiến lược chia đoạn văn bản trong RAG 81
5.3 Ba bộ dữ liệu của chương và quy mô kho tài liệu 83
5.4 Quy mô KB, KG và thời gian dựng của ba miền 86
5.5 Số câu hỏi đánh giá theo miền và loại 88
5.6 Chất lượng truy xuất của mười cấu hình, gộp ba miền 90
5.7 Quét trọng số nhánh đồ thị w_G và số ứng viên rerank n 92

5.8 Độ chính xác câu trả lời và độ trễ của bốn hệ hỏi đáp 93
5.9 Số lần tác tử chọn mỗi công cụ 94

6.1 So sánh ba framework đa tác tử phổ biến 98
6.2 Độ chính xác, khoảng tin cậy 95% và chi phí trung bình mỗi câu của bốn hệ trên GSM8K 102
6.3 Hệ đa tác tử so với hai tác tử đơn: số câu sửa đúng, làm hỏng và chi phí phụ trội 103
6.4 Phân loại câu sai của từng hệ 104
KL.1 Tổng hợp kết quả các thực nghiệm (chỉ số trên tập kiểm thử) 108

Mở đầu

Bối cảnh và lý do chọn đề tài

Chỉ trong hơn một thập niên, trí tuệ nhân tạo (Artificial Intelligence – AI) đã đi từ phòng thí nghiệm vào đời sống: nhận diện khuôn mặt, dịch tự động, phát hiện gian lận, trợ lý hội thoại. Bước ngoặt có ý nghĩa quyết định là chuyển từ *viết tay tri thức* sang *học từ dữ liệu*. Các hệ chuyên gia mã hoá quy tắc của con người chỉ tốt trong miền hẹp và khó bảo trì; học máy (machine learning) cung cấp ví dụ để thuật toán tự điều chỉnh tham số. Trong thực tế, một hệ thống thông minh phải xử lý nhiều hình thái thông tin, và mỗi hình thái đòi hỏi một họ kỹ thuật riêng:

- **Dữ liệu bảng:** các thuật toán học máy cổ điển (hồi quy, cây quyết định, SVM, gradient boosting) và mạng nơ-ron đa tầng (MLP) nắm bắt quan hệ phi tuyến giữa các đặc trưng có cấu trúc.
- **Dữ liệu ảnh:** mạng nơ-ron tích chập (CNN) khai thác tính cục bộ và chia sẻ trọng số.
- **Dữ liệu chuỗi:** mạng nơ-ron hồi quy (RNN) cùng các biến thể có cổng LSTM, GRU ghi nhớ quan hệ theo thời gian.
- **Tri thức có cấu trúc và tác tử:** đồ thị tri thức (Knowledge Graph – KG), sinh tăng cường truy xuất (Retrieval-Augmented Generation – RAG) [1] và hệ đa tác tử giúp mô hình ngôn ngữ lớn (LLM) trả lời có căn cứ và phối hợp nhiều bước xử lý [2].

Xuất phát từ đó, tiểu luận chọn đề tài **phân tích các hệ thống thông minh**, đi từ lịch sử hình thành, qua học máy cơ bản, CNN, RNN, đến KG, RAG và hệ đa tác tử. Mỗi chương đều có thực nghiệm tự cài đặt, chạy trên dữ liệu thật, đánh giá định lượng và đóng gói thành ứng dụng web.

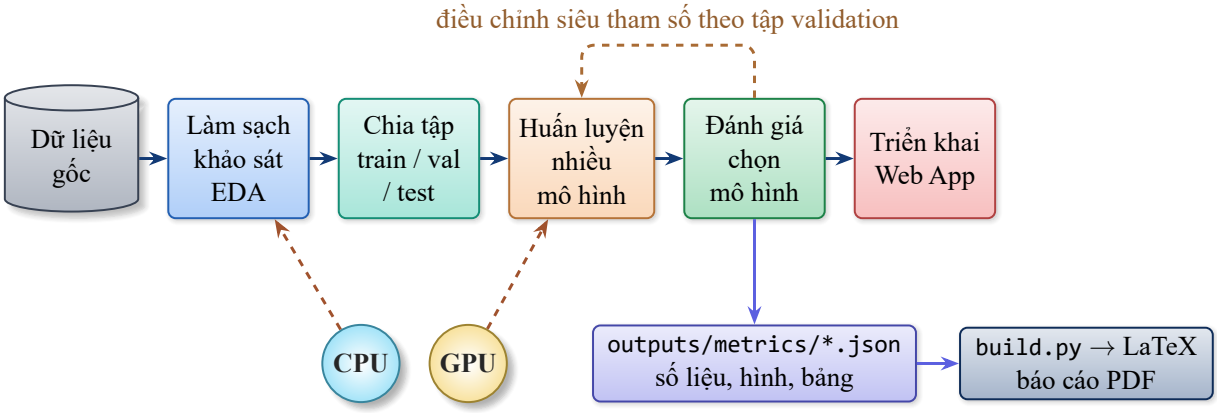
Mục tiêu và nhiệm vụ

Mục tiêu tổng quát là xây dựng nền tảng kiến thức học thuật gắn với năng lực thực hành trên toàn bộ tiến trình phát triển của các hệ thống thông minh. Sáu nhiệm vụ cụ thể gồm:

1. **Khảo cứu lịch sử AI:** các giai đoạn phát triển, hai mùa đông AI, cuộc cách mạng học sâu và kỷ nguyên tác tử.
2. **Nghiên cứu cơ sở lý thuyết:** công thức, hàm mất mát và tối ưu của học máy, tích chập, cổng nhớ LSTM/GRU, bộ ba tri thức, truy xuất lai và các mô hình phối hợp tác tử.
3. **Khảo sát thực nghiệm đa miền:** y tế (tiểu đường), bất động sản, thương mại điện tử, ảnh, tài chính (cổ phiếu, vàng) và bài toán suy luận GSM8K.
4. **Hiện thực đa nền tảng:** cài đặt từ đầu bằng NumPy [3], TensorFlow–Keras [4, 5], PyTorch [6], cùng scikit-learn [7], XGBoost [8], NetworkX [9] và Hugging Face Transformers [10].
5. **Đánh giá, đối chuẩn:** so sánh mô hình và nền tảng trên cùng dữ liệu, cùng phép chia, có đường cơ sở (baseline); so sánh tác tử đơn với hệ đa tác tử.
6. **Triển khai:** tích hợp mô hình vào một Web App Flask có biểu mẫu nhập liệu và chatbot.

Phương pháp và môi trường thực nghiệm

Mọi thực nghiệm đi theo cùng một quy trình (Hình 1). Hai nguyên tắc được giữ xuyên suốt: mô hình và ngưỡng chỉ được chọn trên tập kiểm định (validation), tập kiểm thử (test) chỉ dùng một lần; mọi con số trong báo cáo được đọc tự động từ tệp kết quả JSON và mã in trong báo cáo được trích nguyên văn từ mã đã chạy. Seed được cố định bằng 42.



Hình 1. Quy trình thực nghiệm chung của tiểu luận

Thực nghiệm chạy trên máy CPU AMD Ryzen 9 7945HX (32 luồng), GPU NVIDIA RTX 4060 Laptop 8 GB, RAM 40 GB, Windows 11, Python 3.13. TensorFlow từ bản 2.11 không còn hỗ trợ GPU trực tiếp trên Windows nên Keras chạy trên CPU, còn PyTorch và LLM chạy trên GPU; nhờ vậy hai nhóm việc chạy song song được với nhau.

Bố cục báo cáo

Ngoài Mở đầu và Kết luận, báo cáo gồm sáu chương:

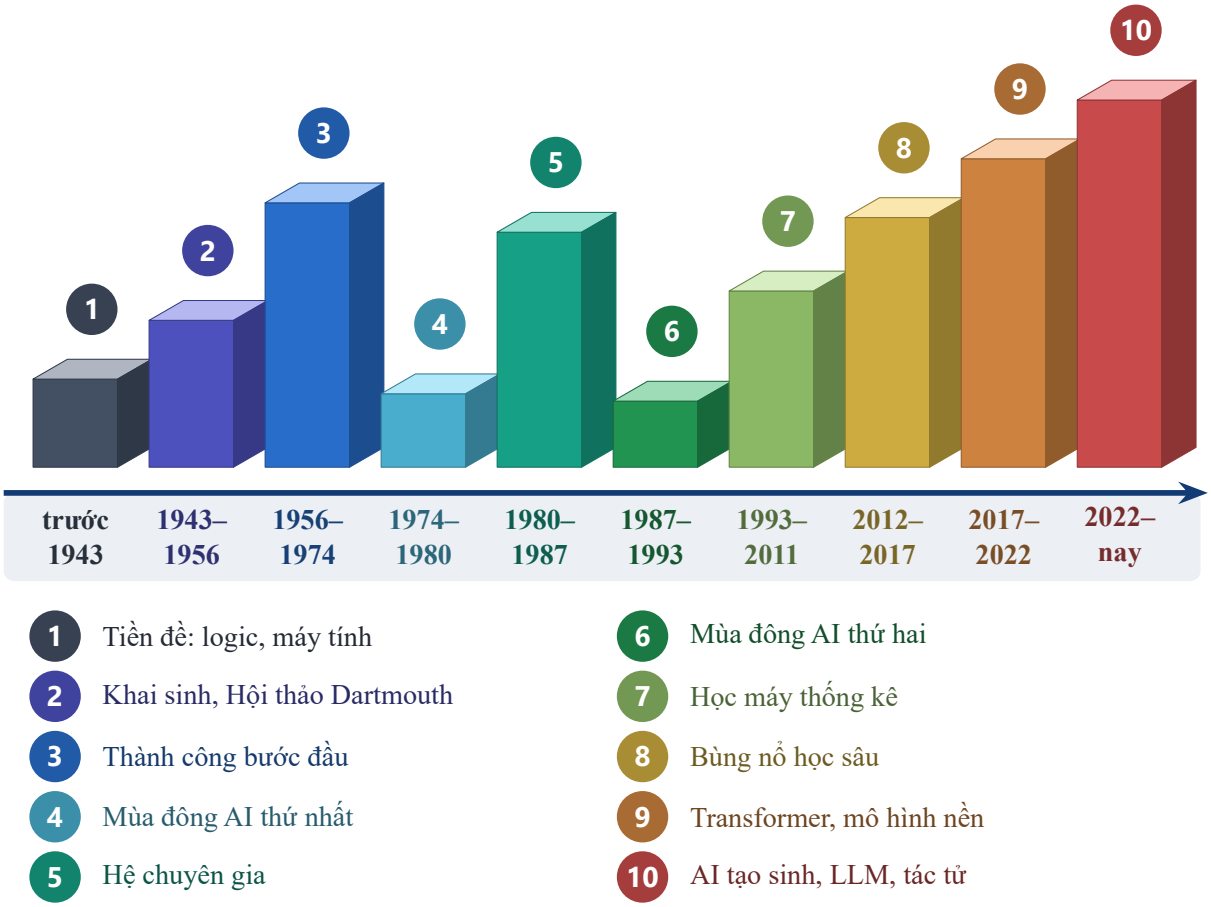
- **Chương 1. Lịch sử phát triển của AI:** các giai đoạn từ 1940 đến 2026, hai mùa đông AI và sự chuyển dịch mô hình tính toán.
- **Chương 2. Các kỹ thuật học máy cơ bản:** lý thuyết thuật toán cốt lõi; EDA ba bộ dữ liệu; cài đặt, đối chuẩn MLP trên ba nền tảng; triển khai.
- **Chương 3. Mạng nơ-ron tích chập:** tích chập, pooling, vùng tiếp nhận, tiến hoá kiến trúc; thực nghiệm MNIST, Fashion-MNIST, Rock/Paper/Scissors, CIFAR-10 trên ba nền tảng.
- **Chương 4. Mạng nơ-ron hồi quy:** BPTT, tiêu biến đạo hàm, LSTM, GRU, BiLSTM; dự báo doanh thu, cổ phiếu AAPL và quỹ vàng GLD.
- **Chương 5. Knowledge Graph, RAG và tác tử:** bộ ba tri thức, truy xuất lai, chia đoạn; hỏi đáp trên ba miền dữ liệu và tác tử định tuyến công cụ.
- **Chương 6. Hệ đa tác tử:** cơ sở hình thức, kiến trúc phối hợp, giao thức; so sánh định lượng tác tử đơn và đa tác tử trên GSM8K.

Chương 1

Lịch sử phát triển của trí tuệ nhân tạo

1.1 Tổng quan

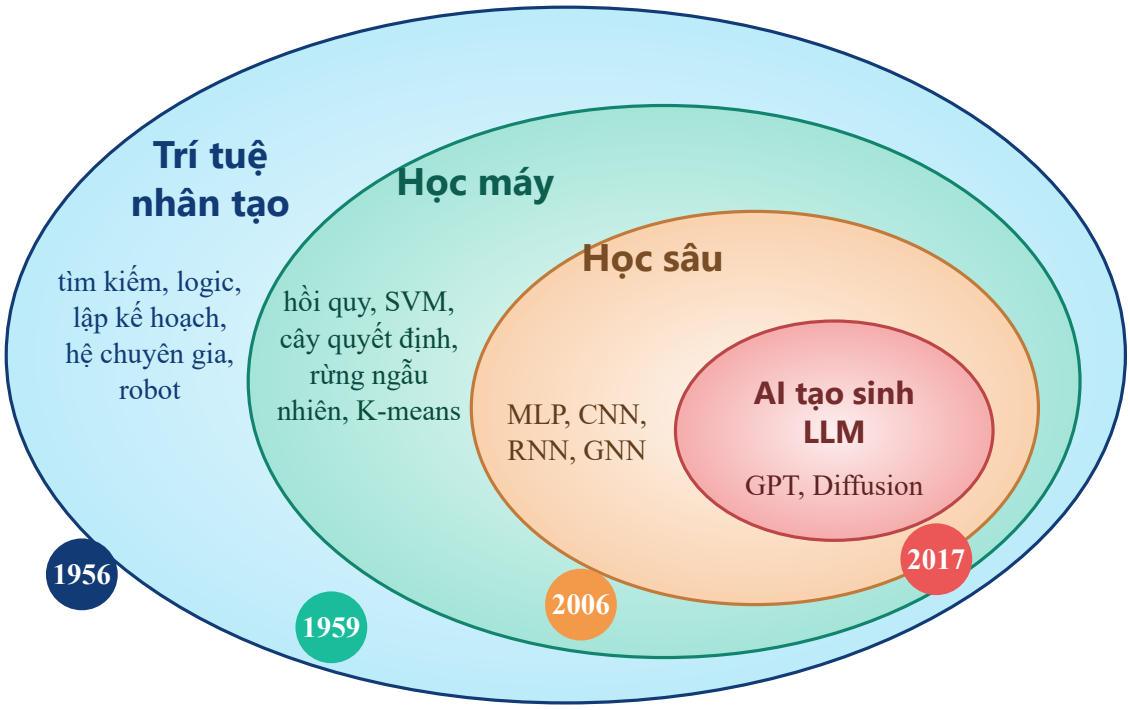
Trí tuệ nhân tạo là lĩnh vực nghiên cứu cách xây dựng các hệ thống tính toán thực hiện những nhiệm vụ mà khi con người làm thì ta gọi là "thông minh": nhận biết, suy luận, học từ kinh nghiệm, lập kế hoạch, giao tiếp và hành động có mục đích [11]. Lịch sử của AI không phải một đường thẳng đi lên mà là chuỗi chu kỳ: lời hứa lớn, thành công trong phạm vi hẹp, thất vọng khi mở rộng, cắt giảm tài trợ, rồi phục hồi dưới một phương pháp mới [12]. Hình 1.1 tóm tắt các giai đoạn chính; chiều cao mỗi khối thể hiện định tính mức kỳ vọng và đầu tư của xã hội.



Hình 1.1. Mười giai đoạn phát triển của AI; chiều cao khối biểu diễn định tính mức kỳ vọng và đầu tư; hai khối thấp số 4 và 6 là hai "mùa đông"

Cần phân biệt các khái niệm thường bị dùng lẫn lộn (Hình 1.2). AI là lĩnh vực rộng nhất, gồm cả các phương pháp không học từ dữ liệu như tìm kiếm và logic; học máy xây dựng mô hình từ dữ liệu; học sâu dùng mạng nơ-ron nhiều tầng; AI tạo sinh và LLM là một phần của học sâu, chuyên sinh nội dung mới. Russell và Norvig xếp các định nghĩa AI theo hai trục (suy nghĩ

hay hành động; chuẩn con người hay tính duy lý); giáo trình hiện đại lấy *tác tử duy lý*, tức chủ thể chọn hành động tối đa hoá kết quả kỳ vọng theo thước đo hiệu năng, làm khung chung [11].



Hình 1.2. Quan hệ bao hàm giữa AI, học máy, học sâu và AI tạo sinh; con số là năm hình thành thuật ngữ hoặc bước ngoặt tương ứng

1.2 Khởi nguồn và kỷ nguyên logic biểu tượng (1940 – 1960)

1.2.1 Máy tính có tư duy và phép thử Turing

Năm 1950, Alan Turing đặt lại câu hỏi "máy có thể suy nghĩ không?" thành một thí nghiệm kiểm chứng được: nếu người thẩm vấn trò chuyện qua văn bản mà không phân biệt được máy với người thì máy được xem là có hành vi thông minh [13]. Searle sau đó phản biện bằng thí nghiệm căn phòng tiếng Trung: thao tác đúng trên ký hiệu chưa chắc là hiểu [14]. Cùng giai đoạn, điều khiển học của Wiener [15] và lý thuyết thông tin của Shannon [16] cung cấp ngôn ngữ toán học để nói về tín hiệu, phản hồi và thông tin.

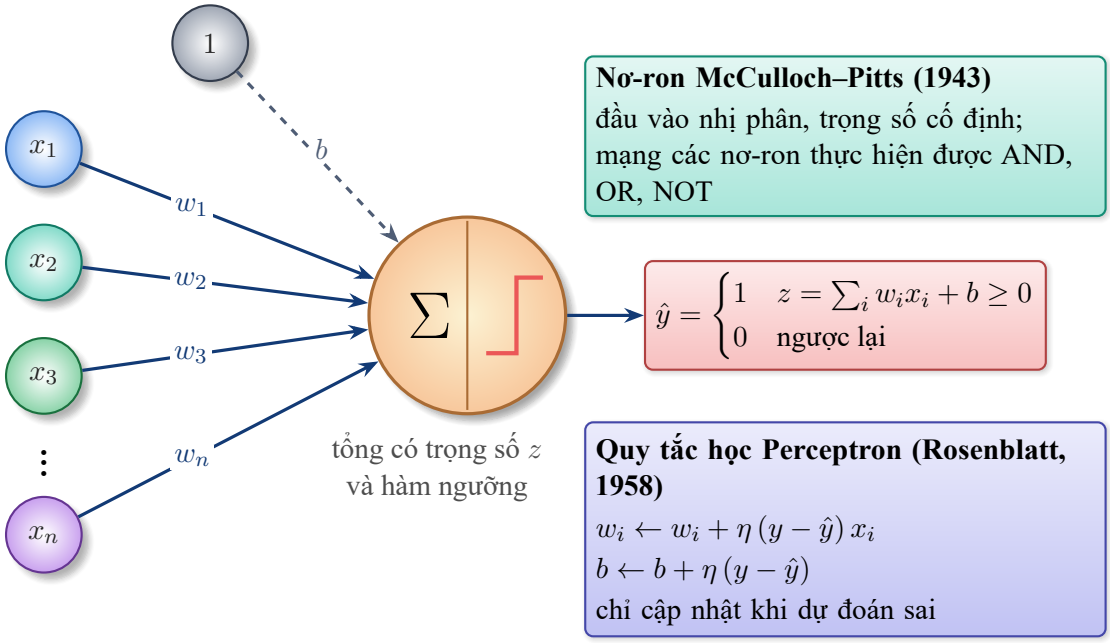
1.2.2 Nơ-ron hình thức và Hội thảo Dartmouth

McCulloch và Pitts (1943) mô tả nơ-ron như một đơn vị ngưỡng tính được các hàm logic [17]. Rosenblatt (1958) thêm khả năng học: Perceptron điều chỉnh trọng số khi dự đoán sai [18], với

$$\hat{y} = \operatorname{sign}(\mathbf{w}^{\top}\mathbf{x} + b), \qquad \mathbf{w} \leftarrow \mathbf{w} + \eta\,(y - \hat{y})\,\mathbf{x}. \tag{1.1}$$

Hội thảo Dartmouth mùa hè 1956 do McCarthy, Minsky, Rochester và Shannon đề xuất đã đặt tên cho lĩnh vực, dựa trên giả định rằng mọi khía cạnh của trí thông minh đều có thể mô tả chính

xác để máy mô phỏng [19]. Cũng tại đây, Newell và Simon giới thiệu Logic Theorist, chương trình chứng minh được các định lý trong *Principia Mathematica* [20].



Hình 1.3. Nơ-ron McCulloch–Pitts (1943) và quy tắc học Perceptron (1958): tổng có trọng số đi qua hàm ngưỡng; trọng số được cập nhật theo sai số

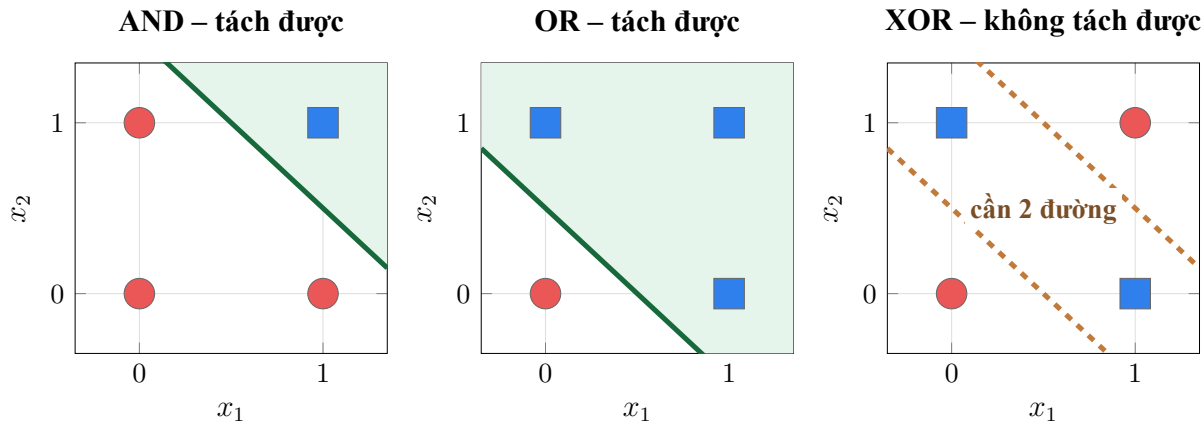
1.2.3 Thành công bước đầu và trường phái biểu tượng

Giai đoạn 1956–1974 chứng kiến các chương trình giải đố bằng tìm kiếm, ELIZA mô phỏng hội thoại [21], SHRDLU hiểu lệnh trong "thế giới khối" [22], và chương trình cờ đam tự học của Samuel [23]. Giả thuyết hệ ký hiệu vật lý của Newell và Simon [24] xem thao tác trên ký hiệu là điều kiện cần và đủ của trí thông minh. Thành công trong các thế giới thu nhỏ dẫn tới những dự đoán quá lạc quan về thời điểm máy đạt tới trí tuệ con người.

1.3 Mùa đông AI lần thứ nhất (1974 – 1980)

Năm 1969, Minsky và Papert chứng minh Perceptron một tầng chỉ biểu diễn được các hàm tách tuyến tính [25]. Ví dụ kinh điển là XOR (Hình 1.4): bốn điểm $(0,0), (1,1)$ nhãn 0 và $(0,1), (1,0)$ nhãn 1 không thể tách bằng một đường thẳng, vì hệ bất phương trình $b < 0$, $w_1 + b > 0$, $w_2 + b > 0$, $w_1 + w_2 + b < 0$ vô nghiệm. Cuốn sách làm nghiên cứu mạng nơ-ron gần như ngừng lại.

Trước đó, báo cáo ALPAC (1966) đã kết luận dịch máy không đáp ứng kỳ vọng [26]. Báo cáo Lighthill (1973) phê phán AI không vượt được bùng nổ tổ hợp ngoài các bài toán đồ chơi [27], dẫn tới cắt giảm tài trợ ở Anh; DARPA tại Mỹ cũng chuyển sang chỉ cấp vốn cho dự án có mục tiêu ứng dụng rõ ràng. Nguyên nhân kỹ thuật có ba: năng lực tính toán thấp, bùng nổ tổ hợp, và thiếu tri thức thường thức mà một đứa trẻ vẫn có.



Hình 1.4. AND và OR tách được bằng một đường thẳng, XOR thì không: giới hạn cốt lõi của Perceptron một tầng

1.4 Hệ chuyên gia và mùa đông AI lần thứ hai (1980 – 1993)

1.4.1 Kỷ nguyên vàng của hệ chuyên gia

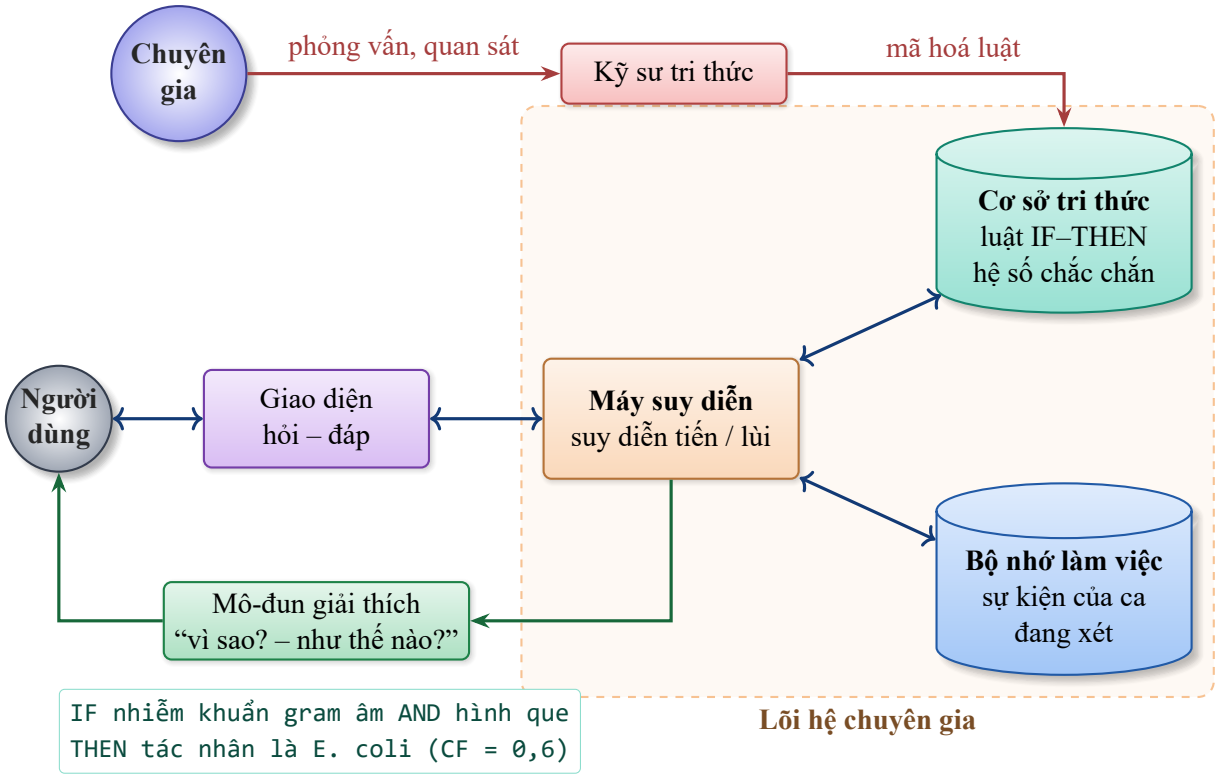
Hướng đi mới là giới hạn miền và mã hoá tri thức chuyên gia thành luật `NẾU -- THÌ` (Hình 1.5). MYCIN chẩn đoán nhiễm khuẩn huyết [28, 29]; R1/XCON cấu hình máy tính VAX cho DEC và là một trong những hệ thương mại thành công đầu tiên [30]. Nhật Bản khởi động dự án Máy tính thế hệ thứ năm (1982), kéo theo các chương trình đầu tư cấp quốc gia ở Mỹ và châu Âu. Dự án Cyc đặt mục tiêu mã hoá tri thức thường thức [31].

1.4.2 Sự tái sinh của mạng nơ-ron

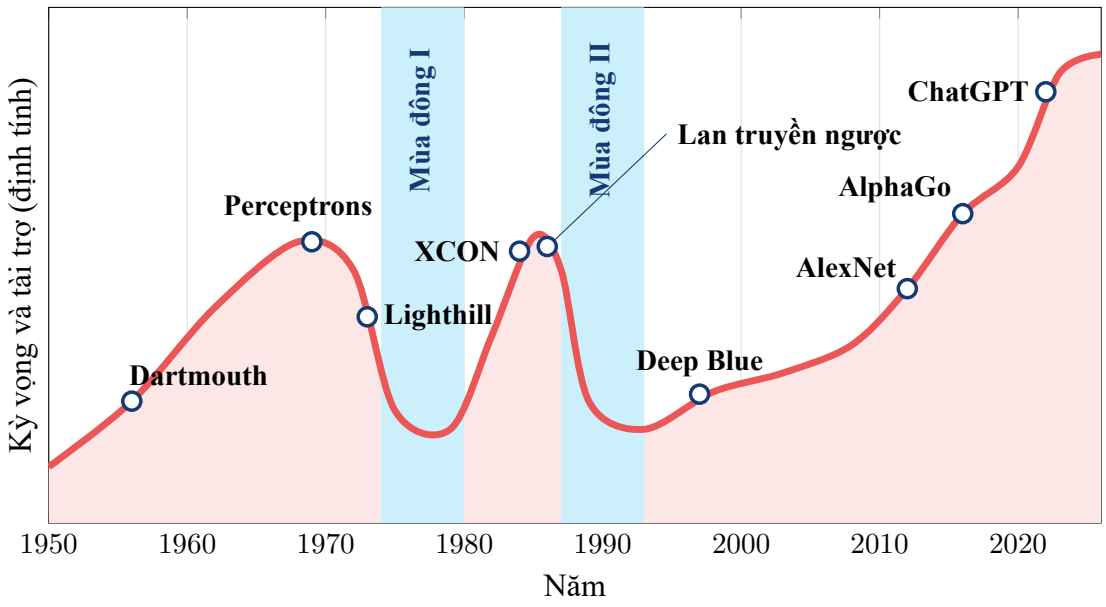
Mạng Hopfield (1982) đưa cơ học thống kê vào mạng nơ-ron [32]. Năm 1986, Rumelhart, Hinton và Williams phổ biến **lan truyền ngược** [33]: dùng quy tắc chuỗi để tính đạo hàm của mất mát theo mọi trọng số trong mạng nhiều tầng, nhờ đó vượt được giới hạn XOR.

1.4.3 Mùa đông thứ hai

Năm 1987 thị trường máy LISP sụp đổ khi máy trạm phổ thông rẻ và mạnh hơn. Hệ chuyên gia lộ các điểm yếu: đắt bảo trì, giòn ở ngoài miền đã mã hoá, nút thắt thu thập tri thức. Dự án thế hệ thứ năm không đạt mục tiêu; tài trợ lại bị cắt. Trong khi đó, Brooks đề xuất robot phản xạ không cần biểu diễn tri thức [34], Pearl đưa suy luận xác suất vào AI [35]. Hình 1.6 khái quát hai chu kỳ này.



Hình 1.5. Kiến trúc hệ chuyên gia: tri thức chuyên gia được kỹ sư tri thức mã hoá thành luật; bộ suy diễn áp dụng luật lên dữ kiện



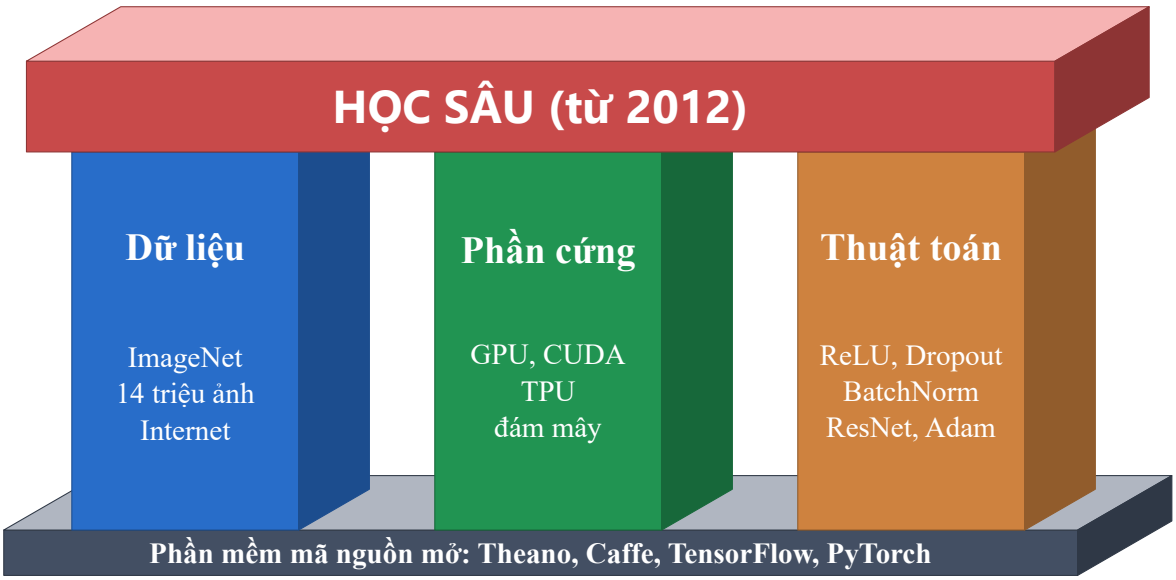
Hình 1.6. Chu kỳ kỳ vọng và tài trợ cho AI (minh hoạ định tính): hai mùa đông xen giữa các đợt hưng phấn

1.5 Kỷ nguyên học máy thống kê (1993 – 2011)

AI trở lại bằng toán học, thực nghiệm và tiêu chuẩn đánh giá khách quan. Máy vector hỗ trợ tìm siêu phẳng có lề lớn nhất [36]; ID3 [37] mở đường cho cây quyết định và các phương pháp tổ hợp; học tăng cường được hệ thống hoá [38] với TD-Gammon [39] là minh chứng sớm. LSTM (1997) giải quyết việc nhớ xa trong chuỗi [40], còn LeNet-5 (1998) đọc séc ngân hàng bằng mạng tích chập [41]. Các cột mốc công chúng gồm Deep Blue thắng Kasparov năm 1997 [42], Stanley vượt sa mạc trong DARPA Grand Challenge 2005 [43] và Watson thắng *Jeopardy!* năm 2011 [44].

1.6 Cuộc cách mạng học sâu (2012 – 2020)

Học sâu bùng nổ khi ba điều kiện hội tụ (Hình 1.7): dữ liệu lớn như ImageNet [45], GPU tính song song, và các cải tiến thuật toán từ huấn luyện từng tầng [46] đến ReLU, dropout, chuẩn hoá theo lô [47, 48].



Hình 1.7. Ba trụ cột của học sâu: dữ liệu lớn, phần cứng song song và thuật toán cải tiến

1.6.1 ImageNet 2012 và tiến hoá kiến trúc thị giác

Tại ILSVRC 2012, AlexNet đạt lỗi top-5 khoảng 15,3%, thấp hơn hẳn mức 26,2% của đội xếp thứ hai [49, 50]. ResNet (2015) dùng kết nối tắt để huấn luyện mạng 152 tầng, đưa lỗi xuống 3,57% [51]. Năm 2016, AlphaGo thắng Lee Sedol [52], rồi AlphaGo Zero tự học hoàn toàn từ tự chơi [53]. GAN [54] và VAE [55] mở đầu cho mô hình tạo sinh hiện đại; AlphaFold 2 giải bài toán dự đoán cấu trúc protein [56].

1.6.2 Từ RNN đến Transformer

Word2vec biểu diễn từ thành vector [57]; seq2seq [58] và attention [59] đưa mạng hồi quy vào dịch máy. Transformer (2017) bỏ hẳn hồi quy, chỉ dùng tự chú ý [60]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{1.2}$$

cho phép xử lý song song và nắm quan hệ xa. BERT [61] và GPT [62, 63] đặt nền móng cho các mô hình nền (foundation model) [64].

1.7 AI tạo sinh và kỷ nguyên tác tử (2020 – 2026)

1.7.1 Mô hình ngôn ngữ lớn và định luật tỷ lệ

GPT-3 (2020) cho thấy quy mô tạo ra chất lượng: sai số giảm theo luỹ thừa của lượng tính toán và dữ liệu [65, 66], trong khi tính toán huấn luyện tăng nhanh hơn nhiều so với định luật Moore [67]. ChatGPT ra mắt cuối năm 2022 nhờ học tăng cường từ phản hồi người [68, 69] đưa AI tới đại chúng. Mô hình khuếch tán [70] đem lại bước nhảy cho sinh ảnh; Gemini [71] và Llama [72, 73] mở rộng sang đa phương thức và trọng số mở.

1.7.2 Suy luận, mô hình nhỏ và giới hạn của mô hình thuần tham số

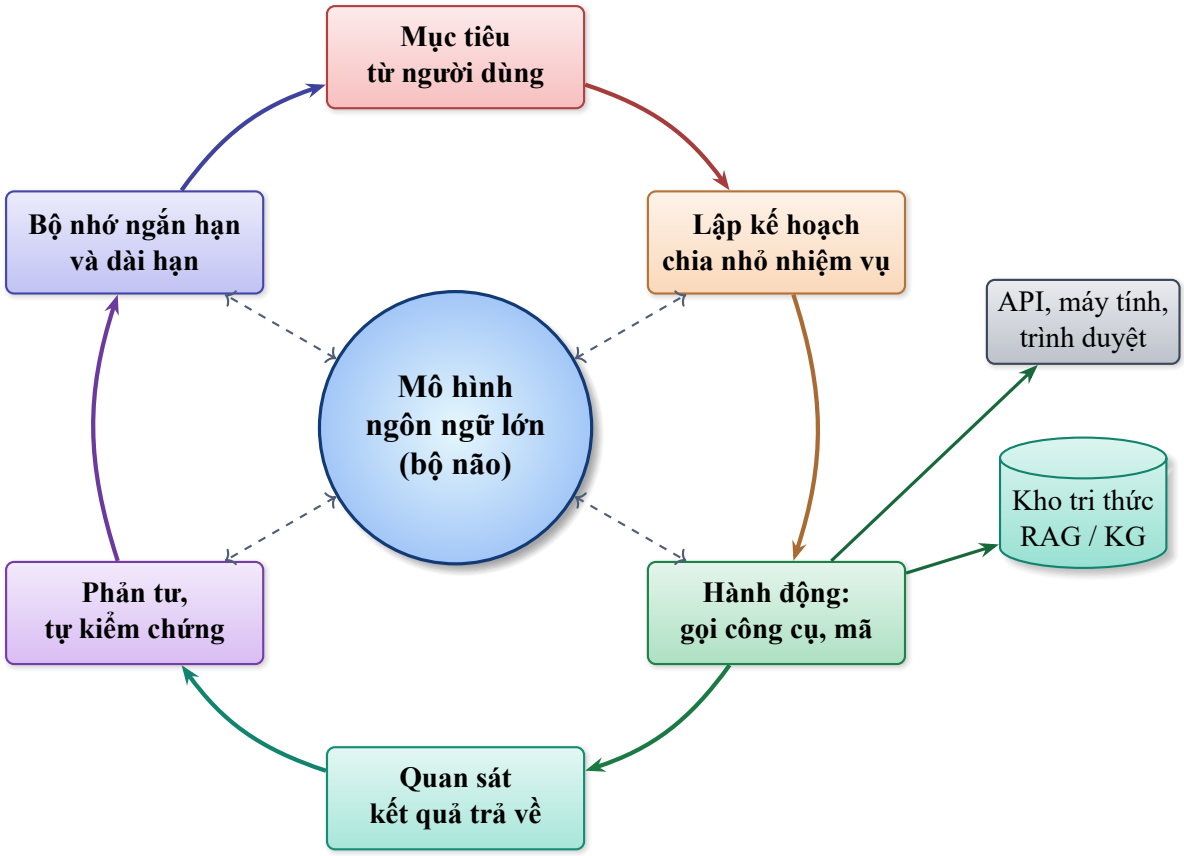
Từ năm 2024, các mô hình suy luận như o1 [74] và DeepSeek-R1 [75] dành thời gian "suy nghĩ" trước khi trả lời; mô hình ngôn ngữ nhỏ chạy được trên máy cá nhân, như Qwen dùng trong tiểu luận [76]. Tuy vậy, LLM vẫn có giới hạn nội tại: tri thức đóng băng tại thời điểm huấn luyện, có thể sinh thông tin sai với giọng tự tin, và không dẫn được nguồn. RAG [1] và đồ thị tri thức bù vào các điểm yếu đó bằng cách đưa bằng chứng bên ngoài vào lúc suy luận (Chương 5).

1.7.3 Tác tử tự chủ và hệ đa tác tử

Tác tử kết hợp LLM với công cụ theo vòng lặp suy nghĩ – hành động – quan sát (Hình 1.8) [2, 77]. Khi nhiệm vụ phức tạp, nhiều tác tử chuyên biệt phối hợp với nhau theo các kiến trúc được trình bày ở Chương 6.

1.7.4 An toàn, quản trị và AGI

Sự bùng nổ đi kèm các rủi ro: thiên lệch, tiêu tốn năng lượng [78], an toàn hệ thống [79, 80]. Liên minh châu Âu thông qua Đạo luật AI năm 2024 [81]; Việt Nam có Chiến lược quốc gia về AI theo Quyết định 127/QĐ-TTg [82]. AI tổng quát (AGI) vẫn là chủ đề tranh luận: ngay cách đo trí thông minh cũng chưa có đồng thuận [83], và tính tái lập của các kết quả học máy vẫn là vấn đề được cộng đồng quan tâm [84].



Hình 1.8. Vòng lặp của một tác tử AI: mục tiêu, lập kế hoạch, hành động bằng công cụ, quan sát kết quả

1.8 Kết luận chương

Bảng 1.1 tóm tắt sự chuyển dịch mô hình tính toán. Ba bài học được rút ra: mỗi lần phục hồi đều gắn với một thay đổi về dữ liệu, phần cứng hoặc thuật toán; phương pháp bị bỏ rơi có thể trở lại khi điều kiện chín muồi (mạng nơ-ron); và các lời hứa vượt quá năng lực luôn phải trả giá. Các chương sau đi vào từng họ kỹ thuật hiện đại, bắt đầu từ học máy cơ bản.

Bảng 1.1. Sự chuyển dịch mô hình tính toán qua các giai đoạn

Giai đoạn	Phương pháp chính	Thành tựu tiêu biểu	Giới hạn
1950–1974	Tìm kiếm, logic biểu tượng	Logic Theorist, ELIZA	Bùng nổ tổ hợp
1980–1987	Hệ chuyên gia, luật	MYCIN, XCON	Giòn, khó bảo trì
1993–2011	Học máy thống kê	SVM, Deep Blue, Watson	Phải tự thiết kế đặc trưng
2012–2020	Học sâu	AlexNet, ResNet, AlphaGo	Cần nhiều dữ liệu có nhãn
2020–2026	LLM, RAG, tác tử	GPT, ChatGPT, DeepSeek-R1	Ảo giác, chi phí, an toàn

Chương 2

Các kỹ thuật học máy cơ bản

Theo Mitchell [85], một chương trình *học* từ kinh nghiệm E đối với nhiệm vụ T và độ đo P nếu hiệu năng trên T, đo bằng P, cải thiện nhờ E. Chương này trình bày các thuật toán cốt lõi, khảo sát ba bộ dữ liệu (tiểu đường, giá nhà, nhận xét khách hàng), cài đặt và đối chuẩn mô hình, mở rộng sang MLP trên ba nền tảng, rồi đóng gói thành dịch vụ web.

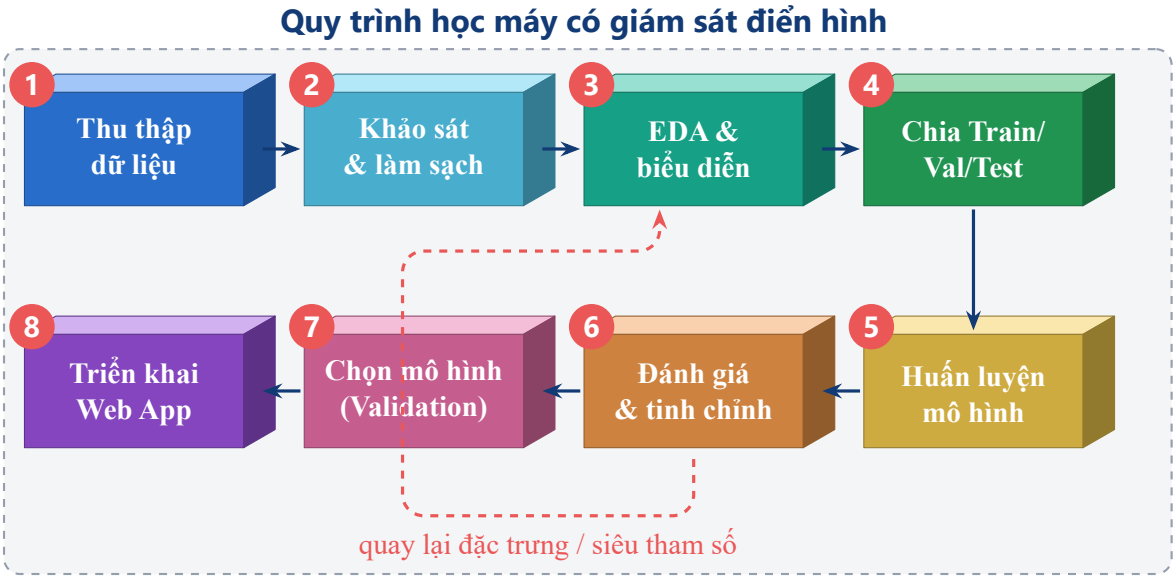
2.1 Cơ sở lý thuyết

2.1.1 Bài toán học có giám sát và quy trình

Tập dữ liệu gồm n cặp $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$ với $\mathbf{x}_i \in \mathbb{R}^d$. Nhãn y_i rời rạc ứng với phân loại, liên tục ứng với hồi quy. Huấn luyện là cực tiểu hoá rủi ro thực nghiệm, thường có thêm hạng chính quy hoá:

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^{n} \ell\big(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i\big) + \lambda R(\boldsymbol{\theta}). \tag{2.1}$$

Khi không có nghiệm đóng, tham số được cập nhật theo Gradient Descent: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J$. Quy trình đầy đủ ở Hình 2.1; dữ liệu được chia 70/15/15 thành Train, Validation, Test, trong đó Test chỉ dùng một lần sau khi đã cố định mô hình.



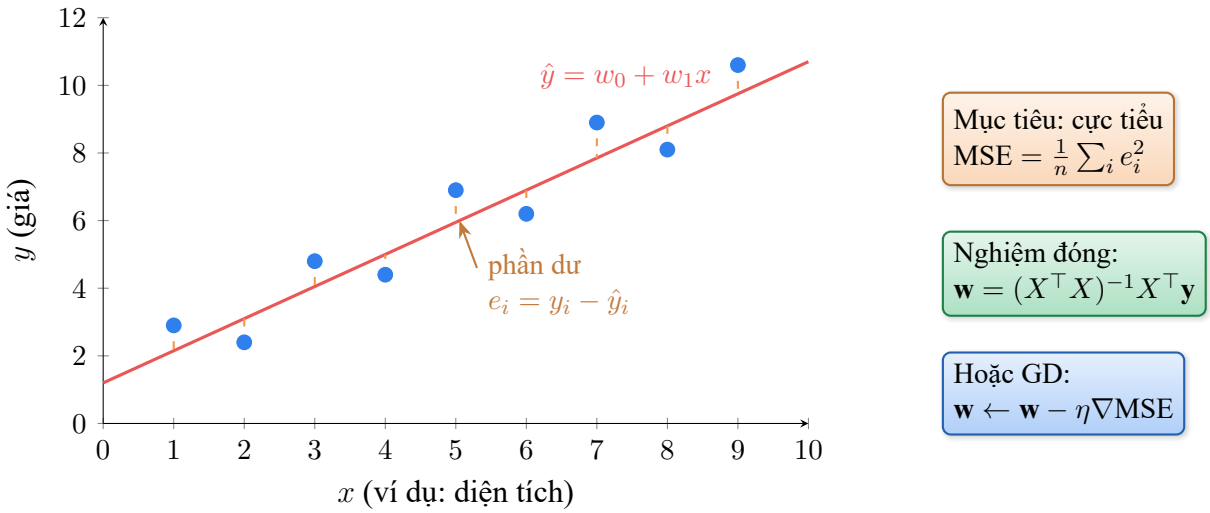
Hình 2.1. Quy trình học máy có giám sát tám bước; vòng đứt nét là vòng lặp điều chỉnh

2.1.2 Hồi quy tuyến tính, Ridge và Lasso

Mô hình $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$ cực tiểu sai số bình phương trung bình. Với ma trận thiết kế X (đã thêm cột 1), nghiệm đóng là phương trình chuẩn:

$$J(\mathbf{w}) = \frac{1}{n}\|X\mathbf{w} - \mathbf{y}\|_2^2, \qquad \mathbf{w}^* = (X^\top X)^{-1} X^\top \mathbf{y}. \tag{2.2}$$

Ridge [86] thêm phạt $\lambda\|\mathbf{w}\|_2^2$, co đều các hệ số và cho nghiệm $(X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$. Lasso [87] thêm phạt $\lambda\|\mathbf{w}\|_1$, đưa một phần hệ số về đúng 0 nên có tác dụng chọn đặc trưng.



Hình 2.2. Hồi quy tuyến tính một biến: đường $\hat{y} = w_0 + w_1 x$ cực tiểu tổng bình phương phần dư

Đoạn mã 2.1. Hồi quy tuyến tính bằng Gradient Descent trên MSE (NumPy)

```python
# tl3/ch2_scratch.py
def linreg_gd(X, y, lr=0.05, epochs=500):
    """Hồi quy tuyến tính bằng Gradient Descent trên MSE: ∇w = (2/n)·Xᵀ(Xw − y)."""
    Xb = np.hstack([np.ones((len(X), 1)), X])
    w = np.zeros(Xb.shape[1])
    hist = []
    for _ in range(epochs):
        r = Xb @ w - y
        w -= lr * 2 / len(y) * (Xb.T @ r)
        hist.append(float(np.mean(r ** 2)))
    return w, hist
```

2.1.3 Hồi quy Logistic và Softmax

Hồi quy logistic [88] đưa tổ hợp tuyến tính qua hàm sigmoid và ước lượng bằng hợp lý cực đại, tương đương cực tiểu entropy chéo:

$$p = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}, \qquad J = -\frac{1}{n} \sum_{i=1}^{n} \big[y_i \log p_i + (1 - y_i) \log(1 - p_i) \big]. \tag{2.3}$$

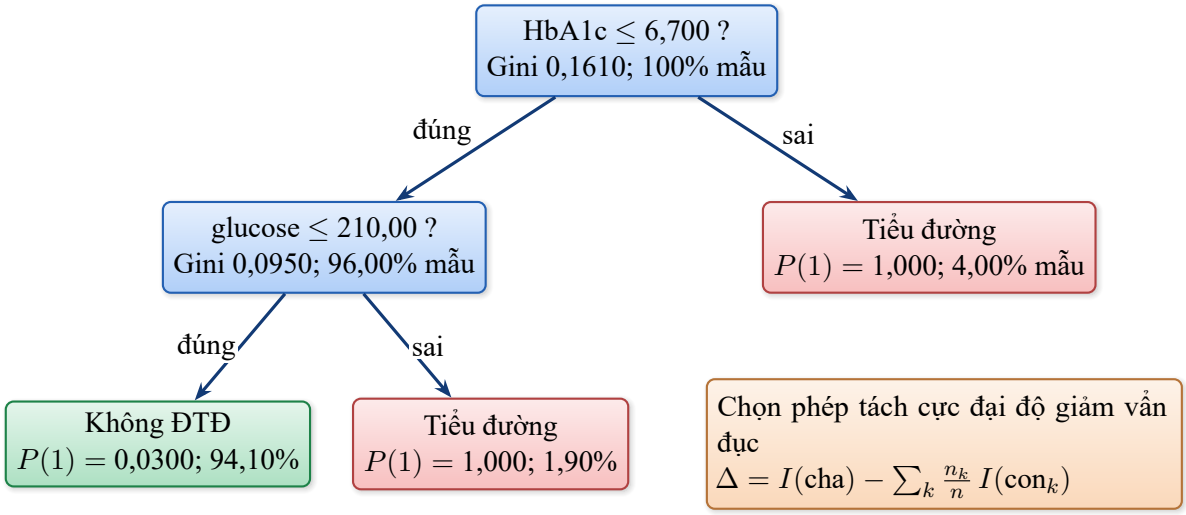
Với K lớp, Softmax Regression thay sigmoid bằng $P(y = k \mid \mathbf{x}) = \exp(\mathbf{w}_k^\top \mathbf{x}) / \sum_j \exp(\mathbf{w}_j^\top \mathbf{x})$. Nhãn được quyết định bằng cách so xác suất với một ngưỡng; ngưỡng 0,5 không nhất thiết tốt nhất khi lớp mất cân bằng.

2.1.4 Cây quyết định và phương pháp tổ hợp

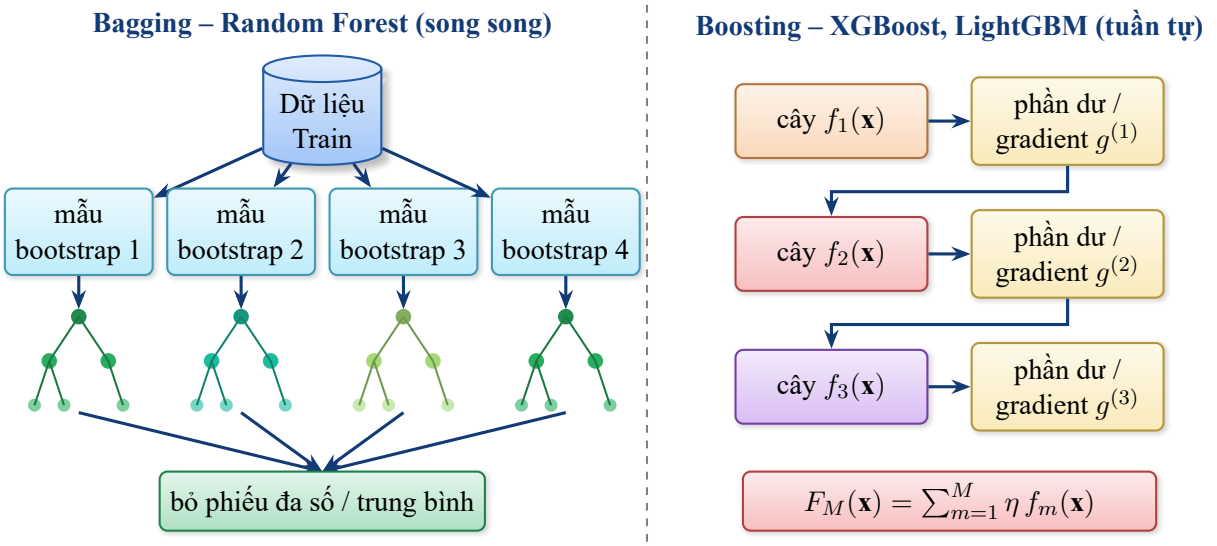
Cây quyết định [89, 90] chia không gian đặc trưng theo các ngưỡng, ở mỗi nút chọn phép tách giảm độ vẩn đục nhiều nhất. Với p_k là tỷ lệ lớp k trong nút, chỉ số Gini và entropy là

$$G = 1 - \sum_k p_k^2, \qquad H = -\sum_k p_k \log_2 p_k. \tag{2.4}$$

Cây dễ diễn giải nhưng phương sai cao. Hai hướng tổ hợp khắc phục điểm này (Hình 2.4): **Bagging** và Random Forest [91, 92] huấn luyện các cây độc lập trên mẫu bootstrap rồi bỏ phiếu, giảm phương sai; **Boosting** [93] xây cây tuần tự, cây sau khớp sai số của các cây trước, giảm thiên lệch. XGBoost [94] và LightGBM [95] là hai cài đặt gradient boosting hiệu quả.



Hình 2.3. Cây quyết định độ sâu 2 huấn luyện trên hồ sơ tiểu đường; ngưỡng và tỷ lệ được đọc từ mô hình thật



Hình 2.4. Bagging (trái): các cây độc lập trên mẫu bootstrap, kết quả bỏ phiếu. Boosting (phải): mỗi cây sửa sai cho các cây trước

2.1.5 Máy vector hỗ trợ và các thuật toán khác

SVM [96] tìm siêu phẳng có lề lớn nhất, cho phép vi phạm qua biến nới lỏng ξ_i:

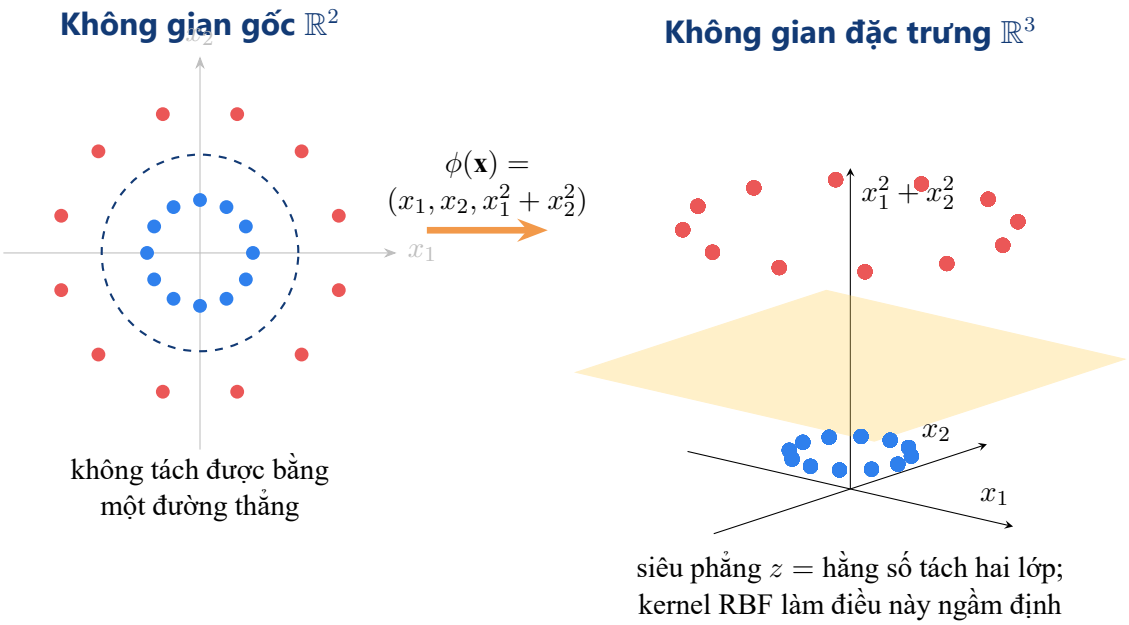
$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i \quad \text{với} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i,\ \ \xi_i \geq 0. \tag{2.5}$$

Thủ thuật kernel thay tích vô hướng bằng $k(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$ để có biên phi tuyến; SVR [97] áp dụng ý tưởng này cho hồi quy. KNN [98] dự đoán theo k láng giềng gần nhất; Naive Bayes [99] áp dụng định lý Bayes với giả định các đặc trưng độc lập có điều kiện theo lớp.

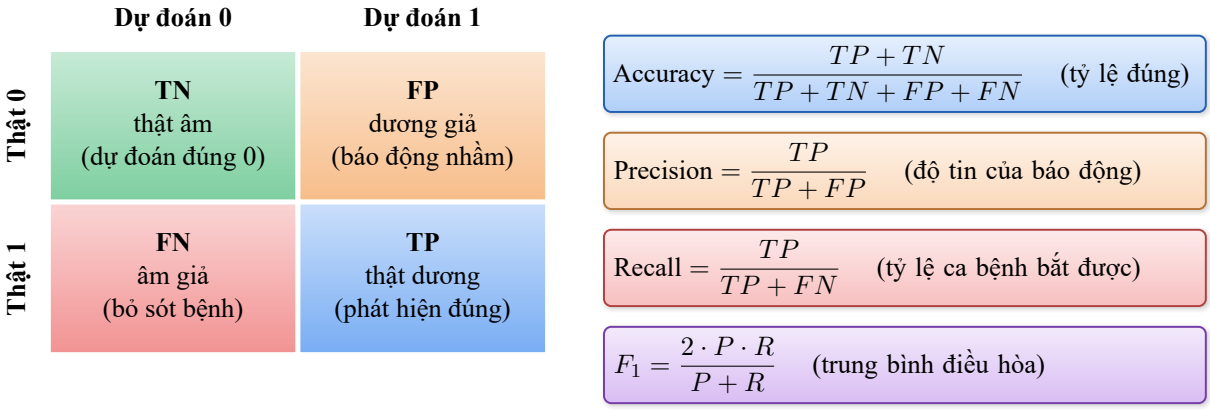
2.1.6 Đánh giá mô hình và quá khớp

Từ ma trận nhầm lẫn (Hình 2.6) ta có Precision $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$, Recall $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$ và F1 là trung bình điều hoà của hai đại lượng này. ROC-AUC [100] đo khả năng xếp hạng; với dữ liệu mất cân bằng, đường Precision–Recall có tính thông tin cao hơn [101]. Cho hồi quy, ta dùng MAE, RMSE và hệ số xác định $R^2 = 1 - \sum_i (y_i - \hat{y}_i)^2 / \sum_i (y_i - \bar{y})^2$.

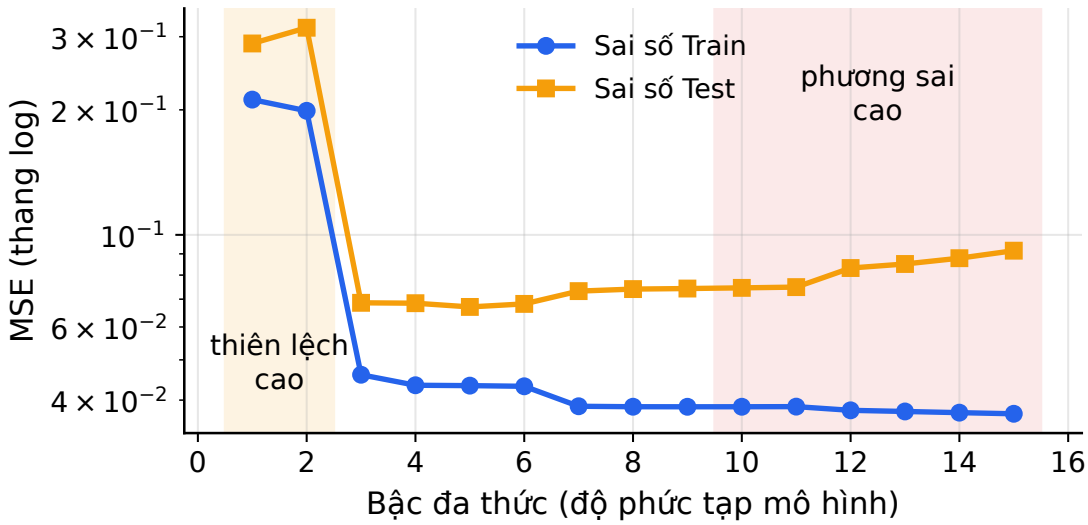
Sai số kỳ vọng phân rã thành thiên lệch bình phương, phương sai và nhiễu không giảm được [102]. Hình 2.7 cho thấy sai số Test có dạng chữ U theo độ phức tạp: mô hình quá đơn giản thì chưa khớp, quá phức tạp thì quá khớp.



Hình 2.5. Kernel trick: hai vòng tròn đồng tâm không tách được trong $\mathbb{R}^2$ trở nên tách được khi ánh xạ lên chiều cao hơn



Hình 2.6. Ma trận nhầm lẫn và bốn chỉ số phân loại suy ra từ nó



Hình 2.7. Sai số Train giảm đơn điệu theo bậc đa thức, còn sai số Test có dạng chữ U

2.2 Khảo sát và tiền xử lý ba bộ dữ liệu

2.2.1 Tiểu đường: sàng lọc nguy cơ

Bộ *Diabetes Prediction Dataset* [103] gồm 100.000 hồ sơ, 9 cột: `gender`, `smoking_history`, `hypertension`, `heart_disease`, `age`, `bmi`, `HbA1c_level`, `blood_glucose_level` và nhãn `diabetes`. Bài toán là phân loại nhị phân, trong đó bỏ sót người bệnh nghiêm trọng hơn báo động nhầm; theo ADA, HbA1c $\geq 6{,}5\%$ là ngưỡng chẩn đoán [104].

Dữ liệu không có ô trống (0 giá trị thiếu) nhưng có 3.854 bản ghi trùng. Sau khi bỏ trùng còn 96.146 hồ sơ, tỷ lệ dương 8,82%. Các giá trị ngoại lai theo IQR được giữ lại vì đó là chỉ số lâm sàng có thật của người bệnh, không phải lỗi nhập liệu (Đoạn mã 2.2).

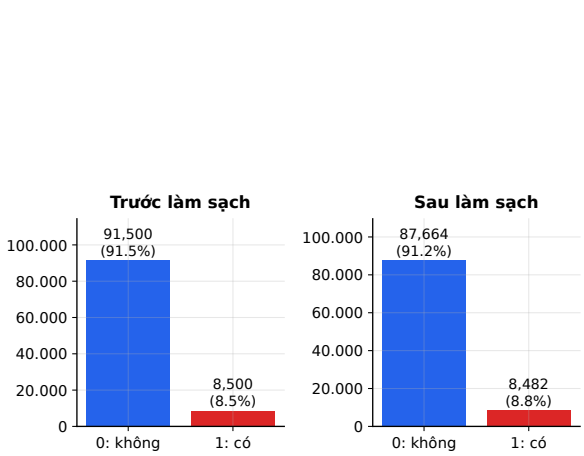
Bảng 2.1. Thống kê mô tả các thuộc tính số của dữ liệu tiểu đường (trước làm sạch)

Thuộc tính	Trung bình	Độ lệch chuẩn	Min	Trung vị	Max	Q3
age	41,89	22,52	0,08	43,00	80,00	60,00
hypertension	0,07	0,26	0,00	0,00	1,00	0,00
heart_disease	0,04	0,20	0,00	0,00	1,00	0,00
bmi	27,32	6,64	10,01	27,32	95,69	29,58
HbA1c_level	5,53	1,07	3,50	5,80	9,00	6,20
blood_glucose_level	138,06	40,71	80,00	140,00	300,00	159,00

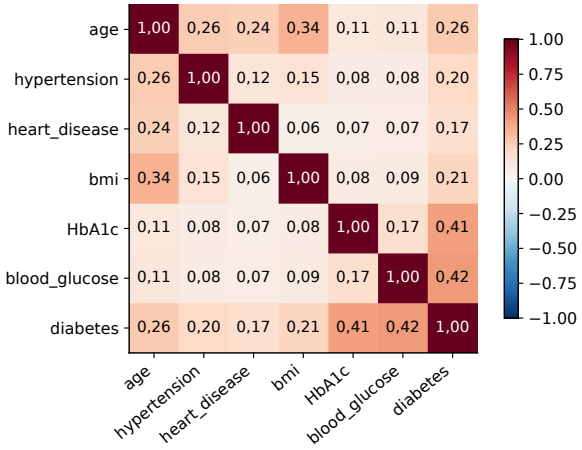
Đoạn mã 2.2. Làm sạch dữ liệu tiểu đường: bỏ trùng lặp, chỉ loại giá trị bất khả thi

```python
# tl3/ch2_data.py
def clean_diabetes(df):
    """Bỏ bản ghi trùng; kiểm tra miền giá trị hợp lệ; giữ ngoại lai (có ý nghĩa lâm sàng);
    'No Info' của smoking_history giữ là một hạng mục riêng thay vì đoán giá trị."""
    out = df.drop_duplicates().reset_index(drop=True)
    valid = (out["age"].between(0, 120) & out["bmi"].between(10, 100) &
             out["HbA1c_level"].between(3, 15)
             & out["blood_glucose_level"].between(50, 400)
             & out[["hypertension", "heart_disease", DIAB_TARGET]].isin([0, 1]).all(axis=1))
    return out[valid].reset_index(drop=True)
```

Nhận xét: Glucose (0,4243) và HbA1c (0,4064) tương quan với nhãn mạnh nhất, tiếp theo là tuổi (0,2649) và BMI (0,2149). Hình 2.9 cho thấy quan hệ có dạng ngưỡng: vượt một mức HbA1c hay glucose nhất định thì gần như toàn bộ là ca bệnh. Dạng quan hệ này báo trước lợi thế của mô hình cây. Tỷ lệ bệnh ở người tăng huyết áp là 27,96%, so với 7,21% ở người không tăng huyết áp.

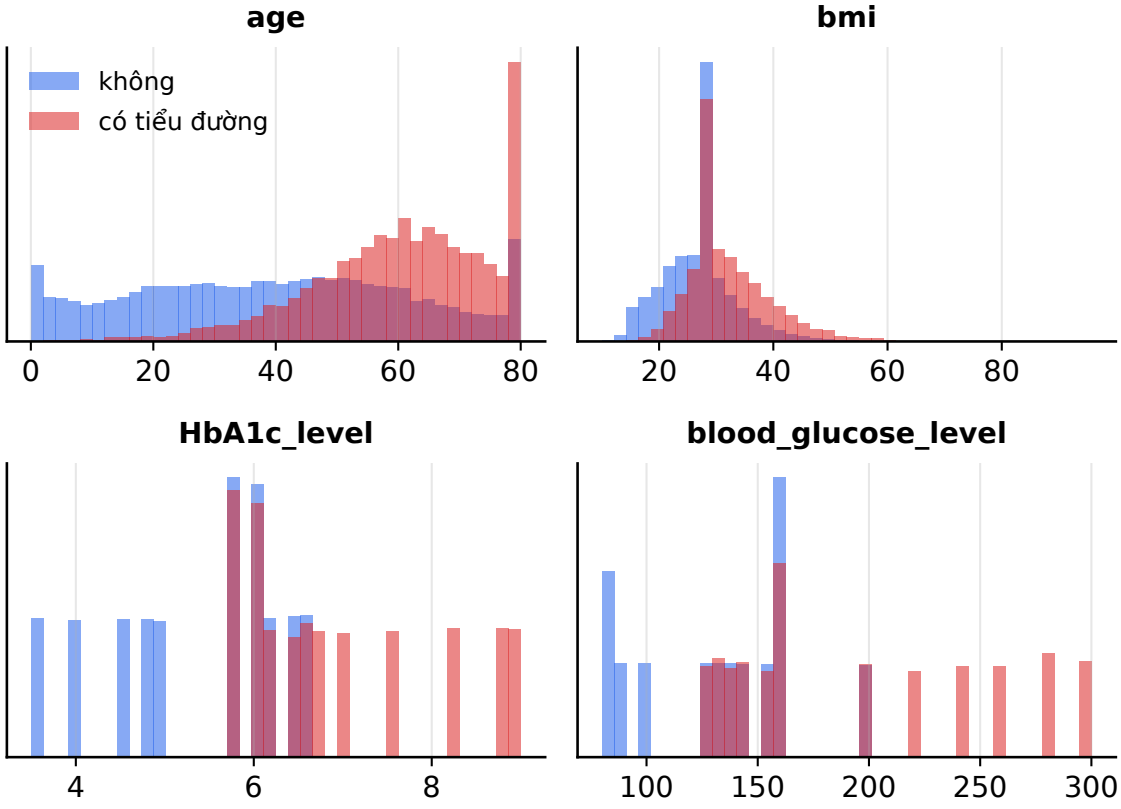


(a) Phân bố nhãn

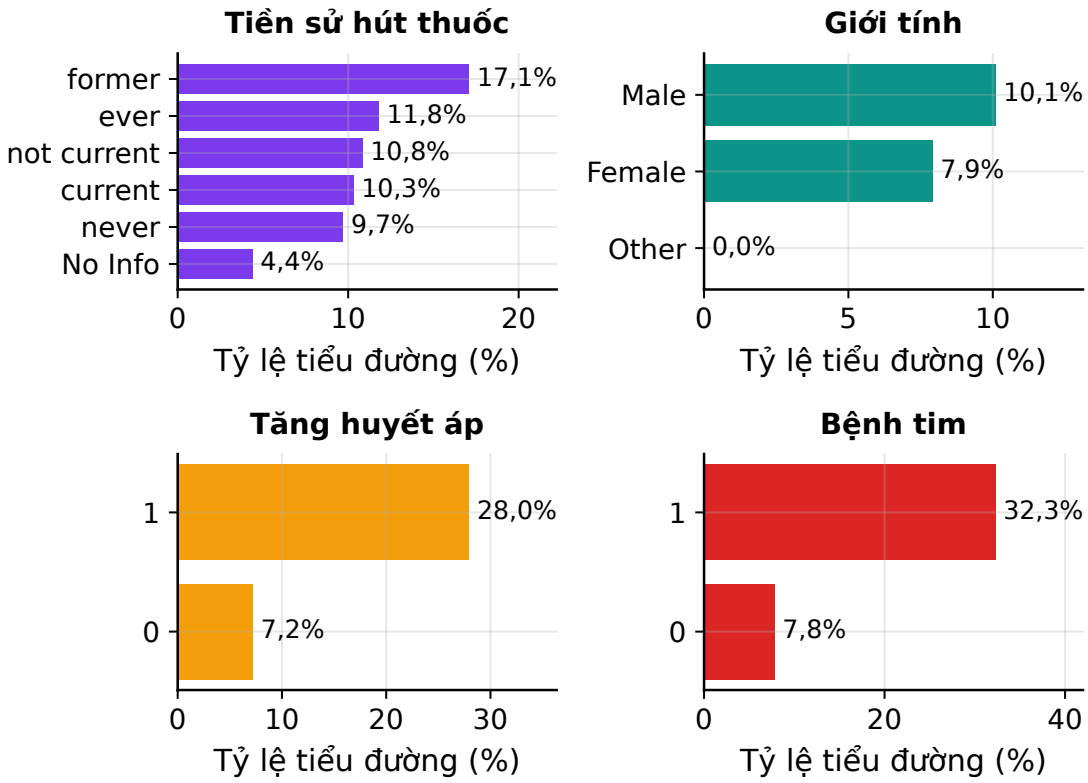


(b) Tương quan Pearson

Hình 2.8. Dữ liệu tiểu đường: mất cân bằng lớp và tương quan giữa các thuộc tính số với nhãn



Hình 2.9. Phân bố bốn thuộc tính liên tục theo nhãn (mật độ chuẩn hoá)



Hình 2.10. Tỷ lệ tiểu đường theo giới tính, tiền sử hút thuốc, tăng huyết áp và bệnh tim

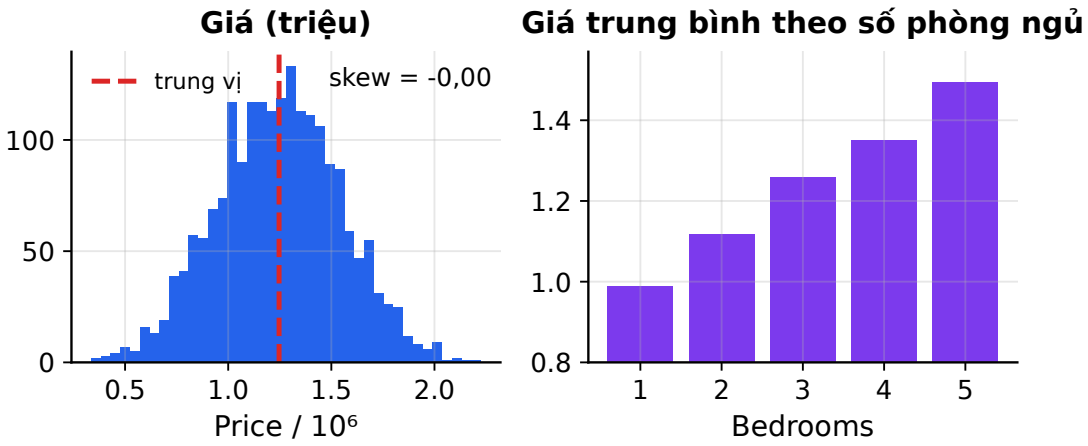
2.2.2 Giá nhà: định giá bất động sản

Bộ *House Price Prediction* [105] gồm 2.000 căn nhà tại bảy thành phố Ấn Độ, 16 cột, với bảy thuộc tính số (diện tích, phòng ngủ, phòng tắm, số tầng, chỗ đỗ xe, tuổi nhà, điểm khu vực), tám thuộc tính phân loại và nhãn `Price`. Dữ liệu có 0 giá trị thiếu và 0 bản ghi trùng.

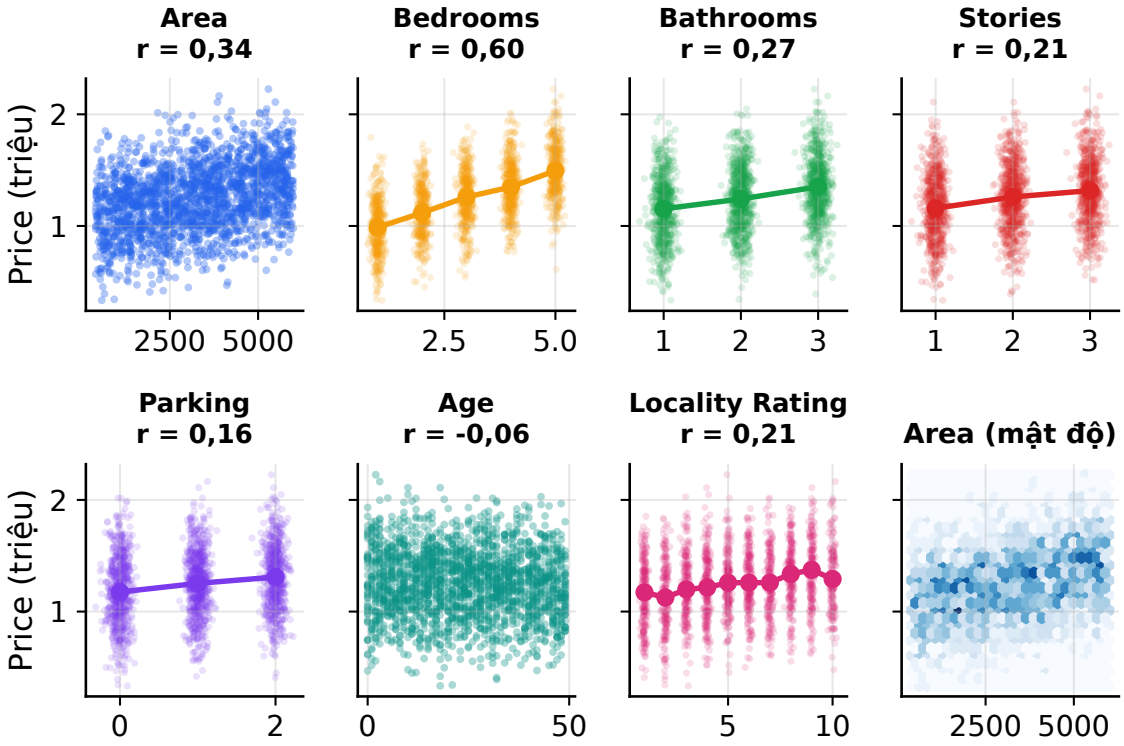
Bảng 2.2. Thống kê mô tả các thuộc tính số của dữ liệu giá nhà

Thuộc tính	Trung bình	Độ lệch chuẩn	Min	Trung vị	Max
Area	3.238,08	1.604,51	401	3.272	5.996
Bedrooms	3,03	1,43	1	3	5
Bathrooms	2,02	0,81	1	2	3
Stories	2,00	0,81	1	2	3
Parking	0,99	0,81	0	1	2
Age	24,01	14,32	0	24	49
Locality Rating	5,36	2,84	1	5	10
Price	1.245.013,64	300.174,38	334.635	1.246.602	2.225.409

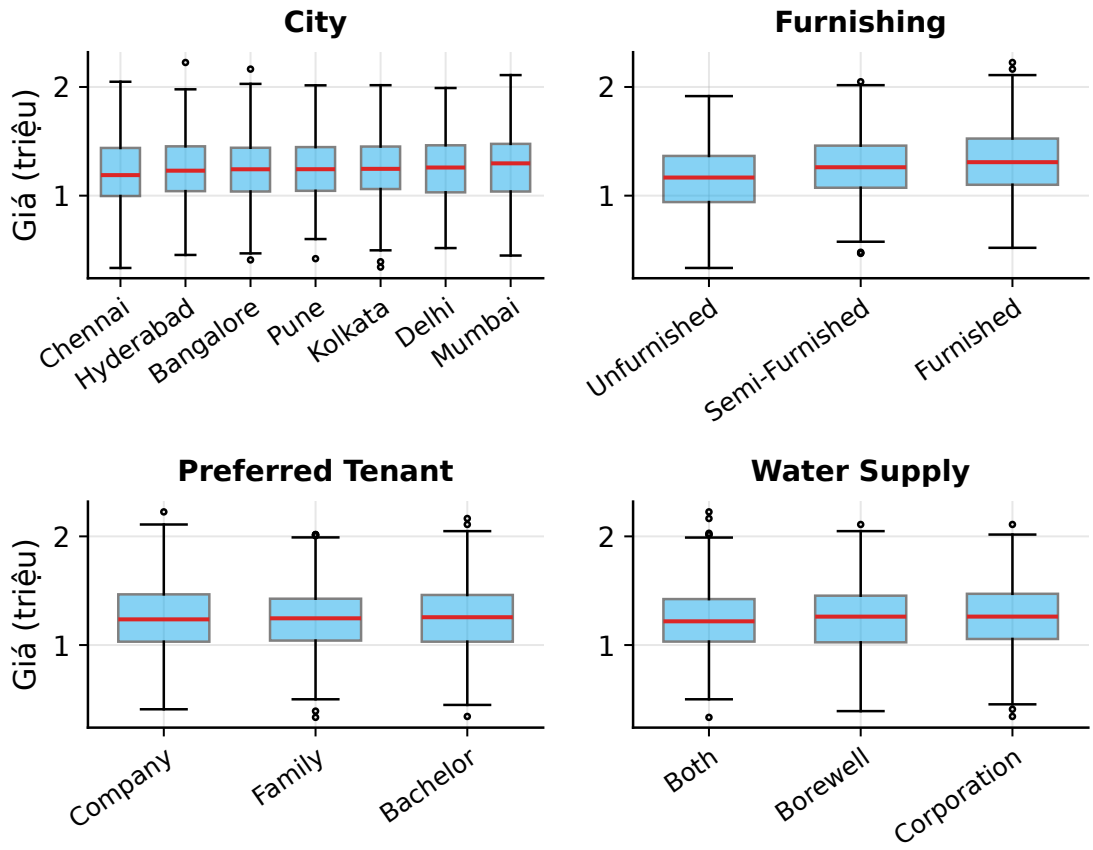
Nhận xét: Số phòng ngủ (0,6006) và diện tích (0,3391) tương quan với giá mạnh nhất, tuổi nhà tương quan âm yếu (−0,0566). Các đám mây điểm có dạng dải thẳng với nhiễu đồng đều, thành phố hầu như không ảnh hưởng tới giá. Đó là dấu hiệu của dữ liệu tổng hợp gần tuyến tính, nên



Hình 2.11. Phân bố giá nhà (trái) và giá trung bình theo số phòng ngủ (phải)



Hình 2.12. Quan hệ giữa giá và các thuộc tính số; đường đỏ là xu hướng tuyến tính



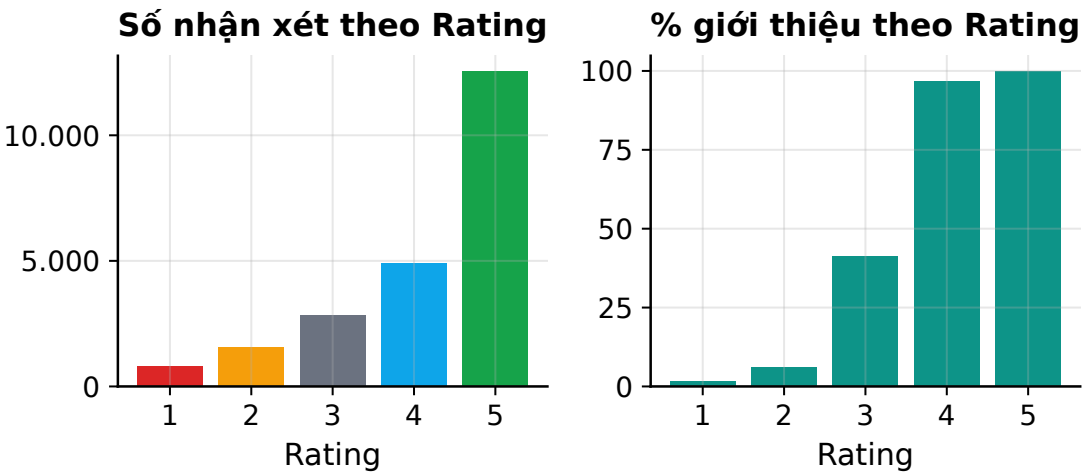
Hình 2.13. Phân bố giá theo thành phố, mức độ nội thất, đối tượng thuê và nguồn nước

cần thận trọng khi suy rộng kết luận ra thị trường thật.

2.2.3 Nhận xét khách hàng: phân loại văn bản

Bộ *Women's E-Commerce Clothing Reviews* [106] gồm 23.486 nhận xét. Sau khi loại 845 nhận xét rỗng còn 22.641 nhận xét, trong đó 81,89% được khách hàng giới thiệu. Lớp cần quan tâm là lớp thiểu số "không giới thiệu". Văn bản được chuẩn hoá rồi biểu diễn bằng TF-IDF [107] unigram + bigram:

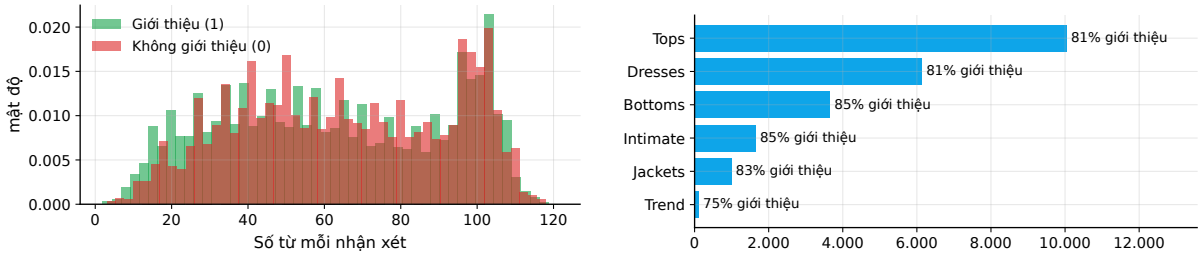
$$\mathrm{tfidf}(t,d) = \big(1+\log \mathrm{tf}(t,d)\big)\cdot\left(\log\frac{1+n}{1+\mathrm{df}(t)}+1\right). \tag{2.6}$$



Hình 2.14. Trái: phân bố số sao. Phải: tỷ lệ giới thiệu theo số sao



Hình 2.15. Đám mây từ của nhận xét được giới thiệu (trái) và không được giới thiệu (phải)



(a) Độ dài nhận xét theo nhãn **(b)** Số nhận xét theo nhóm sản phẩm

Hình 2.16. Khảo sát dữ liệu nhận xét khách hàng

2.3 Cài đặt và kết quả các mô hình học máy

2.3.1 Đường ống tiền xử lý và mô hình

Thuộc tính số được chuẩn hoá bằng StandardScaler, thuộc tính phân loại mã hoá One-Hot; bộ tiền xử lý chỉ học trên Train và được gói cùng mô hình trong một `Pipeline` để tránh rò rỉ dữ liệu. Tập tiểu đường có 67.302 mẫu Train, 14.422 Validation, 14.422 Test và 15 đặc trưng sau biến đổi.

Đoạn mã 2.3. Bộ tiền xử lý dùng chung: StandardScaler cho thuộc tính số, One-Hot cho thuộc tính phân loại

```
# tl3/ch2_data.py
def diabetes_preprocessor():
    """Chuẩn hóa z-score cho 6 thuộc tính số và One-Hot cho 2 thuộc tính phân loại."""
    return ColumnTransformer([
        ("num", StandardScaler(), DIAB_NUM),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), DIAB_CAT),
    ])
```

Để kiểm chứng lý thuyết, hồi quy tuyến tính và logistic còn được cài đặt từ đầu bằng NumPy. Bảng 2.3 cho thấy kết quả trùng với scikit-learn tới nhiều chữ số thập phân.

Bảng 2.3. Đối chiếu cài đặt NumPy từ đầu với scikit-learn trên tập Test

Thuật toán	Chỉ số (Test)	NumPy tự viết	scikit-learn
Hồi quy tuyến tính (phương trình chuẩn)	R^2	0,791895	0,791895
Hồi quy tuyến tính (GD)	R^2	0,792014	0,791895
Hồi quy logistic (mini-batch GD)	ROC-AUC	0,959543	0,959531
Hồi quy logistic (mini-batch GD)	F1 (ngưỡng 0,5)	0,7323	0,7323
Hồi quy logistic	Thời gian (s)	6,807	0,079

2.3.2 Kết quả trên tiểu đường

Nhận xét: Baseline đạt accuracy 91,18% mà F1 bằng 0,0000: accuracy không phản ánh khả năng phát hiện bệnh. XGBoost được chọn theo Validation, đạt F1 0,8115, ROC-AUC 0,9778, PR-AUC 0,8826 trên Test. Precision 0,9705 nghĩa là gần như mọi cảnh báo đều đúng; Recall 0,6973 là giới hạn chính, vì các ca bị bỏ sót có HbA1c và glucose nằm trong vùng bình thường. Quan hệ dạng ngưỡng là sở trường của cây, còn mô hình tuyến tính phải xấp xỉ một bậc thang bằng một mặt phẳng. Naive Bayes thấp nhất do giả định độc lập bị vi phạm.

Đoạn mã 2.4. Khai báo các bộ phân loại với siêu tham số dùng trong thực nghiệm

```python
# tl3/ch2_models.py
def classifiers(pos_weight=1.0):
    """Bộ mô hình phân loại cho Diabetes; pos_weight = n_âm / n_dương dùng cho XGBoost khi cân bằng lớp."""
    return {
        "baseline": DummyClassifier(strategy="most_frequent"),
        "logreg": LogisticRegression(C=1.0, max_iter=2000),
        "knn": KNeighborsClassifier(n_neighbors=25, weights="distance", n_jobs=8),
        "nb": GaussianNB(),
        "tree": DecisionTreeClassifier(max_depth=8, min_samples_leaf=20, random_state=SEED),
        "rf": RandomForestClassifier(n_estimators=300, min_samples_leaf=2, n_jobs=8, random_state=SEED),
        "xgb": XGBClassifier(n_estimators=400, max_depth=5, learning_rate=0.05, subsample=0.9,
                             colsample_bytree=0.9, eval_metric="logloss", n_jobs=8, random_state=SEED),
        "lgbm": LGBMClassifier(n_estimators=400, learning_rate=0.05, num_leaves=31, n_jobs=8,
                               random_state=SEED, verbose=-1),
        "svm": SVC(C=1.0, kernel="rbf", gamma="scale", probability=True, random_state=SEED),
    }
```

Bảng 2.4. Tiểu đường: kết quả trên tập Test, ngưỡng cố định từ Validation

Mô hình	Ngưỡng	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Baseline (lớp đa số)	0,50	0,9118	0,0000	0,0000	0,0000	0,5000	0,0882
Logistic Regression	0,37	0,9530	0,7614	0,6800	0,7184	0,9595	0,8152
KNN (k = 25)	0,39	0,9591	0,8756	0,6250	0,7294	0,9462	0,8093
Gaussian Naive Bayes	0,95	0,8280	0,3180	**0,8294**	0,4597	0,9052	0,5023
Decision Tree	0,41	**0,9718**	**0,9887**	0,6879	0,8113	0,9723	0,8557
Random Forest	0,46	0,9705	0,9618	0,6934	0,8058	0,9669	0,8632
XGBoost	0,48	0,9714	0,9705	0,6973	**0,8115**	**0,9778**	**0,8826**
LightGBM	0,47	0,9707	0,9521	0,7036	0,8092	0,9762	0,8773
SVM (RBF)	0,14	0,9615	0,8316	0,7068	0,7641	0,9099	0,7993
MLP (PyTorch)	0,40	0,9694	0,9369	0,7005	0,8016	0,9742	0,8733

Đoạn mã 2.5. Khai báo các mô hình hồi quy

```
# tl3/ch2_models.py
def regressors():
    """Bộ mô hình hồi quy cho House Price. SVR học trên giá đã chuẩn hóa (TransformedTargetRegressor)."""
    return {
        "baseline": DummyRegressor(strategy="mean"),
        "linear": LinearRegression(),
        "ridge": Ridge(alpha=10.0),
        "lasso": Lasso(alpha=1000.0, max_iter=50000),
        "poly": make_pipeline(PolynomialFeatures(degree=2, include_bias=False), StandardScaler(),
                              Ridge(alpha=30.0)),
        "knn": KNeighborsRegressor(n_neighbors=10, weights="distance"),
        "tree": DecisionTreeRegressor(max_depth=6, min_samples_leaf=10, random_state=SEED),
        "rf": RandomForestRegressor(n_estimators=400, min_samples_leaf=2, n_jobs=8, random_state=SEED),
        "xgb": XGBRegressor(n_estimators=600, max_depth=3, learning_rate=0.03, subsample=0.9,
                            colsample_bytree=0.9, n_jobs=8, random_state=SEED),
        "svr": TransformedTargetRegressor(SVR(C=3.0, epsilon=0.05, kernel="rbf"), transformer=StandardScaler()),
    }
```

Đoạn mã 2.6. Bộ biến đổi TF-IDF unigram + bigram

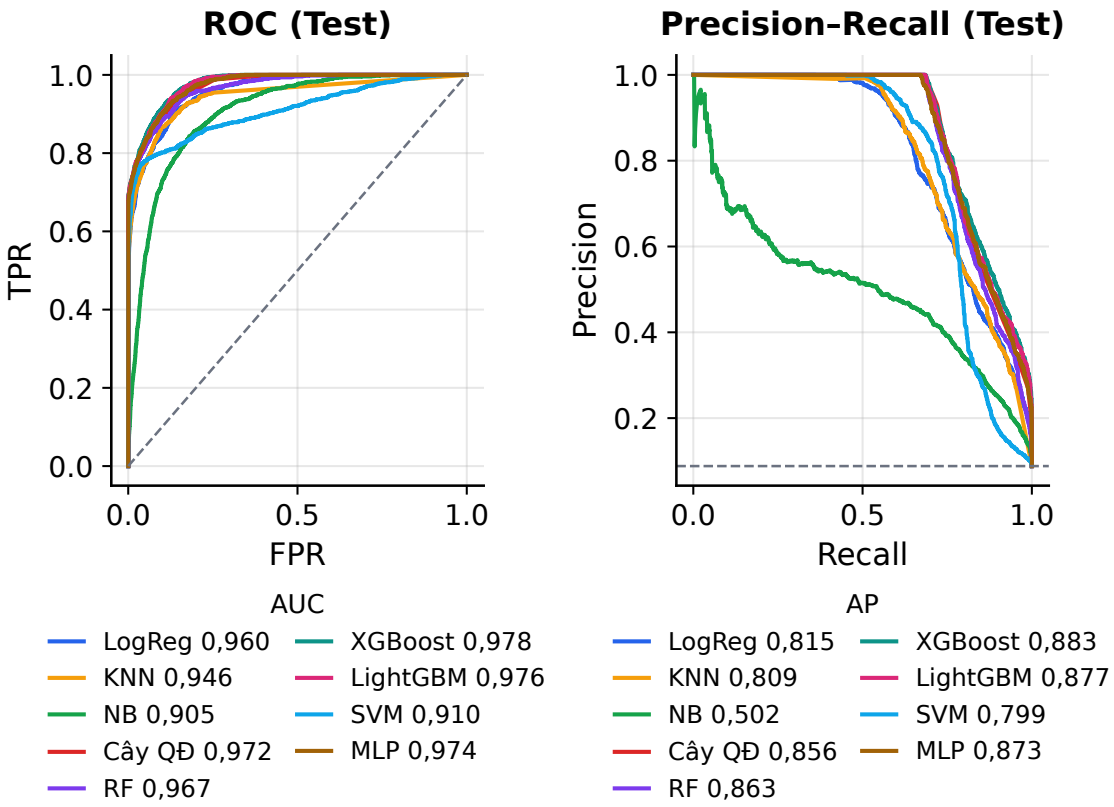
```
# tl3/ch2_text.py
def tfidf(max_features=40000):
    """TF-IDF unigram + bigram, bỏ từ quá hiếm (min_df=3), log-scaling tần suất (sublinear_tf)."""
    return TfidfVectorizer(preprocessor=normalize, ngram_range=(1, 2), min_df=3, max_df=0.9,
                           sublinear_tf=True, max_features=max_features)
```

Đoạn mã 2.7. Các mô hình phân loại văn bản trên TF-IDF

```python
# tl3/ch2_text.py
def text_models():
    """Các Pipeline TF-IDF -> mô hình; class_weight='balanced' bù lớp 'không giới thiệu' chiếm ~18%."""
    return {
        "baseline": Pipeline([("tfidf", tfidf(2000)), ("clf", DummyClassifier(strategy="most_frequent"))]),
        "logreg": Pipeline([("tfidf", tfidf()), ("clf", LogisticRegression(
            C=4.0, class_weight="balanced", max_iter=3000))]),
        "nb": Pipeline([("tfidf", tfidf()), ("clf", MultinomialNB(alpha=0.3))]),
        "cnb": Pipeline([("tfidf", tfidf()), ("clf", ComplementNB(alpha=0.3))]),
        "svm": Pipeline([("tfidf", tfidf()), ("clf", CalibratedClassifierCV(
            LinearSVC(C=0.5, class_weight="balanced"), cv=3))]),
        "softmax": Pipeline([("tfidf", tfidf()), ("clf", LogisticRegression(C=4.0, max_iter=3000))]),
    }
```

Đoạn mã 2.8. Hồi quy logistic bằng mini-batch Gradient Descent trên BCE (NumPy)

```python
# tl3/ch2_scratch.py
def logreg_gd(X, y, lr=0.5, epochs=400, l2=1e-4, batch=4096, seed=0):
    """Hồi quy logistic bằng mini-batch Gradient Descent trên Binary Cross-Entropy + L2:
    ∇w = Xᵀ(σ(Xw) − y)/m + λw. Trả w và lịch sử mất mát toàn tập."""
    rng = np.random.default_rng(seed)
    Xb = np.hstack([np.ones((len(X), 1)), X])
    w = np.zeros(Xb.shape[1])
    hist = []
    for _ in range(epochs):
        idx = rng.permutation(len(y))
        for s in range(0, len(y), batch):
            b = idx[s:s + batch]
            g = Xb[b].T @ (sigmoid(Xb[b] @ w) - y[b]) / len(b) + l2 * np.r_[0, w[1:]]
            w -= lr * g
        p = sigmoid(Xb @ w)
        hist.append(float(-np.mean(y * np.log(p + 1e-12) + (1 - y) * np.log(1 - p + 1e-12))))
    return w, hist
```



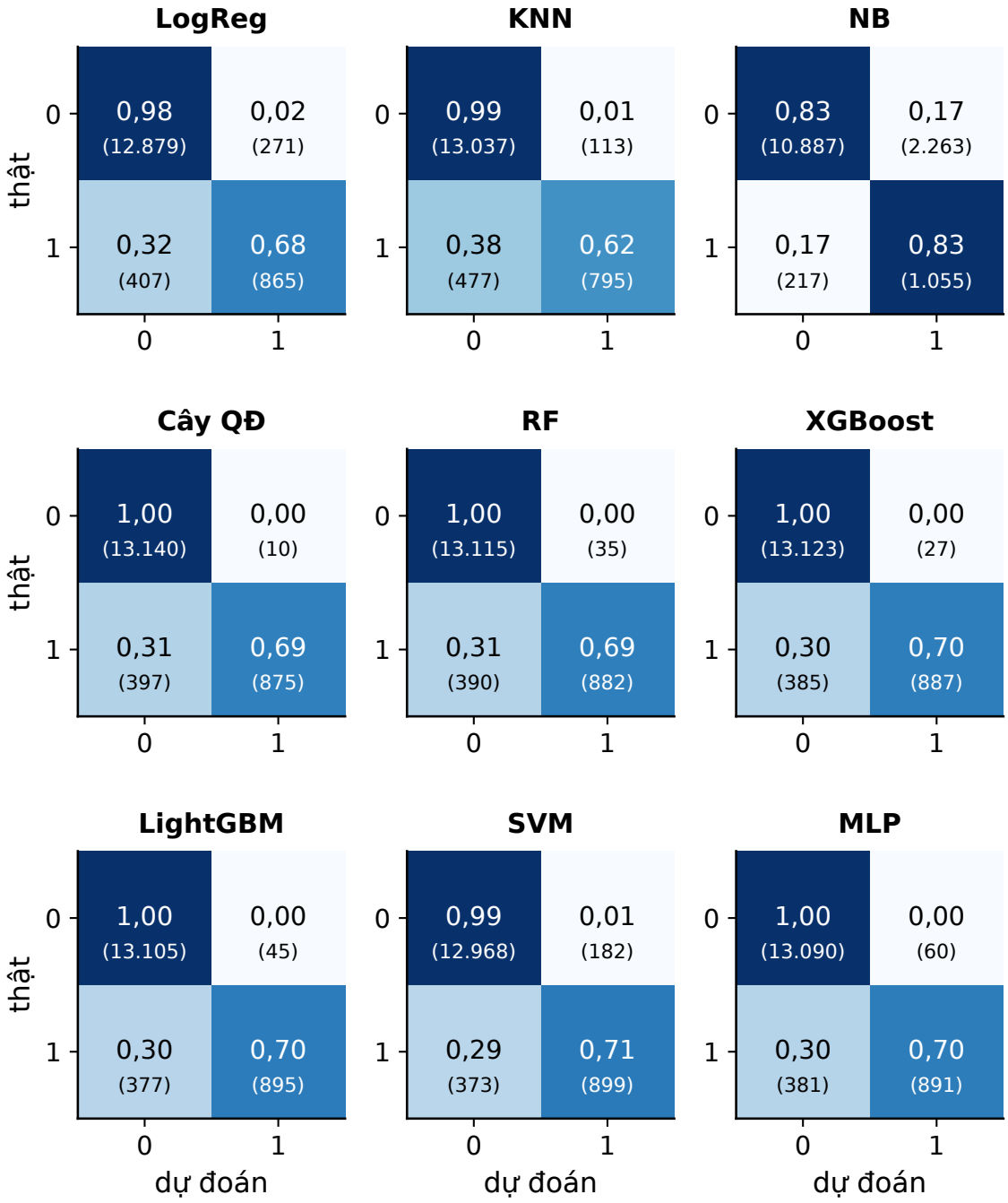
Hình 2.17. Đường ROC (trái) và Precision–Recall (phải) trên tập Test; chú giải ghi AUC và AP

2.3.2.1 Chọn ngưỡng quyết định

Ngưỡng được quét trên Validation theo F1. Chọn ngưỡng giúp nhiều nhất cho các mô hình có xác suất lệch chuẩn: ngưỡng tối ưu của Naive Bayes lên tới 0,9500, còn XGBoost chỉ 0,4800. Hạ ngưỡng tăng Recall nhưng Precision giảm nhanh; nếu mục tiêu sàng lọc cần Recall cao hơn, ta chấp nhận nhiều ca phải xét nghiệm lại hơn.

Bảng 2.5. Precision, Recall, F1 ở ngưỡng 0,5 và ở ngưỡng tối ưu F1 (*) chọn trên Validation

Mô hình	P (0,5)	R (0,5)	F1 (0,5)	Ngưỡng*	P*	R*	F1*	ΔF1
Logistic Regression	0,8517	0,6274	0,7225	0,37	0,7728	0,6871	0,7274	+0,0049
KNN (k = 25)	0,9320	0,5708	0,7079	0,39	0,8555	0,6376	0,7306	+0,0227
Gaussian Naive Bayes	0,1879	0,9678	0,3146	0,95	0,3184	0,8514	0,4635	+0,1489
Decision Tree	0,9918	0,6690	0,7991	0,41	0,9918	0,6690	0,7991	+0,0000
Random Forest	0,9781	0,6682	0,7940	0,46	0,9589	0,6785	0,7947	+0,0006
XGBoost	0,9786	0,6824	0,8041	0,48	0,9701	0,6887	0,8055	+0,0014
LightGBM	0,9570	0,6832	0,7972	0,47	0,9492	0,6903	0,7993	+0,0020
SVM (RBF)	0,9246	0,6171	0,7402	0,14	0,8156	0,7060	0,7568	+0,0166
MLP (PyTorch)	0,9929	0,6588	0,7921	0,40	0,9449	0,7005	0,8045	+0,0125

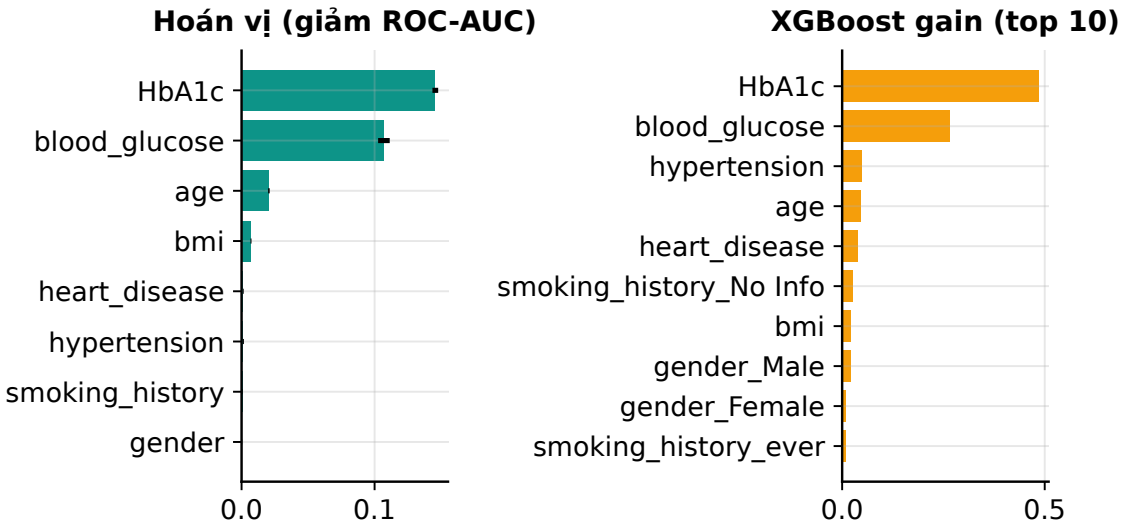


Hình 2.18. Ma trận nhầm lẫn của các mô hình trên tập Test ở ngưỡng chọn trên Validation (tỷ lệ theo hàng, số lượng trong ngoặc)

Đoạn mã 2.9. Chọn ngưỡng quyết định theo F1 trên tập Validation

```python
# tl3/ch2_metrics.py
def best_threshold(y, proba, grid=None):
    """Quét ngưỡng 0,05–0,95 trên Validation, trả ngưỡng cho F1 lớn nhất (không bao giờ chọn trên Test)."""
    grid = np.round(np.arange(0.05, 0.951, 0.01), 2) if grid is None else grid
    f1s = [f1_score(y, (proba >= t).astype(int), zero_division=0) for t in grid]
    i = int(np.argmax(f1s))
    return float(grid[i]), {"grid": grid.tolist(), "f1": [float(v) for v in f1s]}
```

2.3.2.2 Diễn giải mô hình



Hình 2.19. Độ quan trọng đặc trưng: hoán vị (mức giảm ROC-AUC trên Validation) và gain của XGBoost

Hoán vị HbA1c làm ROC-AUC của XGBoost giảm 0,1456, glucose giảm 0,1070, tuổi chỉ giảm 0,0205. Mô hình học đúng các yếu tố y khoa dùng để chẩn đoán, nên đủ tin cậy làm công cụ sàng lọc, không thay thế chẩn đoán. Kiểm định chéo 5-fold cho ROC-AUC $0{,}9789 \pm 0{,}0020$, xác nhận thứ hạng ổn định [108].

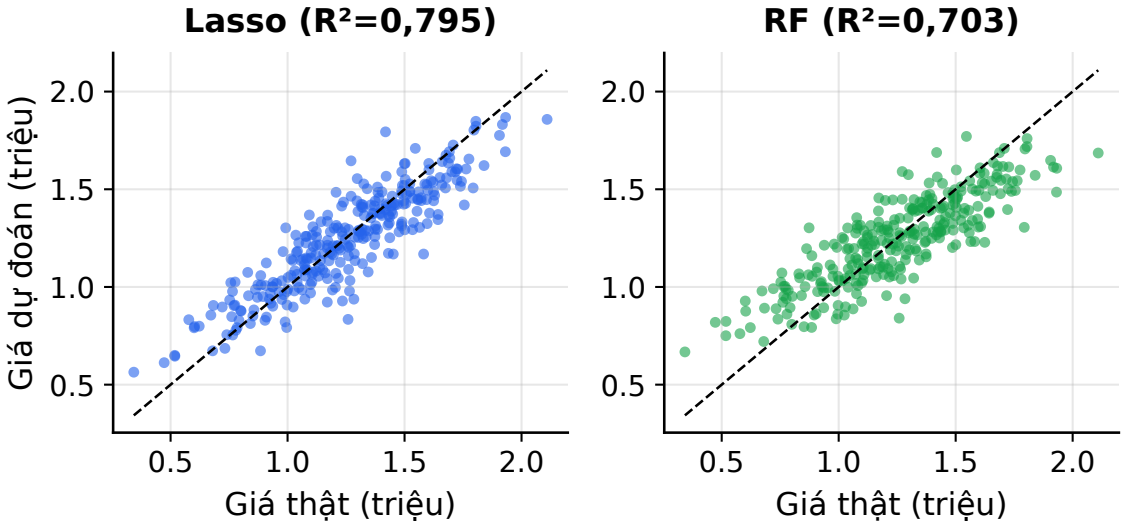
Bảng 2.6. Kiểm định chéo 5-fold trên tập Train: trung bình ± độ lệch chuẩn

Mô hình	ROC-AUC	F1	Accuracy
Logistic Regression	0,9618 ± 0,0023	0,7323 ± 0,0118	0,9591 ± 0,0015
Decision Tree	0,9724 ± 0,0026	0,8008 ± 0,0127	0,9706 ± 0,0015
Random Forest	0,9690 ± 0,0040	0,8015 ± 0,0113	0,9704 ± 0,0014
XGBoost	0,9789 ± 0,0020	0,8041 ± 0,0116	0,9706 ± 0,0014

2.3.3 Kết quả trên giá nhà

Bảng 2.7. Giá nhà: kết quả hồi quy trên tập Test

Mô hình	MAE	RMSE	R^2	MAPE (%)
Baseline (trung bình)	247.833	306.260	−0,0008	23,03
Linear Regression	109.556	139.654	0,7919	9,49
Ridge (L2)	109.420	139.329	0,7929	9,49
Lasso (L1)	**108.540**	**138.568**	**0,7951**	**9,42**
Polynomial bậc 2 + Ridge	127.958	159.112	0,7299	11,15
KNN (k = 10)	141.713	178.835	0,6587	12,82
Decision Tree	165.725	204.916	0,5519	14,40
Random Forest	135.465	166.850	0,7030	12,07
XGBoost	121.109	149.416	0,7618	10,57
SVR (RBF)	138.210	170.313	0,6905	12,15
MLP (PyTorch)	117.269	146.663	0,7705	10,33

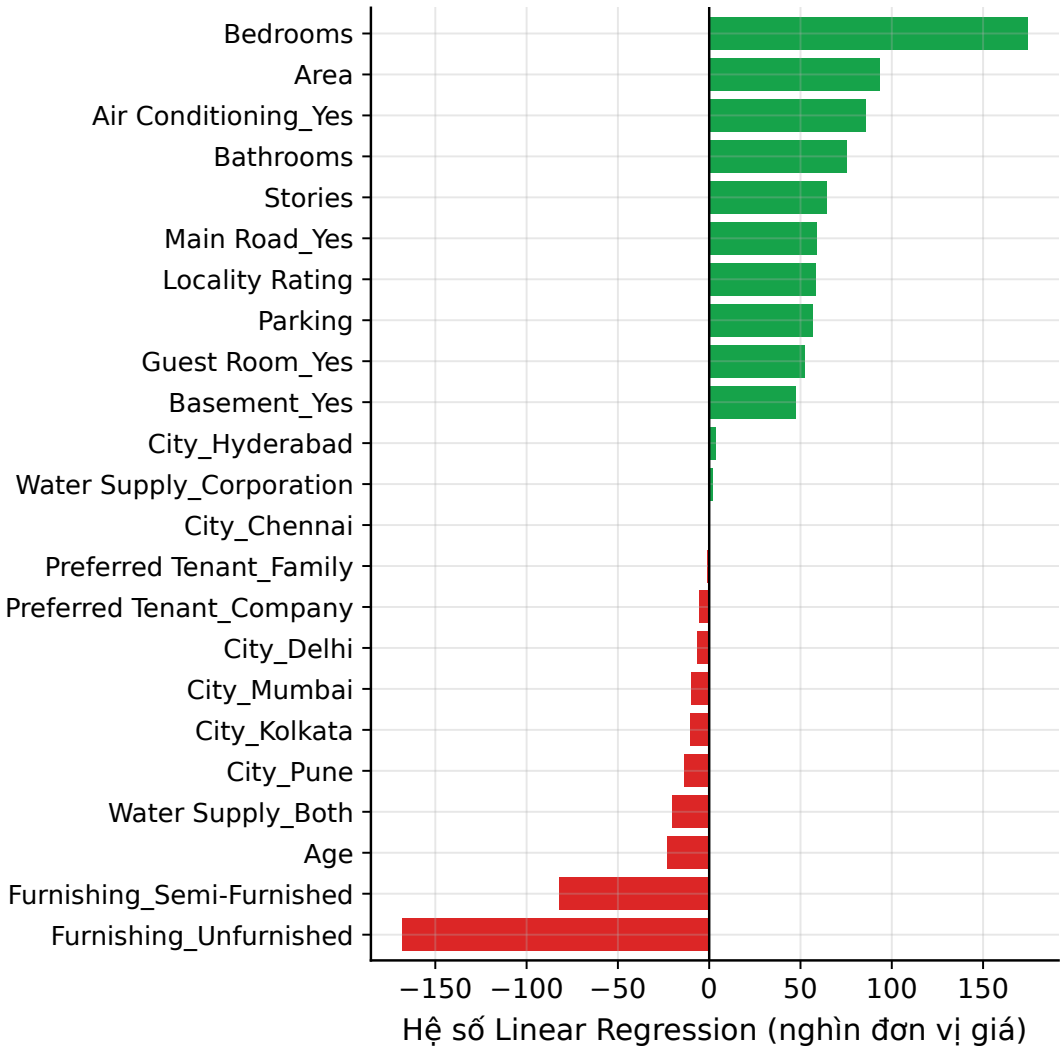


Hình 2.20. Giá dự đoán so với giá thật trên tập Test của Lasso và Random Forest

Nhận xét: Lasso đứng đầu với R^2 0,7951, MAPE 9,42%, giảm MAE hơn một nửa so với baseline. Khi quan hệ thật gần tuyến tính, mô hình tuyến tính có thiên lệch thấp và phương sai nhỏ; cây và KNN phải xấp xỉ một mặt phẳng trên chỉ 1.400 mẫu Train nên kém hơn. Mô hình phức tạp hơn không phải lúc nào cũng tốt hơn.

2.3.4 Kết quả trên nhận xét khách hàng

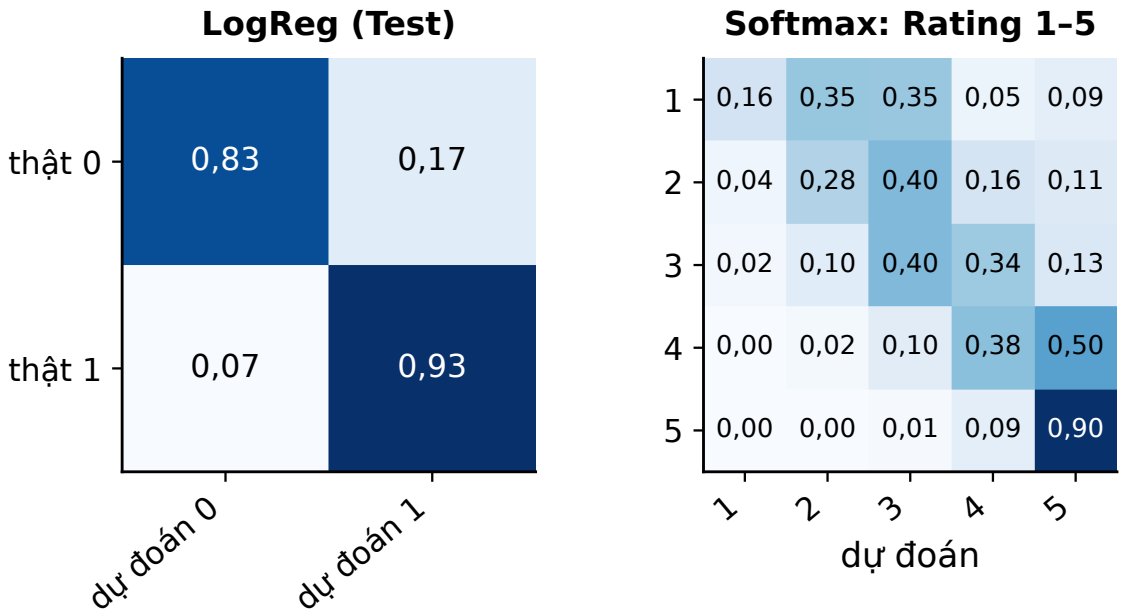
Nhận xét: Logistic Regression đạt macro-F1 0,8542 và tìm ra 82,76% số nhận xét không hài lòng, trong khi baseline có accuracy 81,90% nhưng không phát hiện được nhận xét tiêu cực nào. Multinomial NB tụt lại vì lớp thiểu số bị lấn át; Complement NB khắc phục đúng điểm này [109].



Hình 2.21. Hệ số của Linear Regression (đơn vị tiền trên một độ lệch chuẩn của thuộc tính số, hoặc so với hạng mục tham chiếu)

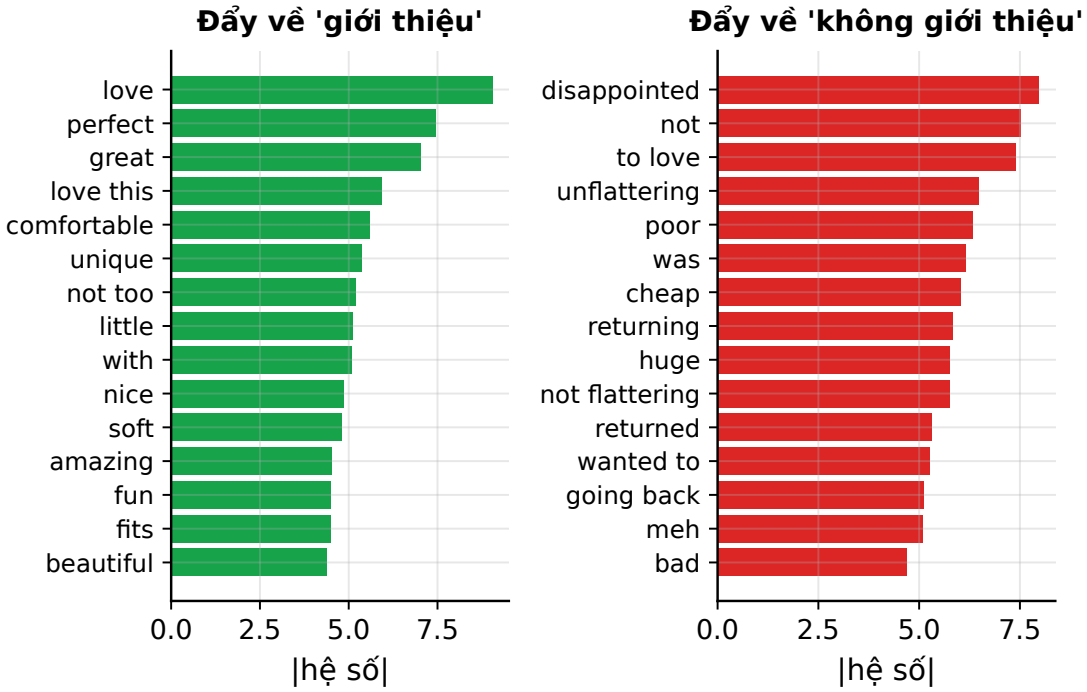
Bảng 2.8. Nhận xét khách hàng: kết quả trên tập Test

Mô hình	Accuracy	Macro-F1	F1 lớp 0	F1 lớp 1	ROC-AUC
Baseline (lớp đa số)	0,8190	0,4502	0,0000	0,9005	0,5000
TF-IDF + Logistic Regression	**0,9082**	**0,8542**	**0,7654**	0,9429	0,9534
TF-IDF + Multinomial NB	0,8855	0,7618	0,5901	0,9334	0,9503
TF-IDF + Complement NB	0,9029	0,8480	0,7566	0,9393	0,9503
TF-IDF + Linear SVM	0,9079	0,8340	0,7233	**0,9447**	**0,9540**
Softmax Regression (5 lớp)	0,9011	0,8139	0,6866	0,9413	0,9538



Hình 2.22. Trái: ma trận nhầm lẫn của Logistic Regression trên Test. Phải: Softmax Regression dự đoán số sao

Phần lớn lỗi còn lại là nhận xét 3 sao và nhận xét mỉa mai, giới hạn của biểu diễn túi từ vốn bỏ qua thứ tự từ.



Hình 2.23. Các từ/cụm từ có hệ số dương (giới thiệu) và âm (không giới thiệu) lớn nhất của Logistic Regression

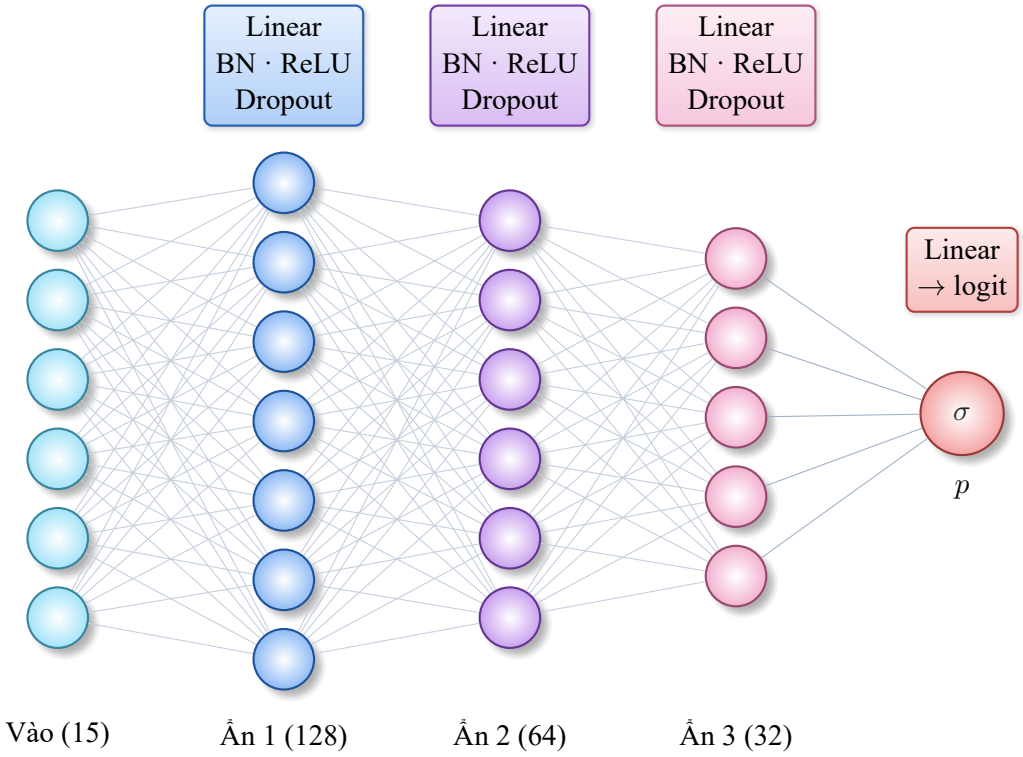
2.4 Mạng nơ-ron đa tầng trên ba nền tảng

2.4.1 Kiến trúc và lan truyền ngược

MLP xếp chồng các tầng $\mathbf{h}^{(l)} = \phi(W^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$ với hàm kích hoạt phi tuyến ReLU. Lan truyền ngược [33] áp dụng quy tắc chuỗi từ đầu ra về đầu vào:

$$\frac{\partial J}{\partial W^{(l)}} = \boldsymbol{\delta}^{(l)}\,\mathbf{h}^{(l-1)\top}, \qquad \boldsymbol{\delta}^{(l-1)} = \phi'\big(\mathbf{z}^{(l-1)}\big) \odot W^{(l)\top}\boldsymbol{\delta}^{(l)}, \tag{2.7}$$

với $\boldsymbol{\delta}^{(l)} = \partial J/\partial \mathbf{z}^{(l)}$ và $\mathbf{z}^{(l)}$ là đầu vào của hàm kích hoạt tầng l. Tối ưu dùng Adam [110]; BatchNorm [111] và Dropout [112] ổn định huấn luyện và giảm quá khớp. Cùng kiến trúc (Hình 2.24) được cài đặt trên ba nền tảng để đối chiếu.



Hình 2.24. MLP cho dữ liệu bảng: ba lớp ẩn 128–64–32 (Linear, BatchNorm, ReLU, Dropout) và một đầu ra

2.4.2 Hiện thực bằng NumPy

Bản từ đầu tự viết lan truyền thuận, lan truyền ngược và Adam, không có BatchNorm và Dropout. Gradient được kiểm tra bằng sai phân số trước khi huấn luyện, sai số tương đối dưới 10^{-9}.

2.4.3 Hiện thực bằng Keras và PyTorch

Đoạn mã 2.10. Lan truyền ngược viết tay của MLP (NumPy)

```python
# tl3/ch2_mlp_numpy.py
def backward(self, z, cache, y):
    """Lan truyền ngược: từ đạo hàm mất mát theo đầu ra, nhân dần qua từng tầng về đầu vào."""
    if self.task == "clf":
        p = 1 / (1 + np.exp(-z))
        g = (self.pos_weight * y * (p - 1) + (1 - y) * p)[:, None]
    else:
        g = 2 * (z - y)[:, None]
    gW, gb = [None] * len(self.W), [None] * len(self.b)
    for k in reversed(range(len(self.W))):
        gW[k] = cache[k].T @ g / len(y)
        gb[k] = g.mean(axis=0)
        g = g @ self.W[k].T * (cache[k] > 0) if k else g
    return gW, gb
```

Đoạn mã 2.11. Cùng kiến trúc MLP bằng Keras: Dense, BatchNorm, ReLU, Dropout

```python
# scripts/ch2_mlp_platforms.py
def keras_mlp(n_in, hidden, dropout, lr, task):
    """Cùng kiến trúc với TabularMLP của PyTorch: Dense, BatchNorm, ReLU, Dropout."""
    net = tf.keras.Sequential([tf.keras.layers.Input((n_in,))])
    for h in hidden:
        net.add(tf.keras.layers.Dense(h))
        net.add(tf.keras.layers.BatchNormalization())
        net.add(tf.keras.layers.ReLU())
        net.add(tf.keras.layers.Dropout(dropout))
    net.add(tf.keras.layers.Dense(1))
    loss = tf.keras.losses.BinaryCrossentropy(from_logits=True) if task == "clf" else "mse"
    net.compile(tf.keras.optimizers.AdamW(lr, weight_decay=1e-4), loss)
    return net
```

Đoạn mã 2.12. Kiến trúc MLP cho dữ liệu bảng bằng PyTorch

```python
# tl3/ch2_mlp.py
class TabularMLP(nn.Module):
    """MLP: [Linear -> BatchNorm -> ReLU -> Dropout] x len(hidden) -> Linear(1).
    Đầu ra là logit (phân loại nhị phân) hoặc giá trị chuẩn hóa (hồi quy)."""

    def __init__(self, n_in, hidden=(128, 64, 32), dropout=0.2):
        super().__init__()
        layers, d = [], n_in
        for h in hidden:
            layers += [nn.Linear(d, h), nn.BatchNorm1d(h), nn.ReLU(),
            ↪  nn.Dropout(dropout)]
            d = h
        layers.append(nn.Linear(d, 1))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x).squeeze(-1)
```

2.4.4 Đối chuẩn ba nền tảng

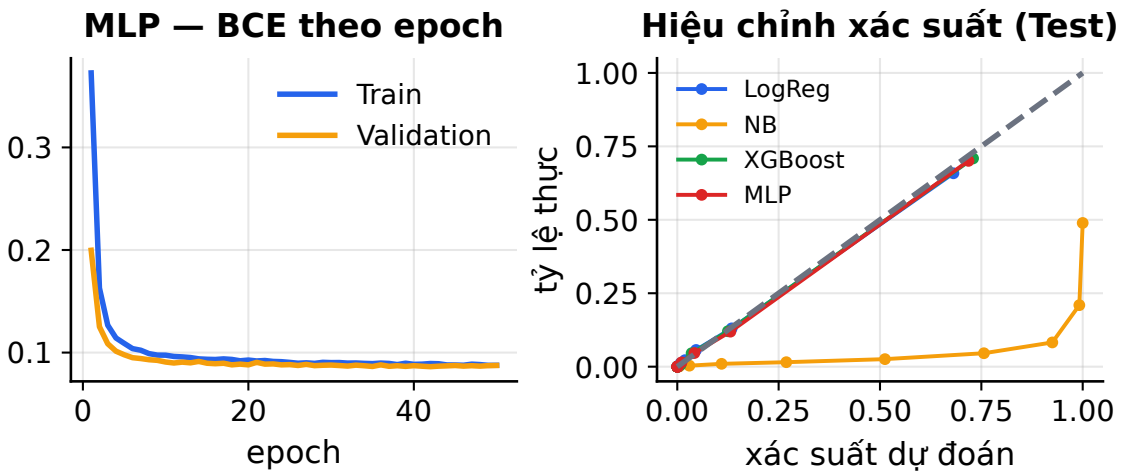
Tập nhận xét dùng TF-IDF nén bằng SVD xuống 300 chiều để cả ba nền tảng dùng chung một ma trận đặc; ngưỡng quyết định được chọn trên Validation cho từng nền tảng.

Bảng 2.9. Đối chuẩn MLP trên ba nền tảng và ba tập dữ liệu (tập Test)

Nền tảng	F1	AUC	Giây	R^2	MAE (nghìn)	Giây	F1	AUC	Giây
NumPy (từ đầu)	0,8071	0,9744	4,6	0,7142	129,3	0,2	0,7043	0,9307	1,5
Keras	0,8031	0,9740	17,1	0,7520	121,9	7,9	0,6923	0,9327	4,5
PyTorch	0,8016	0,9742	190,3	0,7705	117,3	8,3	0,7191	0,9403	7,4

Ba cột đầu là tiểu đường, ba cột tiếp theo là giá nhà, ba cột cuối là nhận xét (F1 của lớp không giới thiệu). Bản NumPy không có BatchNorm và Dropout; PyTorch chạy GPU, hai nền tảng kia chạy CPU.

Nhận xét: Trên tiểu đường, ba nền tảng gần như trùng nhau: F1 từ 0,8016 đến 0,8071, AUC đều xấp xỉ 0,9740, thua XGBoost không đáng kể. Trên giá nhà chỉ có 1.400 mẫu, bản NumPy (không có BatchNorm và Dropout) kém nhất (R^2 0,7142), PyTorch tốt nhất (0,7705), và cả ba đều chưa bằng Lasso. Trên nhận xét, PyTorch đạt F1 lớp không giới thiệu 0,7191, thấp hơn Logistic Regression trên TF-IDF đầy đủ (0,7654), một phần có thể do bước nén SVD. Thời gian chỉ có tính tham khảo vì các lần chạy không cùng thời điểm và thiết bị. Kết luận: MLP không phải lựa chọn mặc định cho dữ liệu bảng; khi cùng kiến trúc và dữ liệu, nền tảng ảnh hưởng tới kết quả ít hơn so với việc chọn đúng họ mô hình.

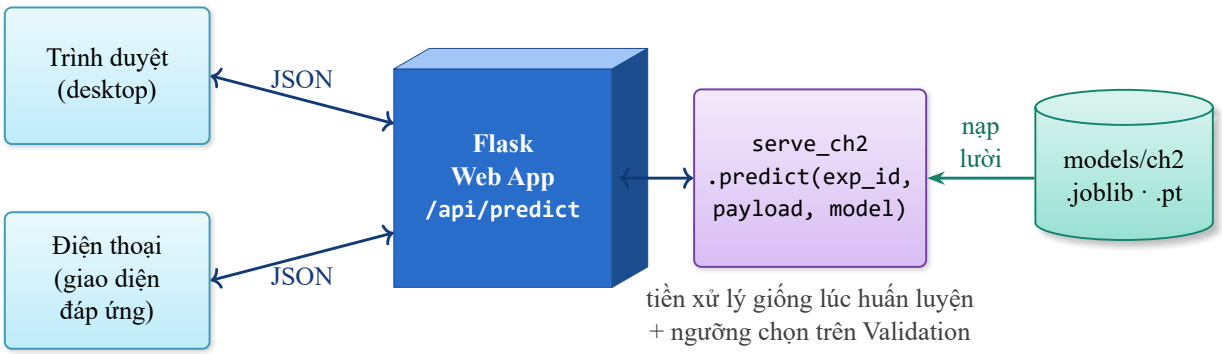


Hình 2.25. Trái: mất mát Train/Validation của MLP theo epoch (dừng sớm). Phải: biểu đồ hiệu chuẩn xác suất trên tập Test

Biểu đồ hiệu chuẩn [113] kiểm tra xác suất dự đoán có "trung thực" hay không. Logistic Regression, XGBoost và MLP bám sát đường chéo nên xác suất dùng được như mức nguy cơ; Naive Bayes thì không, đó là lý do ngưỡng phải được chọn riêng cho từng mô hình.

2.5 Triển khai Web App

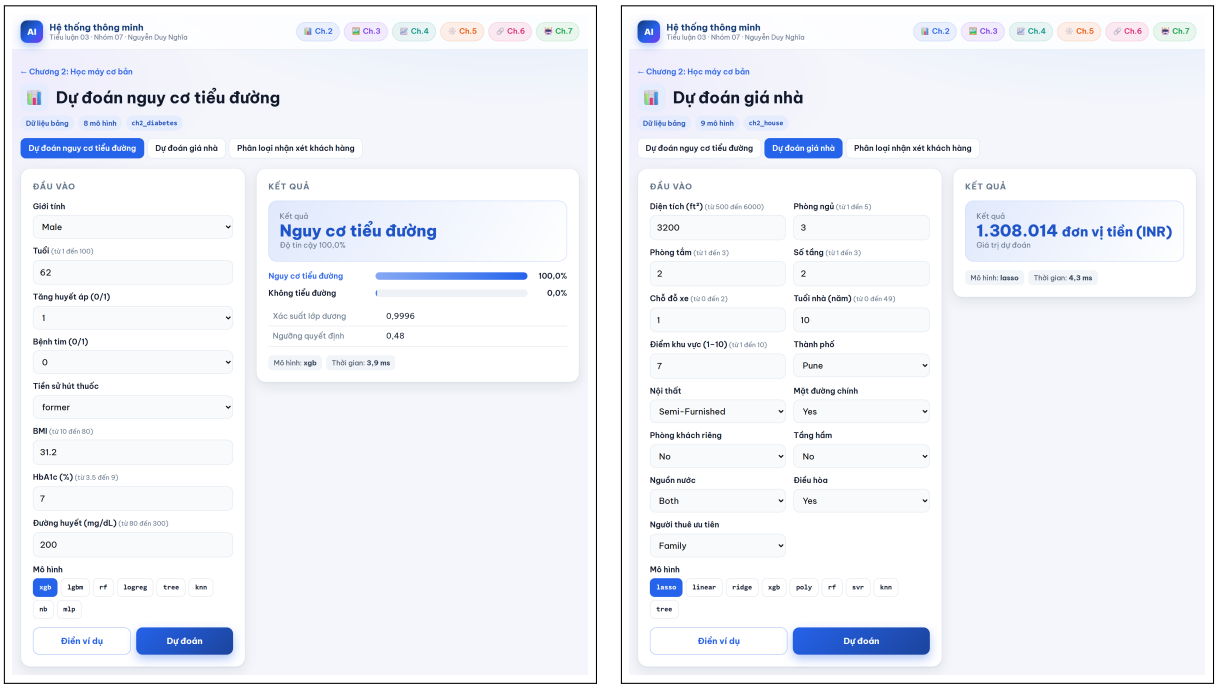
Mô hình được lưu cùng bộ tiền xử lý rồi phục vụ qua mô-đun `serve_ch2.py` với một hàm duy nhất `predict(exp_id, payload, model_id)`. Web App Flask gọi hàm này qua `POST /api/predict`; mô hình được nạp lười một lần rồi giữ trong bộ nhớ (Hình 2.26). Hình 2.27 và Hình 2.28 là giao diện chạy thật.



Hình 2.26. Kiến trúc triển khai: trình duyệt gửi JSON tới Web App, Web App gọi dịch vụ dự đoán đã nạp mô hình

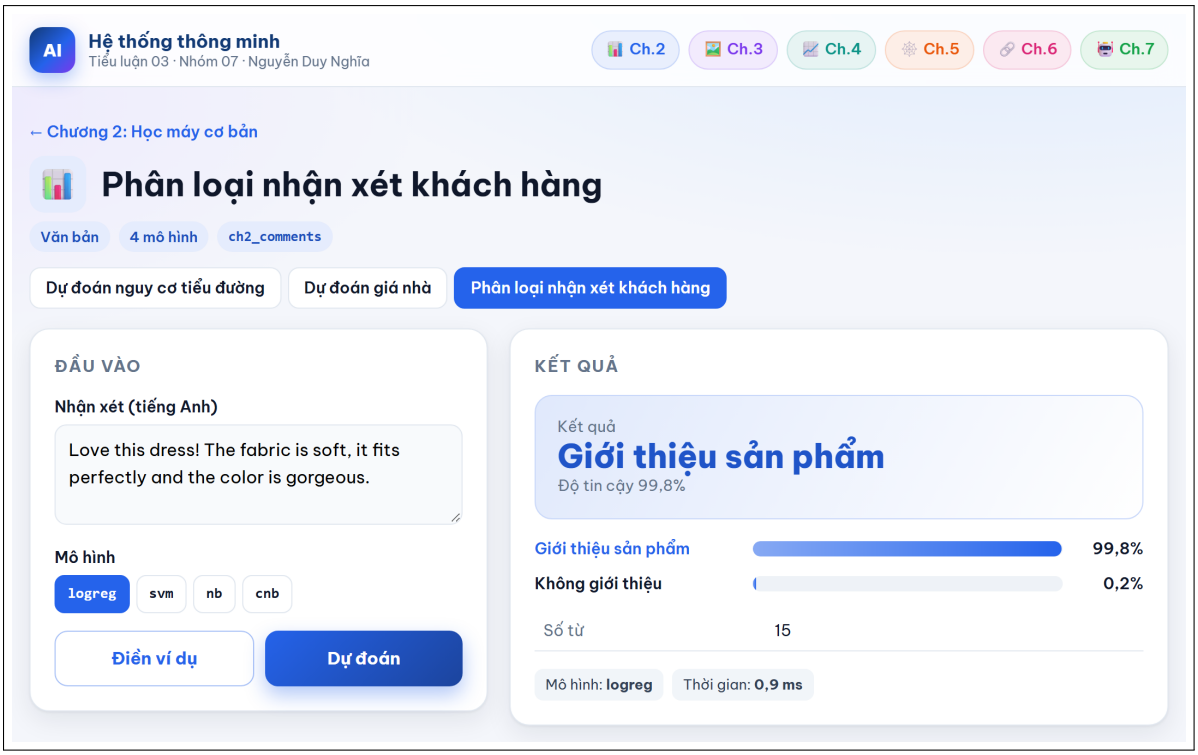
Đoạn mã 2.13. Dự đoán nguy cơ tiểu đường: tiền xử lý, xác suất, so với ngưỡng đã chọn

```python
# tl3/serve_ch2.py
def predict_diabetes(payload, model_id):
    """Trả nhãn, xác suất và ngưỡng quyết định (chọn theo F1 trên Validation) của mô hình được chọn."""
    row = pd.DataFrame([{k: payload[k] for k in SPECS["ch2_diabetes"]["example"]}])
    if model_id == "mlp":
        import torch  # chỉ MLP cần torch; nạp torch mất hàng chục giây nên không nạp cho mô hình cổ điển
        pre, model = _diab_mlp()
        with torch.no_grad():
            p = float(torch.sigmoid(model(torch.tensor(pre.transform(row), dtype=torch.float32)))[0])
    else:
        p = float(_load(f"diabetes_{model_id}").predict_proba(row)[0, 1])
    thr = float(_load("diabetes_meta")["thresholds"][model_id])
    label = "Nguy cơ tiểu đường" if p >= thr else "Không tiểu đường"
    return {"label": label, "proba": {"Không tiểu đường": 1 - p, "Nguy cơ tiểu đường": p},
            "details": {"threshold": thr, "probability": p}}
```



(a) Dự đoán nguy cơ tiểu đường **(b)** Dự đoán giá nhà

Hình 2.27. Giao diện Web App cho hai thực nghiệm dữ liệu bảng



Hình 2.28. Giao diện Web App phân loại nhận xét khách hàng

2.6 Tổng kết chương

Ba bài học được rút ra. Thứ nhất, chỉ số phải khớp với bài toán: accuracy cao có thể vô nghĩa trên dữ liệu mất cân bằng. Thứ hai, không có mô hình tốt nhất cho mọi dữ liệu: gradient boosting thắng khi quan hệ có dạng ngưỡng, Lasso thắng khi quan hệ gần tuyến tính, Logistic Regression trên TF-IDF thắng với văn bản ngắn. Thứ ba, quy trình quan trọng không kém thuật toán: làm sạch có hiểu biết miền, tránh rò rỉ, chọn ngưỡng trên Validation.

Bảng 2.10. Tóm tắt ba thực nghiệm của Chương 2

Thực nghiệm	Kiểu bài toán	Mô hình tốt nhất	Kết quả Test
Tiểu đường	Phân loại nhị phân, mất cân bằng	XGBoost	F1 0,8115, AUC 0,9778
Giá nhà	Hồi quy	Lasso (L1)	R^2 0,7951, MAPE 9,42%
Nhận xét	Phân loại văn bản	Logistic Regression	Macro-F1 0,8542

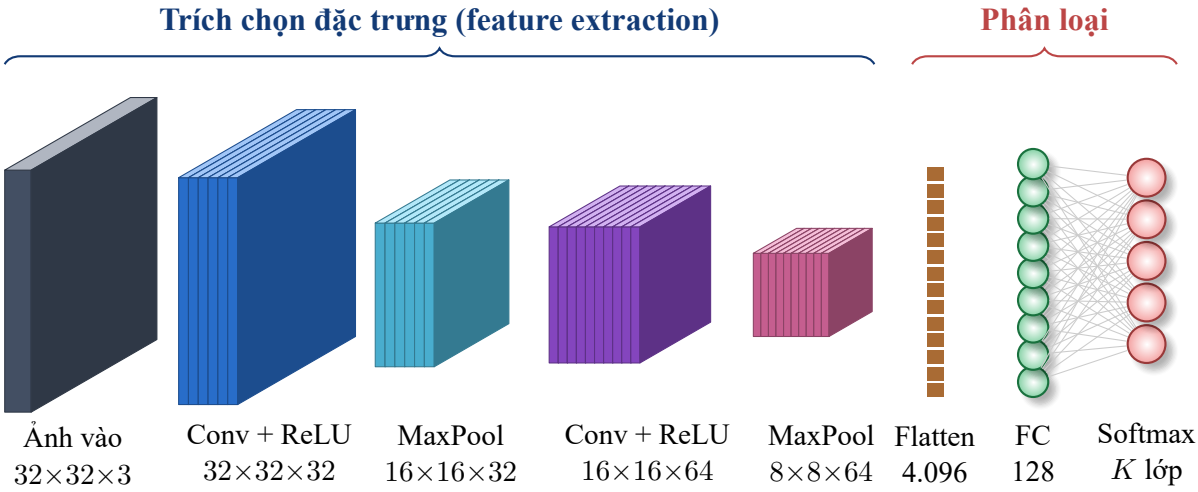
Giới hạn chung của các mô hình trong chương là làm việc trên đặc trưng do con người thiết kế. Với ảnh và chuỗi, việc đó trở nên bất khả thi; hai chương tiếp theo trình bày cách mạng sâu tự học đặc trưng.

Chương 3

Mạng nơ-ron tích chập (CNN)

Ảnh $224 \times 224 \times 3$ có hơn 150 nghìn điểm giá trị; nối đầy đủ vào một tầng 1000 nơ-ron đã cần hơn 150 triệu tham số và làm mất cấu trúc không gian. CNN giải quyết việc này bằng hai ý tưởng lấy cảm hứng từ vỏ não thị giác [114, 115]: *kết nối cục bộ* và *chia sẻ trọng số* [116, 41]. Chương này trình bày cơ sở toán học, tiến hoá kiến trúc, thực nghiệm trên bốn bộ ảnh với ba nền tảng (NumPy, PyTorch, TensorFlow–Keras), rồi triển khai lên Web App.

3.1 Cơ sở lý thuyết



Hình 3.1. Cấu trúc tổng quát của một CNN phân loại ảnh màu: hai khối Conv – ReLU – Pool trích đặc trưng, tiếp theo là các tầng kết nối đầy đủ

3.1.1 Phép tích chập hai chiều

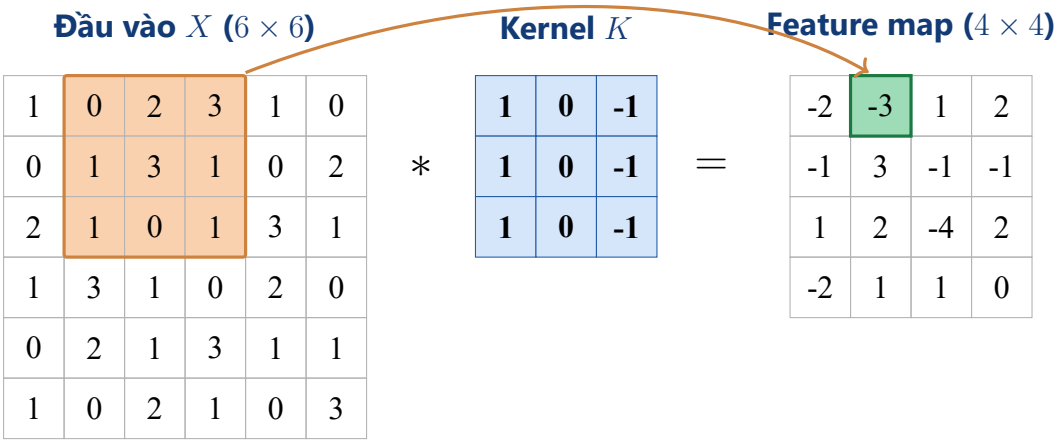
Với ảnh vào I và kernel K kích thước $k \times k$, bản đồ đặc trưng tại vị trí (i, j) là

$$S(i,j) = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} I(i+u, j+v)\, K(u,v). \tag{3.1}$$

Với kích thước vào n, đệm p, bước s, kích thước ra và số tham số của một tầng có c_{in} kênh vào, c_{out} kênh ra là

$$n' = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1, \qquad \#\text{tham số} = (k^2 c_{\text{in}} + 1)\, c_{\text{out}}. \tag{3.2}$$

Số tham số không phụ thuộc kích thước ảnh, đây là lợi ích của chia sẻ trọng số. Hình 3.2 minh hoạ một kernel phát hiện cạnh dọc.



Ô xanh lá: $\sum_{i,j} X_{i,1+j}\, K_{i,j} = (0{\cdot}1{+}2{\cdot}0{+}3{\cdot}(-1)){+}(1{\cdot}1{+}3{\cdot}0{+}1{\cdot}(-1)){+}(1{\cdot}1{+}0{\cdot}0{+}1{\cdot}(-1)) = -3$.
Tổng quát: $Z_{u,v} = b + \sum_{c=1}^{C} \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} X_{c,\,us+i,\,vs+j}\, K_{c,i,j}$, kích thước đầu ra $\lfloor (H + 2p - k)/s \rfloor + 1$.

Hình 3.2. Phép tích chập trên ảnh 6×6 với kernel phát hiện cạnh dọc 3×3, bước 1, không đệm

3.1.2 Pooling, vùng tiếp nhận và hàm kích hoạt

Pooling giảm kích thước không gian và tạo tính bất biến nhỏ với phép dịch (Hình 3.3). *Vùng tiếp nhận* (receptive field) là phần ảnh gốc mà một nơ-ron "nhìn thấy"; qua các tầng nó tăng theo

$$r_l = r_{l-1} + (k_l - 1) \prod_{i=1}^{l-1} s_i, \qquad r_0 = 1. \tag{3.3}$$

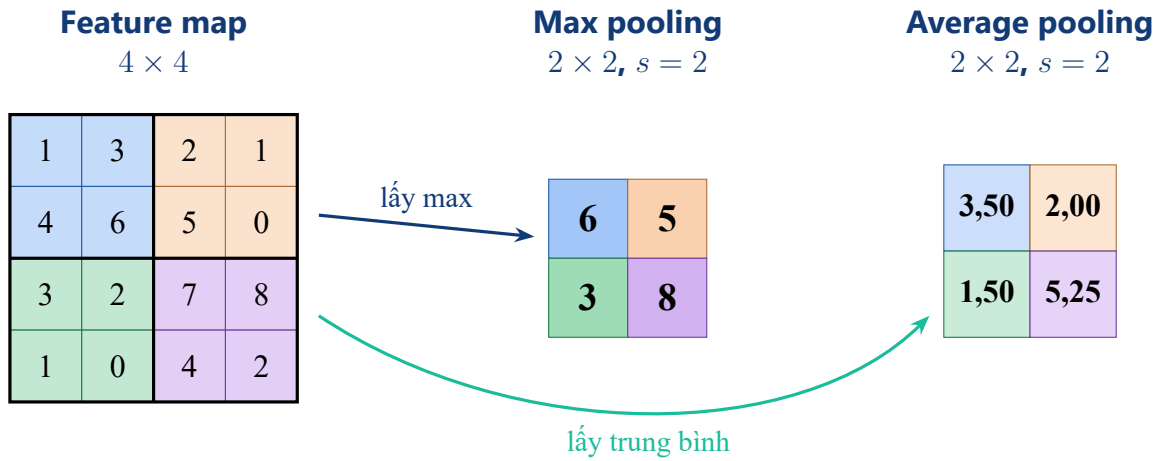
Hai tầng 3×3 có vùng tiếp nhận 5×5 với ít tham số hơn một tầng 5×5, cơ sở thiết kế của VGG. ReLU $\max(0, x)$ [117] tránh bão hoà gradient; tầng cuối dùng Softmax kết hợp entropy chéo. Batch Normalization [111] chuẩn hoá theo lô, Dropout [112] tắt ngẫu nhiên nơ-ron, còn tăng cường dữ liệu [118] tạo biến thể ảnh để giảm quá khớp.

3.1.3 Tiến hoá kiến trúc

LeNet-5 [41] đặt khuôn mẫu Conv – Pool – FC. AlexNet [49] chứng minh sức mạnh của mạng sâu với ReLU, Dropout và GPU trên ImageNet [119]. VGG [120] chỉ dùng kernel 3×3 xếp chồng (Hình 3.5); GoogLeNet [121] tính song song nhiều cỡ kernel.

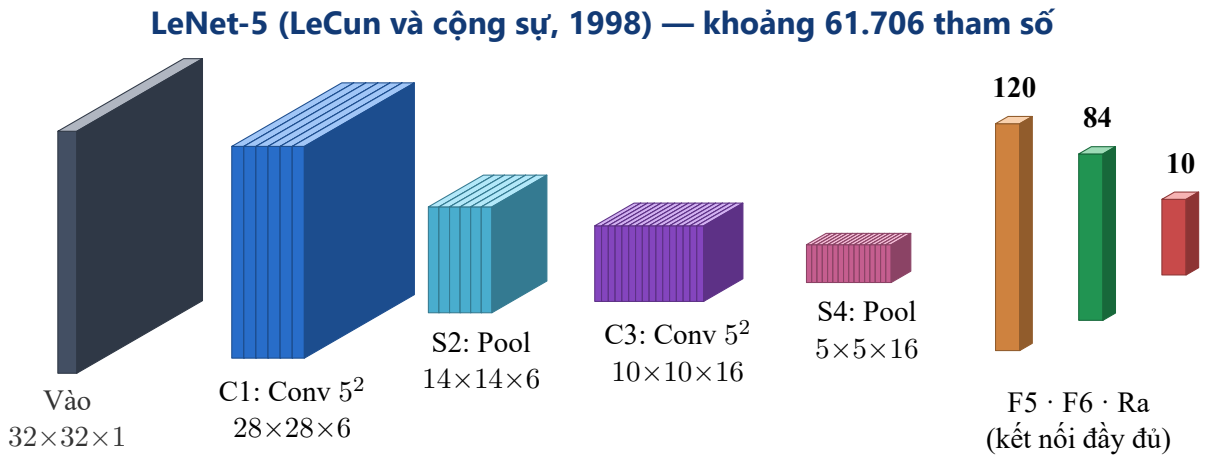
Khi mạng quá sâu, sai số huấn luyện lại tăng vì khó tối ưu. ResNet [122] cho mỗi khối học phần dư $F(\mathbf{x})$ thay vì ánh xạ đầy đủ:

$$\mathbf{y} = F(\mathbf{x}, \{W_i\}) + \mathbf{x}. \tag{3.4}$$



Pooling không có tham số học; giảm H, W một nửa, giữ nguyên số kênh. Lan truyền ngược của max pooling chỉ chuyển gradient về *đúng ô đạt cực đại* (mặt nạ), các ô còn lại nhận 0; average pooling chia đều gradient cho k^2 ô.

Hình 3.3. Max pooling và average pooling 2×2, bước 2 trên bản đồ 4×4

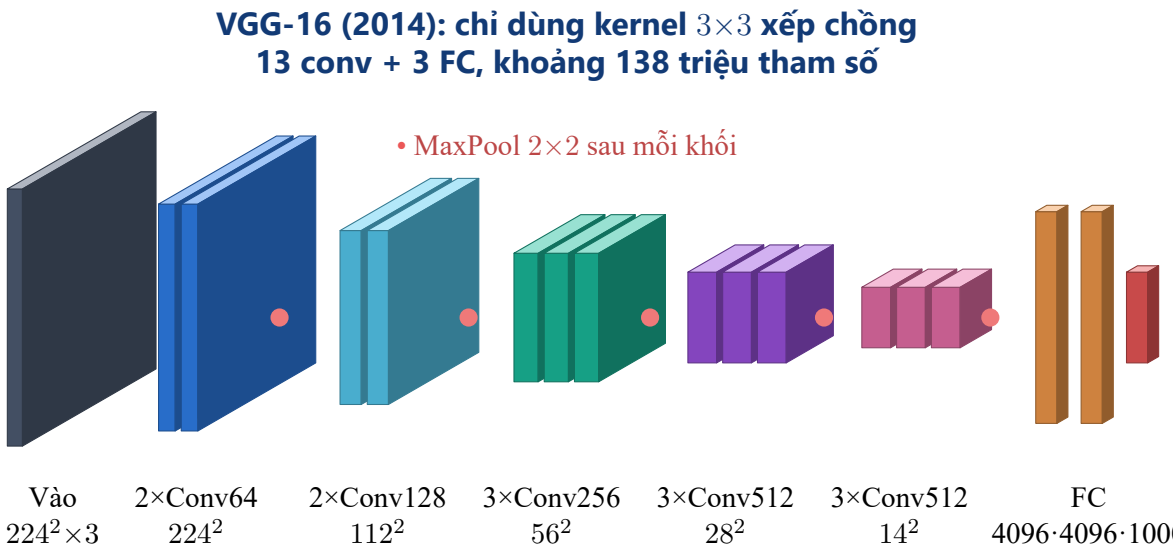


Hình 3.4. Kiến trúc LeNet-5: C1, C3 là tích chập 5×5; S2, S4 là giảm mẫu; F5, F6 và tầng ra kết nối đầy đủ

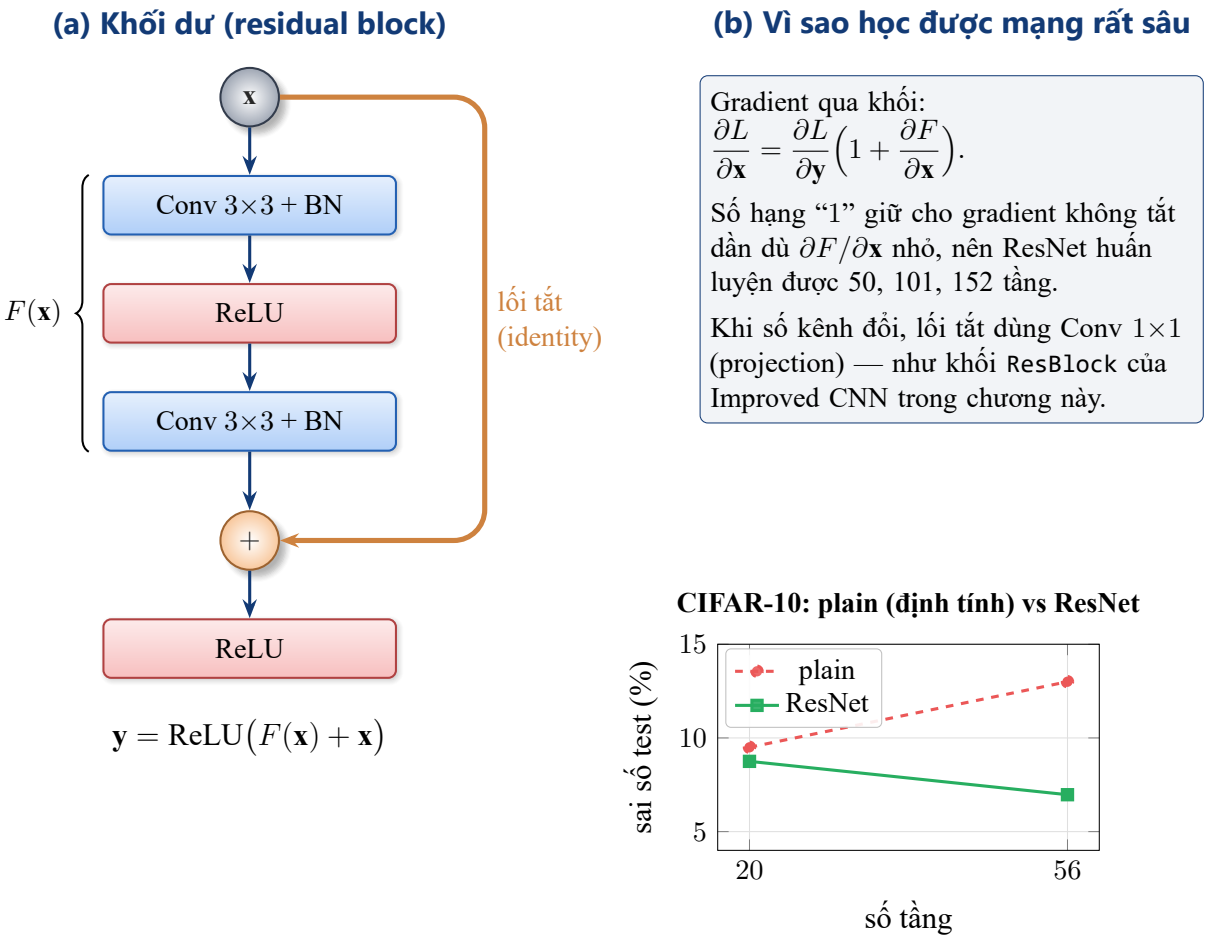
Lối tắt cho gradient đi thẳng về các tầng đầu (Hình 3.6). DenseNet [123] nối mọi tầng với nhau. MobileNet [124, 125] tách tích chập thành depthwise và point-wise, giảm chi phí theo tỷ lệ

$$\frac{D_k^2 M D_F^2 + M N D_F^2}{D_k^2 M N D_F^2} = \frac{1}{N} + \frac{1}{D_k^2}. \tag{3.5}$$

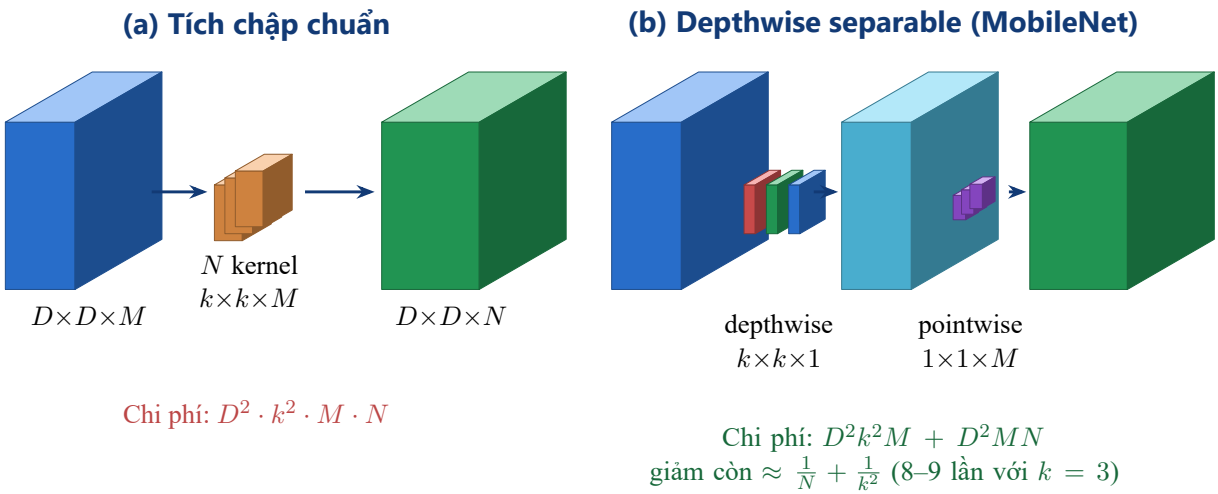
EfficientNet [126] co giãn đồng thời độ sâu, độ rộng và độ phân giải. Với kernel 3×3 và 256 kênh, MobileNet chỉ tốn khoảng 11,5% phép nhân so với tích chập chuẩn (Hình 3.7).



Hình 3.5. Kiến trúc VGG-16: năm khối gồm 2–3 tầng Conv 3×3, sau mỗi khối là Max Pool, cuối cùng là ba tầng FC



Hình 3.6. Khối dư: nhánh $F(\mathbf{x})$ cộng với lối tắt, cho gradient đi thẳng về các tầng đầu



Hình 3.7. (a) Tích chập chuẩn; (b) tích chập tách được theo chiều sâu của MobileNet

3.2 Khảo sát dữ liệu

Mọi mô hình đọc cùng một tệp dữ liệu và cùng một phép chia. 10% tập huấn luyện gốc được tách thành Validation theo từng lớp; tập kiểm tra gốc chỉ dùng một lần ở bước cuối (Bảng 3.1).

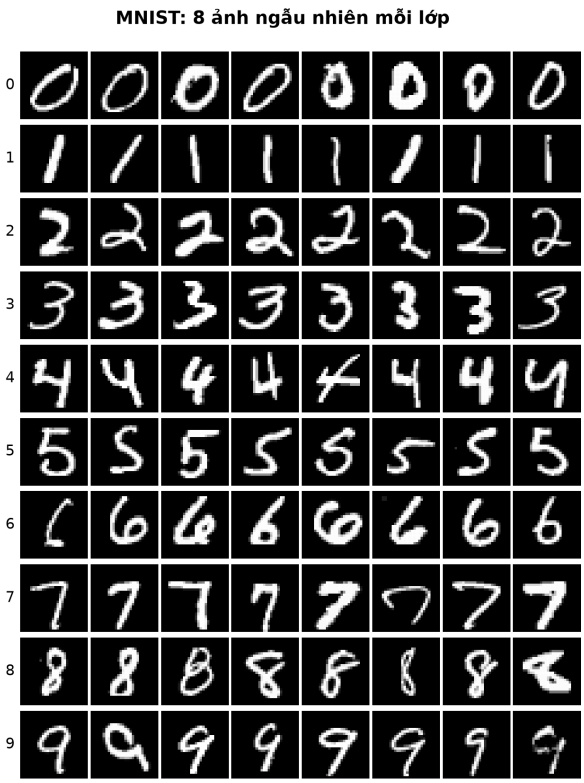
Bảng 3.1. Bốn bộ dữ liệu ảnh của chương sau khi tải và chia tập

Bộ dữ liệu	Huấn luyện	Validation	Kiểm tra	Kích thước gốc	Số lớp	Ảnh vào B/I
MNIST	54.001	5.999	10.000	$28 \times 28 \times 1$	10	28 / 28
Fashion-MNIST	54.000	6.000	10.000	$28 \times 28 \times 1$	10	28 / 28
Rock/Paper/Scissors	2.268	252	372	$128 \times 128 \times 3$	3	64 / 96
CIFAR-10	45.000	5.000	10.000	$32 \times 32 \times 3$	10	32 / 32

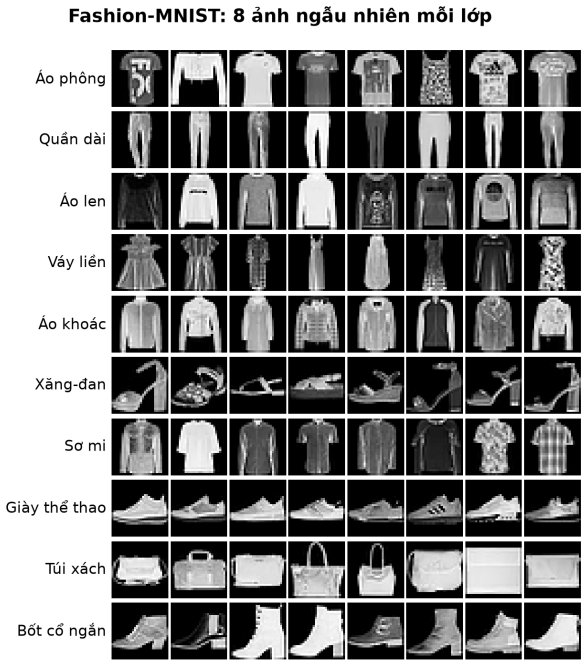
Validation tách phân tầng 10 % từ tập huấn luyện gốc (hạt giống cố định). Ảnh vào B/I: cạnh ảnh đưa vào Basic CNN / Improved CNN.

- **MNIST** [41]: chữ số viết tay, ảnh xám, nền đen sạch, các lớp cân bằng; là bài toán dễ dùng để kiểm chứng cài đặt.
- **Fashion-MNIST** [127]: cùng định dạng nhưng là trang phục; các lớp áo sơ mi, áo phông, áo khoác dễ nhầm với nhau.
- **Rock/Paper/Scissors** [128]: ảnh bàn tay dựng bằng đồ hoạ máy tính, tập kiểm tra có kiểu bàn tay không xuất hiện khi huấn luyện, nên độ dịch chuyển phân phối lớn.
- **CIFAR-10** [129]: ảnh màu 32×32 của vật thể thật, nền phức tạp, độ phân giải thấp, là bộ khó nhất.

Tiền xử lý gồm đưa điểm ảnh về $[0, 1]$, chuẩn hoá theo trung bình và độ lệch chuẩn của từng kênh tính trên tập huấn luyện. Improved CNN dùng thêm tăng cường dữ liệu theo lô trên GPU, như lật, dịch, xoay (Hình 3.10).



(a) MNIST

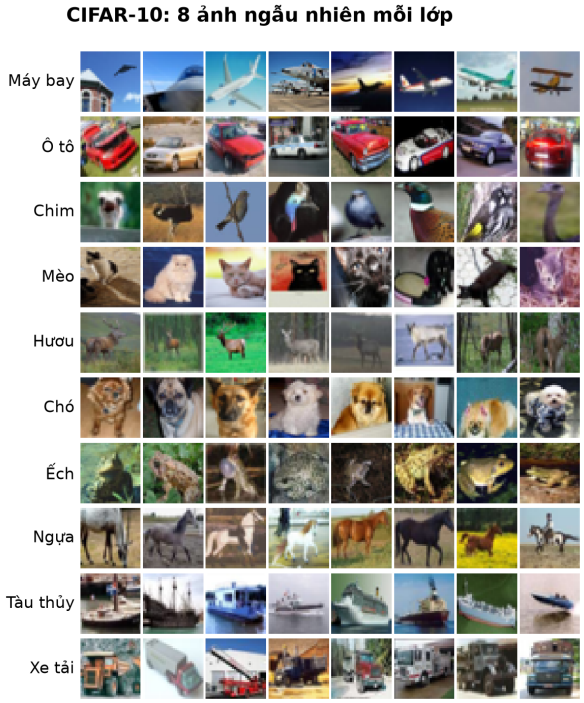


(b) Fashion-MNIST

Hình 3.8. Ảnh ngẫu nhiên theo lớp của hai bộ ảnh xám



(a) Rock/Paper/Scissors



(b) CIFAR-10

Hình 3.9. Ảnh ngẫu nhiên theo lớp của hai bộ ảnh màu



Hình 3.10. Tăng cường dữ liệu cho CIFAR-10: cột đầu là ảnh gốc, các cột sau là biến thể

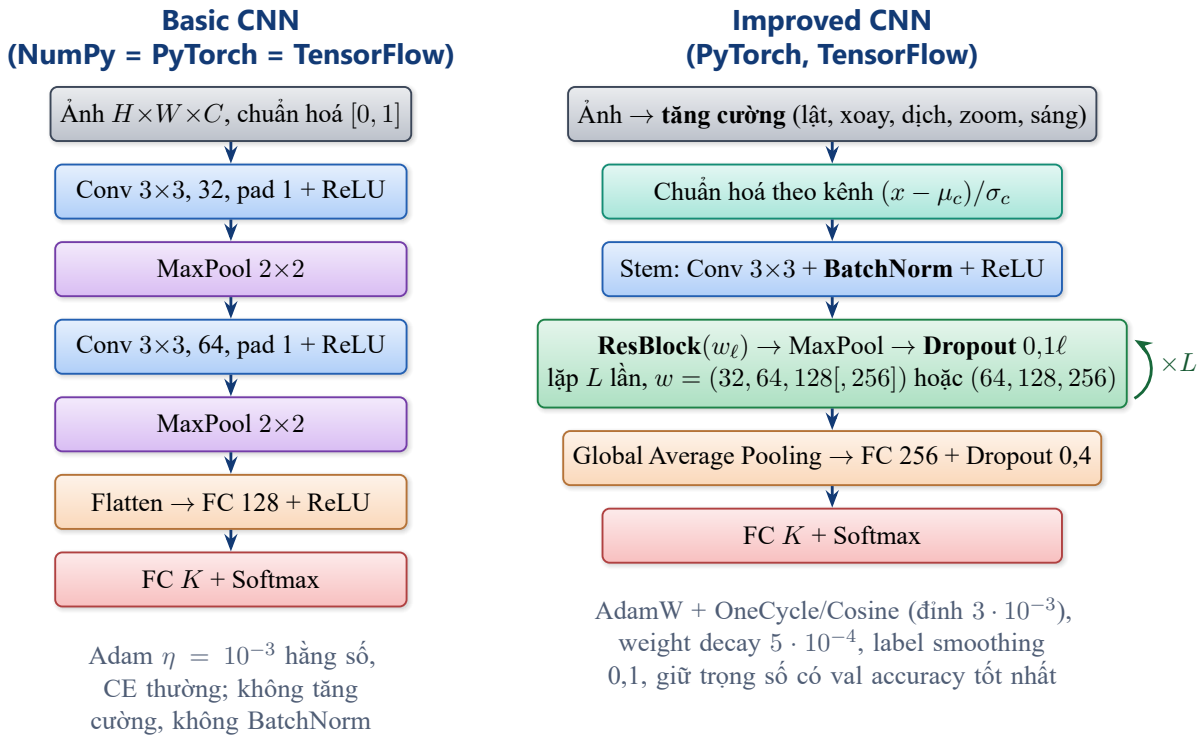
3.3 Hiện thực trên ba nền tảng

3.3.1 Hai họ mô hình

Basic CNN có hai khối tích chập và tầng Dense, cùng kiến trúc trên cả ba nền tảng. Improved CNN thêm BatchNorm, khối dư kiểu ResNet, Global Average Pooling, tăng cường dữ liệu và lịch học OneCycle [130] (Hình 3.11).

Bảng 3.2. Kiến trúc Basic CNN cho ảnh xám 28×28

Tầng	Kích thước đầu ra	Số tham số
Đầu vào	$28 \times 28 \times 1$	0
Conv 3×3, 32, pad 1 + ReLU	$28 \times 28 \times 32$	320
MaxPool 2×2	$14 \times 14 \times 32$	0
Conv 3×3, 64, pad 1 + ReLU	$14 \times 14 \times 64$	18.496
MaxPool 2×2	$7 \times 7 \times 64$	0
Flatten	3.136	0
Dense 128 + ReLU	128	401.536
Dense K + Softmax	10	1.290
Tổng		**421.642**



Hình 3.11. Hai họ mô hình trong thực nghiệm: Basic CNN và Improved CNN

3.3.2 Tích chập từ đầu bằng NumPy

Vòng lặp lồng trên điểm ảnh rất chậm trong Python. Kỹ thuật *im2col* trải mọi cửa sổ thành một hàng của ma trận, biến tích chập thành phép nhân ma trận vector hoá [131].

Đoạn mã 3.1. Hàm `im2col`: trải mọi cửa sổ $k \times k$ thành các hàng của ma trận

```python
# tl3/ch3_scratch.py
def im2col(X, kh, kw, stride=1, pad=0):
    """X [N, C, H, W] -> ma trận cột [N·Ho·Wo, C·kh·kw]: mỗi hàng là một cửa sổ
    ↪ (receptive field) đã duỗi thẳng.

    Thay vì 4 vòng lặp lồng nhau, chỉ lặp kh·kw lần (9 lần với kernel 3×3); mỗi lần
    ↪ cắt một lát ảnh dịch chuyển."""
    N, C, H, W = X.shape
    Ho = (H + 2 * pad - kh) // stride + 1
    Wo = (W + 2 * pad - kw) // stride + 1
    Xp = np.pad(X, ((0, 0), (0, 0), (pad, pad), (pad, pad))) if pad else X
    cols = np.empty((N, C, kh, kw, Ho, Wo), dtype=X.dtype)
    for i in range(kh):
        for j in range(kw):
            cols[:, :, i, j] = Xp[:, :, i:i + stride * Ho:stride, j:j + stride *
            ↪ Wo:stride]
    return cols.transpose(0, 4, 5, 1, 2, 3).reshape(N * Ho * Wo, -1), Ho, Wo
```

Đoạn mã 3.2. Lan truyền thuận của tầng tích chập bằng phép nhân ma trận

```python
# tl3/ch3_scratch.py
def conv_forward(X, W, b, pad=1):
    """Z = X * W + b. W [F, C, kh, kw]; trả Z [N, F, Ho, Wo] và cache cho lan truyền ngược."""
    F, C, kh, kw = W.shape
    cols, Ho, Wo = im2col(X, kh, kw, 1, pad)
    Z = cols @ W.reshape(F, -1).T + b                       # [N·Ho·Wo, F]
    Z = Z.reshape(X.shape[0], Ho, Wo, F).transpose(0, 3, 1, 2)
    return Z, (X.shape, cols, W, pad, Ho, Wo)
```

Đoạn mã 3.3. Lan truyền ngược của tầng tích chập

```python
# tl3/ch3_scratch.py
def conv_backward(dZ, cache, need_dx=True):
    """Từ dL/dZ tính dL/dW = dZᵀ·cols, dL/db = Σ dZ và dL/dX = col2im(dZ·W).
    Tầng tích chập đầu tiên không cần dL/dX (đầu vào là ảnh) nên bỏ qua để tiết kiệm bộ nhớ và thời gian."""
    X_shape, cols, W, pad, Ho, Wo = cache
    F, C, kh, kw = W.shape
    dZ2 = dZ.transpose(0, 2, 3, 1).reshape(-1, F)           # [N·Ho·Wo, F]
    dW = (dZ2.T @ cols).reshape(W.shape)
    db = dZ2.sum(axis=0)
    dX = col2im(dZ2 @ W.reshape(F, -1), X_shape, kh, kw, Ho, Wo, 1, pad) if need_dx else None
    return dX, dW, db
```

Lan truyền ngược tự viết được kiểm tra bằng sai phân số, sai số tương đối lớn nhất $2{,}37 \times 10^{-10}$, ở mức sai số làm tròn.

3.3.3 Khối dư bằng PyTorch và Keras

Đoạn mã 3.4. Khối dư bằng PyTorch: Conv – BN – ReLU – Conv – BN cộng lối tắt

```python
# tl3/ch3_torch.py
class ResBlock(nn.Module):
    """Khối dư (residual): Conv-BN-ReLU-Conv-BN cộng lối tắt (1×1 khi đổi số kênh), rồi ReLU."""

    def __init__(self, cin, cout):
        super().__init__()
        self.conv1 = nn.Conv2d(cin, cout, 3, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(cout)
        self.conv2 = nn.Conv2d(cout, cout, 3, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(cout)
        self.short = (nn.Identity() if cin == cout else
                      nn.Sequential(nn.Conv2d(cin, cout, 1, bias=False),
                      nn.BatchNorm2d(cout)))

    def forward(self, x):
        y = F.relu(self.bn1(self.conv1(x)))
        y = self.bn2(self.conv2(y))
        return F.relu(y + self.short(x))
```

Đoạn mã 3.5. Improved CNN bằng PyTorch: stem, các khối dư, Global Average Pooling

```python
# tl3/ch3_torch.py
class ImprovedCNN(nn.Module):
    """Stem Conv-BN-ReLU → [ResBlock → MaxPool → Dropout tăng dần] × len(widths) → GAP → FC256 → Dropout → FC(K)."""

    def __init__(self, in_ch, n_classes, widths=(32, 64, 128)):
        super().__init__()
        self.stem = nn.Sequential(nn.Conv2d(in_ch, widths[0], 3, padding=1, bias=False),
                                  nn.BatchNorm2d(widths[0]), nn.ReLU())
        blocks, cin = [], widths[0]
        for i, w in enumerate(widths):
            blocks += [ResBlock(cin, w), nn.MaxPool2d(2), nn.Dropout(0.1 + 0.1 * i)]
            cin = w
        self.blocks = nn.Sequential(*blocks)
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Linear(cin, 256), nn.ReLU(),
                                  nn.Dropout(0.4), nn.Linear(256, n_classes))

    def forward(self, x):
        return self.head(self.blocks(self.stem(x)))
```

Đoạn mã 3.6. Khối dư bằng Keras Functional API

```python
# tl3/ch3_keras.py
def res_block(x, filters):
    """Khối dư Keras: Conv-BN-ReLU-Conv-BN + lối tắt (Conv 1×1 + BN khi đổi số kênh) → ReLU."""
    from keras import layers
    y = layers.Conv2D(filters, 3, padding="same", use_bias=False)(x)
    y = layers.ReLU()(layers.BatchNormalization()(y))
    y = layers.BatchNormalization()(layers.Conv2D(filters, 3, padding="same", use_bias=False)(y))
    if x.shape[-1] != filters:
        x = layers.BatchNormalization()(layers.Conv2D(filters, 1, use_bias=False)(x))
    return layers.ReLU()(layers.Add()([x, y]))
```

Đoạn mã 3.7. Basic CNN bằng Keras `Sequential`

```python
# tl3/ch3_keras.py
def build_basic(in_shape, n_classes):
    """Basic CNN Keras tương đương bản NumPy và PyTorch: Conv32 → Pool → Conv64 → Pool → Dense128 → Dense(K)."""
    from keras import Sequential, layers
    return Sequential([
        layers.Input(in_shape),
        layers.Conv2D(32, 3, padding="same", activation="relu"), layers.MaxPooling2D(2),
        layers.Conv2D(64, 3, padding="same", activation="relu"), layers.MaxPooling2D(2),
        layers.Flatten(), layers.Dense(128, activation="relu"), layers.Dense(n_classes, activation="softmax"),
    ], name="basic_cnn")
```

Đoạn mã 3.8. Improved CNN bằng Keras: tăng cường, chuẩn hoá, khối dư

```python
# tl3/ch3_keras.py
def build_improved(in_shape, n_classes, cfg, mean, std):
    """Improved CNN Keras: tăng cường → chuẩn hoá theo kênh → stem → [ResBlock →
    ↪  MaxPool → Dropout] → GAP → Dense."""
    import keras
    from keras import layers
    inp = layers.Input(in_shape)
    x = augmentation(cfg)(inp)
    x = layers.Normalization(mean=mean, variance=std ** 2)(x)
    x = layers.Conv2D(cfg["widths"][0], 3, padding="same", use_bias=False)(x)
    x = layers.ReLU()(layers.BatchNormalization()(x))
    for i, w in enumerate(cfg["widths"]):
        x = res_block(x, w)
        x = layers.Dropout(0.1 + 0.1 * i)(layers.MaxPooling2D(2)(x))
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.4)(layers.Dense(256, activation="relu")(x))
    out = layers.Dense(n_classes, activation="softmax")(x)
    return keras.Model(inp, out, name="improved_cnn")
```

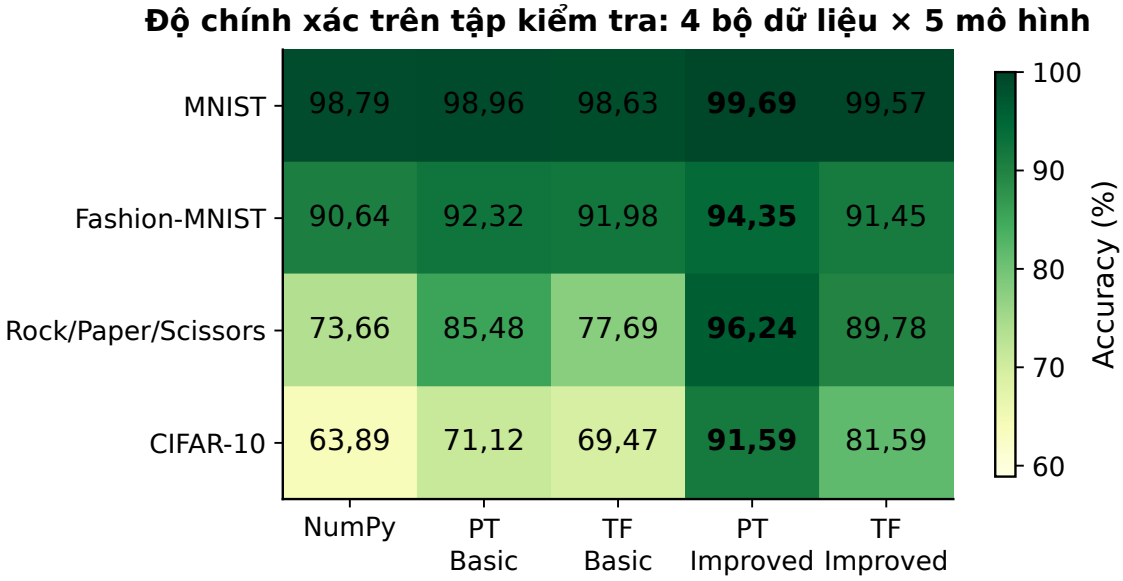
3.4 Đánh giá và đối chuẩn

3.4.1 Kết quả tổng hợp

Bảng 3.3. Độ chính xác (%) trên tập kiểm tra của mọi mô hình trên bốn bộ dữ liệu

Bộ dữ liệu	NumPy	PT Basic	TF Basic	PT Improved	TF Improved
MNIST	98,79	98,96	98,63	**99,69**	99,57
Fashion-MNIST	90,64	92,32	91,98	**94,35**	91,45
Rock/Paper/Scissors	73,66	85,48	77,69	**96,24**	89,78
CIFAR-10	63,89	71,12	69,47	**91,59**	81,59

Độ chính xác (%) trên tập kiểm tra gốc của mỗi bộ dữ liệu.



Hình 3.12. Bản đồ nhiệt độ chính xác: bốn bộ dữ liệu (hàng) × năm mô hình (cột)

Nhận xét: Trên MNIST, ba bản Basic CNN cùng kiến trúc cho kết quả rất sát nhau: NumPy 98,79%, PyTorch 98,96%, TensorFlow 98,63%, xác nhận bản tự viết đúng; chênh lệch đến từ khởi tạo ngẫu nhiên và việc bản NumPy chỉ học trên 30.000 ảnh. Improved CNN PyTorch tốt nhất trên cả bốn bộ. Mức cải thiện tăng theo độ khó: 0,73 điểm trên MNIST, 2,03 điểm trên Fashion-MNIST, 10,75 điểm trên Rock/Paper/Scissors và 20,47 điểm trên CIFAR-10. Bản Improved TensorFlow thấp hơn vì chạy trên CPU với số epoch ít hơn nhiều (6 epoch trên CIFAR-10), không phải do framework: với cùng số epoch, hai bản Basic chỉ chênh 0,33 điểm trên MNIST.

3.4.2 Kết quả chi tiết theo bộ dữ liệu

Trên Rock/Paper/Scissors, Improved CNN PyTorch đạt 100,00% trên Validation nhưng 96,24% trên tập kiểm tra. Validation lấy từ cùng phân phối với Train có thể báo kết quả rất cao trong khi

Bảng 3.4. Mức cải thiện của Improved CNN so với Basic CNN trong cùng framework

Bộ dữ liệu	Framework	Basic (%)	Improved (%)	ΔAcc (đ.%)	ΔF1 (đ.%)	Giảm lỗi (%)
MNIST	PyTorch	98,96	99,69	+0,73	+0,74	70,2
MNIST	TensorFlow	98,63	99,57	+0,94	+0,94	68,6
Fashion-MNIST	PyTorch	92,32	94,35	+2,03	+2,09	26,4
Fashion-MNIST	TensorFlow	91,98	91,45	−0,53	−0,38	−6,6
Rock/Paper/Scissors	PyTorch	85,48	96,24	+10,75	+11,19	74,1
Rock/Paper/Scissors	TensorFlow	77,69	89,78	+12,10	+11,63	54,2
CIFAR-10	PyTorch	71,12	91,59	+20,47	+20,47	70,9
CIFAR-10	TensorFlow	69,47	81,59	+12,12	+11,80	39,7

Giảm lỗi = $1 - \text{lỗi}_{\text{Improved}}/\text{lỗi}_{\text{Basic}}$; đ.%: điểm phần trăm.

Bảng 3.5. Kết quả các mô hình trên tập kiểm tra MNIST

Mô hình	Acc (%)	Prec. (%)	Recall (%)	F1 (%)	Tham số	Ep. tốt/ tổng	Huấn luyện (s)	Suy luận (ms/ảnh)
Basic CNN – NumPy	98,79	98,78	98,78	98,78	421.642	5/5	2.007	5,085
Basic CNN – PyTorch	98,96	98,94	98,95	98,94	421.642	4/8	**99**	**0,022**
Basic CNN – TensorFlow	98,63	98,63	98,63	98,62	421.642	3/8	243	0,309
Improved CNN – PyTorch	**99,69**	**99,69**	**99,68**	**99,69**	342.378	14/15	444	0,048
Improved CNN – TensorFlow	99,57	99,58	99,56	99,57	343.722	6/6	3.147	1,644

Precision, Recall, F1 là trung bình macro trên tập kiểm tra. Thời gian suy luận đo trên toàn tập kiểm tra (PyTorch: GPU; NumPy, TensorFlow: CPU). Chữ đậm: tốt nhất cột.

Bảng 3.6. Kết quả các mô hình trên tập kiểm tra Fashion-MNIST

Mô hình	Acc (%)	Prec. (%)	Recall (%)	F1 (%)	Tham số	Ep. tốt/ tổng	Huấn luyện (s)	Suy luận (ms/ảnh)
Basic CNN – NumPy	90,64	90,81	90,64	90,58	421.642	6/6	2.426	6,674
Basic CNN – PyTorch	92,32	92,34	92,32	92,28	421.642	10/10	**100**	**0,022**
Basic CNN – TensorFlow	91,98	92,05	91,98	91,88	421.642	8/10	389	0,232
Improved CNN – PyTorch	**94,35**	**94,48**	**94,35**	**94,37**	342.378	22/25	905	0,082
Improved CNN – TensorFlow	91,45	91,68	91,45	91,50	343.722	7/7	3.659	1,029

Precision, Recall, F1 là trung bình macro trên tập kiểm tra. Thời gian suy luận đo trên toàn tập kiểm tra (PyTorch: GPU; NumPy, TensorFlow: CPU). Chữ đậm: tốt nhất cột.

Bảng 3.7. Kết quả các mô hình trên tập kiểm tra Rock/Paper/Scissors

Mô hình	Acc (%)	Prec. (%)	Recall (%)	F1 (%)	Tham số	Ep. tốt/ tổng	Huấn luyện (s)	Suy luận (ms/ảnh)
Basic CNN – NumPy	73,66	75,22	73,66	73,74	2.117.059	12/12	2.435	38,402
Basic CNN – PyTorch	85,48	86,36	85,48	85,04	2.117.059	3/15	**26**	**0,303**
Basic CNN – TensorFlow	77,69	77,83	77,69	77,63	2.117.059	4/15	112	2,938
Improved CNN – PyTorch	**96,24**	**96,62**	**96,24**	**96,22**	1.292.963	8/25	310	1,513
Improved CNN – TensorFlow	89,78	91,28	89,78	89,27	1.295.843	4/6	1.587	13,211

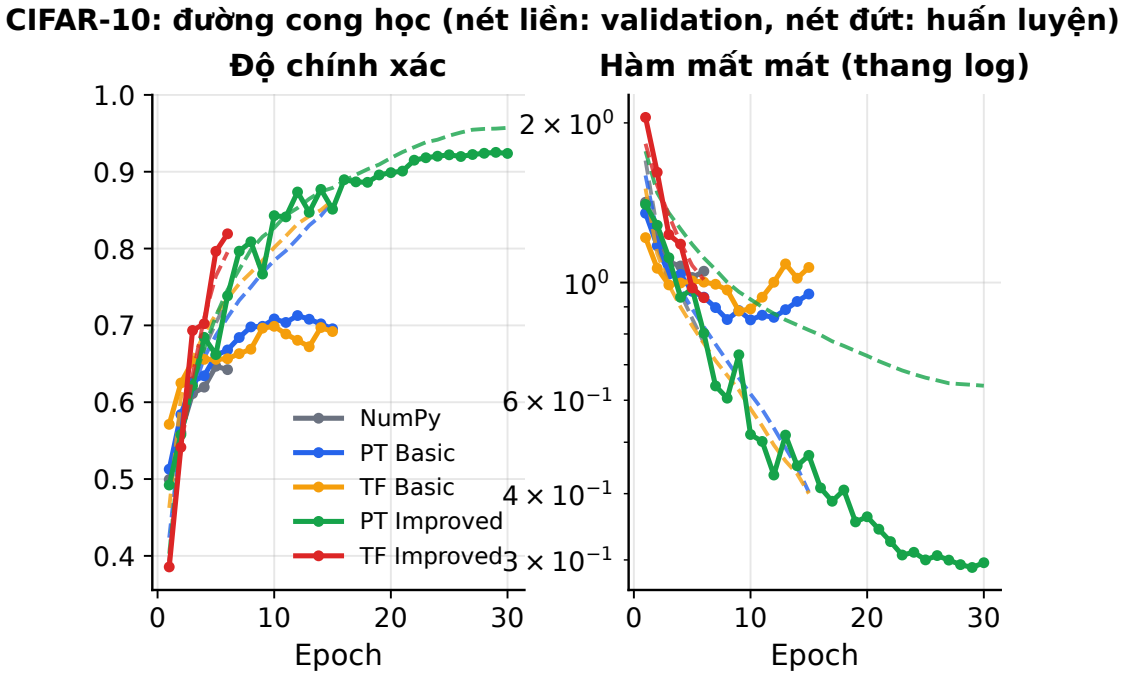
Precision, Recall, F1 là trung bình macro trên tập kiểm tra. Thời gian suy luận đo trên toàn tập kiểm tra (PyTorch: GPU; NumPy, TensorFlow: CPU). Chữ đậm: tốt nhất cột.

tập kiểm tra cho thấy khác biệt: đây là minh hoạ rõ cho dịch chuyển phân phối.

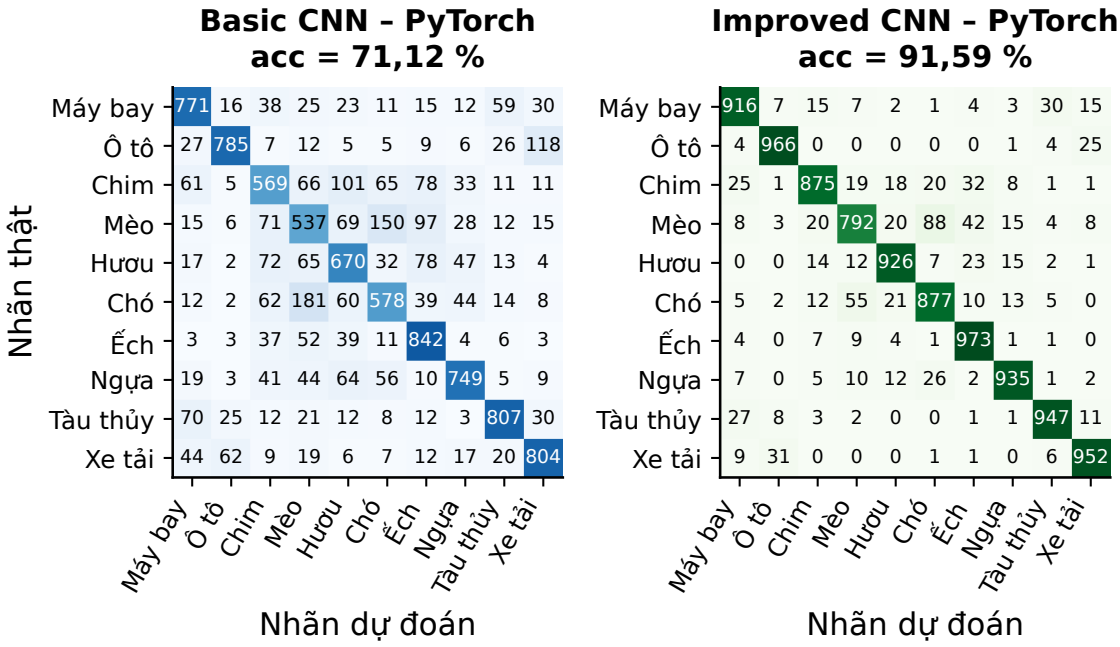
Bảng 3.8. Kết quả chi tiết trên tập kiểm tra CIFAR-10

Mô hình	Acc (%)	Prec. (%)	Recall (%)	F1 (%)	Tham số	Ep. tốt/ tổng	Huấn luyện (s)	Suy luận (ms/ảnh)
Basic CNN – NumPy	63,89	66,57	63,89	63,32	545.098	6/6	2.229	7,327
Basic CNN – PyTorch	71,12	71,25	71,12	71,07	545.098	12/15	**113**	**0,056**
Basic CNN – TensorFlow	69,47	70,24	69,47	69,57	545.098	10/15	434	0,326
Improved CNN – PyTorch	**91,59**	**91,58**	**91,59**	**91,54**	1.293.386	29/30	1.557	0,297
Improved CNN – TensorFlow	81,59	81,84	81,59	81,37	1.296.074	6/6	4.779	1,611

Precision, Recall, F1 là trung bình macro trên tập kiểm tra. Thời gian suy luận đo trên toàn tập kiểm tra (PyTorch: GPU; NumPy, TensorFlow: CPU). Chữ đậm: tốt nhất cột.



Hình 3.13. Đường cong học trên CIFAR-10



Hình 3.14. Ma trận nhầm lẫn trên tập kiểm tra CIFAR-10

Nhận xét: Basic CNN nhanh chóng quá khớp, còn Improved CNN nhờ tăng cường dữ liệu và BatchNorm duy trì khoảng cách nhỏ giữa Train và Validation. Basic CNN PyTorch đạt 86,12% trên Train nhưng chỉ 71,28% trên Validation. Lớp yếu nhất của Improved CNN là Mèo (F1 0,8311).

3.4.3 Chi phí tính toán

Mỗi epoch PyTorch trên GPU nhanh hơn bản NumPy khoảng 41,9 lần và nhanh hơn TensorFlow trên CPU khoảng 3,2 lần trên MNIST. Khác biệt đến từ phần cứng, không phải thuật toán. Improved CNN cho ảnh xám có ít tham số hơn Basic CNN nhờ Global Average Pooling thay tầng Dense lớn, nhưng tốn thời gian hơn vì sâu hơn và chạy nhiều epoch hơn.

3.4.4 Bên trong mô hình

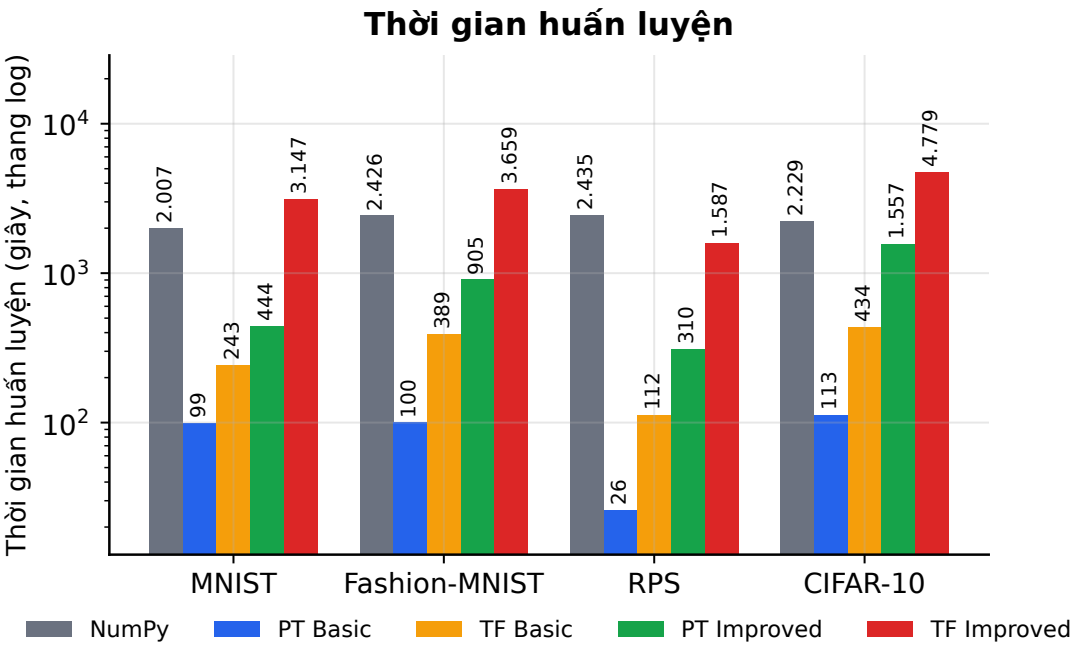
Tầng đầu học các bộ lọc cạnh và nét theo nhiều hướng, tầng sau cho các tổ hợp thưa hơn, đúng thứ bậc đặc trưng mà lý thuyết mô tả.

3.4.5 Diễn giải mô hình bằng Grad-CAM

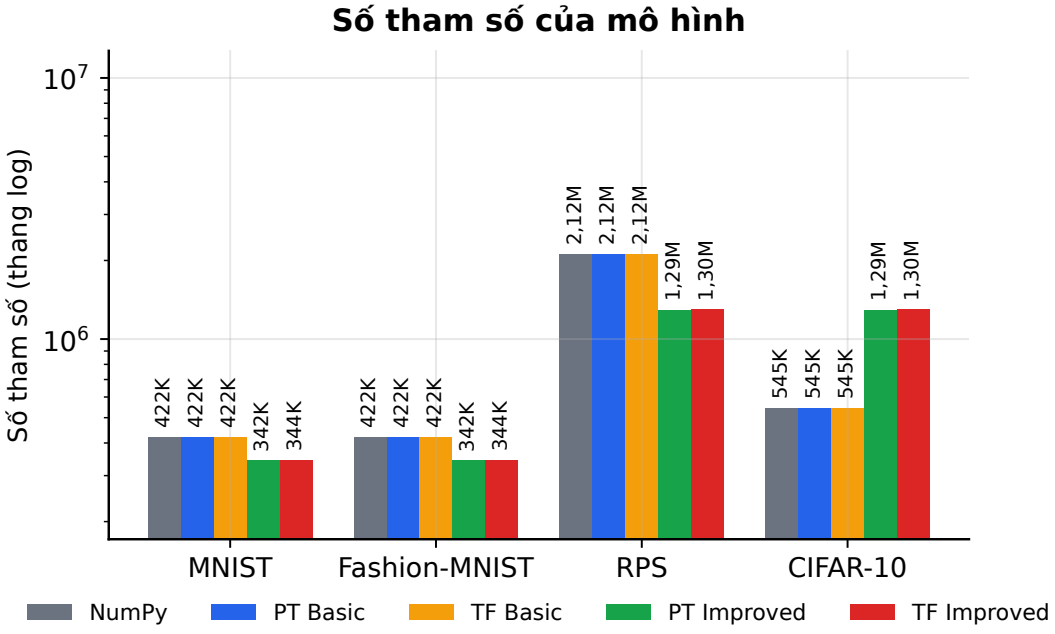
Grad-CAM [132] tính bản đồ nhiệt từ gradient của điểm lớp theo bản đồ đặc trưng A^k tầng tích chập cuối:

$$\alpha_k^c = \frac{1}{Z}\sum_{i,j}\frac{\partial y^c}{\partial A_{ij}^k}, \qquad L^c = \mathrm{ReLU}\Big(\sum_k \alpha_k^c A^k\Big). \tag{3.6}$$

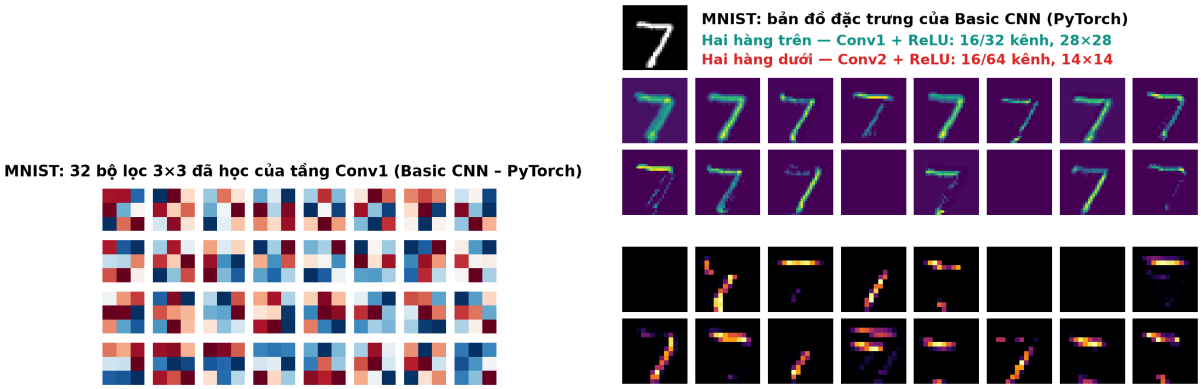
Nhận xét: Với “bao” và “kéo”, vùng nóng nằm đúng ở các ngón tay; cổ tay và nền không đóng



Hình 3.15. Thời gian huấn luyện của các mô hình (giây, thang log)

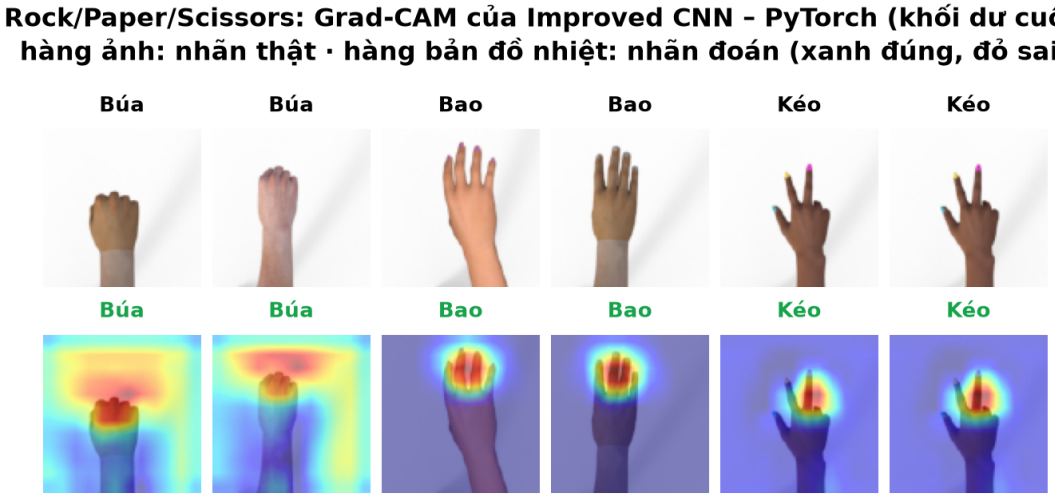


Hình 3.16. Số tham số của các mô hình (thang log)



(a) 32 bộ lọc Conv1 **(b)** Bản đồ đặc trưng Conv1, Conv2

Hình 3.17. Bên trong Basic CNN PyTorch trên MNIST: bộ lọc đã học (a) và bản đồ đặc trưng (b)

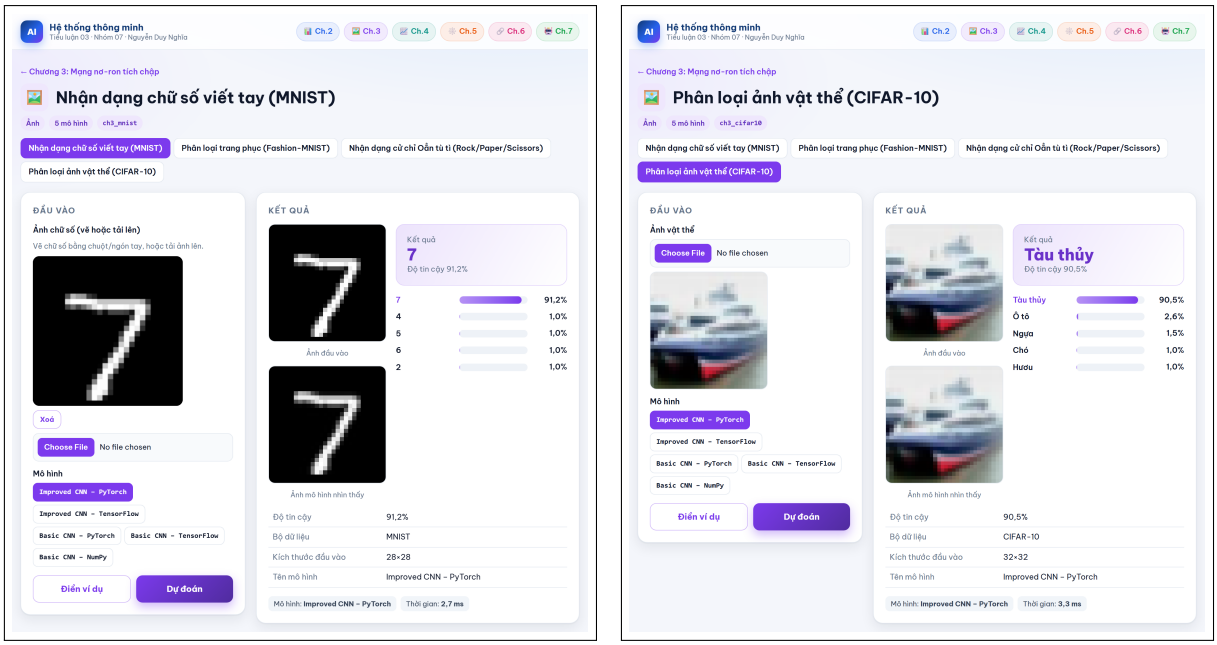


Hình 3.18. Grad-CAM của Improved CNN PyTorch trên Rock/Paper/Scissors

góp. Với "búa", vùng ảnh hưởng lan lên khoảng trống phía trên nắm tay, hợp lý vì điều phân biệt lớp này chính là việc không có ngón tay ở đó. Trên CIFAR-10, mô hình còn dựa vào cả nền, một rủi ro cần lưu ý khi suy rộng.

3.5 Triển khai Web App

Bốn mô hình tốt nhất được phục vụ qua `serve_ch3.py` với cùng tiền xử lý như lúc huấn luyện. Web App nhận ảnh tải lên hoặc nét vẽ tay, trả nhãn, xác suất top-5 và ảnh đầu vào mà mô hình thực sự nhìn thấy (Hình 3.19).



(a) Nhận dạng chữ số MNIST **(b)** Phân loại ảnh CIFAR-10

Hình 3.19. Giao diện Web App thị giác máy tính

3.6 Tổng kết chương

Bốn kết luận chính được rút ra. Thứ nhất, framework không làm mô hình thông minh hơn: Basic CNN viết bằng NumPy đạt độ chính xác ngang bản PyTorch và TensorFlow cùng kiến trúc trên MNIST, framework chỉ làm mô hình nhanh hơn và dễ mở rộng hơn. Thứ hai, kiến trúc quan trọng hơn khi bài toán khó: khối dư, BatchNorm và tăng cường dữ liệu mang lại 20,47 điểm trên CIFAR-10 (đạt 91,59%). Thứ ba, dữ liệu quyết định giới hạn: lỗi trên Fashion-MNIST là các cặp áo có nhãn mơ hồ, lỗi trên Rock/Paper/Scissors chủ yếu là kiểu bàn tay chưa từng thấy khi huấn luyện. Thứ tư, CNN giải thích được một phần bằng Grad-CAM. CNN xử lý tốt dữ liệu không gian nhưng không có trí nhớ theo thời gian; chương sau trình bày mạng hồi quy cho dữ liệu chuỗi.

Chương 4

Mạng nơ-ron hồi quy (RNN)

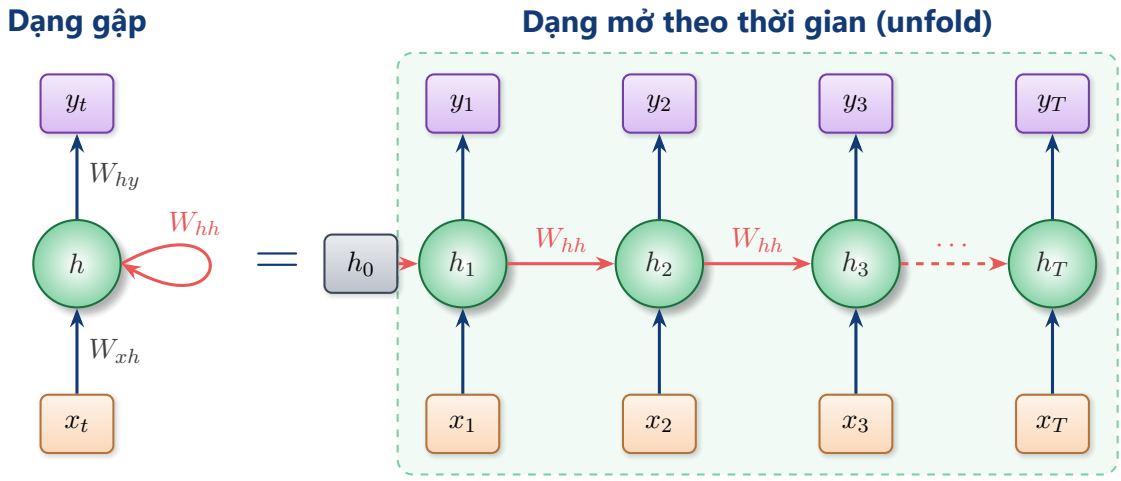
Dữ liệu chuỗi có thứ tự, độ dài thay đổi và phụ thuộc theo thời gian, những điều MLP và CNN không biểu diễn tự nhiên. RNN [133, 33] giữ một trạng thái ẩn đóng vai trò bộ nhớ. Chương này trình bày cơ sở toán học, hiện tượng tiêu biến gradient, các kiến trúc có cổng, rồi thực nghiệm dự báo trên ba chuỗi thời gian thật với PyTorch, Keras và bản NumPy từ đầu.

4.1 Cơ sở lý thuyết

4.1.1 Simple RNN và đồ thị tính toán mở rộng

Tại bước t, mô hình kết hợp đầu vào $\mathbf{x}_t$ với trạng thái trước đó, dùng chung tham số cho mọi bước:

$$\mathbf{h}_t = \tanh\left(W_{hh}\mathbf{h}_{t-1} + W_{xh}\mathbf{x}_t + \mathbf{b}_h\right), \qquad \hat{\mathbf{y}}_t = W_{hy}\mathbf{h}_t + \mathbf{b}_y. \tag{4.1}$$



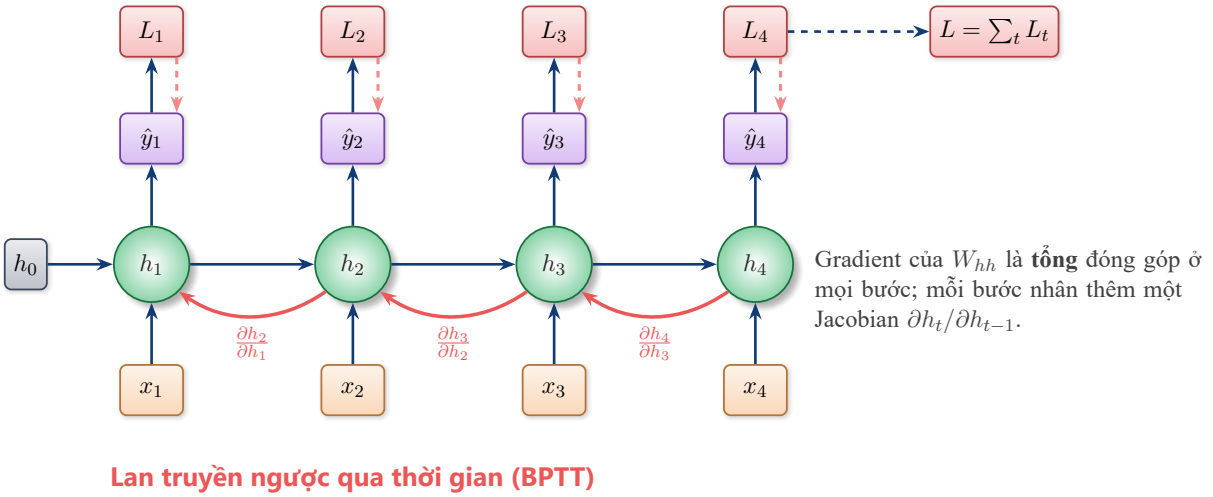
Hình 4.1. Mở rộng RNN theo thời gian: cùng một ô với cùng trọng số được lặp lại ở mọi bước

4.1.2 BPTT và tiêu biến gradient

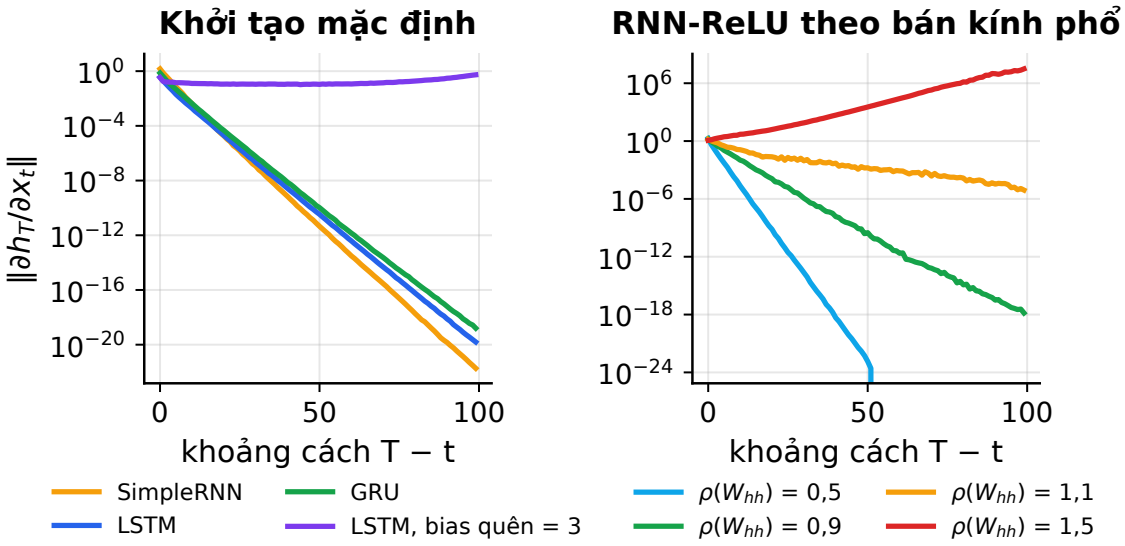
Lan truyền ngược qua thời gian (BPTT) [134] mở rộng mạng theo các bước rồi áp dụng quy tắc chuỗi (Hình 4.2). Gradient của mất mát tại T theo trạng thái t chứa tích các Jacobian:

$$\frac{\partial L_T}{\partial \mathbf{h}_t} = \frac{\partial L_T}{\partial \mathbf{h}_T}\prod_{k=t+1}^{T}\frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}, \qquad \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}} = \operatorname{diag}\left(\tanh'(\cdot)\right) W_{hh}. \tag{4.2}$$

Vì $|\tanh'| \le 1$, nếu chuẩn phổ của W_{hh} nhỏ hơn 1 thì tích giảm theo hàm mũ (tiêu biến), nếu lớn hơn nhiều thì có thể tăng theo hàm mũ (bùng nổ) [135, 136]. Hình 4.3 đo trực tiếp hiện tượng này: SimpleRNN mất gần hết gradient sau 50 bước, LSTM giữ được.



Hình 4.2. BPTT: mất mát tại mỗi bước lan truyền ngược qua toàn bộ chuỗi



Hình 4.3. Chuẩn gradient theo khoảng cách thời gian (thang log)

4.1.3 Cắt chuẩn gradient

Với hiện tượng bùng nổ, ta co gradient khi chuẩn vượt ngưỡng θ mà vẫn giữ nguyên hướng [136]:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{\theta}{\|\mathbf{g}\|}\right). \tag{4.3}$$

4.1.4 LSTM và GRU

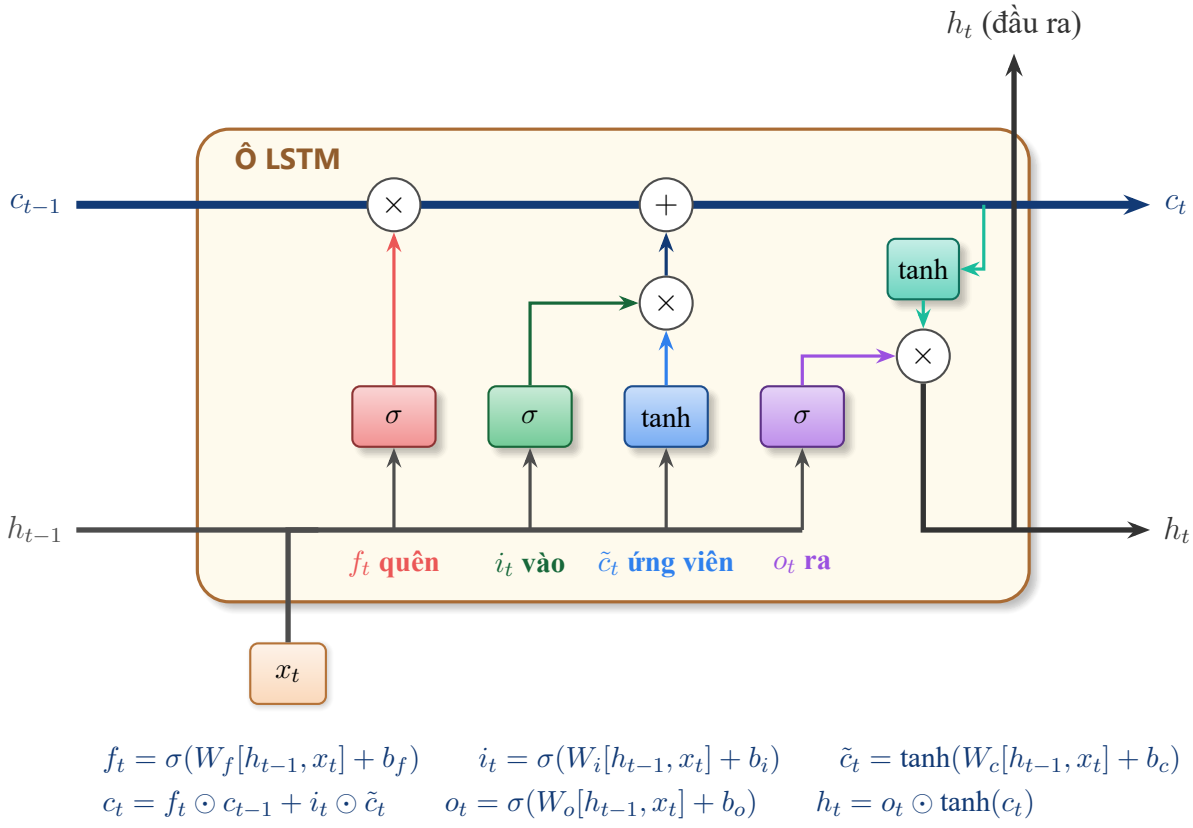
LSTM [40, 137] thêm trạng thái ô $\mathbf{c}_t$ được cập nhật theo kiểu cộng, điều khiển bởi ba cổng (Hình 4.4):

$$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f), \qquad \mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i), \tag{4.4}$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c), \qquad \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{4.5}$$

$$\mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o), \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \tag{4.6}$$

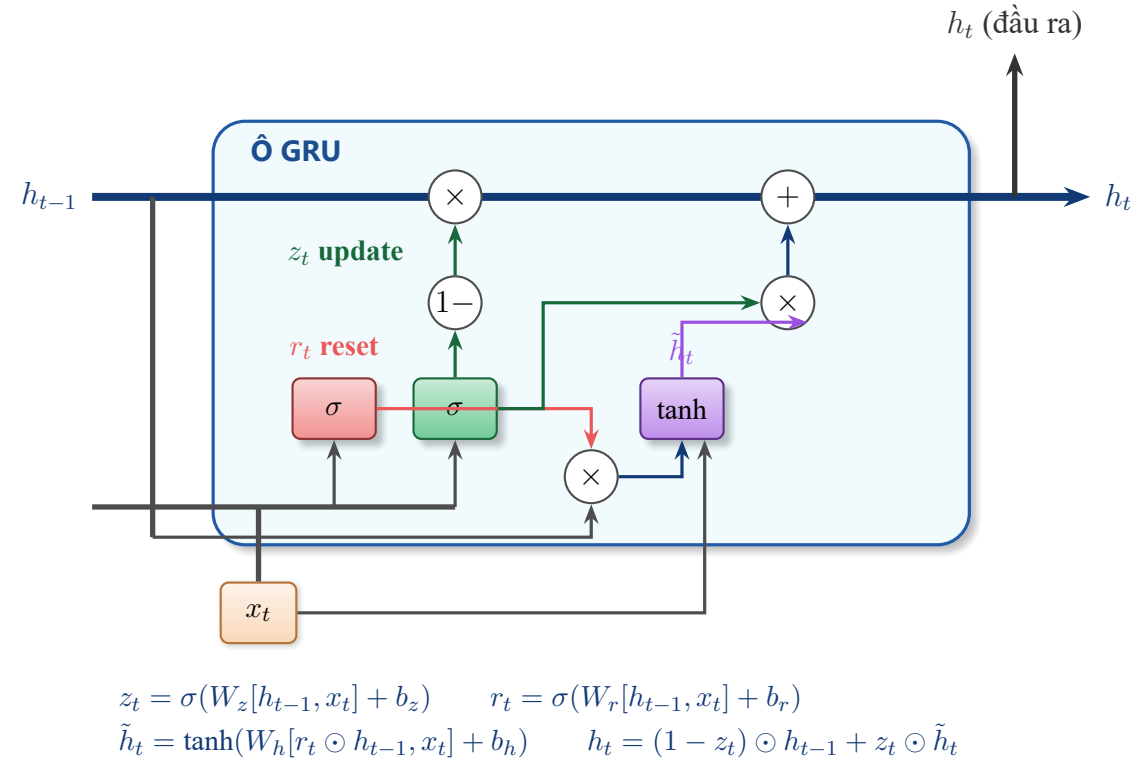
Khi cổng quên gần 1, $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} \approx I$ nên gradient đi qua nhiều bước mà không suy giảm. GRU [138, 139] gộp lại còn hai cổng, cổng cập nhật $\mathbf{z}_t$ và cổng đặt lại $\mathbf{r}_t$, không có trạng thái ô riêng nên ít tham số hơn. Nghiên cứu đối chiếu cho thấy GRU và LSTM thường tương đương [139].



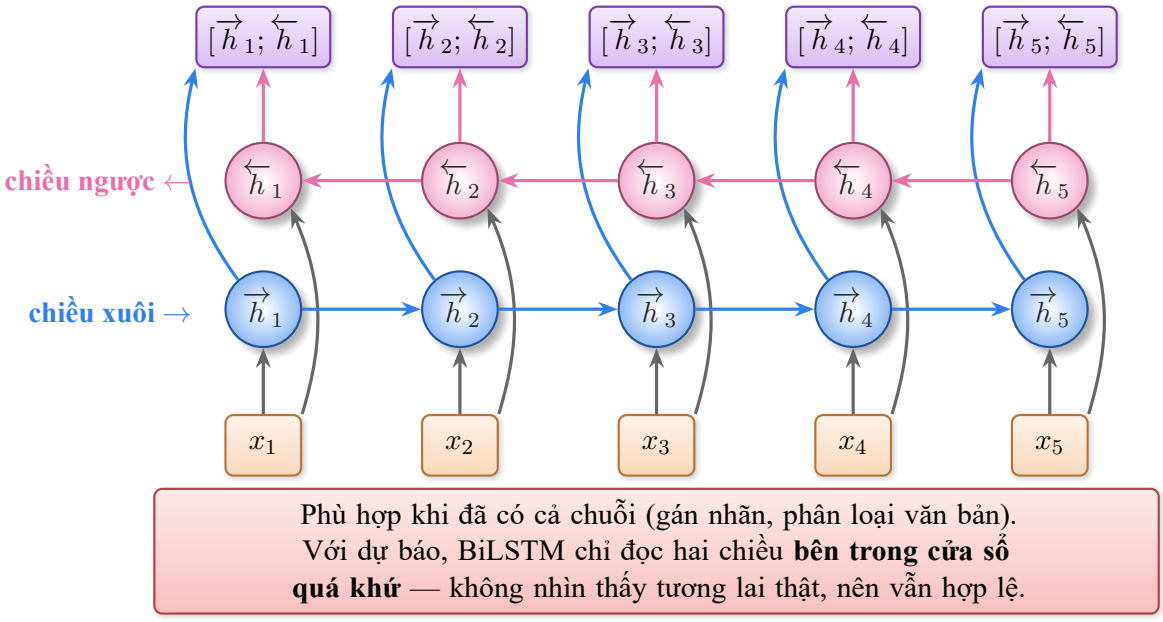
Hình 4.4. Ô LSTM với ba cổng và trạng thái ô chạy xuyên suốt

4.1.5 BiLSTM và chống quá khớp

BiLSTM [140, 141] chạy hai lớp LSTM theo hai chiều rồi ghép trạng thái (Hình 4.6). Với dự báo, cần lưu ý chiều ngược chỉ đọc lại bên trong cửa sổ quá khứ, không dùng dữ liệu tương lai. Dropout [112] và dừng sớm được dùng để chống quá khớp.



Hình 4.5. Ô GRU với cổng cập nhật và cổng đặt lại



Hình 4.6. RNN hai chiều: trạng thái hai chiều được ghép để tạo đầu ra

4.2 Khảo sát dữ liệu

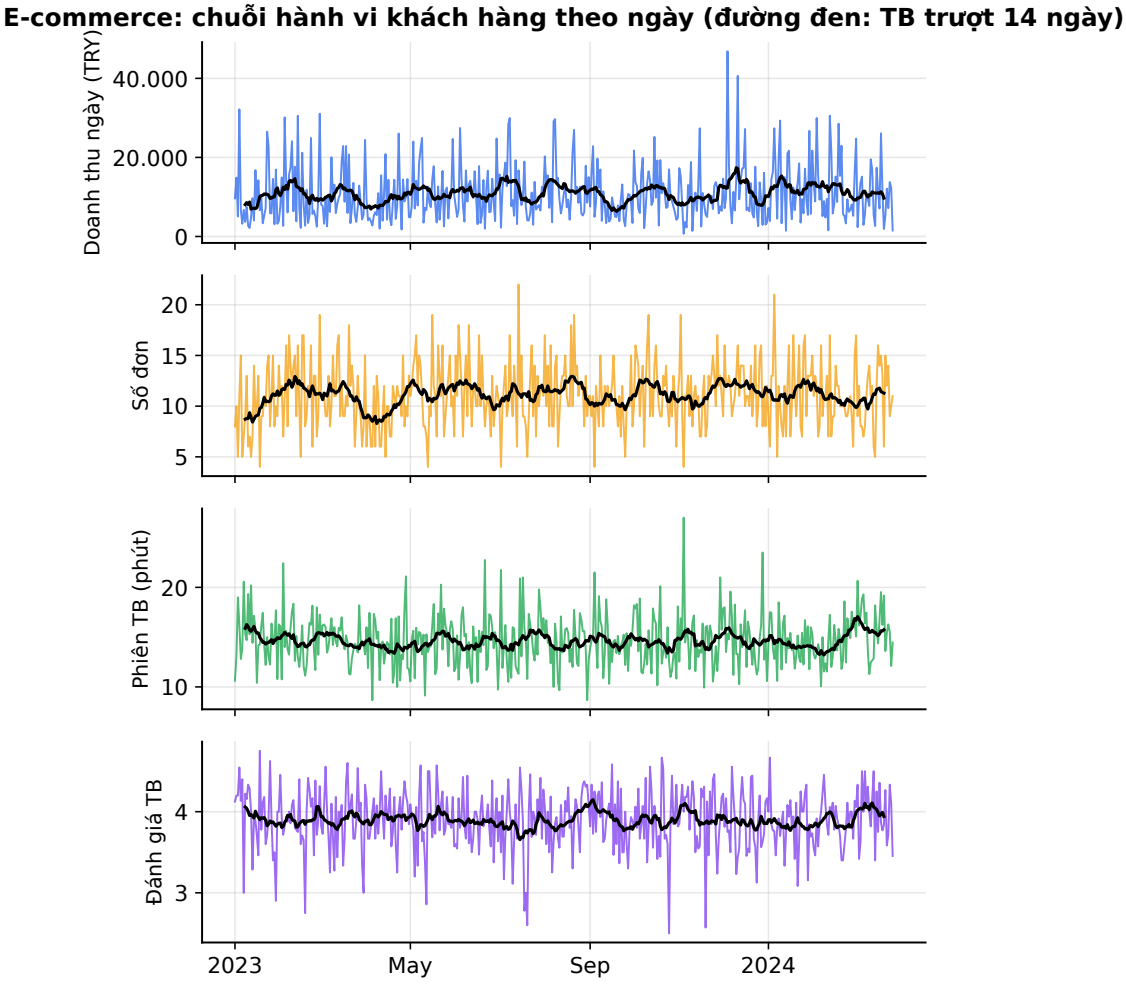
Ba chuỗi thời gian thật được tải từ Kaggle (Bảng 4.1).

Bảng 4.1. Ba bộ dữ liệu chuỗi thời gian của chương

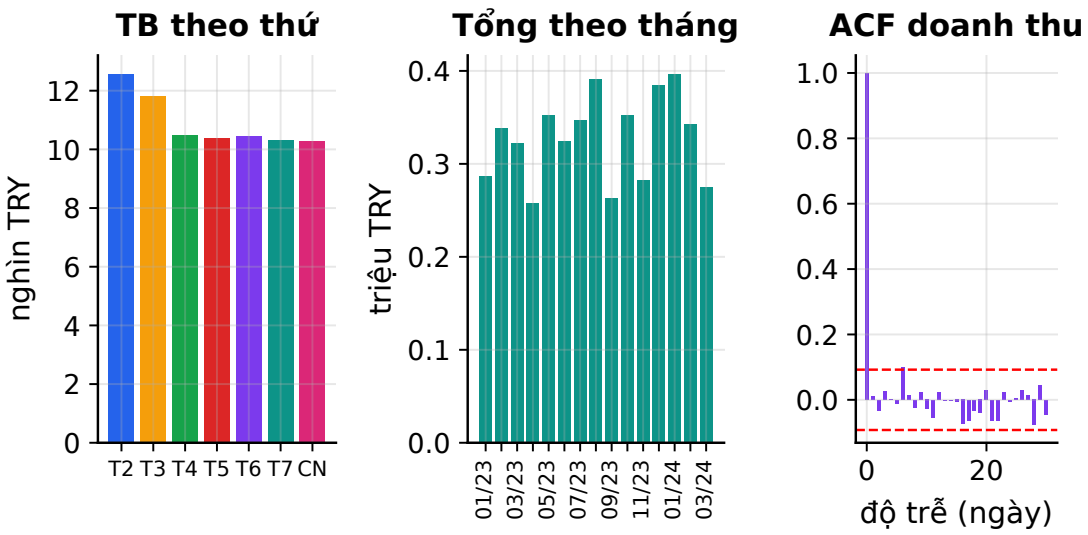
Bộ dữ liệu	Khoảng thời gian	Số bước	Đặc trưng	Mục tiêu	Đơn vị
E-commerce (TR)	2023-01-01 → 2024-03-26	451 ngày	9	Total_Amount	TRY
AAPL	1980-12-12 → 2024-06-20	10.971 phiên	6	close	USD
Vàng (FINAL_USO)	2011-12-15 → 2018-12-31	1.718 phiên	13	GLD_Close	USD

4.2.1 Doanh thu ngày của sàn thương mại điện tử

Bộ dữ liệu [142] gồm 5.000 đơn hàng của một sàn tại Thổ Nhĩ Kỳ, 18 cột, được gộp thành chuỗi 451 ngày với chín đặc trưng. Doanh thu dao động mạnh quanh một mức ổn định, tự tương quan gần 0 (Hình 4.7). Mục tiêu là dự báo doanh thu ngày kế tiếp.



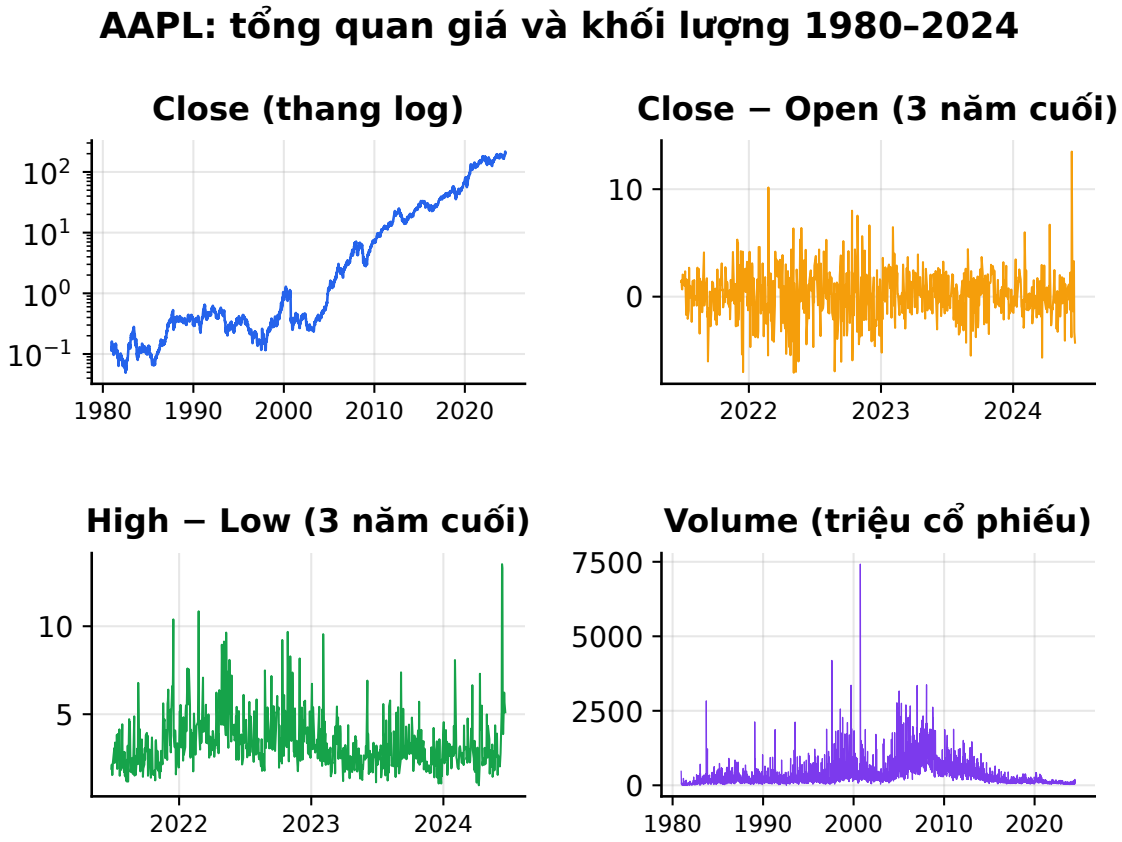
Hình 4.7. Chuỗi doanh thu ngày của sàn thương mại điện tử



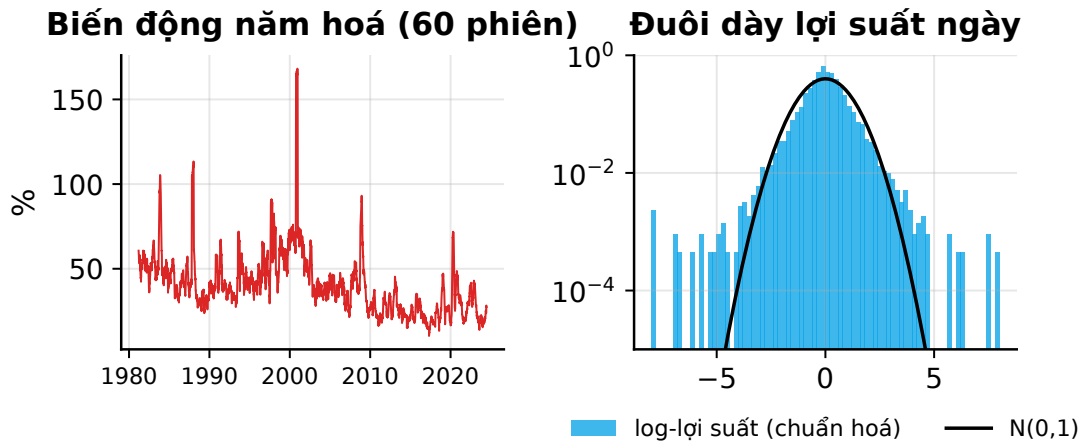
Hình 4.8. Tính mùa vụ của doanh thu theo ngày trong tuần và theo tháng

4.2.2 Cổ phiếu Apple

Bộ dữ liệu [143] gồm 10.971 phiên từ 1980-12-12 đến 2024-06-20. Giá đóng cửa tăng từ 0,1283 USD lên 209,65 USD, gấp 1.633 lần, tức chuỗi không dừng.



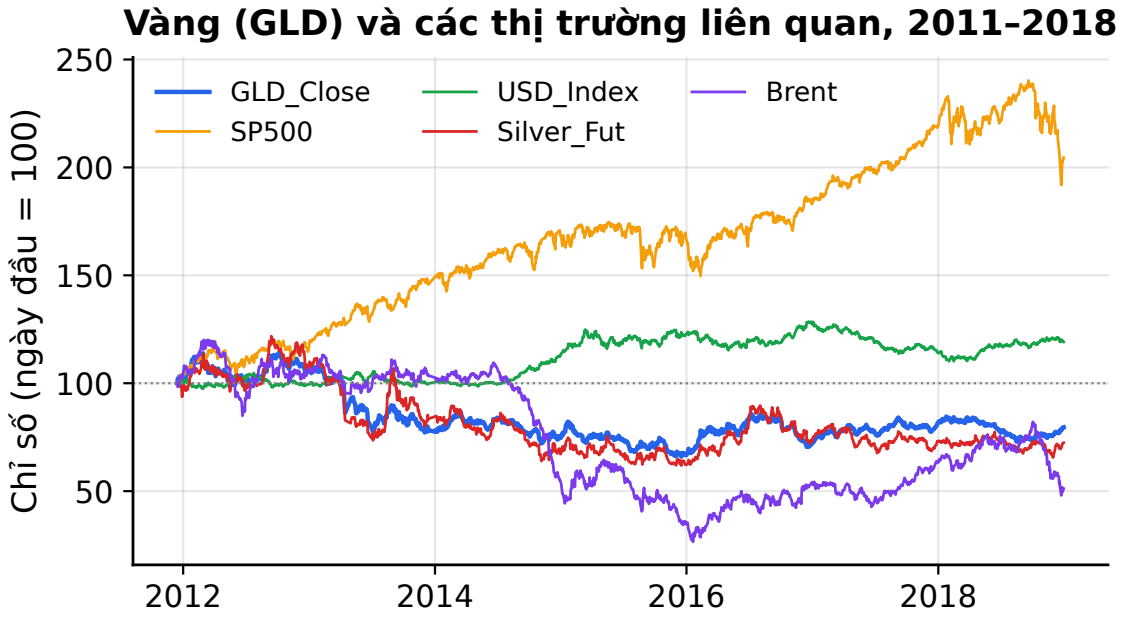
Hình 4.9. Giá đóng cửa và khối lượng giao dịch của cổ phiếu AAPL



Hình 4.10. Lợi suất ngày của cổ phiếu AAPL

4.2.3 Quỹ vàng GLD

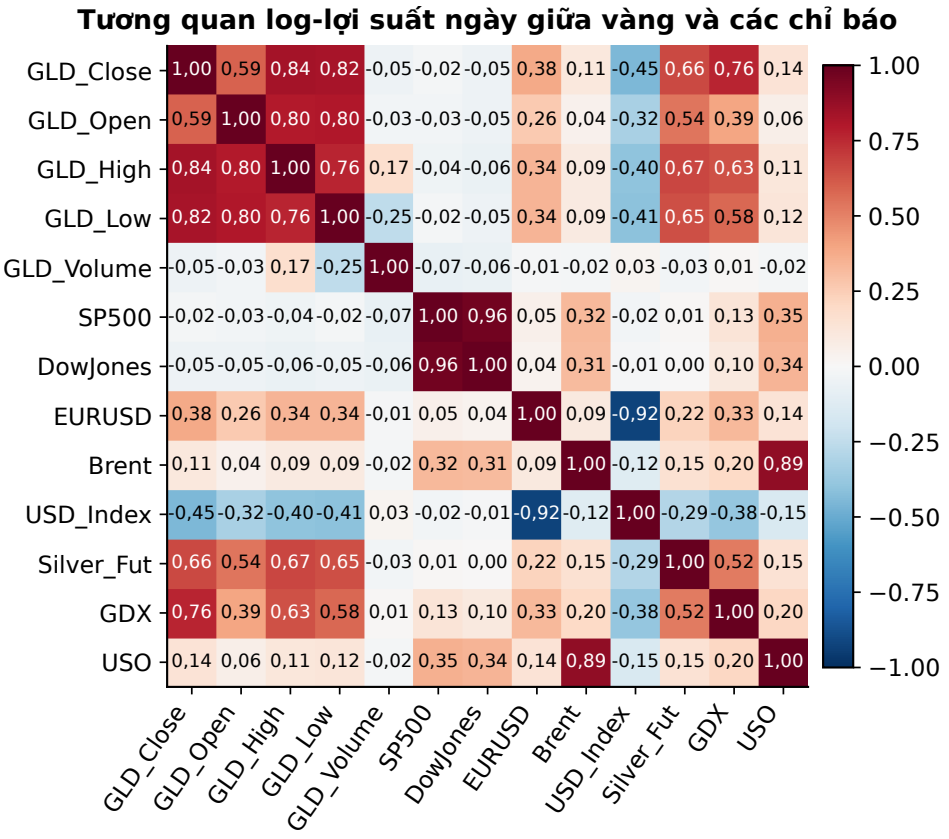
Bộ dữ liệu [144] có 1.718 phiên của quỹ GLD với 81 cột thị trường, trong đó 13 cột được dùng làm đặc trưng.



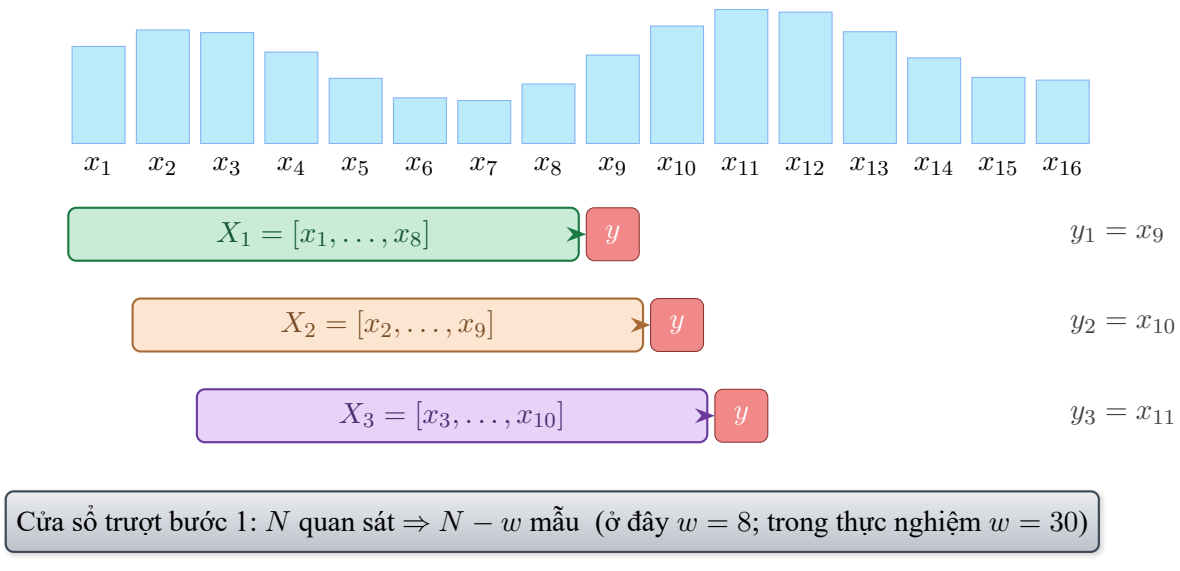
Hình 4.11. Giá quỹ vàng GLD trong giai đoạn khảo sát

4.2.4 Tiền xử lý và cửa sổ trượt

Chuỗi được cắt thành các cửa sổ 30 bước (Hình 4.13), chia 70/15/15 theo đúng thứ tự thời gian, không xáo trộn; thống kê chuẩn hoá chỉ lấy trên Train. Doanh thu dùng z-score trên mức. Giá dùng **chuẩn hoá tương đối**: đầu vào là log-tỷ số so với bước cuối cửa sổ, mục tiêu là log-lợi suất, nhờ đó mô hình không gặp mức giá chưa từng thấy.



Hình 4.12. Tương quan giữa quỹ GLD và các cột thị trường



Hình 4.13. Cửa sổ trượt: mỗi khung màu là một mẫu đầu vào, ô đỏ là giá trị cần dự báo

Đoạn mã 4.1. Chuẩn hoá tương đối theo cửa sổ: log-tỉ số so với bước cuối, mục tiêu là log-lợi suất

```python
# tl3/ch4_data.py
def _relative(self, vals, a_row):
    level = [self.cols.index(c) for c in self.exp.level_cols]
    price = [i for i in range(len(self.cols)) if i not in level]
    lv = np.log1p(vals[:, level])
    self.lv_mu, self.lv_sd = lv[:a_row].mean(0), lv[:a_row].std(0) + 1e-9
    lvz = (lv - self.lv_mu) / self.lv_sd
    logp = np.log(vals[:, price])
    X = []
    for i in range(self.window, len(vals)):
        w = logp[i - self.window:i] - logp[i - 1]            # log-tỉ số so với bước cuối cửa sổ
        X.append(np.concatenate([w, lvz[i - self.window:i]], axis=1))
    X = np.stack(X)
    tgt = np.log(vals[self.window:, self.t_idx]) - np.log(vals[self.window - 1:-1, self.t_idx])
    self.ret_sd = tgt[:a_row - self.window].std() + 1e-12     # chia để mục tiêu có thang ~1 khi huấn luyện
    self.price_cols = [self.cols[i] for i in price] + [self.cols[i] for i in level]
    return X.astype(np.float32) * 10.0, (tgt / self.ret_sd).astype(np.float32)
```

4.3 Hiện thực trên ba nền tảng

4.3.1 RNN từ đầu bằng NumPy

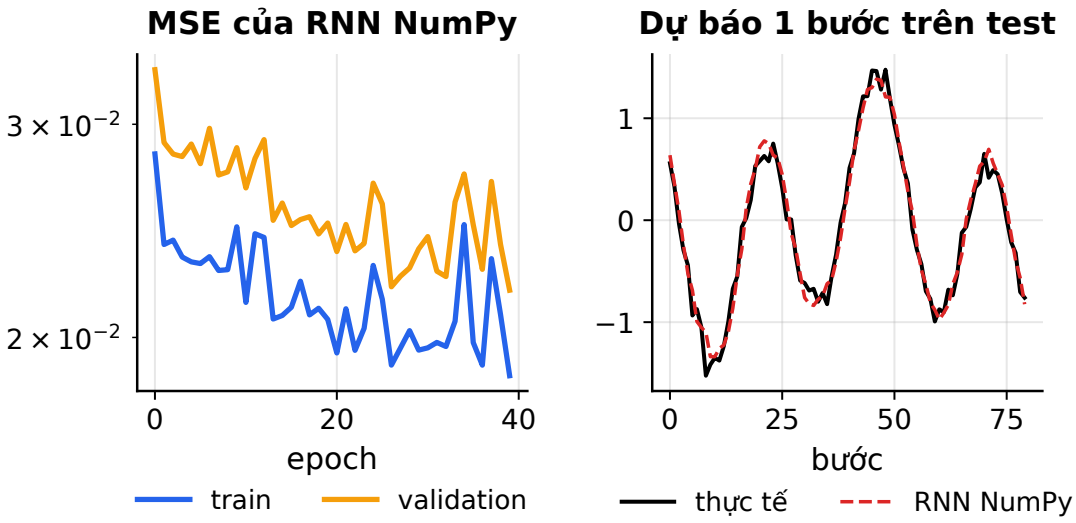
Bản từ đầu viết tay lan truyền xuôi và BPTT. Gradient khớp sai phân số tới $3{,}26 \times 10^{-9}$ và đầu ra khớp `torch.nn.RNN` khi dùng chung trọng số, chênh lệch lớn nhất $1{,}31 \times 10^{-8}$. Trên chuỗi sin có nhiễu, RMSE test đạt 0,1256, so với 0,2251 của dự báo ngây thơ.

Đoạn mã 4.2. Lan truyền xuôi: lặp qua các bước, lưu mọi h_t để dùng cho BPTT

```python
# tl3/ch4_scratch.py
def forward(self, x):
    """x: (T, n_in). Lưu mọi h_t để dùng lại khi lan truyền ngược."""
    p = self.p
    hs = [np.zeros(p["Whh"].shape[0])]
    for t in range(len(x)):
        hs.append(np.tanh(p["Wxh"] @ x[t] + p["Whh"] @ hs[-1] + p["bh"]))
    self.x, self.hs = x, hs
    return (p["Why"] @ hs[-1] + p["by"])[0]
```

Đoạn mã 4.3. BPTT viết tay: cộng dồn gradient theo thời gian và ghi chuẩn $\|\delta_t\|$

```python
# tl3/ch4_scratch.py
def backward(self, yhat, y):
    """BPTT: δ_T = W_hyᵀ (ŷ − y); δ_{t-1} = W_hhᵀ [(1 − h_t²) ⊙ δ_t]; cộng dồn
    ↪ gradient của trọng số dùng chung."""
    p, hs, x = self.p, self.hs, self.x
    g = {k: np.zeros_like(v) for k, v in p.items()}
    dy = yhat - y
    g["Why"] = dy * hs[-1][None, :]
    g["by"] = np.array([dy])
    dh = p["Why"][0] * dy
    self.dh_norms = []
    for t in range(len(x), 0, -1):
        self.dh_norms.append(np.linalg.norm(dh))
        da = (1.0 - hs[t] ** 2) * dh            # qua đạo hàm tanh
        g["Wxh"] += np.outer(da, x[t - 1])
        g["Whh"] += np.outer(da, hs[t - 1])
        g["bh"] += da
        dh = p["Whh"].T @ da                    # truyền về bước t − 1
    return g
```



Hình 4.14. RNN NumPy: MSE Train/Validation theo epoch (trái) và dự báo một bước trên test (phải)

4.3.2 Keras và PyTorch

Hai framework dùng cùng kiến trúc: một lớp hồi tiếp 64 đơn vị (SimpleRNN, LSTM, GRU hoặc BiLSTM), lấy trạng thái ở bước cuối, Dropout 0,2 và một lớp tuyến tính. Bản PyTorch [6] chạy GPU, bản Keras [145] chạy CPU. Huấn luyện dùng Adam [146], tối đa 150 epoch, dừng sớm theo mất mát Validation.

Đoạn mã 4.4. Mô hình hồi quy chuỗi trong PyTorch, chọn kiến trúc bằng tham số `arch`

```python
# tl3/ch4_torch.py
class SeqRegressor(nn.Module):
    """Một lớp hồi tiếp (hidden=64) → Dropout → Linear(1): dự báo many-to-one từ trạng thái ẩn cuối."""

    def __init__(self, n_features, arch="lstm", hidden=64, dropout=0.2):
        super().__init__()
        cell = {"rnn": nn.RNN, "lstm": nn.LSTM, "gru": nn.GRU, "bilstm": nn.LSTM}[arch]
        kw = {"nonlinearity": "tanh"} if arch == "rnn" else {}
        self.rnn = cell(n_features, hidden, batch_first=True, bidirectional=arch == "bilstm", **kw)
        self.drop = nn.Dropout(dropout)
        self.head = nn.Linear(hidden * (2 if arch == "bilstm" else 1), 1)

    def forward(self, x):                       # x: (batch, thời gian, đặc trưng)
        out, _ = self.rnn(x)                    # out: (batch, thời gian, hidden × số chiều)
        return self.head(self.drop(out[:, -1])).squeeze(-1)
```

Bảng 4.2. Số tham số và tốc độ huấn luyện của bốn kiến trúc

Kiến trúc	Tham số PyTorch	Tham số Keras	Giây/epoch PyTorch (GPU)	Giây/epoch Keras (CPU)
SimpleRNN	4.673	4.609	1,75	0,86
LSTM	18.497	18.241	2,15	2,63
GRU	13.889	13.889	1,38	2,35
BiLSTM	36.993	36.481	1,28	1,99

Thực nghiệm AAPL: 6 đặc trưng, ẩn 64, cửa sổ 30, batch 64. PyTorch có thêm một vector bias b_{hh} mỗi cổng nên nhiều tham số hơn Keras ở cùng cấu hình.

Đoạn mã 4.5. Một epoch huấn luyện PyTorch có cắt chuẩn gradient

```python
# tl3/ch4_torch.py
def train_epoch(model, loader, opt, loss_fn, clip=1.0):
    """Một epoch: lan truyền xuôi, MSE, lan truyền ngược qua thời gian (BPTT), cắt chuẩn gradient, Adam."""
    model.train()
    total, n = 0.0, 0
    for xb, yb in loader:
        opt.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), clip)
        opt.step()
        total += loss.item() * len(xb)
        n += len(xb)
    return total / n
```

Đoạn mã 4.6. Cùng kiến trúc trong Keras

```python
# tl3/ch4_keras.py
def build_keras_model(n_features, arch="lstm", window=30, hidden=64, dropout=0.2, lr=1e-3):
    """SimpleRNN / LSTM / GRU / Bidirectional(LSTM) 64 đơn vị → Dropout → Dense(1), cùng cấu hình bản PyTorch."""
    import keras
    from keras import layers
    rec = {"rnn": lambda: layers.SimpleRNN(hidden, activation="tanh"),
           "lstm": lambda: layers.LSTM(hidden),
           "gru": lambda: layers.GRU(hidden),
           "bilstm": lambda: layers.Bidirectional(layers.LSTM(hidden))}[arch]
    model = keras.Sequential([
        keras.Input(shape=(window, n_features)),
        rec(),
        layers.Dropout(dropout),
        layers.Dense(1),
    ], name=f"keras_{arch}")
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=lr, clipnorm=1.0), loss="mse")
    return model
```

4.4 Đánh giá và đối chuẩn

Dự báo được đổi về đơn vị gốc trước khi tính MAE, RMSE, MAPE, R^2 và độ chính xác hướng. Đối chứng bắt buộc là dự báo ngây thơ (naive), tức "ngày mai bằng hôm nay"; với giá tài sản, đây là mức rất khó vượt theo giả thuyết thị trường hiệu quả [147].

4.4.1 Kết quả trên ba chuỗi

Bảng 4.3. E-commerce: kết quả dự báo doanh thu ngày trên tập test

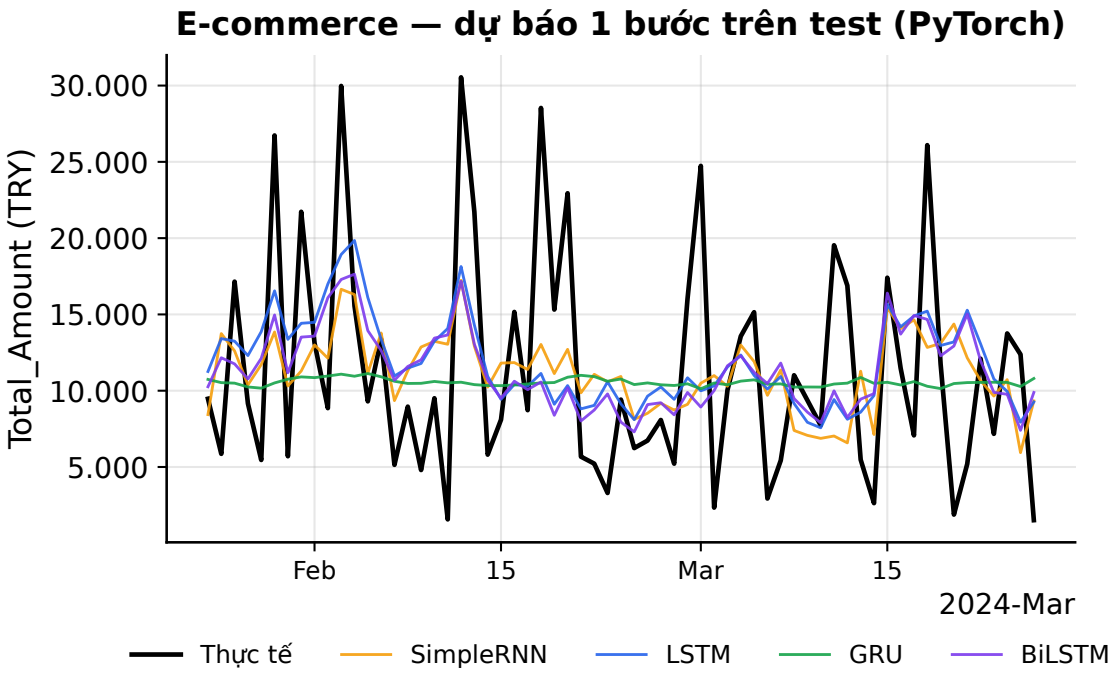
Mô hình	Framework	MAE	RMSE	MAPE (%)	R^2	Đúng hướng (%)	Epoch	Giây/epoch
Naive (hôm qua)	–	8.438	10.596	117,56	−1,0649	–	–	–
TB trượt 7 ngày	–	6.234	7.940	94,15	−0,1596	–	–	–
TB trượt 30 ngày	–	6.090	7.420	97,48	−0,0126	–	–	–
SimpleRNN	PyTorch	5.484	6.861	87,77	0,1343	77,8	40	0,13
LSTM	PyTorch	5.617	**6.836**	91,56	**0,1406**	76,2	44	0,06
GRU	PyTorch	5.773	7.415	85,07	−0,0113	79,4	16	0,12
BiLSTM	PyTorch	**5.478**	6.893	88,16	0,1260	77,8	39	0,12
SimpleRNN	Keras	5.804	7.464	**73,99**	−0,0245	79,4	20	0,38
LSTM	Keras	5.657	7.290	90,43	0,0227	76,2	25	0,49
GRU	Keras	5.613	7.013	86,69	0,0955	79,4	29	0,43
BiLSTM	Keras	5.548	7.133	85,00	0,0641	76,2	25	0,36

Chỉ số tính trên tập test, đơn vị gốc của biến mục tiêu; in đậm là giá trị tốt nhất của cột. Đúng hướng: tỉ lệ ngày mô hình đoán đúng chiều tăng/giảm so với hôm trước.

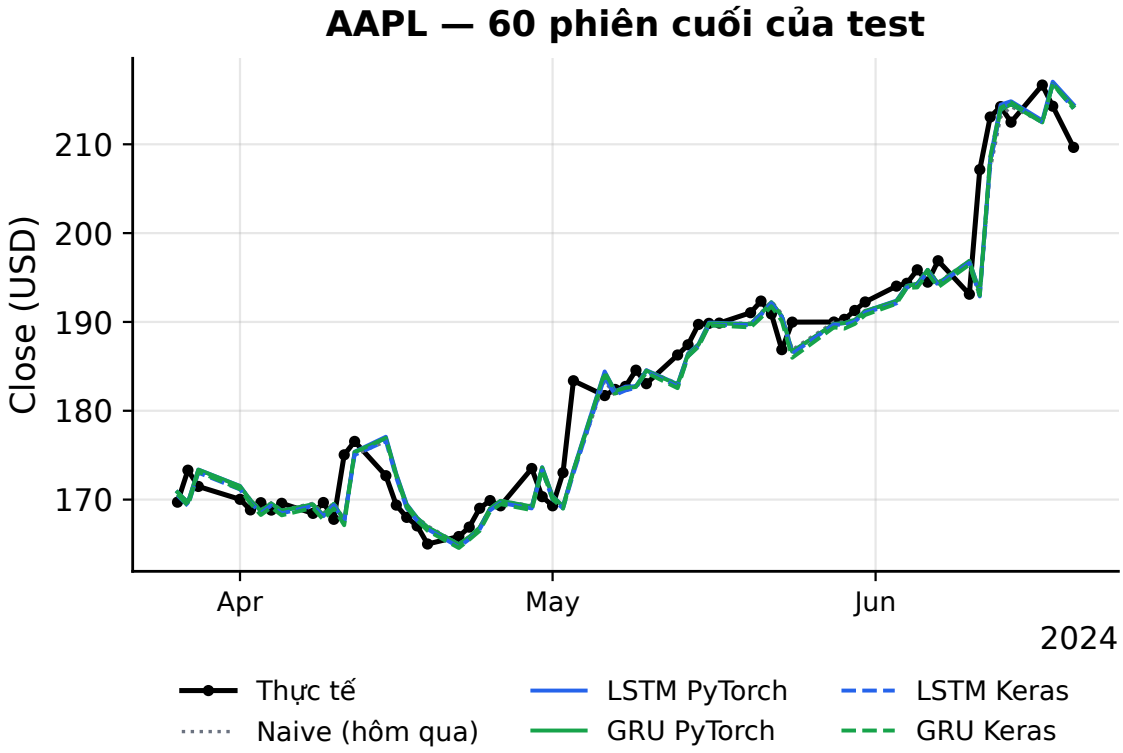
Bảng 4.4. AAPL: kết quả dự báo giá đóng cửa trên tập test

Mô hình	Framework	MAE	RMSE	MAPE (%)	R^2	Đúng hướng (%)	Epoch	Giây/epoch
Naive (hôm qua)	–	1,544	2,259	1,38	0,9982	–	–	–
SimpleRNN	PyTorch	1,542	2,257	1,38	0,9982	52,7	19	1,75
LSTM	PyTorch	1,542	2,259	1,38	0,9982	52,0	19	2,15
GRU	PyTorch	**1,541**	**2,257**	**1,38**	**0,9982**	51,3	19	1,38
BiLSTM	PyTorch	1,542	2,259	1,38	0,9982	52,4	19	1,28
SimpleRNN	Keras	1,548	2,262	1,39	0,9982	50,3	23	0,86
LSTM	Keras	1,564	2,277	1,40	0,9981	46,5	18	2,63
GRU	Keras	1,554	2,272	1,39	0,9982	48,5	20	2,35
BiLSTM	Keras	1,573	2,285	1,41	0,9981	47,0	18	1,99

Chỉ số tính trên tập test, đơn vị gốc của biến mục tiêu; in đậm là giá trị tốt nhất của cột. Đúng hướng: tỉ lệ ngày mô hình đoán đúng chiều tăng/giảm so với hôm trước.



Hình 4.15. Dự báo doanh thu trên tập test của các mô hình PyTorch

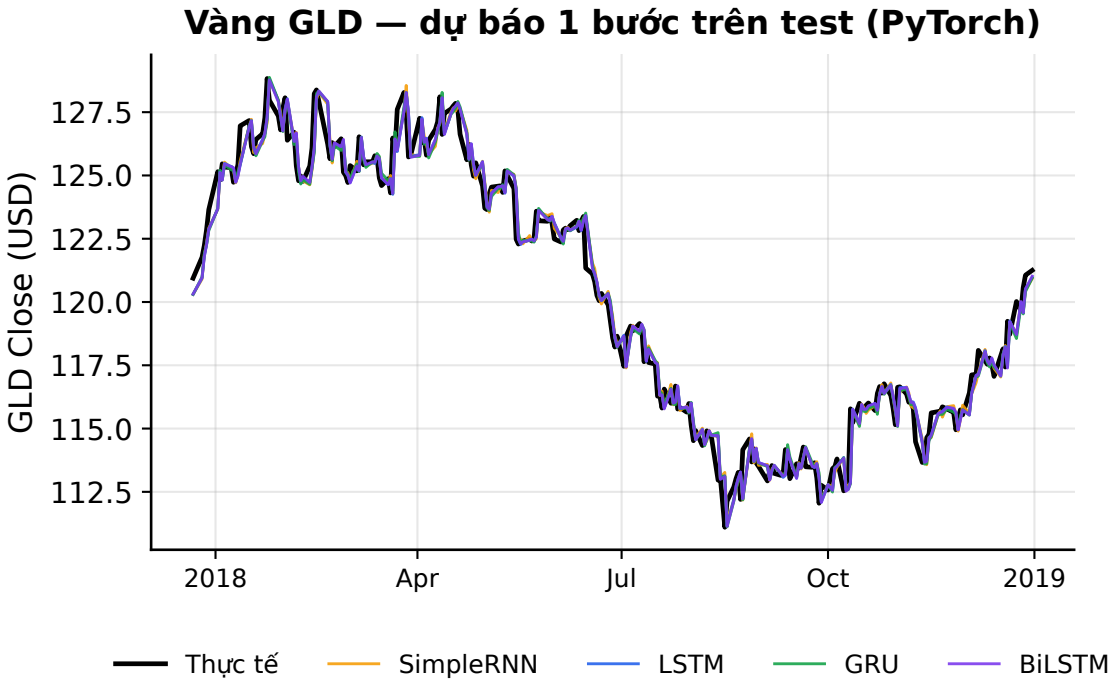


Hình 4.16. AAPL: phóng to dự báo trên một đoạn của tập test

Bảng 4.5. Vàng GLD: kết quả dự báo giá đóng cửa trên tập test

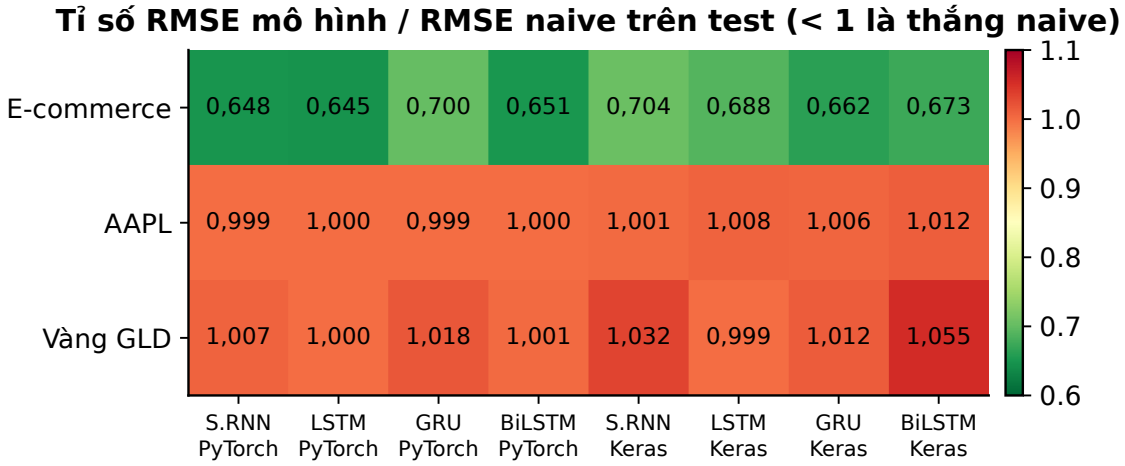
Mô hình	Framework	MAE	RMSE	MAPE (%)	R^2	Đúng hướng (%)	Epoch	Giây/epoch
Naive (hôm qua)	–	0,553	0,738	0,46	0,9794	–	–	–
SimpleRNN	PyTorch	0,563	0,743	0,47	0,9791	49,8	21	0,34
LSTM	PyTorch	**0,552**	0,738	**0,46**	0,9794	53,0	17	0,35
GRU	PyTorch	0,562	0,751	0,47	0,9787	47,8	16	0,24
BiLSTM	PyTorch	0,554	0,739	0,46	0,9793	50,2	17	0,44
SimpleRNN	Keras	0,589	0,762	0,49	0,9781	47,0	19	0,40
LSTM	Keras	0,553	**0,737**	0,46	**0,9794**	50,2	18	1,04
GRU	Keras	0,562	0,747	0,47	0,9789	47,4	26	0,92
BiLSTM	Keras	0,587	0,778	0,49	0,9771	47,8	22	0,65

Chỉ số tính trên tập test, đơn vị gốc của biến mục tiêu; in đậm là giá trị tốt nhất của cột. Đúng hướng: tỉ lệ ngày mô hình đoán đúng chiều tăng/giảm so với hôm trước.



Hình 4.17. Dự báo giá quỹ vàng GLD trên tập test của các mô hình PyTorch

4.4.2 Mô hình nào thắng dự báo ngây thơ?



Hình 4.18. Tỷ số RMSE mô hình / RMSE naive trên test cho 8 mô hình × 3 thực nghiệm

Nhận xét: Trên e-commerce, mọi RNN giảm sai số khoảng một phần ba so với naive (mô hình được chọn theo Validation đạt tỷ số 0,6475). Trên AAPL và vàng, mọi mô hình nằm sát 1 dù R^2 rất cao (R^2 0,9982 trên AAPL). Lý do nằm ở dữ liệu, không ở kiến trúc: doanh thu có mức trung bình ổn định và nhiễu độc lập nên mô hình nào học được mức trung bình cũng thắng "hôm qua"; giá tài sản gần bước ngẫu nhiên nên "hôm qua" đã là dự báo gần tối ưu. Vì vậy, R^2 cao trên chuỗi giá không chứng minh mô hình có năng lực dự báo.

4.4.3 PyTorch so với Keras

Bảng 4.6. RMSE trên test của cùng kiến trúc trong hai framework

Thực nghiệm	Kiến trúc	RMSE PyTorch	RMSE Keras	Keras so với PyTorch (%)	Tốt hơn
E-commerce	SimpleRNN	6.861	7.464	+8,8	PyTorch
	LSTM	6.836	7.290	+6,6	PyTorch
	GRU	7.415	7.013	−5,4	Keras
	BiLSTM	6.893	7.133	+3,5	PyTorch
AAPL	SimpleRNN	2,257	2,262	+0,2	PyTorch
	LSTM	2,259	2,277	+0,8	PyTorch
	GRU	2,257	2,272	+0,7	PyTorch
	BiLSTM	2,259	2,285	+1,2	PyTorch
Vàng GLD	SimpleRNN	0,743	0,762	+2,5	PyTorch
	LSTM	0,738	0,737	−0,1	Keras
	GRU	0,751	0,747	−0,5	Keras
	BiLSTM	0,739	0,778	+5,3	PyTorch

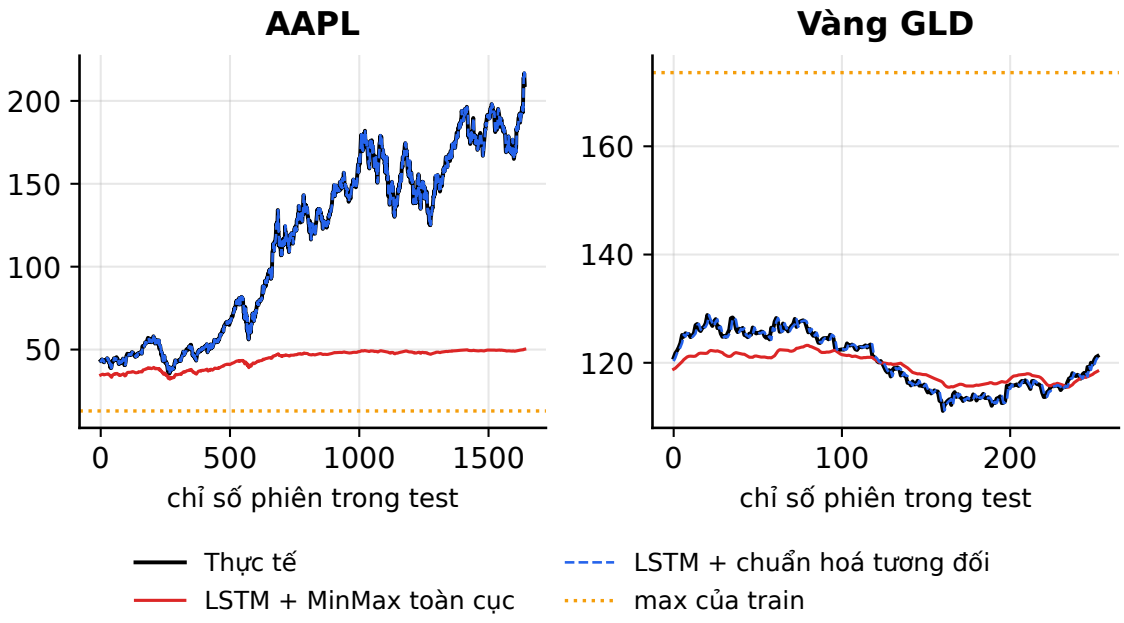
RMSE trung bình của bốn kiến trúc là 7.001 (PyTorch) và 7.225 (Keras) trên e-commerce, 2,258 và 2,274 trên AAPL. Chênh lệch nhỏ đến từ khởi tạo mặc định khác nhau và tính ngẫu nhiên của huấn luyện. Ở quy mô này, lựa chọn framework không ảnh hưởng tới chất lượng.

4.4.4 Ảnh hưởng của cách chuẩn hoá

LSTM và GRU được huấn luyện lại với Min–Max toàn cục, cách làm phổ biến trong nhiều bài tham khảo, để so với chuẩn hoá tương đối.

Bảng 4.7. Ảnh hưởng của cách chuẩn hoá tới dự báo trên test

Dữ liệu	Chuẩn hoá + mô hình	MAE	RMSE	R^2
AAPL	MinMax toàn cục + LSTM	70,809	85,365	−1,6044
	MinMax toàn cục + GRU	68,090	83,671	−1,5020
	Tương đối + LSTM	1,542	2,259	0,9982
	Tương đối + GRU	1,541	2,257	0,9982
Vàng GLD	MinMax toàn cục + LSTM	2,685	3,112	0,6338
	MinMax toàn cục + GRU	3,046	3,722	0,4762
	Tương đối + LSTM	0,552	0,738	0,9794
	Tương đối + GRU	0,562	0,751	0,9787



Hình 4.19. Dự báo của LSTM với Min–Max toàn cục (đỏ) và chuẩn hoá tương đối (xanh, đứt) trên test AAPL (trái) và vàng (phải)

Phát hiện: R^2 âm là lỗi chuẩn hoá, không phải lỗi mô hình

Với Min–Max toàn cục, LSTM trên AAPL có $R^2 = -1{,}604$; cùng mô hình với chuẩn hoá tương đối đạt $R^2 = 0{,}9982$. Giá cao nhất của Train là 12,97 USD trong khi Test lên tới 216,67: đầu vào Test nằm ngoài $[0, 1]$, tanh bão hoà và dự báo "kẹt" gần mức trần.

4.4.5 Độ dài cửa sổ

Bảng 4.8. Tỷ số RMSE/naive theo độ dài cửa sổ

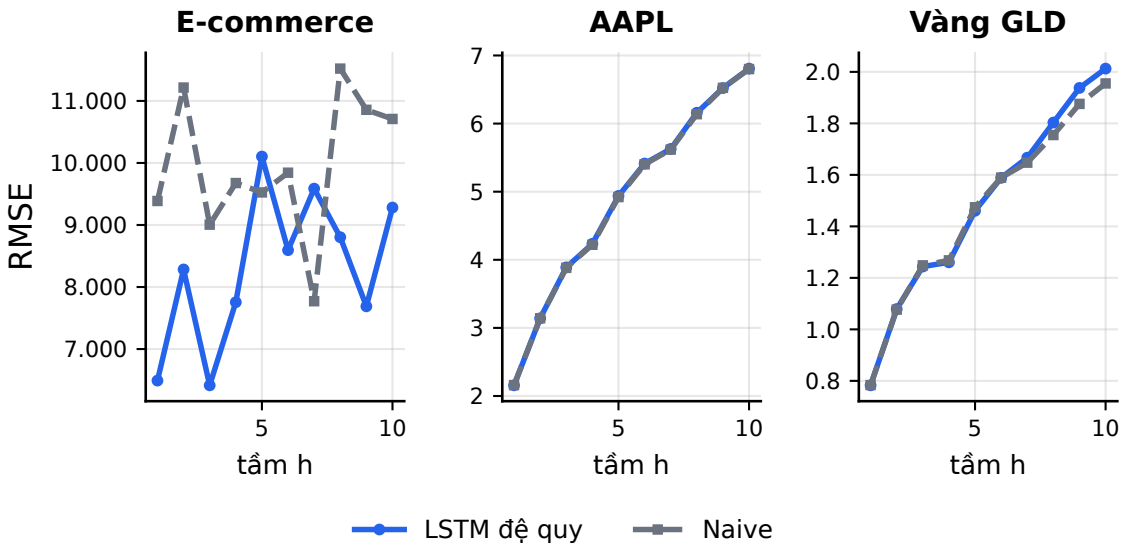
Thực nghiệm	Mô hình	w = 10	w = 20	w = 30	w = 60	w = 90
E-commerce	LSTM	0,677	0,658	0,645	**0,633**	0,705
	GRU	0,699	**0,693**	0,700	0,705	0,714
AAPL	LSTM	1,001	1,001	1,000	**0,999**	1,002
	GRU	1,000	1,001	**0,999**	1,002	1,004
Vàng GLD	LSTM	1,001	1,000	1,000	**0,998**	1,002
	GRU	1,022	1,014	1,018	**1,014**	1,041

Giá trị là RMSE test / RMSE naive (càng nhỏ càng tốt; < 1 là thắng dự báo ngây thơ).

Trên AAPL và vàng, mọi độ dài cửa sổ cho tỷ số sát 1: thêm lịch sử không giúp vì lịch sử không chứa thông tin về hướng. Trên e-commerce, cửa sổ dài giúp ước lượng mức trung bình ổn định hơn; tốt nhất là LSTM với 60 ngày (0,6334), đổi lại số mẫu Train của một chuỗi vốn ngắn giảm đi.

4.4.6 Dự báo nhiều bước

Web App cần dự báo nhiều ngày tới nên dùng chiến lược đệ quy: dự báo một bước, nối vào cửa sổ, lặp lại. Sai số tăng theo tầm dự báo. Với AAPL, RMSE tăng từ 2,157 ở $h = 1$ lên 6,807 ở $h = 10$, sát với naive (6,800), đúng quy luật $\sqrt{h}$ của bước ngẫu nhiên. Do đó, ứng dụng hiển thị dự báo nhiều bước như một xu hướng tham khảo.



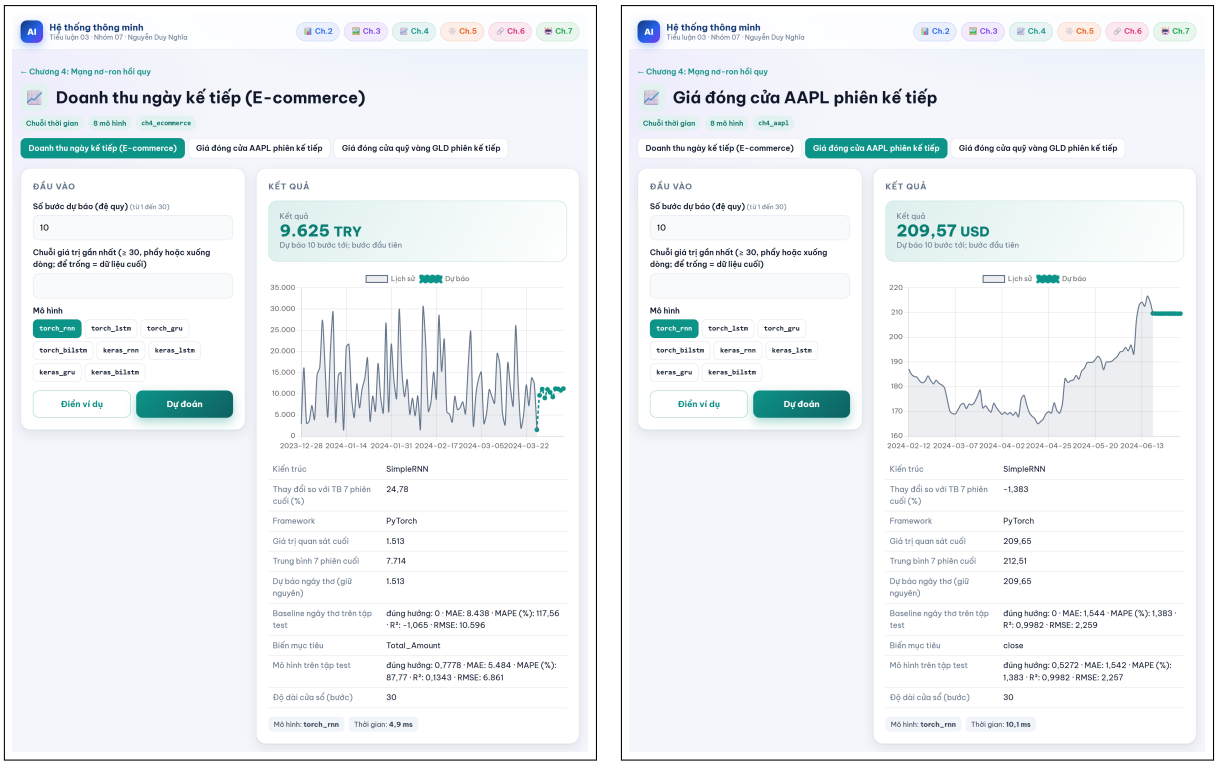
Hình 4.20. RMSE theo tầm dự báo $h = 1..10$ trên ba thực nghiệm

4.5 Triển khai Web App

Ba mô hình được phục vụ qua `serve_ch4.py`. Người dùng chọn thực nghiệm và mô hình; Web App trả dự báo một bước, dự báo nhiều bước và chỉ số test kèm dự báo ngây thơ để đọc kết quả đúng mức (Hình 4.21).

Đoạn mã 4.7. Dự báo đệ quy nhiều bước

```python
# tl3/ch4_forecast.py
def recursive_forecast(prep, predict_fn, rows, steps=10):
    """Dự báo `steps` bước: mỗi bước dự báo một giá trị, nối hàng mới vào cuối cửa sổ, bỏ hàng đầu, lặp lại."""
    rows = np.asarray(rows, dtype=np.float64).copy()
    out = []
    for _ in range(steps):
        v = output_to_value(prep, predict_fn(window_to_input(prep, rows)), rows)
        out.append(v)
        rows = np.vstack([rows[1:], next_row(prep, rows, v)])
    return np.array(out)
```



(a) Dự báo doanh thu **(b)** Dự báo giá AAPL

Hình 4.21. Giao diện Web App dự báo chuỗi thời gian

4.6 Tổng kết chương

Về lý thuyết, tích các Jacobian trong BPTT là nguồn gốc của tiêu biến và bùng nổ gradient, còn LSTM và GRU khắc phục bằng cơ chế cổng. Trên dữ liệu thực, chất lượng dự báo do dữ liệu quyết định nhiều hơn kiến trúc: doanh thu có tự tương quan thấp nên mọi RNN thắng naive, giá AAPL và vàng gần bước ngẫu nhiên nên mọi RNN chỉ ngang naive; SimpleRNN, LSTM, GRU, BiLSTM chênh nhau rất ít, PyTorch và Keras cũng vậy. Thiết kế tiền xử lý quan trọng hơn chọn mô hình: chuẩn hoá tương đối đưa R^2 của LSTM trên AAPL từ $-1{,}604$ lên 0,9982. Bài học thực hành là luôn so với dự báo ngây thơ trước khi tin vào R^2.

Chương 5

Knowledge Graph, RAG và tác tử thông minh

5.1 Giới hạn của mô hình ngôn ngữ thuần tuý

Các chương trước xét những mô hình cất tri thức trong trọng số. LLM thuần có ba giới hạn: tri thức đóng băng tại thời điểm huấn luyện, có thể sinh thông tin sai với giọng tự tin (ảo giác), và không dẫn được nguồn. Hai hướng bổ sung cho nhau: **đồ thị tri thức** lưu dữ kiện dưới dạng tường minh [148], còn **RAG** truy xuất bằng chứng rồi đưa vào lời nhắc [1]. Bảng 5.1 so sánh các phương án.

Bảng 5.1. So sánh đặc tính vận hành của LLM đơn lẻ, fine-tuning và hệ RAG

Tiêu chí	LLM đơn lẻ	Fine-tuning	RAG / tác tử
Cập nhật tri thức	Phải chờ mô hình mới	Phải huấn luyện lại	Cập nhật kho tài liệu
Dẫn nguồn	Không	Không	Có
Dữ kiện cụ thể	Dễ ảo giác	Khá hơn với miền hẹp	Tốt khi truy xuất đúng
Học văn phong, định dạng	Hạn chế	Tốt	Hạn chế
Chi phí chính	Suy luận	Dữ liệu, huấn luyện	Chỉ mục, truy xuất

5.2 Knowledge Graph và biểu diễn tri thức

5.2.1 Mô hình bộ ba

Cơ sở tri thức (Knowledge Base – KB) lưu tri thức có tổ chức; đồ thị tri thức biểu diễn tri thức thành mạng thực thể và quan hệ [149]. Đơn vị cơ bản là bộ ba *(chủ thể, quan hệ, đối tượng)*:

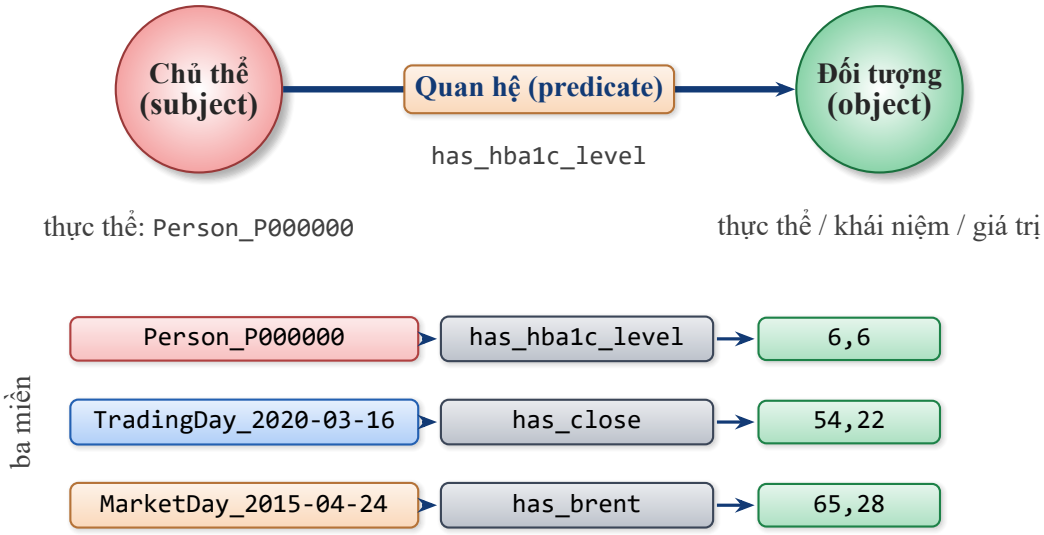
$$\mathcal{G} = \{(h, r, t)\} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}. \tag{5.1}$$

Hai mô hình dữ liệu phổ biến là RDF (bộ ba thuần, truy vấn bằng SPARQL [150]) và đồ thị có nhãn thuộc tính LPG (thuộc tính gắn lên nút và cạnh, truy vấn bằng Cypher [151]), xem Hình 5.2. Lớp ontology định nghĩa kiểu và ràng buộc, cho phép suy luận. Nhúng đồ thị như TransE học biểu diễn sao cho $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$ [152].

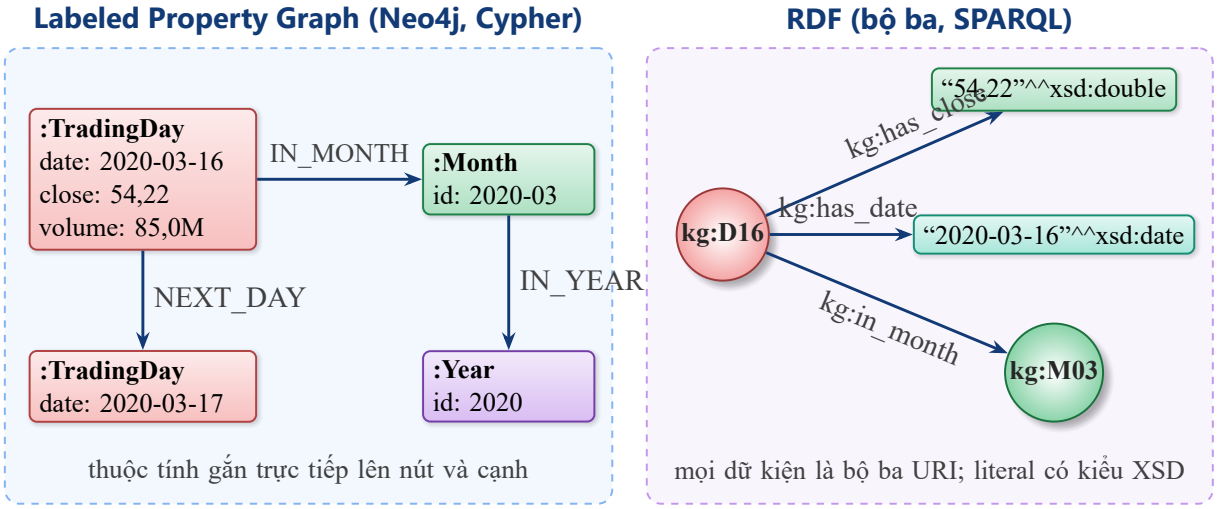
5.2.2 Truy xuất thưa, dày và theo đồ thị

BM25 [153] là mô hình thưa chấm điểm theo tần suất từ, có chuẩn hoá độ dài tài liệu:

$$\mathrm{BM25}(q, d) = \sum_{t \in q} \mathrm{IDF}(t)\, \frac{f(t, d)\,(k_1 + 1)}{f(t, d) + k_1\big(1 - b + b\,|d|/\mathrm{avgdl}\big)}. \tag{5.2}$$



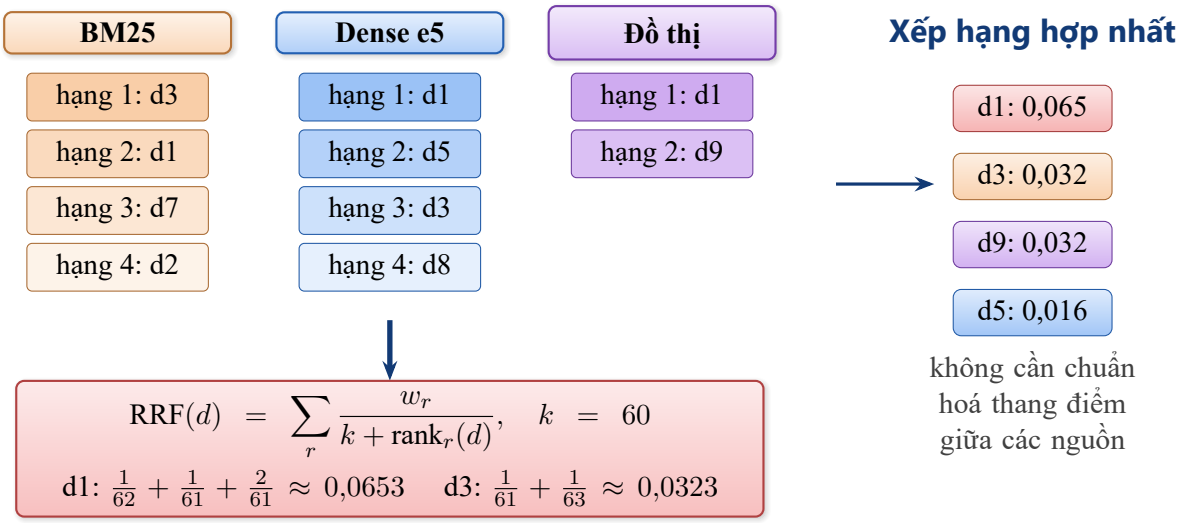
Hình 5.1. Giải phẫu bộ ba (chủ thể, quan hệ, đối tượng) và ví dụ của ba miền dữ liệu trong chương



Hình 5.2. Cùng một dữ kiện giao dịch biểu diễn theo LPG và theo RDF

Truy xuất dày mã hoá câu hỏi và tài liệu thành vector rồi đo độ tương đồng cosin [154], nên hiểu được đồng nghĩa nhưng dễ trượt khi cần khớp chính xác con số. **Truy xuất đồ thị** nhận diện thực thể trong câu hỏi rồi đi theo cạnh, rất chính xác nếu nhận diện đúng. Vì mỗi nguồn có điểm mạnh riêng, ta hợp nhất thứ hạng bằng RRF [155] với trọng số w_r:

$$\mathrm{RRF}(d) = \sum_{r} \frac{w_r}{k + \mathrm{rank}_r(d)}, \qquad k = 60. \tag{5.3}$$

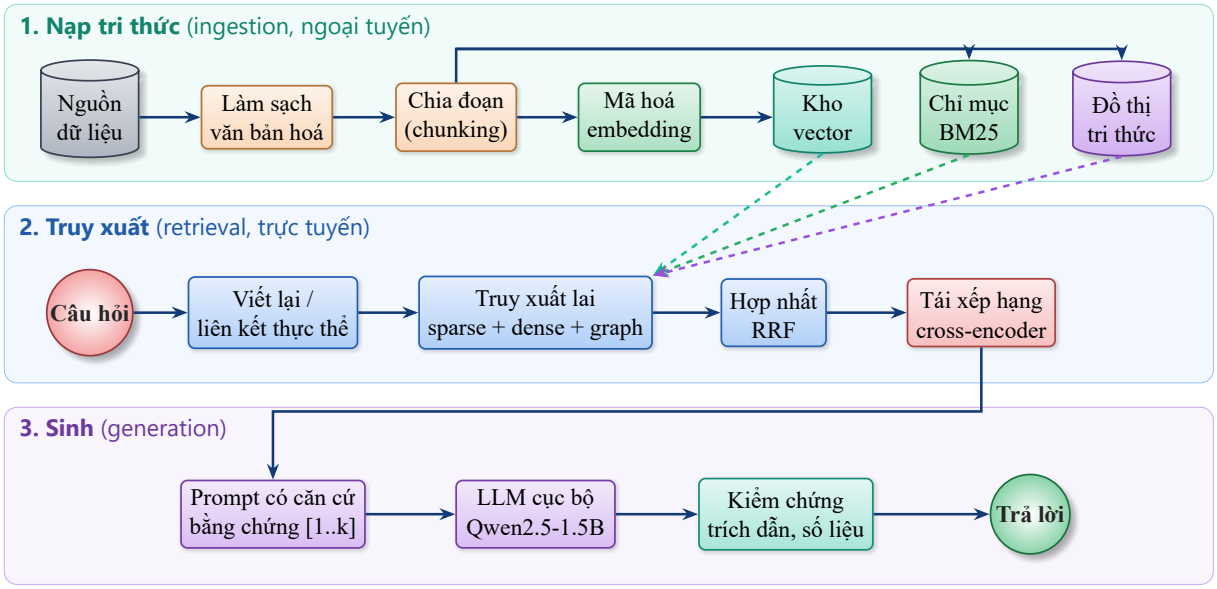


Hình 5.3. Hợp nhất thứ hạng RRF từ ba nguồn BM25, dense và đồ thị (trọng số đồ thị $w_G = 2$)

5.3 RAG nâng cao

5.3.1 Kiến trúc và các thế hệ

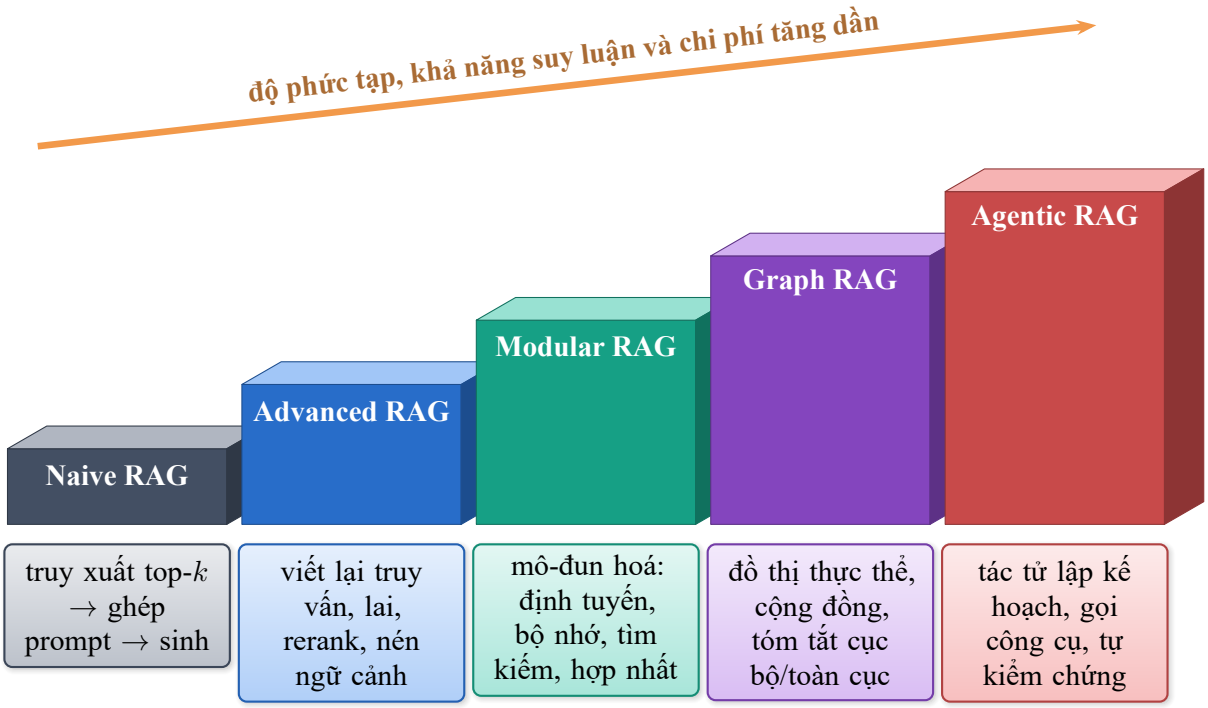
RAG có ba làn: nạp tri thức, truy xuất và sinh có căn cứ (Hình 5.4). Từ RAG cơ bản, các thế hệ sau thêm truy xuất lai, tái xếp hạng bằng cross-encoder [156], GraphRAG [157] và RAG tác tử với vòng tự phản tư [158].



Hình 5.4. Kiến trúc RAG ba làn: nạp tri thức, truy xuất lai và sinh có căn cứ

5.3.2 Chiến lược chia đoạn

Tài liệu dài phải chia đoạn trước khi lập chỉ mục. Đoạn quá dài làm loãng ngữ nghĩa, đoạn quá ngắn làm mất ngữ cảnh (Bảng 5.2). Dữ liệu trong chương là dữ liệu bảng nên chọn *mỗi bản ghi*



Hình 5.5. Năm thế hệ RAG: độ phức tạp, khả năng suy luận và chi phí tăng dần

là một tài liệu, cộng các tài liệu tổng hợp theo nhóm bệnh nhân, tháng và năm.

Bảng 5.2. So sánh các chiến lược chia đoạn văn bản trong RAG

Chiến lược	Ưu điểm	Nhược điểm
Cỡ cố định	Đơn giản, nhanh	Có thể cắt ngang câu
Chồng lấn	Giữ ngữ cảnh ở biên	Tăng số đoạn, trùng thông tin
Theo cấu trúc	Bám tiêu đề, đoạn văn	Phụ thuộc định dạng
Theo ngữ nghĩa	Mỗi đoạn một chủ đề	Tốn tính toán embedding
Phân cấp	Tìm đoạn nhỏ, trả ngữ cảnh lớn	Phức tạp khi lập chỉ mục

5.3.3 Sinh có căn cứ và đánh giá

Bằng chứng được đánh số trong lời nhắc; mô hình phải trả lời dựa trên chúng và trích dẫn nguồn. Với rank_q là hạng của tài liệu đúng, truy xuất được đo bằng

$$\text{Recall@}k = \frac{1}{|Q|}\sum_{q\in Q} \mathbb{1}[\text{rank}_q \le k], \qquad \text{MRR} = \frac{1}{|Q|}\sum_{q\in Q}\frac{1}{\text{rank}_q}. \tag{5.4}$$

Chất lượng câu trả lời được đo bằng tỷ lệ đáp án đúng, ngoài ra có thể dùng các độ đo như RAGAS [159].

Đoạn mã 5.1. Dựng prompt có bằng chứng đánh số

```python
# tl3/llm.py
def grounded_prompt(question, evidence):
    """evidence: list[(nhãn nguồn, văn bản)] -> prompt đánh số [1]..[n]."""
    lines = [f"[{i}] ({src}) {txt}" for i, (src, txt) in enumerate(evidence, 1)]
    return "BẰNG CHỨNG:\n" + "\n".join(lines) + f"\n\nCÂU HỎI: {question}\n\nTRẢ LỜI (tiếng Việt, có trích dẫn [n]):"
```

5.4 Thiết kế hệ thống thực nghiệm

Hệ thống có năm tầng (Hình 5.6) và chạy hoàn toàn cục bộ: LLM Qwen2.5-1.5B [160], embedding e5 [161], reranker bge-reranker-v2-m3 [162], NetworkX [9]. Ba miền dữ liệu được dùng: tiểu đường [163], cổ phiếu Alphabet [164] và vàng FINAL_USO [144] (Bảng 5.3).



Hình 5.6. Kiến trúc năm tầng của hệ thống KG – RAG trong chương

Hiểu đúng dữ liệu trước khi dựng tri thức

Cột `Close` của FINAL_USO là giá quỹ SPDR Gold Shares (GLD), không phải giá vàng giao ngay; nhóm cột USO là quỹ dầu. Nếu dựng KG theo tên tệp, mọi câu trả lời về vàng sẽ sai về đơn vị ngay từ đầu.

Bảng 5.3. Ba bộ dữ liệu của chương và quy mô kho tài liệu

Bộ dữ liệu	Nguồn	Giai đoạn	Số dòng	Số cột	Tài liệu bản ghi	Tài liệu tổng hợp	Tổng tài liệu
Tiểu đường	Kaggle ghnshymsaini	không có thời gian	96.146	9	96.146	42	96.188
Cổ phiếu Google	Kaggle jillanisofttech (V2)	02/01/2015 – 10/01/2024	2.271	6	2.271	119	2.390
Vàng (FINAL_USO)	Kaggle sid321axn	15/12/2011 – 31/12/2018	1.718	81	1.718	93	1.811

Tiểu đường: sau khi bỏ dòng trùng lặp. Tài liệu tổng hợp: theo nhóm (tiểu đường) hoặc theo tháng, năm (thị trường).

5.4.1 Mô hình hoá KB và KG

Mỗi bản ghi được văn bản hoá thành một câu tiếng Việt, kèm các khái niệm suy ra từ tri thức miền: phân loại HbA1c theo ADA [165], BMI theo WHO [166], kiểu biến động theo lợi suất ngày. Giá trị số là thuộc tính của nút, các khái niệm là nút dùng chung, nhờ đó câu hỏi tổng hợp được trả lời bằng phép giao các tập láng giềng. Với dữ liệu thị trường, nút ngày nối với nhau bằng `next_day` và với nút tháng, năm.

Đoạn mã 5.2. Văn bản hoá một hồ sơ tiểu đường

```
# tl3/ch5_data.py
def diabetes_doc(r):
    """Một hồ sơ -> câu mô tả giữ đủ tên thuộc tính, giá trị và khái niệm suy ra (nhóm tuổi, phân loại BMI…)."""
    yes = lambda v: "có" if v else "không"
    return (f"Hồ sơ {r.pid}: bệnh nhân {GENDER_VN[r.gender]}, {vnum(r.age, 0 if r.age >= 2 else 2)} tuổi "
            f"(nhóm tuổi {r.age_group}), {yes(r.hypertension)} tăng huyết áp, {yes(r.heart_disease)} bệnh tim, "
            f"hút thuốc: {SMOKE_VN[r.smoking_history]}. Chỉ số BMI {vnum(r.bmi)} ({r.bmi_class}), "
            f"HbA1c {vnum(r.HbA1c_level, 1)}% ({r.hba1c_class}), đường huyết {r.blood_glucose_level} mg/dL "
            f"(nhóm {r.glucose_class}). Nhãn: {'mắc tiểu đường' if r.diabetes else 'không mắc tiểu đường'}.")
```

5.4.2 Bộ truy xuất và bộ câu hỏi đánh giá

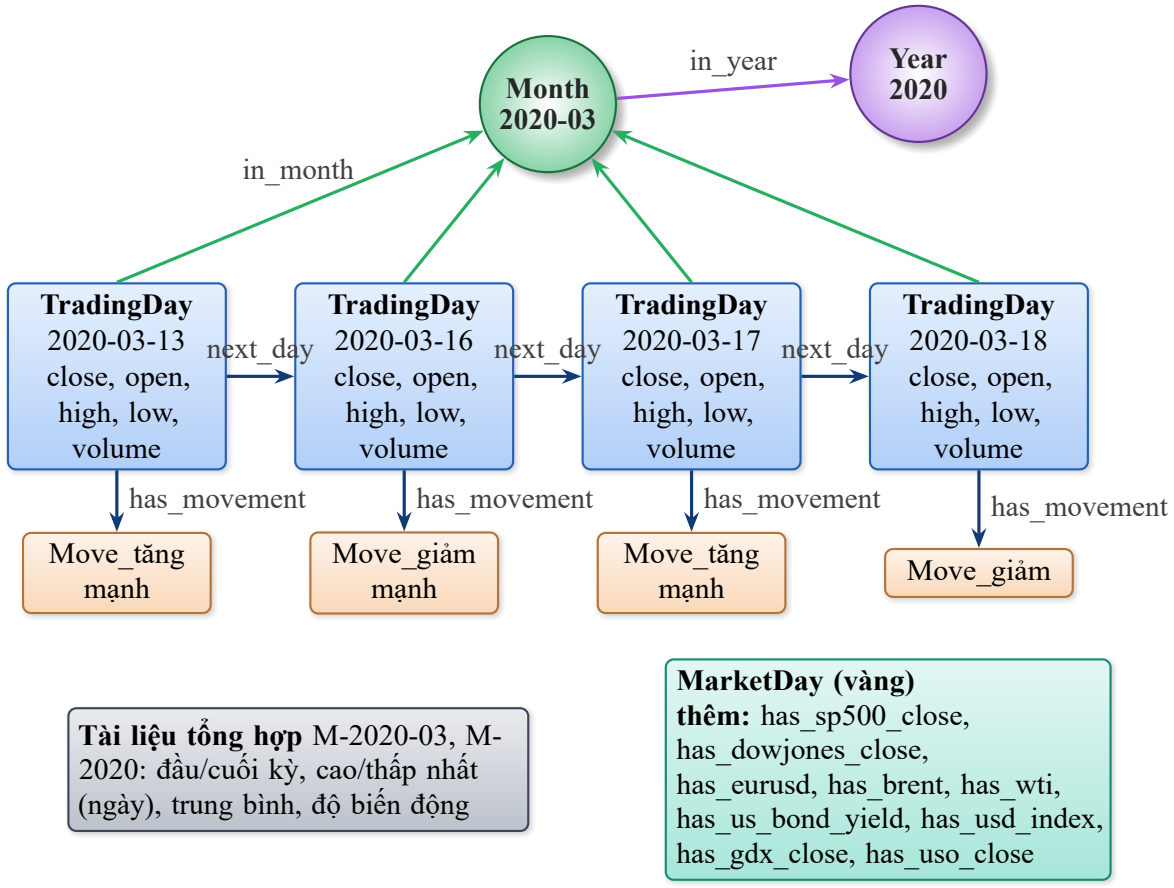
Mười cấu hình truy xuất có chung giao diện `search(q, k)`. Câu hỏi được sinh tự động từ chính dữ liệu nên biết trước đáp án và tài liệu đúng (Bảng 5.5), trong đó có câu diễn đạt lại để không khớp mặt chữ.

Đoạn mã 5.3. Sinh bộ ba dữ kiện, khái niệm và tầng ontology cho KG tiểu đường

```python
# tl3/ch5_kg.py
def diabetes_triples(df):
    """Mỗi hồ sơ sinh 9 bộ ba dữ kiện (has_*) + 5 bộ ba khái niệm (in_age_group, bmi_class…)."""
    out = []
    for r in df.itertuples():
        s = f"Person_{r.pid}"
        out += [(s, "has_gender", f"Gender_{r.gender}"), (s, "has_age", float(r.age)),
                (s, "has_hypertension", int(r.hypertension)), (s, "has_heart_disease", int(r.heart_disease)),
                (s, "has_smoking_history", f"Smoking_{r.smoking_history}"), (s, "has_bmi", float(r.bmi)),
                (s, "has_hba1c_level", float(r.HbA1c_level)), (s, "has_blood_glucose_level", int(r.blood_glucose_level)),
                (s, "has_diabetes", int(r.diabetes)),
                (s, "in_age_group", f"AgeGroup_{r.age_group}"), (s, "bmi_class", f"BMI_{r.bmi_class}"),
                (s, "hba1c_class", f"HbA1c_{r.hba1c_class}"), (s, "glucose_class", f"Glucose_{r.glucose_class}"),
                (s, "diabetes_status", "Diabetes_yes" if r.diabetes else "Diabetes_no")]
        if r.hypertension:
            out.append((s, "has_condition", "Condition_Hypertension"))
        if r.heart_disease:
            out.append((s, "has_condition", "Condition_HeartDisease"))
    # tầng ontology: khái niệm -> yếu tố nguy cơ (tri thức nền theo ADA/WHO, không suy ra từ dữ liệu)
    out += [("HbA1c_ngưỡng chẩn đoán", "indicates", "Diabetes_yes"),
            ("HbA1c_tiền tiểu đường", "risk_factor_of", "Diabetes_yes"),
            ("Glucose_từ 200", "indicates", "Diabetes_yes"), ("BMI_béo phì", "risk_factor_of", "Diabetes_yes"),
            ("Condition_Hypertension", "risk_factor_of", "Diabetes_yes"),
            ("Condition_HeartDisease", "comorbid_with", "Diabetes_yes"),
            ("AgeGroup_60-79", "risk_factor_of", "Diabetes_yes"), ("AgeGroup_từ 80", "risk_factor_of", "Diabetes_yes")]
    return out
```

Đoạn mã 5.4. Dựng đồ thị có hướng: chuỗi thành cạnh, số thành thuộc tính

```python
# tl3/ch5_kg.py
def build_graph(triples):
    """Đối tượng là chuỗi -> cạnh tới nút thực thể/khái niệm; đối tượng là số -> thuộc tính của nút chủ thể."""
    g = nx.DiGraph()
    for s, p, o in triples:
        if s not in g:
            g.add_node(s, kind=s.split("_", 1)[0])
        if isinstance(o, str) and not re.fullmatch(r"\d{4}-\d{2}-\d{2}", o):
            if o not in g:
                g.add_node(o, kind=o.split("_", 1)[0])
            g.add_edge(s, o, rel=p)
        else:
            g.nodes[s][p] = o
    return g
```

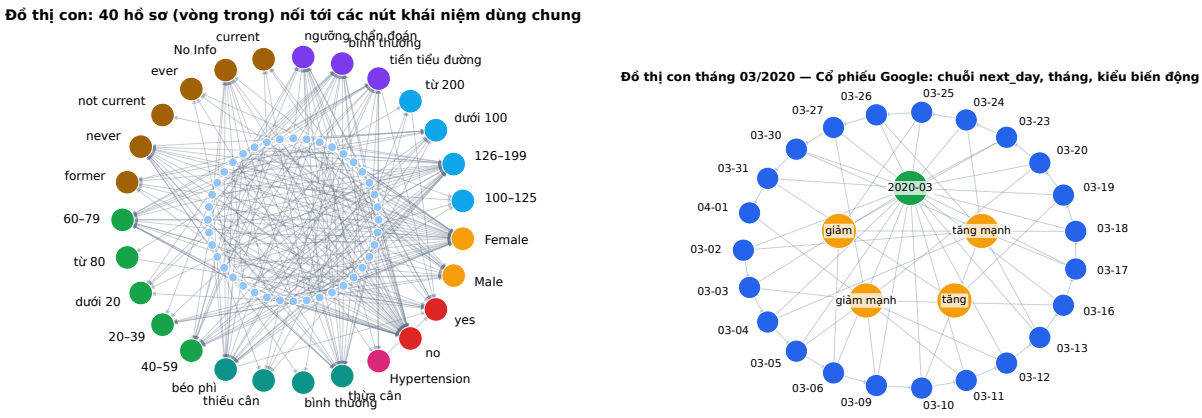


Hình 5.7. Lược đồ KG thị trường: ngày – tháng – năm, chuỗi `next_day` và nút khái niệm biến động

Bảng 5.4. Quy mô KB, KG và thời gian dựng của ba miền

Chỉ số	Tiểu đường	Google	Vàng
Số tài liệu	96.188	2.390	1.811
Độ dài TB (âm tiết)	45,3	43,1	76,5
Độ dài lớn nhất	49	57	78
Đặc trưng TF-IDF khác 0	256.676	12.212	23.611
Số bộ ba	1.357.436	22.817	32.725
Số nút KG	96.175	2.396	1.817
Số cạnh KG	684.414	6.921	5.238
Thuộc tính literal trên nút	673.022	15.896	27.487
Số loại quan hệ	11	4	4
Văn bản hoá (s)	16,25	0,50	2,24
TF-IDF (s)	48,90	0,96	1,79
BM25 (s)	30,11	0,64	0,84
Dựng KG (s)	30,65	0,37	0,95
Embedding MiniLM-L12 (s)	428,5	50,6	88,0
Embedding e5-small (s)	447,0	17,5	30,5
Embedding e5-base (s)	1.056,3	27,8	48,6

Thời gian đo trên CPU 4 luồng (thưa, đồ thị) và GPU RTX 8 GB (embedding). Độ dài tính theo khoảng trắng.



(a) KG tiểu đường **(b)** KG cổ phiếu, tháng 3/2020

Hình 5.8. Đồ thị con của hai KG: hồ sơ nối tới các nút khái niệm dùng chung; ngày giao dịch nối theo chuỗi `next_day`

Đoạn mã 5.5. BM25 Okapi

```python
# tl3/ch5_retrieval.py
class BM25Retriever:
    name = "BM25"

    def __init__(self, texts, k1=1.5, b=0.75):
        self.bm25 = BM25Okapi([tokenize(t) for t in texts], k1=k1, b=b)

    def scores(self, q):
        return np.asarray(self.bm25.get_scores(tokenize(q)), dtype=np.float32)

    def search(self, q, k=10):
        return topk(self.scores(q), k)
```

Đoạn mã 5.6. Truy xuất đồ thị: liên kết thực thể, nút KG, tài liệu

```python
# tl3/ch5_retrieval.py
class GraphRetriever:
    """Truy xuất đồ thị: hàm link(q) của miền trả danh sách id tài liệu ứng với nút KG khớp thực thể trong câu hỏi.
    Điểm = 1 cho tài liệu khớp (giảm dần theo thứ tự khớp), 0 cho phần còn lại."""
    name = "Graph"

    def __init__(self, ids, link):
        self.pos = {d: i for i, d in enumerate(ids)}
        self.n, self.link = len(ids), link

    def scores(self, q):
        s = np.zeros(self.n, dtype=np.float32)
        for r, d in enumerate(self.link(q)):
            if d in self.pos:
                s[self.pos[d]] = max(s[self.pos[d]], 1.0 - 1e-3 * r)
        return s

    def search(self, q, k=10):
        return [(i, v) for i, v in topk(self.scores(q), k) if v > 0]
```

Đoạn mã 5.7. Hợp nhất thứ hạng RRF có trọng số

```python
# tl3/ch5_retrieval.py
class HybridRRF:
    """Reciprocal Rank Fusion: s(d) = Σ_r w_r / (k + rank_r(d)); chỉ cộng các tài liệu trong top-depth mỗi nguồn."""

    def __init__(self, retrievers, k=60, depth=100, weights=None):
        self.rs, self.k, self.depth = retrievers, k, depth
        self.w = weights or [1.0] * len(retrievers)
        self.name = "RRF(" + " + ".join(r.name for r in retrievers) + ")"

    def search(self, q, k=10):
        fused = {}
        for r, w in zip(self.rs, self.w):
            for rank, (i, _) in enumerate(r.search(q, self.depth), 1):
                fused[i] = fused.get(i, 0.0) + w / (self.k + rank)
        return sorted(fused.items(), key=lambda x: -x[1])[:k]
```

Bảng 5.5. Số câu hỏi đánh giá theo miền và loại

Miền	Bản ghi	Nhóm	Tháng	Năm	Chỉ báo	Diễn đạt lại	Tổng
Tiểu đường	60	38	0	0	0	30	98
Cổ phiếu Google	60	0	20	8	0	20	88
Vàng (FINAL_USO)	60	0	20	7	12	20	99
Tổng	180	38	40	15	12	70	285

Diễn đạt lại (para): câu hỏi dùng từ đồng nghĩa và định dạng ngày khác tài liệu, kiểm tra truy xuất ngữ nghĩa.

5.5 Kiến trúc tác tử ReAct đơn lẻ

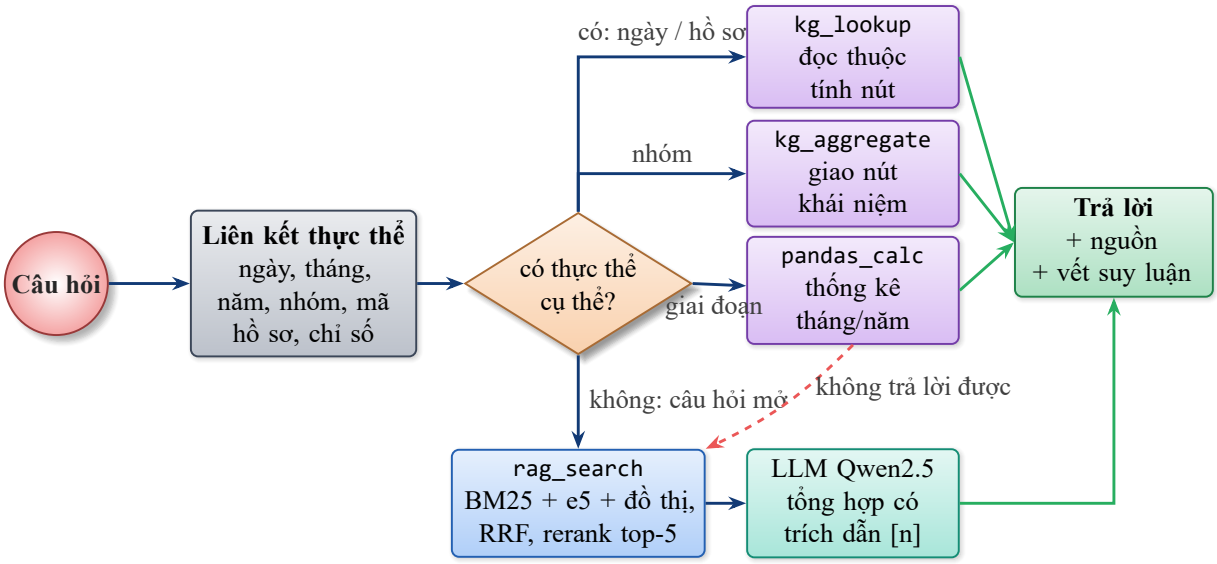
Tác tử theo vòng lặp ReAct [2]: *Thought* chọn công cụ, *Action* gọi công cụ, *Observation* nhận kết quả (Hình 5.9). Nếu liên kết thực thể nhận ra ngày, tháng hay hồ sơ thì tác tử gọi công cụ chính xác trên KG; nếu không, tác tử quay về RAG. Cách phối hợp này tương tự các kiến trúc sinh kèm công cụ [167].

Vết suy luận của tác tử – Cổ phiếu Google, ví dụ 1 (2 ms)

CÂU HỎI Giá mở cửa của cổ phiếu Google (Alphabet) ngày 27/08/2020 là bao nhiêu?

THOUGHT Câu hỏi nêu thực thể cụ thể (D2020-08-27; loại bản ghi) → dùng công cụ tra cứu chính xác trên KG/bảng.

ACTION kg_lookup("Giá mở cửa của cổ phiếu Google (Alphabet) ngày 27/08/2020 là…") (2,0 ms)



Hình 5.9. Tác tử định tuyến: liên kết thực thể quyết định công cụ chính xác hay RAG

Đoạn mã 5.8. Vòng lặp tác tử: định tuyến, gọi công cụ, quay về RAG khi cần

```python
# tl3/ch5_agent.py
def run(self, q, mode="auto"):
    t0 = time.perf_counter()
    trace = [("question", q)]
    route, why = ("rag", "Người dùng chọn chế độ RAG.") if mode == "rag" else self.plan(q)
    if mode == "graph":
        route, why = "exact", "Người dùng chọn chế độ đồ thị."
    trace.append(("thought", why))
    if route == "exact":
        t1 = time.perf_counter()
        f = self.D.facts(q)
        if f is not None:
            trace.append(("action", f"{f['tool']}(\"{q[:60]}…\")", round(1000 * (time.perf_counter() - t1))))
            trace.append(("obs", " | ".join(f["evidence"][:8]) or "(không có bộ ba)"))
            trace.append(("answer", f["answer"]))
            return {"answer": f["answer"], "sources": self.D.link(q)[:5], "tools": [f["tool"]], "trace": trace,
                    "values": f["values"], "latency_ms": 1000 * (time.perf_counter() - t0)}
        trace.append(("thought", "Công cụ chính xác không trả lời được → chuyển sang RAG."))
    out = self.rag(q, self.R.rerank or self.R.hybrid_graph, trace=trace)
    trace.append(("answer", out["answer"]))
    out["latency_ms"] = 1000 * (time.perf_counter() - t0)
    return out
```

OBSERV. (TradingDay_2020-08-27, has_date, 2020-08-27) | (TradingDay_2020-08-27, has_open, 82,68) | (TradingDay_2020-08-27, has_high, 82,75) | (TradingDay_2020-08-27, has_low, 81,29) | (TradingDay_2020-08-27, has_close, 81,72) | (TradingDay_2020-08-27, has_volume, 37232000) | (TradingDay_2020-08-27, has_return_pct, -1,09) | (TradingDay_2020-08-27, in_month, Month_2020-08)

ANSWER Giá mở cửa của cổ phiếu Alphabet ngày 27/08/2020 là 82,68 USD.

5.6 Đánh giá thực nghiệm

5.6.1 Chất lượng truy xuất

Bảng 5.6. Chất lượng truy xuất của mười cấu hình, gộp ba miền

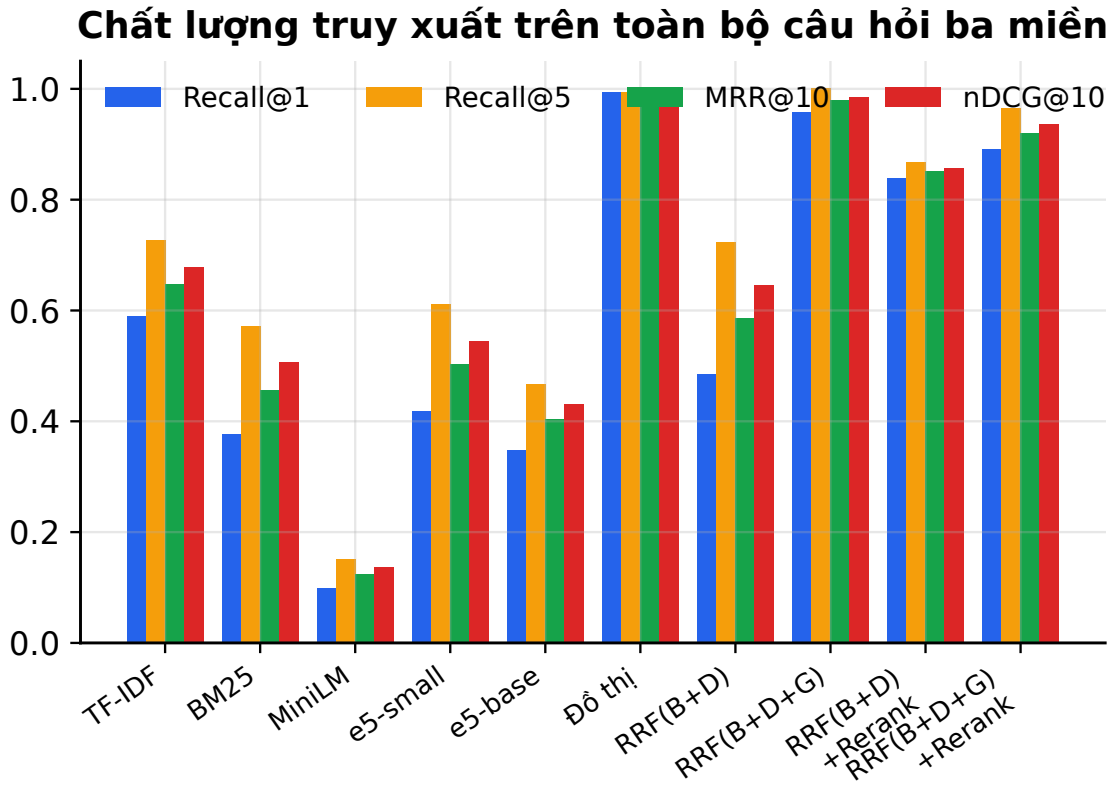
Bộ truy xuất	R@1	R@5	R@10	MRR	nDCG@10	ms / câu
TF-IDF	58,95	72,63	77,54	64,78	67,83	186,2
BM25	37,54	57,19	66,67	45,60	50,58	1.388,7
Dense MiniLM-L12	9,82	15,09	17,19	12,41	13,57	147,2
Dense e5-small	41,75	61,05	68,07	50,20	54,49	164,5
Dense e5-base	34,74	46,67	51,58	40,27	42,99	356,7
Đồ thị (KG)	**99,30**	99,30	99,30	**99,30**	**99,30**	**32,7**
RRF BM25 + Dense	48,42	72,28	83,86	58,63	64,60	1.553,7
RRF BM25 + Dense + KG	95,79	**100,00**	**100,00**	97,89	98,45	1.586,4
RRF + Rerank	83,86	86,67	87,37	85,06	85,62	1.981,2
RRF + KG + Rerank	89,12	96,49	98,25	91,99	93,50	2.045,5

Trung bình trên toàn bộ câu hỏi của ba miền (%). In đậm: tốt nhất mỗi cột. Dense trong RRF là e5-small; rerank: bge-reranker-v2-m3 xếp lại 20 ứng viên.

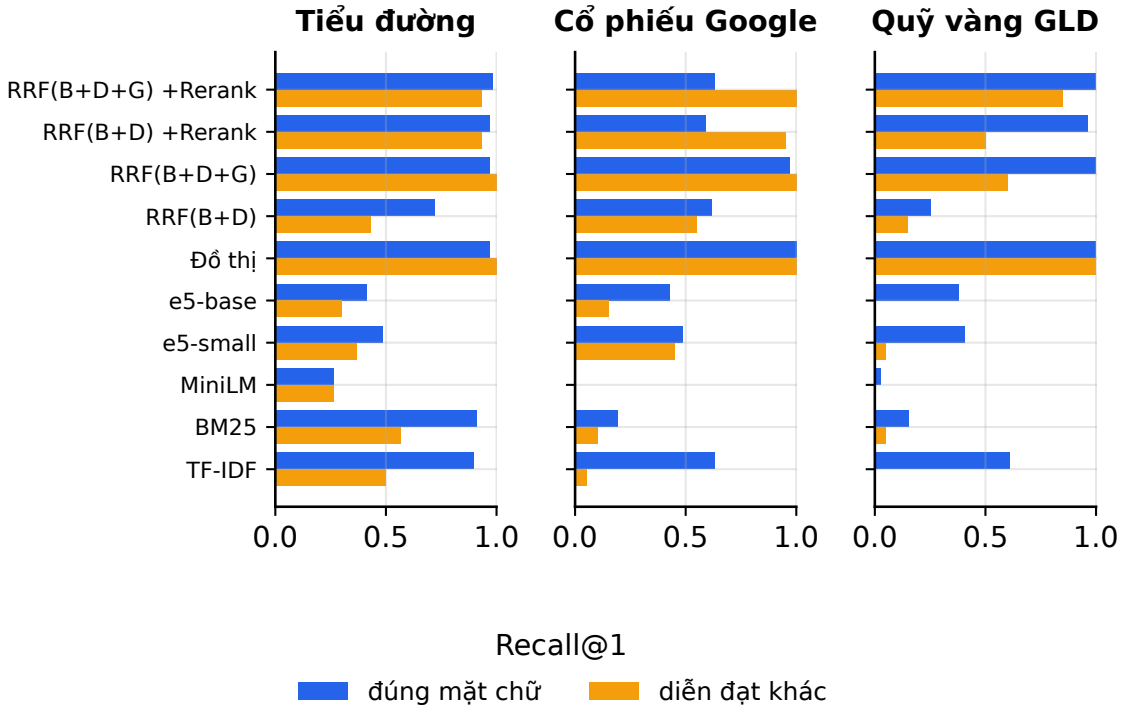
Nhận xét: Đồ thị tốt nhất khi đứng riêng (MRR 99,30%) nhưng chỉ hoạt động khi nhận ra thực thể. Trong các bộ truy xuất văn bản, TF-IDF có bigram (64,78%) hơn BM25 (45,60%) và cả ba mô hình embedding (e5-small 50,20%), vì câu hỏi tra cứu chứa nhiều con số phải khớp chính xác. RRF có nhánh KG đạt 97,89%, gần bằng đồ thị mà vẫn trả kết quả cho câu hỏi đồ thị không hiểu; bỏ nhánh KG chỉ còn 58,63%. Rerank kéo RRF không đồ thị lên 85,06%, nhưng đặt sau RRF có KG thì giảm (91,99%).

Khi chuyển sang câu hỏi diễn đạt lại, TF-IDF giảm mạnh ở cả ba miền, cho thấy mặt trái của truy xuất thưa, trong khi nhánh đồ thị ổn định nhờ bộ liên kết thực thể.

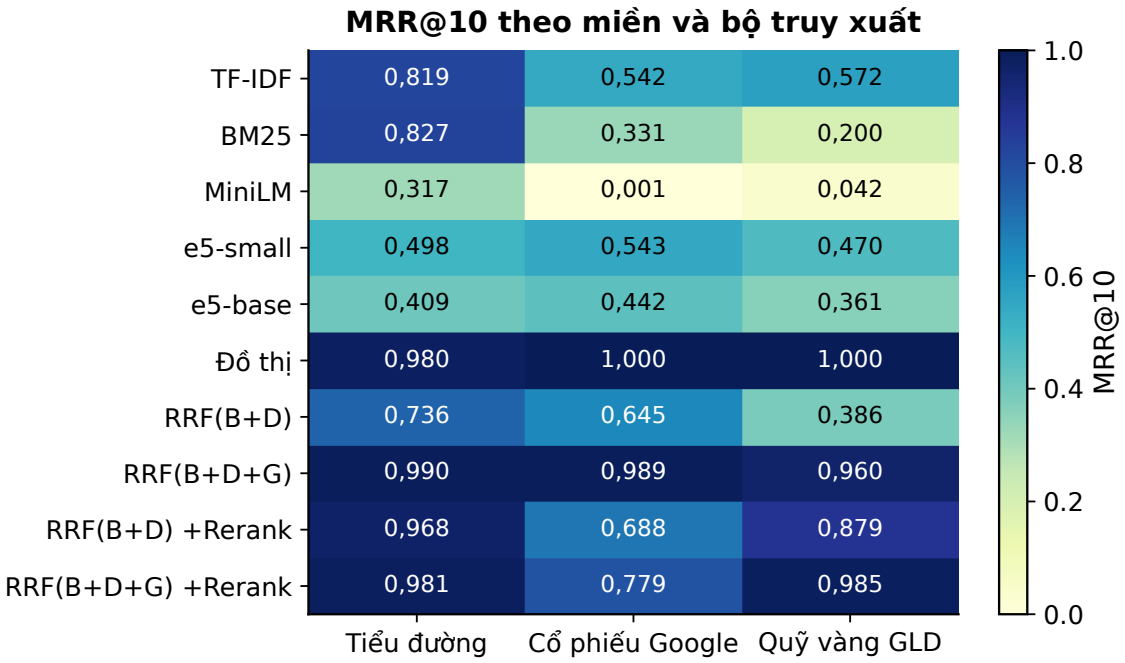
Tăng w_G từ 0 lên 1 cải thiện MRR rất mạnh ở cả ba miền; từ 2 trở lên đường cong gần như bão hoà, nên $w_G = 2$ là lựa chọn hợp lý: đồ thị thắng khi đúng, các nguồn khác vẫn đủ trọng số khi đồ thị không có kết quả.



Hình 5.10. Recall@1, Recall@10, MRR và nDCG@10 của mười cấu hình truy xuất



Hình 5.11. Recall@1 với câu hỏi khớp mặt chữ và câu hỏi diễn đạt lại



Hình 5.12. MRR@10 theo miền và bộ truy xuất

Bảng 5.7. Quét trọng số nhánh đồ thị w_G và số ứng viên rerank n

Miền	w_G=0,0	w_G=0,5	w_G=1,0	w_G=2,0	w_G=4,0	n=5	n=10	n=20	n=40
Tiểu đường	73,64	91,84	98,81	98,98	99,49	85,54	95,75	96,77	97,19
Cổ phiếu Google	64,46	75,76	77,54	98,86	100,00	67,23	68,79	68,84	68,26
Vàng (FINAL_USO)	38,58	87,41	93,55	95,96	95,96	58,59	80,30	87,88	92,42

MRR (%). Trái: trọng số nhánh đồ thị trong RRF BM25 + Dense + KG. Phải: số ứng viên đưa vào cross-encoder (RRF BM25 + Dense + Rerank).

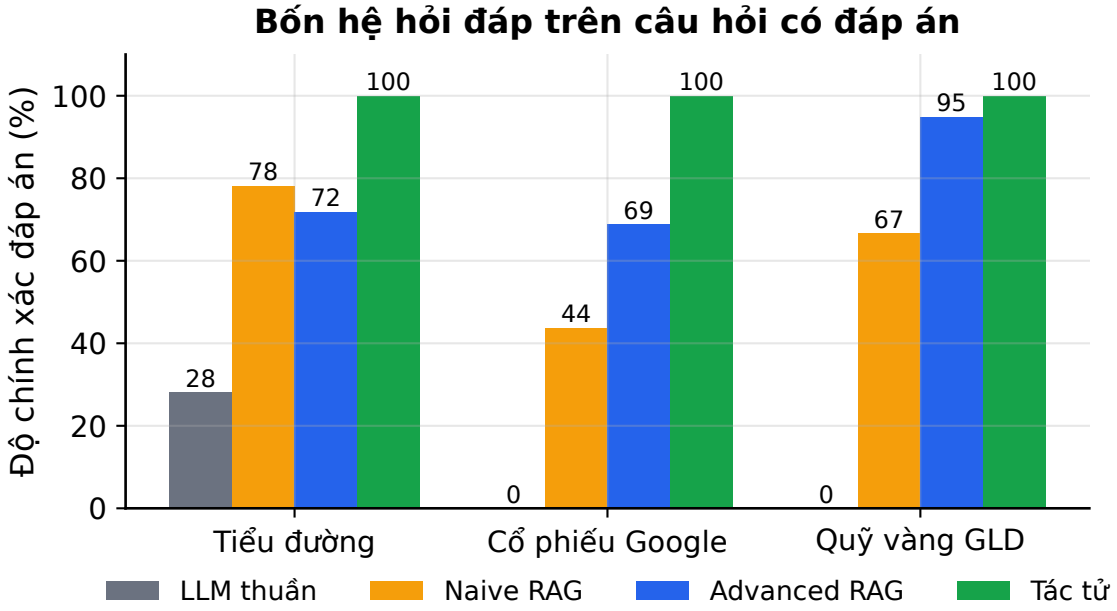
5.6.2 Chất lượng câu trả lời

Nhận xét: LLM thuần gần như không dùng được với dữ kiện cụ thể: 0,00% ở miền cổ phiếu và 0,00% ở miền vàng. RAG là bước cải thiện lớn nhất, nhưng truy xuất đúng mới là điều kiện cần: ở tiểu đường, Advanced RAG có nguồn đúng 100% mà độ chính xác chỉ 71,88% vì LLM 1,5B vẫn có thể đọc sai con số. Tác tử đạt 100,00% với độ trễ mili-giây do các câu có đáp án đều nêu thực thể mà bộ liên kết nhận ra. Kết quả này gắn với bộ câu hỏi tự sinh có cấu trúc đều đặn, không nên suy rộng cho câu hỏi tự do của người dùng thật.

Bảng 5.8. Độ chính xác câu trả lời và độ trễ của bốn hệ hỏi đáp

Miền	Hệ thống	Độ chính xác (%)	Nguồn đúng (%)	Trễ TB (ms)	Trễ trung vị (ms)
Tiểu đường	LLM thuần (không truy xuất)	28,12	–	17.735	12.236
	Naive RAG (dense top-5)	78,12	71,88	15.461	12.551
	Advanced RAG (lai + rerank)	71,88	100,00	15.550	13.009
	Tác tử chọn công cụ	**100,00**	100,00	**171**	87
Cổ phiếu Google	LLM thuần (không truy xuất)	0,00	–	35.688	38.122
	Naive RAG (dense top-5)	43,75	40,62	16.506	15.599
	Advanced RAG (lai + rerank)	68,75	75,00	18.198	16.494
	Tác tử chọn công cụ	**100,00**	100,00	**2**	2
Vàng (FINAL_USO)	LLM thuần (không truy xuất)	0,00	–	40.001	41.156
	Naive RAG (dense top-5)	66,67	69,23	16.610	15.055
	Advanced RAG (lai + rerank)	94,87	100,00	18.054	16.124
	Tác tử chọn công cụ	**100,00**	100,00	**3**	3

Nguồn đúng: tỷ lệ câu mà tài liệu đáp án có trong bằng chứng đưa cho LLM (hoặc nút KG tác tử dùng). LLM: Qwen2.5-1.5B-Instruct cục bộ, giải mã tham lam.



Hình 5.13. Độ chính xác đáp án của bốn hệ trên ba miền

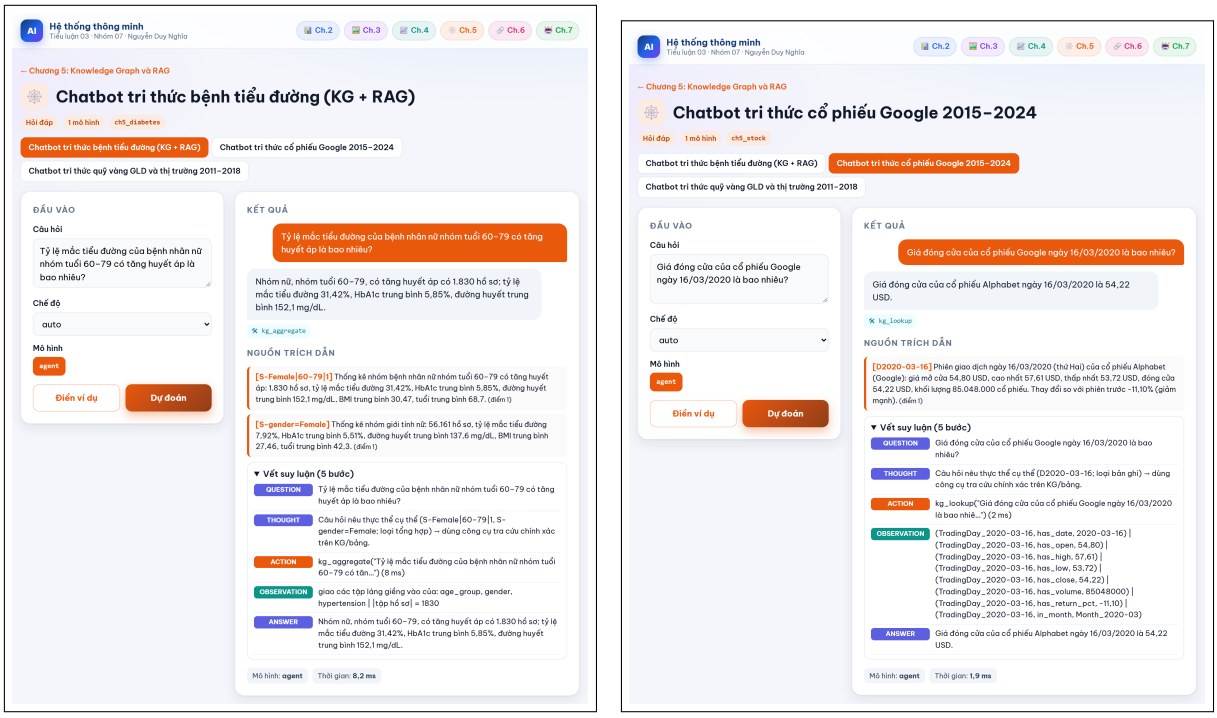
Bảng 5.9. Số lần tác tử chọn mỗi công cụ

Miền	`kg_aggregate`	`kg_lookup`	`pandas_calc`	Số câu
Tiểu đường	16	16	0	32
Cổ phiếu Google	0	16	16	32
Vàng (FINAL_USO)	0	24	15	39

Số câu hỏi đánh giá mà bộ định tuyến chuyển tới mỗi công cụ (công cụ đầu tiên được gọi).

5.7 Triển khai và điểm nghẽn của tác tử đơn

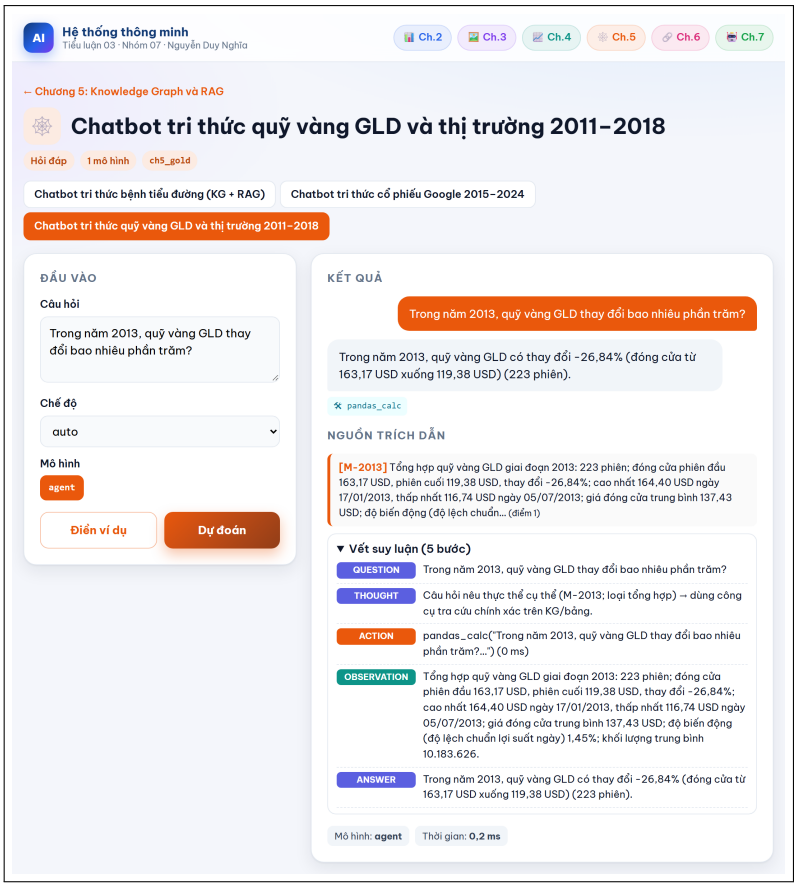
Ba chatbot được phục vụ qua `serve_ch5.py`, hiển thị câu trả lời, nguồn, công cụ đã chọn và vết suy luận (Hình 5.14).



(a) Chatbot tri thức tiểu đường **(b)** Chatbot cổ phiếu Google

Hình 5.14. Giao diện chatbot trên Web App

Thực nghiệm cũng cho thấy các điểm nghẽn của kiến trúc tác tử đơn. Liên kết thực thể bằng luật chỉ phủ các mẫu câu đã thiết kế. Một tác tử phải kiêm mọi vai trò, không có cơ chế tự kiểm chứng; lỗi của LLM nhỏ khi diễn đạt không được phát hiện. Với nhiệm vụ nhiều bước, ngữ cảnh một luồng dễ quá tải. Những hạn chế này dẫn tới hệ đa tác tử ở chương sau.



Hình 5.15. Chatbot quỹ vàng GLD: câu hỏi tổng hợp năm 2013

5.8 Tổng kết chương

Bốn kết luận chính gồm: biểu diễn quyết định chất lượng (văn bản hoá tiếng Việt, khái niệm suy ra, tài liệu tổng hợp); không bộ truy xuất đơn lẻ nào tốt nhất mọi nơi nên cần truy xuất lai; truy xuất đúng là điều kiện cần, chưa đủ để trả lời đúng; và phải hiểu đúng dữ liệu trước khi dựng tri thức.

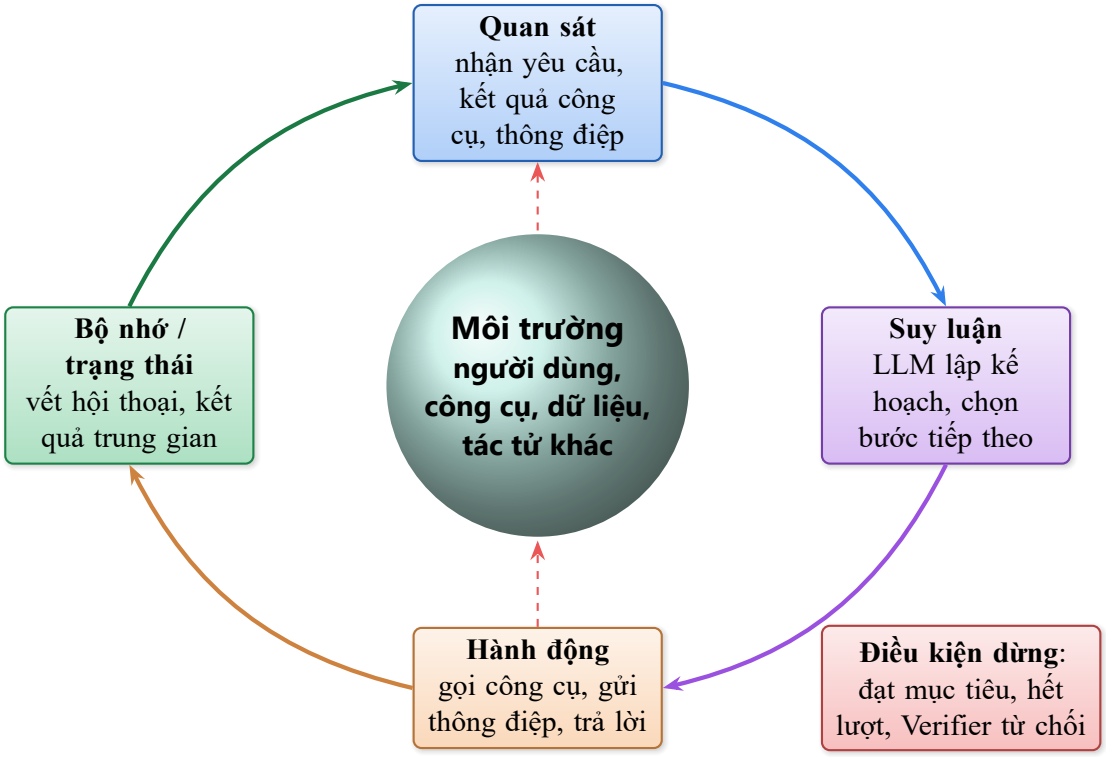
Chương 6

Hệ đa tác tử (Multi-Agent Systems)

6.1 Tổng quan và cơ sở lý thuyết

6.1.1 Từ tác tử đơn sang đa tác tử

Tác tử là thực thể *cảm nhận* môi trường và *tác động* lên môi trường [168], có tính tự chủ, phản ứng, chủ động và khả năng xã hội [169]. Trong hệ hiện đại, bộ não của tác tử thường là LLM, vận hành theo vòng lặp quan sát – suy luận – hành động – ghi nhớ [2] (Hình 6.1). Hệ đa tác tử chia nhiệm vụ cho nhiều vai trò chuyên biệt, nhằm tách biệt ngữ cảnh, kiểm chứng chéo và song song hoá.



Hình 6.1. Vòng lặp của một tác tử dựa trên LLM: quan sát, suy luận, hành động trên môi trường

6.1.2 Cơ sở hình thức

Khi các tác tử cùng mục tiêu nhưng chỉ quan sát được một phần môi trường, bài toán được mô hình hoá bằng Dec-POMDP, bộ tám phần tử

$$\mathcal{M} = \langle \mathcal{I}, \mathcal{S}, \{\mathcal{A}_i\}, T, R, \{\Omega_i\}, O, \gamma \rangle, \tag{6.1}$$

gồm tập tác tử $\mathcal{I}$, không gian trạng thái $\mathcal{S}$, tập hành động $\mathcal{A}_i$, hàm chuyển $T(s' \mid s, \mathbf{a})$, phần thưởng chung $R(s, \mathbf{a})$, tập quan sát Ω_i, hàm quan sát O và hệ số chiết khấu γ. Mục tiêu là tìm chính sách chung $\boldsymbol{\pi} = (\pi_1, \ldots, \pi_n)$ cực đại

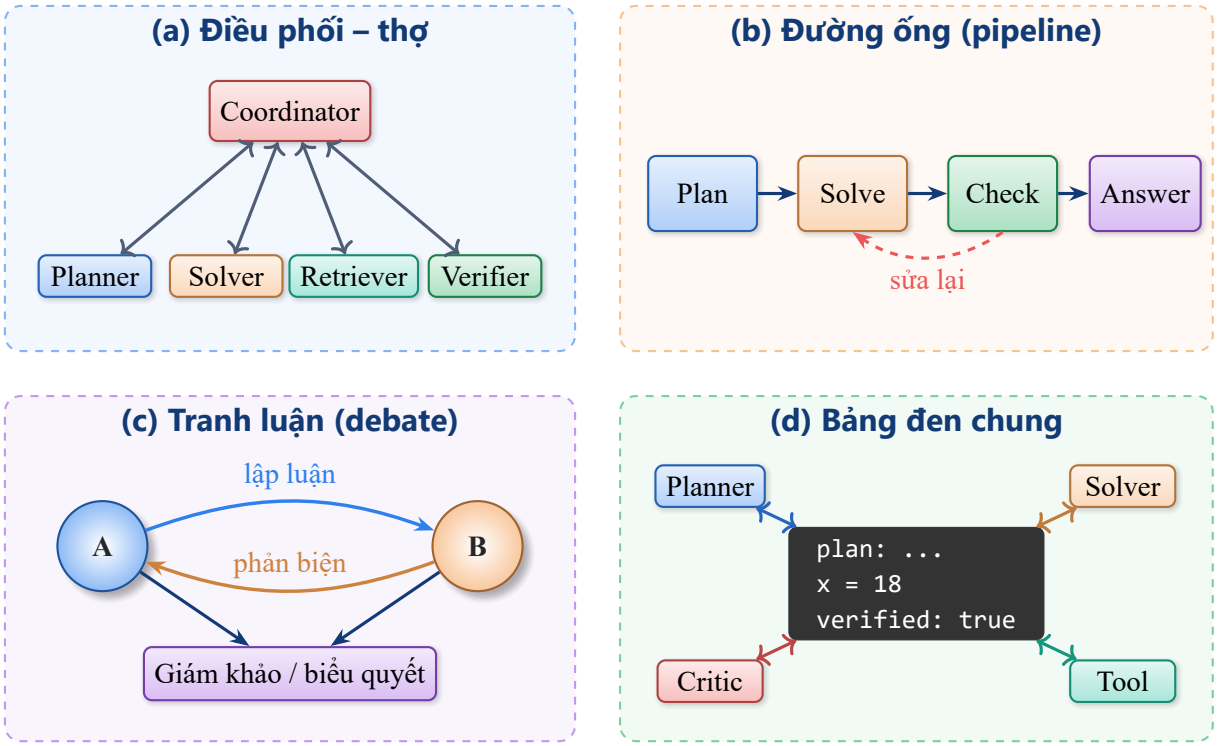
$$J(\boldsymbol{\pi}) = \mathbb{E}\Big[\sum_{t=0}^{\infty} \gamma^t R(s_t, \mathbf{a}_t)\Big]. \tag{6.2}$$

Nếu mỗi tác tử có phần thưởng riêng, mô hình trở thành trò chơi Markov. Ở mức nhận thức, mô hình BDI mô tả tác tử bằng niềm tin (Belief), mong muốn (Desire) và ý định (Intention) [169]: trong hệ LLM, ba thành phần này tương ứng trạng thái bộ nhớ, mục tiêu và kế hoạch đã cam kết.

6.2 Kiến trúc phối hợp và giao thức

6.2.1 Các cấu trúc phối hợp

Cách nối các tác tử quyết định luồng thông tin, điểm nghẽn và chế độ hỏng (Hình 6.2). Mô hình **điều phối – thợ** có một Supervisor phân việc và tổng hợp; **đường ống** truyền kết quả tuần tự; **tranh luận** cho các tác tử phản biện nhau [170]; **bảng đen** dùng vùng nhớ chung [171].

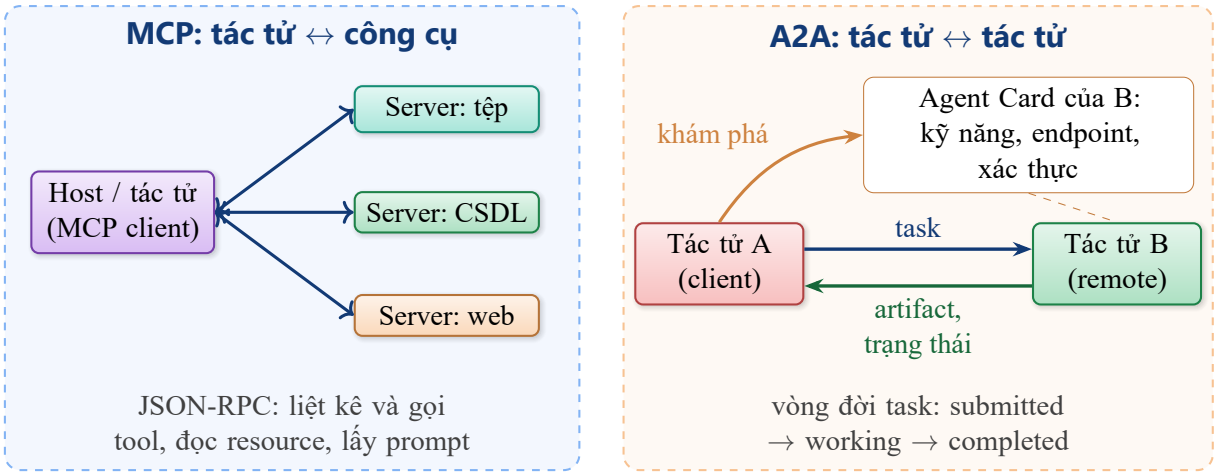


Hình 6.2. Bốn mẫu tổ chức hệ đa tác tử: điều phối – thợ, đường ống, tranh luận và bảng đen

6.2.2 Giao thức truyền thông

Hai giao thức bổ sung cho nhau (Hình 6.3): MCP nối tác tử với công cụ và dữ liệu [172], A2A nối tác tử với tác tử [173]. Thông điệp có cấu trúc chứa người gửi, người nhận, loại, nội dung và

định danh tác vụ, giúp truy vết toàn bộ hội thoại.



Hình 6.3. Hai giao thức bổ sung cho nhau: MCP nối tác tử với công cụ, A2A nối tác tử với tác tử

6.2.3 Các framework hiện đại

Bảng 6.1. So sánh ba framework đa tác tử phổ biến

Tiêu chí	LangGraph	AutoGen [174]	CrewAI
Mô hình	Đồ thị trạng thái	Hội thoại giữa tác tử	Nhóm theo vai trò
Điều khiển	Tường minh, có nhánh	Theo lượt hội thoại	Theo quy trình khai báo
Thế mạnh	Luồng phức tạp, có trạng thái	Thử nghiệm nhanh	Dễ dựng nhóm vai trò
Điểm yếu	Viết nhiều mã hơn	Khó kiểm soát luồng	Kém linh hoạt

Tiểu luận tự cài đặt phần điều phối bằng Python thay vì dùng framework, để đo chính xác từng lượt gọi và token. Các nghiên cứu về lỗi cho thấy hệ đa tác tử hỏng nhiều ở khâu đặc tả và phối hợp, không chỉ ở mô hình [175].

6.3 Thực nghiệm: đa tác tử giải toán lời văn GSM8K

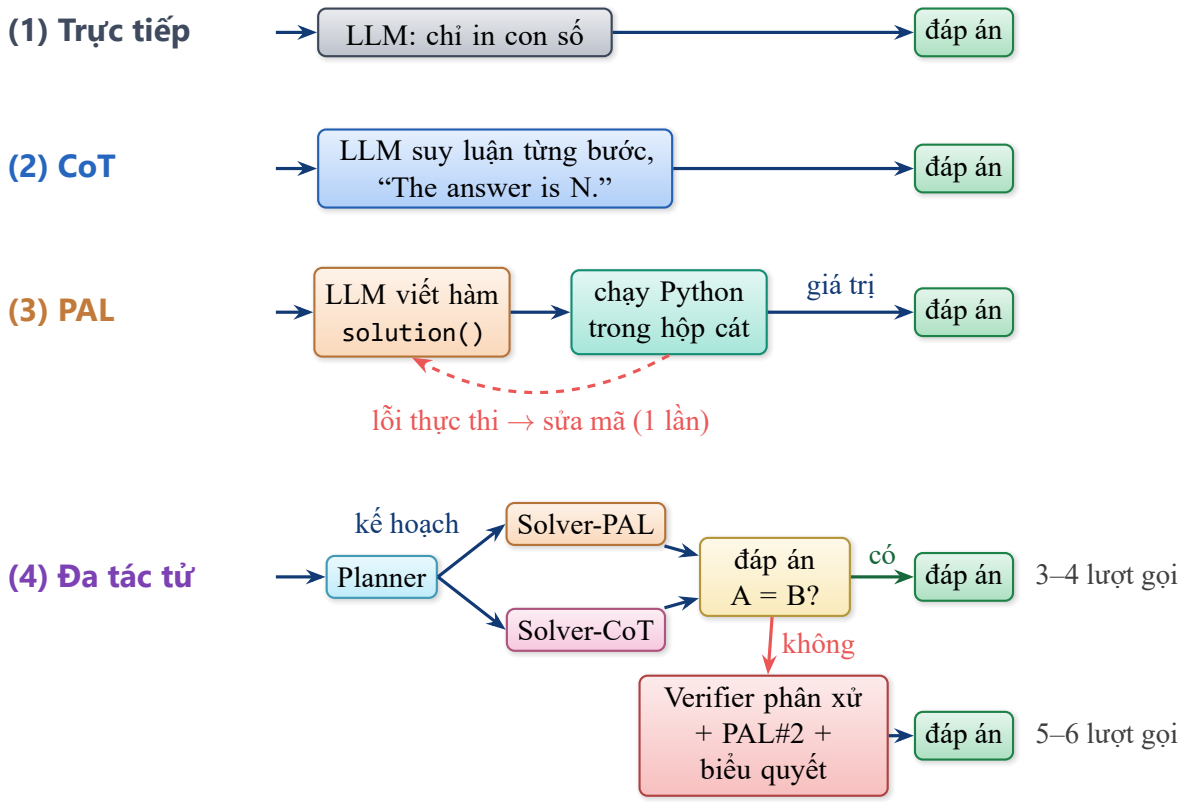
6.3.1 Bài toán và câu hỏi nghiên cứu

GSM8K [176] gồm các bài toán lời văn bậc tiểu học, mỗi bài cần 2–8 phép tính, đáp án là một số; chỉ một bước sai là sai cả câu. Mọi hệ dùng chung LLM Qwen2.5-1.5B-Instruct [160] chạy cục bộ trên GPU, giải mã tham lam. Chọn mô hình nhỏ là có chủ ý: tác dụng hay tác hại của việc thêm tác tử sẽ hiện ra rõ. Thực nghiệm trả lời ba câu hỏi: suy luận từng bước và công cụ cải thiện bao nhiêu; hệ đa tác tử có tốt hơn tác tử đơn tốt nhất không; phần cải thiện phải trả bằng bao nhiêu chi phí.

6.3.2 Thiết kế bốn hệ

Bốn hệ được so sánh trên cùng tập câu và cùng cách chấm (Hình 6.4):

- **Trực tiếp:** một lượt gọi, chỉ in ra con số; là mốc dưới.
- **CoT:** một tác tử suy luận từng bước [177].
- **PAL + Python:** một tác tử viết hàm `solution()`, công cụ chạy mã trong hộp cát [178]; nếu lỗi thì gửi lại để sửa một lần.
- **Đa tác tử:** Planner lập kế hoạch; Solver-PAL viết mã, Solver-CoT giải bằng lời độc lập. Nếu hai đáp án khớp, Verifier chấp nhận (cổng đồng thuận). Nếu không, Verifier phân xử, một Solver-PAL thứ hai lấy mẫu ở nhiệt độ 0,7, rồi Coordinator biểu quyết đa số, ý tưởng gần với self-consistency [179].



Hình 6.4. Bốn hệ được so sánh, từ một lượt gọi đến một nhóm năm vai trò có cổng đồng thuận

Đoạn mã 6.3. Hệ đa tác tử: Planner, hai Solver độc lập, cổng đồng thuận, Verifier và biểu quyết

```python
# tl3/ch7_agents.py
def solve_multi(llm, q):
    """Planner -> hai Solver độc lập (PAL theo kế hoạch, CoT) -> Verifier.
    Đồng thuận: chấp nhận ngay. Bất đồng: Critic phân xử, thêm một mẫu PAL nhiệt độ
    ↪ 0,7, rồi biểu quyết đa số."""
    tr = Trace()
    plan = Agent(llm, "Planner", SYS_PLANNER, max_new_tokens=160).act(q, tr)
    a_pal = pal_once(Agent(llm, "Solver-PAL", SYS_PAL, max_new_tokens=360), q, tr,
    ↪ plan=plan)
```

Đoạn mã 6.1. Một tác tử là một vai trò (lời nhắc hệ thống) trên LLM dùng chung

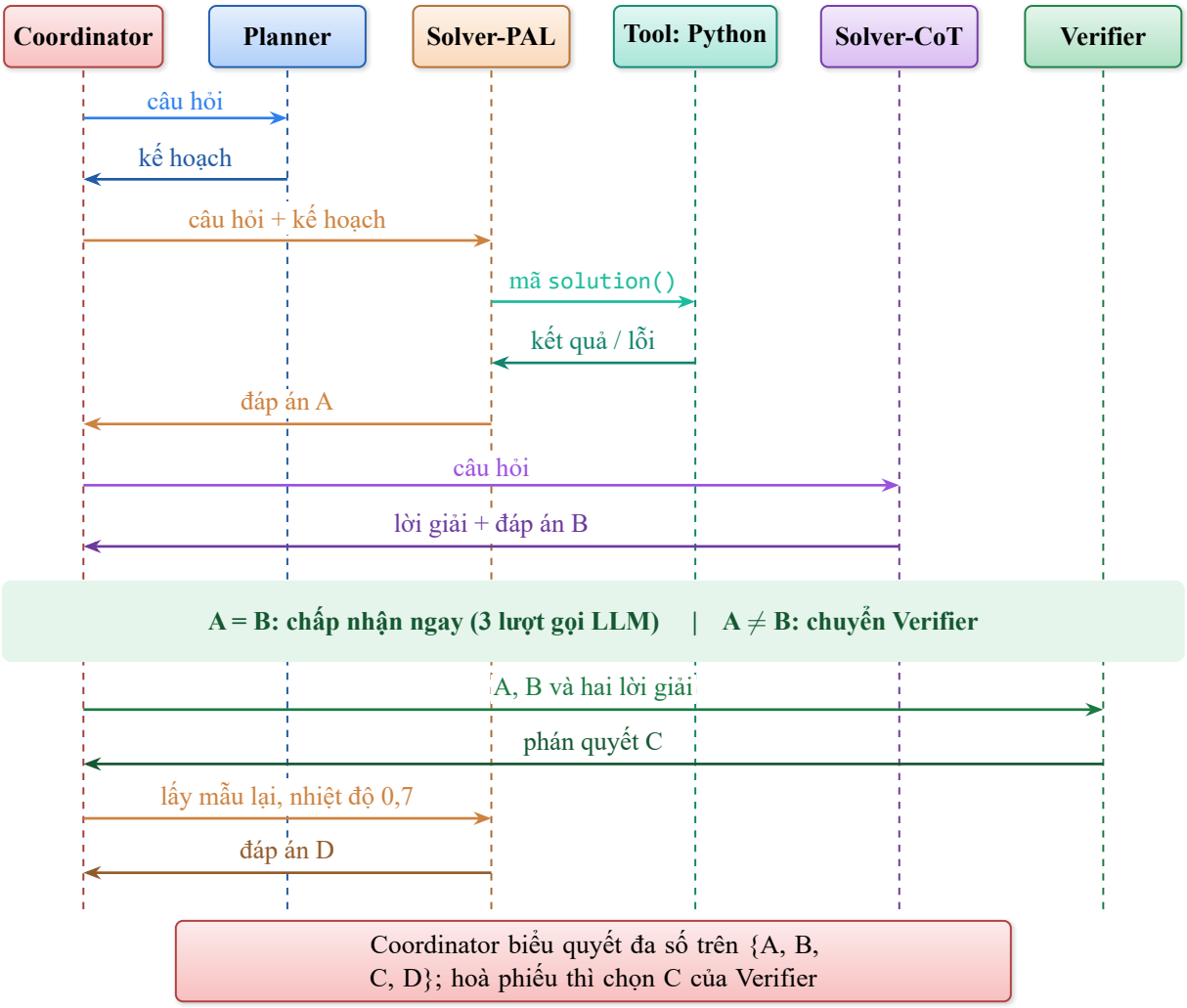
```
# tl3/ch7_agents.py
class Agent:
    """Một tác tử = một vai trò (system prompt) dùng chung LLM; mỗi lần act() là một lượt gọi được ghi vết."""

    def __init__(self, llm, role, system, max_new_tokens=320, temperature=0.0):
        self.llm, self.role, self.system = llm, role, system
        self.max_new_tokens, self.temperature = max_new_tokens, temperature

    def act(self, message, trace):
        text, n_out, sec = self.llm.chat(message, system=self.system,
            max_new_tokens=self.max_new_tokens,
                                         temperature=self.temperature)
        trace.calls += 1
        trace.tokens_in += len(self.llm.tok.encode(self.system + message))
        trace.tokens_out += n_out
        trace.seconds += sec
        trace.add(self.role, text)
        return text
```

Đoạn mã 6.2. Công cụ python: chạy mã không tin cậy trong tiến trình con có giới hạn

```
# tl3/ch7_agents.py
def run_python(code, timeout=5.0):
    """Chạy mã do LLM sinh trong tiến trình con riêng (-I, thư mục tạm, chặn import ngoài danh sách, giới hạn
    thời gian). Mã phải định nghĩa solution(); trả (giá trị, lỗi)."""
    program = SANDBOX_PRELUDE + "\n" + code + "\n\nprint(repr(solution()))\n"
    with tempfile.TemporaryDirectory() as tmp:
        try:
            p = subprocess.run([sys.executable, "-I", "-c", program], cwd=tmp,
                capture_output=True, text=True,
                               timeout=timeout, env={"PYTHONIOENCODING": "utf-8",
                                "SYSTEMROOT": "C:\\Windows"})
        except subprocess.TimeoutExpired:
            return None, "timeout"
    if p.returncode != 0:
        return None, (p.stderr.strip().splitlines() or ["lỗi"])[-1][:200]
    return to_number(p.stdout.strip().splitlines()[-1] if p.stdout.strip() else None), None
```



Hình 6.5. Trình tự thông điệp của hệ đa tác tử ở nhánh bất đồng

```
    cot_text = Agent(llm, "Solver-CoT", SYS_COT, max_new_tokens=400).act(q, tr)
    a_cot = to_number(cot_text)
    if same(a_pal, a_cot):
        tr.add("Verifier", f"hai solver đồng thuận: {a_pal} -> chấp nhận")
        return a_pal, tr
    pal_code = next((s["text"] for s in reversed(tr.steps) if s["role"] ==
    ↪  "Solver-PAL"), "")
    critic_msg = (f"Problem: {q}\n\nSolver A (Python program) returned
    ↪  {a_pal}:\n{pal_code}\n\n"
                  f"Solver B (reasoning) returned {a_cot}:\n{cot_text}\n\nWhich is
                   ↪  correct?")
    a_critic = to_number(Agent(llm, "Verifier", SYS_CRITIC,
    ↪  max_new_tokens=360).act(critic_msg, tr))
    a_extra = pal_once(Agent(llm, "Solver-PAL#2", SYS_PAL, max_new_tokens=360,
    ↪  temperature=0.7), q, tr, plan=plan,
                       retries=0)
    votes = Counter(v for v in (a_pal, a_cot, a_critic, a_extra) if v is not None)
    if not votes:
        tr.add("Coordinator", "không tác tử nào ra số -> bỏ trống")
```

```
        return None, tr
    best, n = votes.most_common(1)[0]
    if n == 1 and a_critic is not None:          # hoà phiếu: tin Verifier vì nó đã
    ↪ đọc cả hai lời giải
        best = a_critic
    tr.add("Coordinator", f"phiếu {dict(votes)} -> chọn {best}")
    return best, tr
```

6.4 Đánh giá định lượng: tác tử đơn và đa tác tử

Cỡ mẫu

Tốc độ sinh thực tế trên GPU dùng chung chỉ khoảng 2,456 token/giây nên mỗi hệ chạy được số câu khác nhau. Mọi so sánh dùng *tập chung* gồm 35 câu mà cả bốn hệ đều đã chạy. Đây là mẫu nhỏ: chênh lệch nhỏ hơn bề rộng khoảng tin cậy bootstrap ở Bảng 6.2 không nên được diễn giải như khác biệt thật.

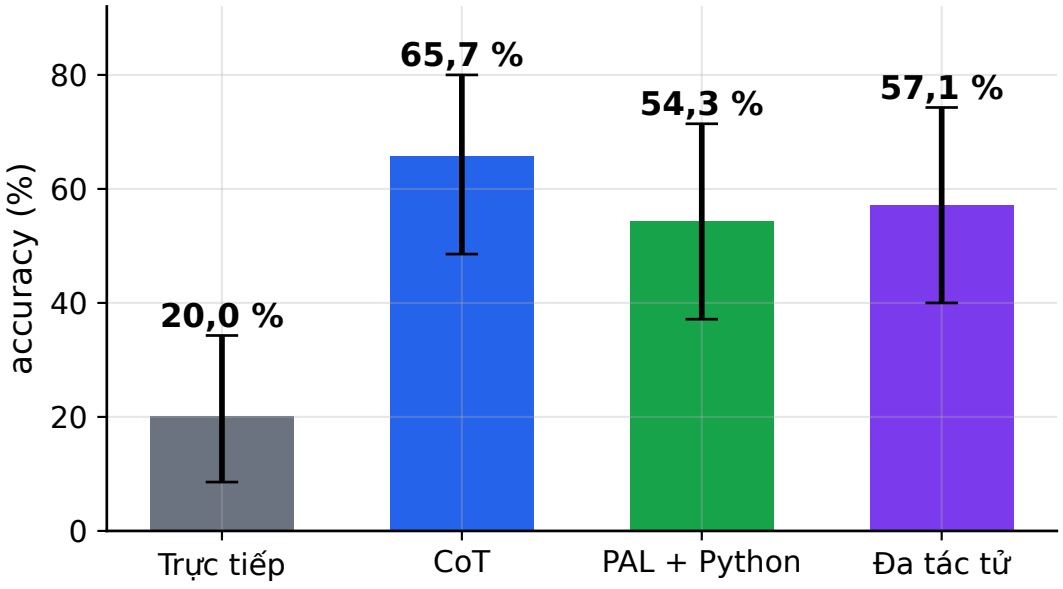
Bảng 6.2. Độ chính xác, khoảng tin cậy 95% và chi phí trung bình mỗi câu của bốn hệ trên GSM8K

Hệ	Acc (%)	KTC 95 %	Lượt gọi	Token vào	Token ra	Giây (trung vị)	Giây (TB)
Trực tiếp	20,0	[8,6; 34,3]	1,00	80	4	1,8	1,7
CoT	**65,7**	[48,6; 80,0]	1,00	98	265	101,2	108,0
PAL + Python	54,3	[37,1; 71,4]	1,29	236	113	53,1	64,9
Đa tác tử	57,1	[40,0; 74,3]	5,09	1.366	765	128,8	176,6

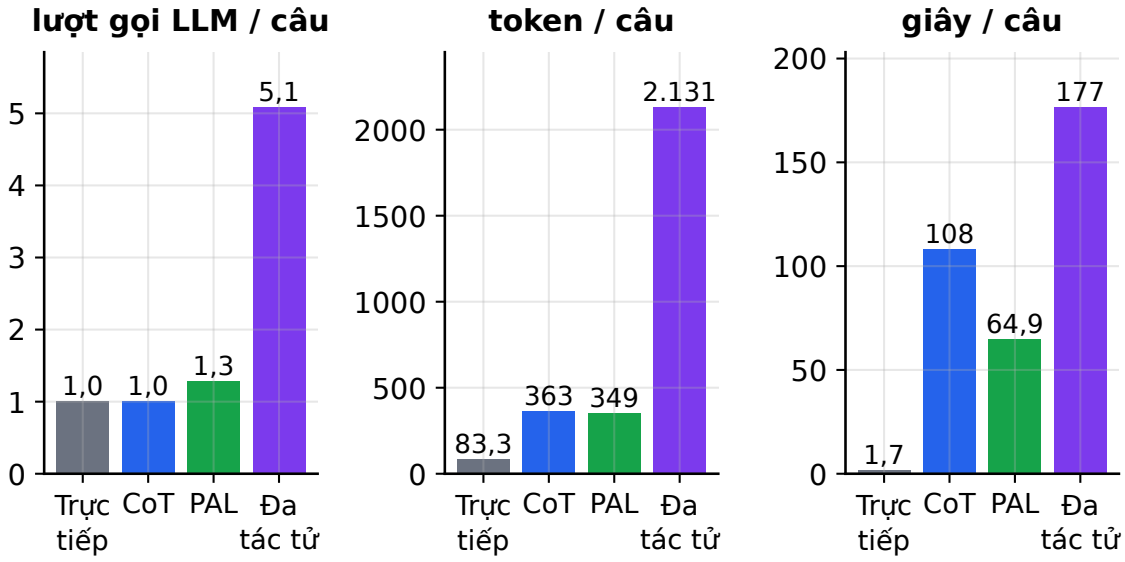
Trung bình trên cùng 35 câu GSM8K test (mẫu ngẫu nhiên, hạt giống 42), Qwen2.5-1.5B-Instruct chạy cục bộ, giải mã tham lam. KTC: khoảng tin cậy bootstrap 2.000 lần lấy mẫu lại.

Nhận xét: Trả lời trực tiếp chỉ đúng 20,00%. Suy luận từng bước nâng lên 65,71%, bước nhảy lớn nhất của thực nghiệm mà chỉ tốn thêm token trong một lượt gọi. PAL đạt 54,29%: Python loại bỏ lỗi số học, nhưng với mô hình nhỏ, viết đúng chương trình và định dạng là một kỹ năng riêng. Hệ đa tác tử đạt 57,14%, tức *không vượt* CoT, trong khi tốn 5,086 lượt gọi và 2.131 token mỗi câu.

Hai Solver đồng ý ở 42,86% số câu với độ chính xác 93,33%: khi hai phương pháp khác nhau cho cùng một con số, con số đó đáng tin hơn hẳn mỗi phương pháp riêng lẻ. Đây là giá trị cốt lõi của kiểm chứng chéo. Ở nhánh bất đồng, hệ chỉ đạt 30,00% trong khi cận trên oracle (ít nhất một tác tử con đúng) là 77,14%. Khoảng cách đó là phần cơ chế tổng hợp để lọt: để một LLM nhỏ tự phân xử gần như khó bằng việc tự giải bài toán [180].



Hình 6.6. Độ chính xác của bốn hệ; thanh lỗi là khoảng tin cậy 95% bootstrap

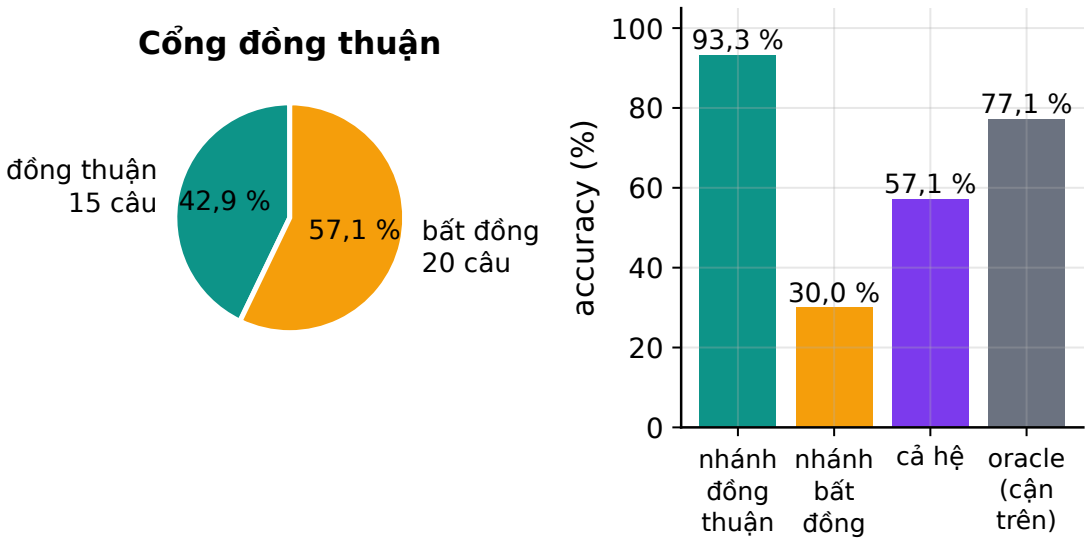


Hình 6.7. Chi phí trung bình mỗi câu: số lượt gọi LLM, số token (vào + ra) và thời gian

Bảng 6.3. Hệ đa tác tử so với hai tác tử đơn: số câu sửa đúng, làm hỏng và chi phí phụ trội

So sánh	Sửa đúng	Làm hỏng	Ròng	Acc (điểm %)	Lượt gọi / câu đúng thêm	Thời gian (lần)
Đa tác tử so với PAL + Python	5	4	+1	+2,9	133,0	2,72
Đa tác tử so với CoT	2	5	−3	−8,6	–	1,63

Sửa đúng: câu hệ tham chiếu sai nhưng đa tác tử đúng; làm hỏng: ngược lại. Lượt gọi / câu đúng thêm: số lượt gọi LLM phụ trội trên toàn tập chia cho số câu đúng tăng thêm.

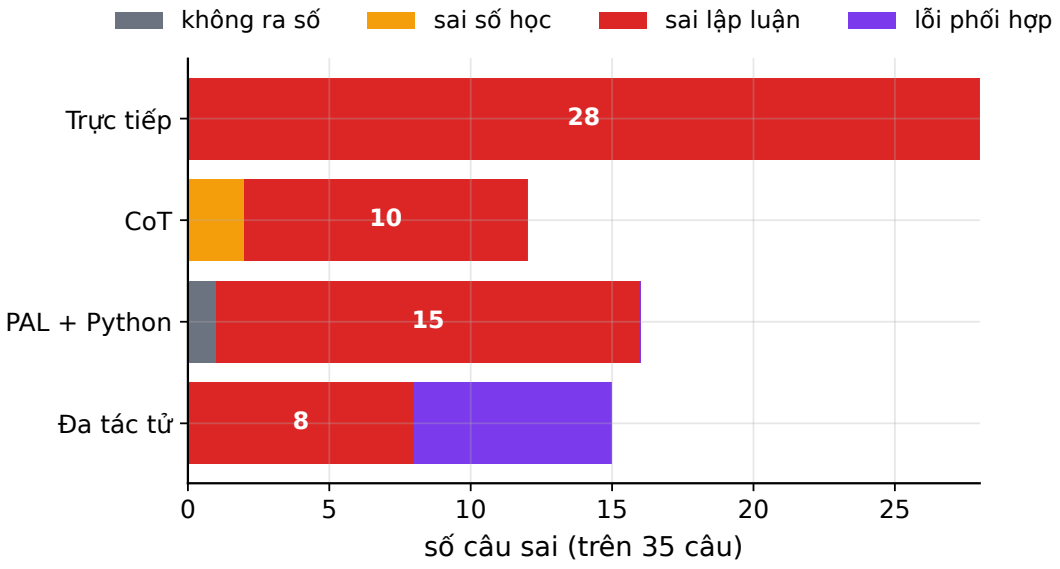


Hình 6.8. Cổng đồng thuận: tỷ lệ câu hai Solver khớp nhau (trái) và độ chính xác ở từng nhánh, của cả hệ và cận trên oracle (phải)

Bảng 6.4. Phân loại câu sai của từng hệ

Hệ	Không ra số	Sai số học	Sai lập luận	Lỗi phối hợp	Tổng sai
Trực tiếp	0	0	28	0	28
CoT	0	2	10	0	12
PAL + Python	1	0	15	0	16
Đa tác tử	0	0	8	7	15

Sai số học: lời giải có một phép tính viết sai kết quả; lỗi phối hợp (chỉ hệ đa tác tử): ít nhất một tác tử đã ra đáp án đúng nhưng bước tổng hợp chọn sai; còn lại là sai lập luận.



Hình 6.9. Cơ cấu lỗi của bốn hệ

Vết tác tử – lỗi phối hợp – một tác tử đã ra đáp án đúng nhưng bị biểu quyết loại (câu 39, đáp án 18, hệ trả lời 16, 6 lượt gọi, 241,5 s)

CÂU HỎI Dana can run at a rate of speed four times faster than she can walk, but she can skip at a rate of speed that is half as fast as she can run. If she can skip at 3 miles per hour, how many miles can she travel in six hours if she spends one-third of the time running and two-thirds of the time walking?

PLANNER 1. Determine Dana's running speed: - Given that skipping at 3 miles per hour is half as fast as running. - Therefore, running speed = 2 × 3 = 6 miles per hour. 2. Determine Dana's walking speed: - Given that running at 6 miles per hour is four times faster than walking. - Therefore, walking speed = 6 / 4 = 1.5 miles per hour. 3. Calculate total distance for each activity: - Time spent running = 6 × (1)/(3) = 2 hours. - Distance running = 2 × 6 = 12 miles. - Time spent walking =

SOLVER-PAL Given data skip_speed = 3 # miles per hour walk_speed = skip_speed / 4 # miles per hour run_speed = skip_speed * 4 # miles per hour Total time available total_time = 6 # hours Time spent on each activity time_running = total_time / 3 # hours time_walking = total_time - time_running # hours Calculate distances distance_running = time_running * run_speed # miles distance_walking = time_walking * walk_speed # miles Total distance traveled total_distance = distance_running + distance_walking # miles

PYTHON lỗi: không có hàm solution()

SOLVER-PAL

```
It seems there was an error in the previous code because it did not define
the `solution` function correctly. Let me correct this and provide
the proper solution:
def solution():
# Given data
total_time = 6  # Total time in hours
time_running_fraction = 1/3  # Fraction of time spent running
time_walking_fraction = 2/3  # Fraction of time spent walking
# Calculate time spent on each activity
time_running = total_time * time_running_fraction
time_walking = total_time * time_walking_fraction
# Given speeds
running_speed = 6  # Running speed in miles per hour
walking_speed = running_speed / 4  # Walking speed in miles per hour
# Calculate distances
distance_running = time_running * running_speed
distance_walking = time_walking * walking_speed
# Calculate total distance traveled
total_distance = distance_running + distance_walking
return total_distance
# Call the function to get the result
```

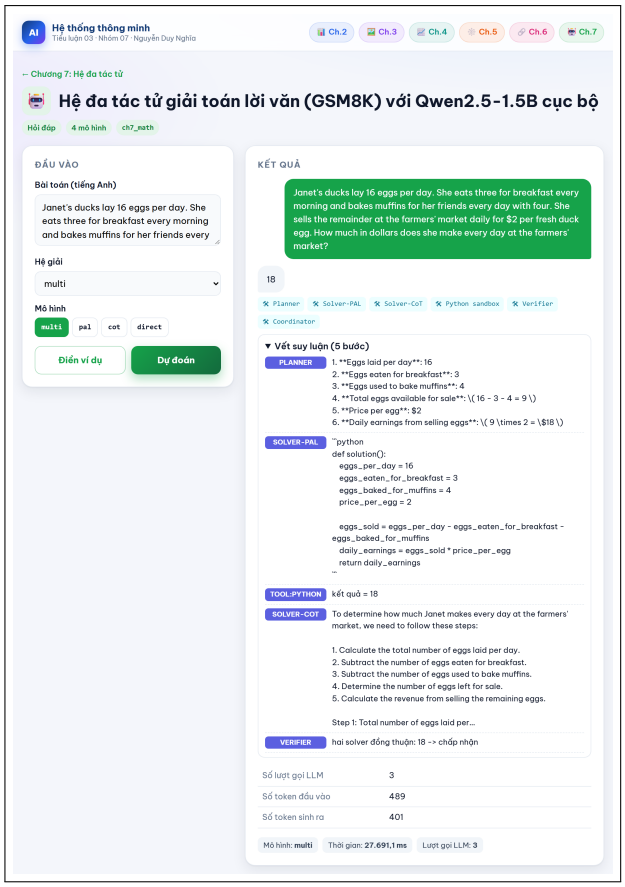
	`result = solution()`
	`print(result)`
	`This code will calculate the total distance Dana travels in six hours, given the fractions of time spent running and walking, and the speeds for both activities.`

PYTHON	kết quả = 18
SOLVER-CoT	To solve this problem, we need to determine Dana's speeds for running, walking, and skipping, then calculate the distance she travels in six hours based on her spending different portions of that time on each activity. 1. Determine Dana's running speed: - Skip rate = 3 mph (half as fast as running) - Running rate = 2 * Skip rate = 2 * 3 mph = 6 mph 2. Calculate the time spent on each activity: - Total time = 6 hours - Time spent running = 1/3 of total time = 6 hours / 3 = 2 hours - Time spent walking = 2/3 of total time = 6 hours * 2/3 = 4 hours 3. Calculate the distance traveled during each a ... [lược bớt]
VERIFIER	The answer is 16.
PAL#2	Given data walking_speed = 1.5 # miles per hour running_speed = 6 # miles per hour skip_speed = 3 # miles per hour Calculate the total time spent on each activity time_running = 6 * (1/3) time_walking = 6 * (2/3) Calculate the distance traveled by each activity distance_running = time_running * running_speed distance_walking = time_walking * walking_speed distance_skipping = time_running * skip_speed Total distance traveled total_distance = distance_running + distance_walking + distance_skipping
PYTHON	lỗi: không có hàm solution()
ĐIỀU PHỐI	phiếu {18: 1, 16: 2} -> chọn 16

Bảng 6.4 cho thấy phối hợp sinh ra loại lỗi mới: 7 câu sai vì khâu tổng hợp gạt đi đáp án đúng đã có trong vết, và kế hoạch của Planner lấn át đặc tả định dạng của Solver-PAL (94,29% lượt chạy đầu thiếu `solution()`).

6.5 Triển khai Web App

Hệ đa tác tử được phục vụ qua `serve_ch7.py`: người dùng nhập bài toán, Web App trả đáp án, vết từng vai trò và công cụ đã gọi (Hình 6.10).



Hình 6.10. Hệ đa tác tử giải bài toán lời văn trên Web App: đáp án và vết từng vai trò

6.6 Tổng kết chương

Trên 35 câu chung, suy luận từng bước là đòn bẩy lớn nhất; công cụ không tự động tốt hơn; hệ đa tác tử không vượt tác tử đơn tốt nhất với mô hình nhỏ, nhưng sự đồng thuận giữa các Solver là tín hiệu tin cậy có giá trị. Bài học thiết kế: bắt đầu từ tác tử đơn mạnh nhất; chỉ thêm tác tử khi có tín hiệu kiểm chứng bên ngoài đáng tin như trình thông dịch, bộ kiểm thử hay người duyệt; và dùng sự bất đồng để chuyển câu khó cho người kiểm tra. Giới hạn của chương là cỡ mẫu nhỏ và chỉ thử một LLM.

Kết luận

Tiểu luận đã đi từ lịch sử của trí tuệ nhân tạo đến các hệ thống thông minh nhiều thành phần. Mỗi chương lý thuyết được kiểm chứng bằng thực nghiệm chạy thật trên máy cá nhân, và mọi mô hình được tích hợp vào một Web App Flask chung. Bảng KL.1 tóm tắt kết quả; mô hình được chọn theo tập kiểm định, không theo tập kiểm thử.

Bảng KL.1. Tổng hợp kết quả các thực nghiệm (chỉ số trên tập kiểm thử)

Ch.	Thực nghiệm	Mô hình được chọn	Chỉ số chính
2	Tiểu đường	XGBoost	F1 0,8115; ROC-AUC 0,9778
2	Giá nhà	Lasso (L1)	R^2 0,7951; MAPE 9,42%
2	Nhận xét khách hàng	TF-IDF + Logistic Regression	macro-F1 0,8542; Acc 90,82%
3	MNIST	Improved CNN (PyTorch)	Acc 99,69%
3	Fashion-MNIST	Improved CNN (PyTorch)	Acc 94,35%
3	Rock/Paper/Scissors	Improved CNN (PyTorch)	Acc 96,24%
3	CIFAR-10	Improved CNN (PyTorch)	Acc 91,59%
4	Doanh thu ngày	SimpleRNN (PyTorch)	RMSE bằng 64,75% dự báo ngây thơ
4	Cổ phiếu AAPL	SimpleRNN (PyTorch)	RMSE bằng 99,90% dự báo ngây thơ
4	Quỹ vàng GLD	SimpleRNN (PyTorch)	RMSE bằng 100,72% dự báo ngây thơ
5	Truy xuất (ba miền)	Lai RRF + KG	MRR 97,89% (e5-small 50,20%)
5	Hỏi đáp cổ phiếu	Tác tử định tuyến công cụ	Acc 100,00% (LLM thuần 0,00%)
6	GSM8K (35 câu)	CoT một tác tử	Acc 65,71% (đa tác tử 57,14%)

Các bài học xuyên suốt

- **Mô hình phải khớp với cấu trúc dữ liệu.** Gradient boosting hợp quan hệ ngưỡng, Lasso hợp quan hệ gần tuyến tính, CNN hợp ảnh, KG hợp tra cứu dữ kiện. Mô hình phức tạp hơn không tự động tốt hơn.
- **Phải có đường cơ sở.** Accuracy 91,18% của baseline tiểu đường và R^2 cao trên chuỗi giá đều vô nghĩa nếu không so với mô hình đối chứng.
- **Framework không quyết định chất lượng.** Trên cùng kiến trúc, NumPy, Keras và PyTorch cho kết quả gần nhau; khác biệt lớn nằm ở tốc độ và công sức.
- **Hiểu dữ liệu quan trọng hơn chọn mô hình.** Chuẩn hoá sai làm hỏng LSTM trên AAPL; hiểu sai cột `Close` của FINAL_USO làm sai cả kho tri thức.
- **Thêm thành phần là thêm chi phí và lỗi mới.** Hệ đa tác tử chỉ đáng dùng khi có tín hiệu kiểm chứng đáng tin; kết luận về thành phần này còn bị giới hạn bởi cỡ mẫu.

Hạn chế và hướng phát triển

Các mô hình dự đoán y tế và tài chính chỉ mang tính học thuật, không thay thế chẩn đoán hay tư vấn đầu tư. Bộ dữ liệu giá nhà là dữ liệu tổng hợp; bộ câu hỏi RAG được sinh tự động; thực nghiệm đa tác tử có mẫu nhỏ và chỉ dùng một LLM 1,5 tỷ tham số. Hướng phát triển gồm: thử nghiệm trên dữ liệu Việt Nam, dự báo xác suất thay vì một điểm, liên kết thực thể bằng mô hình học, đánh giá RAG với câu hỏi người dùng thật, và lặp lại thực nghiệm đa tác tử với mô hình lớn hơn trên toàn bộ tập GSM8K.

Tài liệu tham khảo

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020.

[2] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations*. 2023.

[3] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. “Array Programming with NumPy”. In: *Nature* 585.7825 (2020), pp. 357–362.

[4] Martín Abadi, Paul Barham, Jianmin Chen, et al. “TensorFlow: A System for Large-Scale Machine Learning”. In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 2016, pp. 265–283.

[5] François Chollet et al. *Keras*. `https://keras.io`. 2015.

[6] Adam Paszke, Sam Gross, Francisco Massa, et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019.

[7] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

[8] Tianqi Chen and Carlos Guestrin. “XGBoost: A Scalable Tree Boosting System”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 785–794.

[9] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. “Exploring Network Structure, Dynamics, and Function using NetworkX”. In: *Proceedings of the 7th Python in Science Conference (SciPy)*. 2008, pp. 11–15.

[10] Thomas Wolf, Lysandre Debut, Victor Sanh, et al. “Transformers: State-of-the-Art Natural Language Processing”. In: *Proceedings of EMNLP 2020: System Demonstrations*. 2020, pp. 38–45.

[11] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2021.

[12] Nils J. Nilsson. *The Quest for Artificial Intelligence: A History of Ideas and Achievements*. Cambridge University Press, 2010.

[13] Alan M. Turing. “Computing Machinery and Intelligence”. In: *Mind* 59.236 (1950), pp. 433–460.

[14] John R. Searle. “Minds, Brains, and Programs”. In: *Behavioral and Brain Sciences* 3.3 (1980), pp. 417–424.

[15] Norbert Wiener. *Cybernetics: Or Control and Communication in the Animal and the Machine*. MIT Press, 1948.

[16] Claude E. Shannon. “A Mathematical Theory of Communication”. In: *Bell System Technical Journal* 27.3 (1948), pp. 379–423.

[17] Warren S. McCulloch and Walter Pitts. “A Logical Calculus of the Ideas Immanent in Nervous Activity”. In: *Bulletin of Mathematical Biophysics* 5.4 (1943), pp. 115–133.

[18] Frank Rosenblatt. “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain”. In: *Psychological Review* 65.6 (1958), pp. 386–408.

[19] John McCarthy et al. “A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence, August 31, 1955”. In: *AI Magazine* 27.4 (2006), pp. 12–14.

[20] Allen Newell and Herbert A. Simon. “The Logic Theory Machine: A Complex Information Processing System”. In: *IRE Transactions on Information Theory* 2.3 (1956), pp. 61–79.

[21] Joseph Weizenbaum. “ELIZA—A Computer Program for the Study of Natural Language Communication between Man and Machine”. In: *Communications of the ACM* 9.1 (1966), pp. 36–45.

[22] Terry Winograd. “Understanding Natural Language”. In: *Cognitive Psychology* 3.1 (1972), pp. 1–191.

[23] Arthur L. Samuel. “Some Studies in Machine Learning Using the Game of Checkers”. In: *IBM Journal of Research and Development* 3.3 (1959), pp. 210–229.

[24] Allen Newell and Herbert A. Simon. “Computer Science as Empirical Inquiry: Symbols and Search”. In: *Communications of the ACM* 19.3 (1976), pp. 113–126.

[25] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press, 1969.

[26] Automatic Language Processing Advisory Committee. *Language and Machines: Computers in Translation and Linguistics*. Washington, DC: National Academy of Sciences, National Research Council, 1966.

[27] James Lighthill. “Artificial Intelligence: A General Survey”. In: *Artificial Intelligence: A Paper Symposium*. London: Science Research Council, 1973.

[28] Edward H. Shortliffe. *Computer-Based Medical Consultations: MYCIN*. New York: Elsevier, 1976.

[29] Bruce G. Buchanan and Edward H. Shortliffe. *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project*. Addison-Wesley, 1984.
[30] John McDermott. "R1: A Rule-Based Configurer of Computer Systems". In: *Artificial Intelligence* 19.1 (1982), pp. 39–88.
[31] Douglas B. Lenat. "CYC: A Large-Scale Investment in Knowledge Infrastructure". In: *Communications of the ACM* 38.11 (1995), pp. 33–38.
[32] John J. Hopfield. "Neural Networks and Physical Systems with Emergent Collective Computational Abilities". In: *Proceedings of the National Academy of Sciences* 79.8 (1982), pp. 2554–2558.
[33] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. "Learning Representations by Back-Propagating Errors". In: *Nature* 323.6088 (1986), pp. 533–536.
[34] Rodney A. Brooks. "Intelligence without Representation". In: *Artificial Intelligence* 47.1–3 (1991), pp. 139–159.
[35] Judea Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 1988.
[36] Corinna Cortes and Vladimir Vapnik. "Support-Vector Networks". In: *Machine Learning* 20.3 (1995), pp. 273–297.
[37] J. Ross Quinlan. "Induction of Decision Trees". In: *Machine Learning* 1.1 (1986), pp. 81–106.
[38] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018.
[39] Gerald Tesauro. "Temporal Difference Learning and TD-Gammon". In: *Communications of the ACM* 38.3 (1995), pp. 58–68.
[40] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780.
[41] Yann LeCun et al. "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
[42] Murray Campbell, A. Joseph Hoane, and Feng-hsiung Hsu. "Deep Blue". In: *Artificial Intelligence* 134.1–2 (2002), pp. 57–83.
[43] Sebastian Thrun, Mike Montemerlo, Hendrik Dahlkamp, et al. "Stanley: The Robot that Won the DARPA Grand Challenge". In: *Journal of Field Robotics* 23.9 (2006), pp. 661–692.
[44] David Ferrucci, Eric Brown, Jennifer Chu-Carroll, et al. "Building Watson: An Overview of the DeepQA Project". In: *AI Magazine* 31.3 (2010), pp. 59–79.
[45] Jia Deng et al. "ImageNet: A Large-Scale Hierarchical Image Database". In: *IEEE Conference on Computer Vision and Pattern Recognition*. 2009, pp. 248–255.
[46] Geoffrey E. Hinton, Simon Osindero, and Yee-Whye Teh. "A Fast Learning Algorithm for Deep Belief Nets". In: *Neural Computation* 18.7 (2006), pp. 1527–1554.
[47] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep Learning". In: *Nature* 521.7553 (2015), pp. 436–444.
[48] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
[49] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 25. 2012, pp. 1097–1105.
[50] Olga Russakovsky et al. "ImageNet Large Scale Visual Recognition Challenge". In: *International Journal of Computer Vision* 115.3 (2015), pp. 211–252.
[51] Kaiming He et al. "Deep Residual Learning for Image Recognition". In: *IEEE Conference on Computer Vision and Pattern Recognition*. 2016, pp. 770–778.
[52] David Silver et al. "Mastering the Game of Go with Deep Neural Networks and Tree Search". In: *Nature* 529.7587 (2016), pp. 484–489.
[53] David Silver et al. "Mastering the Game of Go without Human Knowledge". In: *Nature* 550.7676 (2017), pp. 354–359.
[54] Ian Goodfellow et al. "Generative Adversarial Nets". In: *Advances in Neural Information Processing Systems*. Vol. 27. 2014.
[55] Diederik P. Kingma and Max Welling. "Auto-Encoding Variational Bayes". In: *International Conference on Learning Representations*. 2014.
[56] John Jumper, Richard Evans, Alexander Pritzel, et al. "Highly Accurate Protein Structure Prediction with AlphaFold". In: *Nature* 596.7873 (2021), pp. 583–589.
[57] Tomas Mikolov et al. "Efficient Estimation of Word Representations in Vector Space". In: *arXiv preprint arXiv:1301.3781* (2013).

[58] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. “Sequence to Sequence Learning with Neural Networks”. In: *Advances in Neural Information Processing Systems*. Vol. 27. 2014.

[59] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. “Neural Machine Translation by Jointly Learning to Align and Translate”. In: *International Conference on Learning Representations*. 2015.

[60] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 5998–6008.

[61] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of NAACL-HLT*. 2019, pp. 4171–4186.

[62] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. Tech. rep. OpenAI, 2019.

[63] Tom B. Brown et al. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 1877–1901.

[64] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, et al. “On the Opportunities and Risks of Foundation Models”. In: *arXiv preprint arXiv:2108.07258* (2021).

[65] Jared Kaplan, Sam McCandlish, Tom Henighan, et al. “Scaling Laws for Neural Language Models”. In: *arXiv preprint arXiv:2001.08361* (2020).

[66] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. “Training Compute-Optimal Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022.

[67] Jaime Sevilla et al. “Compute Trends Across Three Eras of Machine Learning”. In: *International Joint Conference on Neural Networks (IJCNN)*. 2022.

[68] Long Ouyang, Jeff Wu, Xu Jiang, et al. “Training Language Models to Follow Instructions with Human Feedback”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 27730–27744.

[69] Paul F. Christiano et al. “Deep Reinforcement Learning from Human Preferences”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.

[70] Jonathan Ho, Ajay Jain, and Pieter Abbeel. “Denoising Diffusion Probabilistic Models”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 6840–6851.

[71] Gemini Team, Google. “Gemini: A Family of Highly Capable Multimodal Models”. In: *arXiv preprint arXiv:2312.11805* (2023).

[72] Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. “LLaMA: Open and Efficient Foundation Language Models”. In: *arXiv preprint arXiv:2302.13971* (2023).

[73] Llama Team, AI @ Meta. “The Llama 3 Herd of Models”. In: *arXiv preprint arXiv:2407.21783* (2024).

[74] OpenAI. “OpenAI o1 System Card”. In: *arXiv preprint arXiv:2412.16720* (2024).

[75] DeepSeek-AI. “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning”. In: *arXiv preprint arXiv:2501.12948* (2025).

[76] Qwen Team. “Qwen2.5 Technical Report”. In: *arXiv preprint arXiv:2412.15115* (2024).

[77] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, et al. “Toolformer: Language Models Can Teach Themselves to Use Tools”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023.

[78] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, et al. “Datasheets for Datasets”. In: *Communications of the ACM* 64.12 (2021), pp. 86–92.

[79] Dario Amodei et al. “Concrete Problems in AI Safety”. In: *arXiv preprint arXiv:1606.06565* (2016).

[80] Yoshua Bengio et al. *International AI Safety Report*. Tech. rep. DSIT 2025/001. UK Department for Science, Innovation and Technology, 2025.

[81] European Parliament and Council of the European Union. *Regulation (EU) 2024/1689 Laying Down Harmonised Rules on Artificial Intelligence (Artificial Intelligence Act)*. Official Journal of the European Union, L series. 2024.

[82] Thủ tướng Chính phủ. *Quyết định 127/QĐ-TTg ban hành Chiến lược quốc gia về nghiên cứu, phát triển và ứng dụng trí tuệ nhân tạo đến năm 2030*. Hà Nội, ngày 26/01/2021. 2021.

[83] François Chollet. “On the Measure of Intelligence”. In: *arXiv preprint arXiv:1911.01547* (2019).

[84] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, et al. “Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program)”. In: *Journal of Machine Learning Research* 22.164 (2021), pp. 1–20.

[85] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.

[86] Arthur E. Hoerl and Robert W. Kennard. “Ridge regression: Biased estimation for nonorthogonal problems”. In: *Technometrics* 12.1 (1970), pp. 55–67.

[87] Robert Tibshirani. “Regression shrinkage and selection via the lasso”. In: *Journal of the Royal Statistical Society: Series B* 58.1 (1996), pp. 267–288.

[88] David R. Cox. “The regression analysis of binary sequences”. In: *Journal of the Royal Statistical Society: Series B* 20.2 (1958), pp. 215–242.
[89] Leo Breiman et al. *Classification and Regression Trees*. Wadsworth, 1984.
[90] J. Ross Quinlan. “Induction of decision trees”. In: *Machine Learning* 1.1 (1986), pp. 81–106.
[91] Leo Breiman. “Bagging predictors”. In: *Machine Learning* 24.2 (1996), pp. 123–140.
[92] Leo Breiman. “Random forests”. In: *Machine Learning* 45.1 (2001), pp. 5–32.
[93] Jerome H. Friedman. “Greedy function approximation: A gradient boosting machine”. In: *Annals of Statistics* 29.5 (2001), pp. 1189–1232.
[94] Tianqi Chen and Carlos Guestrin. “XGBoost: A scalable tree boosting system”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 785–794.
[95] Guolin Ke et al. “LightGBM: A highly efficient gradient boosting decision tree”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.
[96] Corinna Cortes and Vladimir Vapnik. “Support-vector networks”. In: *Machine Learning* 20.3 (1995), pp. 273–297.
[97] Harris Drucker et al. “Support vector regression machines”. In: *Advances in Neural Information Processing Systems* 9 (1997), pp. 155–161.
[98] Thomas Cover and Peter Hart. “Nearest neighbor pattern classification”. In: *IEEE Transactions on Information Theory* 13.1 (1967), pp. 21–27.
[99] Andrew McCallum and Kamal Nigam. “A comparison of event models for naive Bayes text classification”. In: *AAAI-98 Workshop on Learning for Text Categorization*. 1998, pp. 41–48.
[100] Tom Fawcett. “An introduction to ROC analysis”. In: *Pattern Recognition Letters* 27.8 (2006), pp. 861–874.
[101] Takaya Saito and Marc Rehmsmeier. “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets”. In: *PLoS ONE* 10.3 (2015), e0118432.
[102] Stuart Geman, Elie Bienenstock, and René Doursat. “Neural networks and the bias/variance dilemma”. In: *Neural Computation* 4.1 (1992), pp. 1–58.
[103] Ghanshyam Saini. *Diabetes Prediction Dataset*. Kaggle, https://www.kaggle.com/datasets/ghnshymsaini/diabetes-prediction-dataset. Truy cập 10/2026. 2024.
[104] American Diabetes Association. *2. Classification and Diagnosis of Diabetes: Standards of Care in Diabetes—2023*. Diabetes Care, 46(Suppl. 1), S19–S40. 2023.
[105] Chershi. *House Price Prediction Dataset (2000 rows)*. Kaggle, https://www.kaggle.com/datasets/chershi/house-price-prediction-dataset-2000-rows. Truy cập 10/2026. 2025.
[106] Nick Brooks. *Women’s E-Commerce Clothing Reviews*. Kaggle, https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews. Truy cập 10/2026. 2018.
[107] Gerard Salton and Christopher Buckley. “Term-weighting approaches in automatic text retrieval”. In: *Information Processing & Management* 24.5 (1988), pp. 513–523.
[108] Ron Kohavi. “A study of cross-validation and bootstrap for accuracy estimation and model selection”. In: *Proceedings of the 14th International Joint Conference on Artificial Intelligence*. 1995, pp. 1137–1145.
[109] Jason D. M. Rennie et al. “Tackling the poor assumptions of naive Bayes text classifiers”. In: *Proceedings of the 20th International Conference on Machine Learning*. 2003, pp. 616–623.
[110] Diederik P. Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. In: *International Conference on Learning Representations*. 2015.
[111] Sergey Ioffe and Christian Szegedy. “Batch normalization: Accelerating deep network training by reducing internal covariate shift”. In: *International Conference on Machine Learning*. 2015, pp. 448–456.
[112] Nitish Srivastava et al. “Dropout: A simple way to prevent neural networks from overfitting”. In: *Journal of Machine Learning Research* 15.1 (2014), pp. 1929–1958.
[113] Alexandru Niculescu-Mizil and Rich Caruana. “Predicting good probabilities with supervised learning”. In: *Proceedings of the 22nd International Conference on Machine Learning*. 2005, pp. 625–632.
[114] David H. Hubel and Torsten N. Wiesel. “Receptive fields, binocular interaction and functional architecture in the cat’s visual cortex”. In: *The Journal of Physiology* 160.1 (1962), pp. 106–154.
[115] Kunihiko Fukushima. “Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position”. In: *Biological Cybernetics* 36.4 (1980), pp. 193–202.
[116] Yann LeCun et al. “Backpropagation applied to handwritten zip code recognition”. In: *Neural Computation* 1.4 (1989), pp. 541–551.
[117] Vinod Nair and Geoffrey E. Hinton. “Rectified linear units improve restricted Boltzmann machines”. In: *International Conference on Machine Learning (ICML)*. 2010, pp. 807–814.

[118] Connor Shorten and Taghi M. Khoshgoftaar. "A survey on image data augmentation for deep learning". In: *Journal of Big Data* 6.1 (2019), p. 60.

[119] Olga Russakovsky et al. "ImageNet large scale visual recognition challenge". In: *International Journal of Computer Vision* 115.3 (2015), pp. 211–252.

[120] Karen Simonyan and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition". In: *International Conference on Learning Representations (ICLR)*. 2015.

[121] Christian Szegedy et al. "Going deeper with convolutions". In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 1–9.

[122] Kaiming He et al. "Deep residual learning for image recognition". In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778.

[123] Gao Huang et al. "Densely connected convolutional networks". In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 4700–4708.

[124] Andrew G. Howard et al. "MobileNets: Efficient convolutional neural networks for mobile vision applications". In: *arXiv preprint arXiv:1704.04861* (2017).

[125] Mark Sandler et al. "MobileNetV2: Inverted residuals and linear bottlenecks". In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 4510–4520.

[126] Mingxing Tan and Quoc V. Le. "EfficientNet: Rethinking model scaling for convolutional neural networks". In: *International Conference on Machine Learning (ICML)*. 2019, pp. 6105–6114.

[127] Han Xiao, Kashif Rasul, and Roland Vollgraf. "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms". In: *arXiv preprint arXiv:1708.07747* (2017).

[128] Laurence Moroney. *Rock, Paper, Scissors Dataset*. Laurence Moroney Datasets. 2019.

[129] Alex Krizhevsky. *Learning Multiple Layers of Features from Tiny Images*. Tech. rep. University of Toronto, 2009.

[130] Leslie N. Smith and Nicholay Topin. "Super-convergence: Very fast training of neural networks using large learning rates". In: *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*. Vol. 11006. Proc. SPIE. 2019.

[131] Kumar Chellapilla, Sidd Puri, and Patrice Simard. "High performance convolutional neural networks for document processing". In: *Tenth International Workshop on Frontiers in Handwriting Recognition* (2006).

[132] Ramprasaath R. Selvaraju et al. "Grad-CAM: Visual explanations from deep networks via gradient-based localization". In: *IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 618–626.

[133] Jeffrey L. Elman. "Finding Structure in Time". In: *Cognitive Science* 14.2 (1990), pp. 179–211.

[134] Paul J. Werbos. "Backpropagation Through Time: What It Does and How to Do It". In: *Proceedings of the IEEE* 78.10 (1990), pp. 1550–1560.

[135] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. "Learning Long-Term Dependencies with Gradient Descent is Difficult". In: *IEEE Transactions on Neural Networks* 5.2 (1994), pp. 157–166.

[136] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. "On the Difficulty of Training Recurrent Neural Networks". In: *Proceedings of ICML*. 2013, pp. 1310–1318.

[137] Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins. "Learning to Forget: Continual Prediction with LSTM". In: *Neural Computation* 12.10 (2000), pp. 2451–2471.

[138] Kyunghyun Cho et al. "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation". In: *Proceedings of EMNLP*. 2014, pp. 1724–1734.

[139] Junyoung Chung et al. "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling". In: *NIPS Deep Learning Workshop*. 2014.

[140] Mike Schuster and Kuldip K. Paliwal. "Bidirectional Recurrent Neural Networks". In: *IEEE Transactions on Signal Processing* 45.11 (1997), pp. 2673–2681.

[141] Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton. "Speech Recognition with Deep Recurrent Neural Networks". In: *IEEE ICASSP*. 2013, pp. 6645–6649.

[142] Umut Tuygur. *E-Commerce Customer Behavior & Sales Analysis – TR*. Kaggle Dataset, `https://www.kaggle.com/datasets/umuttuygurr/e-commerce-customer-behavior-and-sales-analysis-tr`. 2025.

[143] Nitiraj Kulkarni. *AAPL Stock Performance*. Kaggle Dataset, `https://www.kaggle.com/datasets/nitirajkulkarni/aapl-stock-performance`. 2024.

[144] Sid Saini. *Gold Price Prediction Dataset*. Kaggle Dataset, `https://www.kaggle.com/datasets/sid321axn/gold-price-prediction-dataset`. 2019.

[145] François Chollet. *Deep Learning with Python*. 2nd ed. Manning, 2021.

[146] Diederik P. Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations (ICLR)* (2015).
[147] Eugene F. Fama. "Efficient Capital Markets: A Review of Theory and Empirical Work". In: *The Journal of Finance* 25.2 (1970), pp. 383–417.
[148] Aidan Hogan et al. "Knowledge Graphs". In: *ACM Computing Surveys* 54.4 (2021), 71:1–71:37.
[149] Amit Singhal. *Introducing the Knowledge Graph: things, not strings*. Google Official Blog. 16/05/2012. 2012.
[150] Steve Harris and Andy Seaborne. *SPARQL 1.1 Query Language*. W3C Recommendation. 2013.
[151] Nadime Francis et al. "Cypher: An Evolving Query Language for Property Graphs". In: *Proceedings of ACM SIGMOD 2018*. 2018, pp. 1433–1445.
[152] Antoine Bordes et al. "Translating Embeddings for Modeling Multi-relational Data". In: *Advances in Neural Information Processing Systems 26*. 2013, pp. 2787–2795.
[153] Stephen Robertson and Hugo Zaragoza. "The Probabilistic Relevance Framework: BM25 and Beyond". In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389.
[154] Vladimir Karpukhin et al. "Dense Passage Retrieval for Open-Domain Question Answering". In: *Proceedings of EMNLP 2020*. 2020, pp. 6769–6781.
[155] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods". In: *Proceedings of the 32nd ACM SIGIR Conference*. 2009, pp. 758–759.
[156] Rodrigo Nogueira and Kyunghyun Cho. "Passage Re-ranking with BERT". In: *arXiv preprint arXiv:1901.04085* (2019).
[157] Darren Edge et al. "From Local to Global: A Graph RAG Approach to Query-Focused Summarization". In: *arXiv preprint arXiv:2404.16130* (2024).
[158] Akari Asai et al. "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection". In: *International Conference on Learning Representations (ICLR)*. 2024.
[159] Shahul Es et al. "RAGAS: Automated Evaluation of Retrieval Augmented Generation". In: *Proceedings of EACL 2024: System Demonstrations*. 2024, pp. 150–158.
[160] Qwen Team. "Qwen2.5 Technical Report". In: *arXiv preprint arXiv:2412.15115* (2024).
[161] Liang Wang et al. "Text Embeddings by Weakly-Supervised Contrastive Pre-training". In: *arXiv preprint arXiv:2212.03533* (2022).
[162] Jianlv Chen et al. "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation". In: *arXiv preprint arXiv:2402.03216* (2024).
[163] ghnshymsaini. *Diabetes Prediction Dataset*. Kaggle, https : / / www . kaggle . com / datasets / ghnshymsaini/diabetes-prediction-dataset. 2024.
[164] Jillani SofTech. *Google 10 Years Stock Price Dataset*. Kaggle, https://www.kaggle.com/datasets/jillanisofttech/google-10-years-stockprice-dataset. 2024.
[165] American Diabetes Association Professional Practice Committee. "2. Diagnosis and Classification of Diabetes: Standards of Care in Diabetes – 2024". In: *Diabetes Care* 47.Supplement 1 (2024), S20–S42.
[166] World Health Organization. *Obesity: Preventing and Managing the Global Epidemic*. Technical Report Series 894. WHO, 2000.
[167] Grégoire Mialon, Roberto Dessì, Maria Lomeli, et al. "Augmented Language Models: a Survey". In: *Transactions on Machine Learning Research* (2023).
[168] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2021.
[169] Michael Wooldridge. *An Introduction to MultiAgent Systems*. 2nd ed. John Wiley & Sons, 2009.
[170] Yilun Du et al. "Improving Factuality and Reasoning in Language Models through Multiagent Debate". In: *International Conference on Machine Learning (ICML)*. 2024.
[171] Lee D. Erman et al. "The Hearsay-II Speech-Understanding System: Integrating Knowledge to Resolve Uncertainty". In: *ACM Computing Surveys* 12.2 (1980), pp. 213–253.
[172] Anthropic. *Model Context Protocol Specification*. https://modelcontextprotocol.io. 2024.
[173] Google. *Agent2Agent (A2A) Protocol*. https://a2a-protocol.org. 2025.
[174] Qingyun Wu et al. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation". In: *arXiv preprint arXiv:2308.08155* (2023).
[175] Mert Cemri et al. "Why Do Multi-Agent LLM Systems Fail?" In: *arXiv preprint arXiv:2503.13657* (2025).
[176] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, et al. "Training Verifiers to Solve Math Word Problems". In: *arXiv preprint arXiv:2110.14168* (2021).

[177] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022.

[178] Luyu Gao et al. “PAL: Program-aided Language Models”. In: *International Conference on Machine Learning (ICML)*. 2023, pp. 10764–10799.

[179] Xuezhi Wang et al. “Self-Consistency Improves Chain of Thought Reasoning in Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2023.

[180] Jie Huang et al. “Large Language Models Cannot Self-Correct Reasoning Yet”. In: *International Conference on Learning Representations (ICLR)* (2024).